



Universidad Tecnológica de Xicotepec de Juárez

Ing. En Desarrollo y Gestión de Software

Gestión de Proyectos de desarrollo de Software

Documentación para el Desarrollo de la Aplicación
Móvil para la
Dirección General Hospitaliza

10º A

Crespo Alvarado Carlos Iván – 210237

Karen Alyn Fosado Rodríguez - 210764

Contenido

Resumen del Proyecto: Desarrollo de la Aplicación Móvil para el Área de Dirección General Hospitalaria	4
Estado del Arte	5
Objetivo General	14
Objetivos Específicos	15
Planteamiento del problema	17
Hipótesis:	19
Justificación	21
Marco Teórico	23
Diagramas UML	27
Diagrama de Casos de Uso	30
Modelo Entidad Relación	31
Microsoft Visio	39
Microsoft Word	42
Figma	45
Python	51
SQL	54
MySQL	58
Framework	62
FastAPI (Backend Framework)	63
Flutter (Frontend Framework)	64
Overview (Monitoreo de Bases de Datos)	65
Amazon Amplify (Alternativa para Despliegue y Escalabilidad)	69
Render (Despliegue en la Nube)	72
Marco Metodológico	77
Método o Metodologías	77
Metodología Scrum	77
DevOps	78
Presentación de la conexión metodológica	79
Presentación de los Instrumentos de elaboración	82
Antecedentes Epidemiológicos y Estructuración de los Requerimientos Funcionales	82
Coordinación sectorial e Intersectorial	83
Ruta de atención Medica Continua	84
Logística y Operación	85
Conservación Y Mantenimiento	86

Innovaciones en gestión hospitalaria en México.	87
Infraestructura y Sistemas de Información	87
Requerimientos no funcionales	90
Diseño del Proyecto	92
Diagramas de caos de uso	92
Flujo de Trabajo:	95
Conclusión:	96
Contexto del negocio	96
Organización de la Empresa	98
Áreas Hospitalarias Involucradas	99
Proceso De Negocio	103
Maquetación y Bocetos	104
☐ Fondo: Degradado azul cielo a blanco.	105
☐ Campos de texto: Fondo blanco con bordes grises claros.	105
☐ Botones: Azul oscuro con texto blanco.	105
Desarrollo	114
Identificación de Atributos de Cada Entidad	115
Construcción del Back-End	127
Desarrollo de la Aplicación con el entorno Flutter	144
Ejecución de la Aplicación	198
Limitaciones del proyecto	203
Índice de Imágenes	205
Bibliografía	209

Resumen del Proyecto: Desarrollo de la Aplicación Móvil para el Área de Dirección General Hospitalaria

El proyecto se desarrollará para integrar las actividades de gestión y operación en las solicitudes provenientes de otras áreas médicas involucradas en el ejercicio del servicio clínico, en conjunto todas las áreas que operan dentro del hospital Privilege Care, establecerán criterios de evaluación sistemática que agilicen sus demandas evaluadas por un comité especializado que priorice y actualice los estados de dichas solicitudes, mejorando el rendimiento y la tasa de éxito de las operaciones realizadas en un tiempo de gestión más optimizado y organizado. La aplicación a desarrollar para el área de Dirección General mejorará la eficiencia operativa permitiéndole una colaboración fluida entre las áreas del hospital en tiempo real, la plataforma unificará la toma de decisiones estratégicas basadas en datos precisos, promoviendo tanto la atención de alta calidad al paciente como una gestión optima de los recursos médicos y humanos.

Proyecto General:

El sistema abarca funciones clave para todos los departamentos del hospital, tales como:

- Gestión de Personal y Recursos Médicos: Supervisará la disponibilidad de personal, equipos médicos y suministros, generando alertas sobre posibles faltantes o sobrecargas de trabajo.
- Coordinación Interdepartamental: Facilitará la comunicación y colaboración entre áreas críticas, optimizando el flujo de información en situaciones de emergencia o demanda elevada.
- Gestión de Insumos: Facilitará la administración de insumos (Instrumental clínico, medicamentos, etc.).

Módulo Específico: Dirección General

El módulo de Dirección General proporcionará a los responsables del hospital una visión global y en tiempo real de las operaciones, habilitando una supervisión más precisa y un control integral.

Este módulo permitirá:

1. Supervisión de Indicadores Clave de Desempeño: Se monitorizarán datos de peticiones a través de solicitudes del personal, tales como gastos de insumos, información de personal, información de pacientes, etc.

2. Toma de Decisiones Estratégicas: Basada en datos consolidados y reportes generados a través de gráficas automáticas por el sistema, la Dirección General podrá realizar ajustes y mejoras en función de la demanda y los recursos.
3. Monitoreo de Solicitudes: Supervisará el flujo de solicitudes de recursos, personal y equipos médicos que provienen de las áreas operativas, priorizando los casos más urgentes.
4. Mejora en la Comunicación Interna: Mediante a las clasificaciones de las solicitudes, la Dirección General podrá coordinar acciones rápidas ante contingencias, asegurando que todas las áreas respondan de manera oportuna.
5. Permitirá establecer criterios de atención prioritaria a solicitudes de urgencia generada por los departamentos de atención de urgencias clínicas y epidemiológicas que pongan en riesgo a la población.
6. Se manejará un control y calidad del servicio en el análisis de las consultas dentro de la bitácora de actividades registradas en el movimiento de las solicitudes transparentando las peticiones de los servicios involucrados.

Impacto Esperado:

La integración del sistema de gestión hospitalaria ofrecerá un control total sobre las operaciones, optimizando los recursos, mejorando la experiencia del paciente y facilitando la gestión centralizada de la información hospitalaria. La aplicación móvil brindará a la Dirección General acceso inmediato a información crítica, permitiendo decisiones rápidas y fundamentadas en datos actualizados. Esto no solo mejorará la eficiencia operativa del hospital, sino que también fortalecerá el compromiso del hospital PrivilegeCare con un servicio médico de calidad y una atención al paciente oportuna y eficiente.

Estado del Arte

En su mayoría las instituciones del ejercicio de la Salud cuentan con problemas en la gestión de sus procesos de administración y gestión, debido a que el sistema sanitario público no está diseñado para ejercer un suministro constante para la atención de sus derecho habientes, reduciendo el número de atenciones de calidad a quienes son tomados como casos de urgencia o necesitan ser canalizados de manera inmediata para su intervención quirúrgica o tratamiento especial, la atención

y determinación de la estructura en la gestión clínica depende de varios factores importantes que establecen una normativa de seguimiento protocolaria que ejerce confianza y determinación en las actividades de los clínicos como de las asistencias en el manejo de cada caso.

Es por tanto que en la administración y gestión hospitalaria se toman decisiones basadas en hechos y datos precisos, que proviene de las actividades de cada área respectiva, por este medio la información relevante de las áreas de administración, de financiamiento y logística, son vitales para la toma de decisiones mejorando la calidad de la prestación del servicio de salud de los derechohabientes.

Para la recolección de información relevante de los pacientes se realizan testeos o cuestionarios que involucran una determinación en el registro de estos datos, dado que cada registro realizado debe ser procesado y evaluado, tomando en consideración rangos de edad, sexo y patología presentada, esta clasificación genera un nivel de atención probable de acuerdo al índice de morbilidad en la región estableciendo las patologías más comunes en el rango de edad de los pacientes que asisten a consulta médica, de acuerdo con el panorama epidemiológico en México, hasta el año 2014 las principales causas de morbilidad han sido las enfermedades infecciosas, como son las enfermedades agudas, las infecciones intestinales y las infecciones por vías urinarias, a otras causas también se agrega la inflamación del aparato digestivo, como la detección de úlceras por causa de gastritis en el saco estomacal y duodenal, así como Gingivitis y enfermedades periodontales, también se registran la Conjuntivitis ocular, el diagnóstico de diabetes Mellitus tipo 2, siendo la obesidad el detonante principal, entre otras causas bacterianas, virales, fúngicas y parasitarias, se ha detectado la Vulvovaginitis aguda, Asma y estado Asmático, Intoxicación por picadura de alacrán, Amibiasis Intestinal, Candidiasis Urogenital, síndrome febril, varicela, Helmintiasis, Neumonías y Bronconeumonías, faringitis y amigdalitis estreptocócica, y otras causas, No obstante cabe mencionar que al comparar el comportamiento de enfermedades infecciosas frecuentes como las respiratorias y las intestinales, ha habido fluctuaciones en los niveles de atención [1].

Con el registro de las patologías presentadas la gestión de acuerdo con la recopilación de datos registrados en el Sistema Nominal de información en Crónicas (SIC) y determinación en el nivel de atención médica, es crucial para que el estado de los pacientes ingresados mejore de acuerdo con el nivel de tratamiento establecido, es por tanto que si la atención presentada en las áreas

hospitalarias es óptima el flujo de pacientes no presentará estancamientos ni colapsos, evitando una sobrecarga del servicio, esto ha dejado una huella particular en la atención de la población dentro de la prestación del servicio médico público, por lo que la prestación del servicio hospitalario ha dejado un papel muy claro en los casos de atención más relevantes, de acuerdo a datos estadísticos, En el año 2013 se habían registrado en todo el país 5866493 egresos hospitalarios, en su mayoría correspondiendo con un 38.4% al área de Ginecoobstetricia, seguida con un 23.9% a las áreas de Cirugía, sin embargo esto ha ido en declive en la última década, se ha observado un descenso en el porcentaje de estas dos áreas junto con la de pediatría y el incremento se ha notado en medicina interna, donde paso de 18.4 a 20.8% desde el año 2000 [1].

Se establece dentro de la gestión en el control de flujo de datos para el intercambio de información en salud referente al reporte al subsistema de prestación de servicios 'SIS' (Sistema de Información en Salud) y consulta externa, en su justificación se implementa que; En cumplimiento a lo previsto en la NOM-024-SSA3-2012 y la NOM-035-SSA3-2012, para que de manera efectiva se pueda contar con información estandarizada, comparable y oportuna prudente de los Sistemas de Información de Registro Electrónico para la Salud, entre los que se encuentran los Expedientes Clínicos Electrónicos; y mediante esta Guía se habite la interoperabilidad e intercambio con el Sistema Nacional de Información Básica en Materia de Salud para el acopio, conservación, procesamiento, generación de información útil, con base a los requerimientos de los programas de salud de nivel federal, que permita contar con información de consulta externa para la adecuada toma de decisiones, la planeación, asignación de recursos y evaluación de la calidad y los servicios proporcionados; así mismo, sea fuente de información para la investigación clínica y epidemiológica. [2, p. 6]. Estos lineamientos establecidos permiten tomar muestras en los casos de estudio partiendo de poblaciones que pueden estar expuestas a ciertas patologías y sintomatologías, la información que se compila en cada centro de salud es analizada para proporcionar datos estadísticos y con ellos ejercer planes de acción futura para el control de las enfermedades en los sectores más vulnerables de la población.

De acuerdo con los datos epidemiológicos, el nivel de atención se determina de acuerdo con el tipo de patologías presentadas en su población, con esto se han desarrollado sistemas de compilación de datos que abren un panorama claro de cómo se pueden prevenir y actuar, en casos de crisis humanitaria, pandemias o endemias, marcando un margen establecido en la preparación de los hospitales públicos como privados, que obedecen a normativas y convenciones que notifican las

medidas en el tratamiento de nuevas enfermedades, para ello se ha desarrollado una plataforma en la que se informa, La notificación Convencional de Casos Nuevos de Enfermedad se llevará a cabo operativamente con el formato de recolección de datos SUVIE-1 y el programa de cómputo SUAVE (“Sistema Único Automatizado para la Vigilancia Epidemiológica”), es importante señalar que la correcta operación del sistema depende de la participación de todo el personal de salud en sus diferentes escenarios. [3, p. 22]

Dentro de la plataforma SUAVE se realizan los procedimientos que se deben seguir para la realización de notificaciones de aquellos padecimientos sujetos a vigilancia epidemiológica tal como se dispone en la norma oficial mexicana NOM-017 SSA2-2012, hace referencia a los elementos que se llevan a cabo para realizar notificaciones de forma adecuada y oportuna, determinando tiempos y los procedimientos que se deben de seguir en los niveles operativos del Sector Salud, desde la unidad médica hasta la Dirección General, indicando los niveles de evaluación en la notificación y cobertura, derivado de la norma oficial NOM-017 SSA2-2012, para la vigilancia epidemiológica surge la necesidad de información correspondida a los nuevos perfiles de movilidad de la población y sus determinantes de acuerdo a la periodicidad semanal cambiando a convencional [4, pp. 9-10]

El sistema Nacional de Vigilancia Epidemiológica (SINAVE), se considera como la aplicación de medidas estratégicas y acciones en la epidemiología para la producción de datos epidemiológicos permitiendo generar información útil para la atención de la salud pública, dicha información es recopilada de todos los centros de atención del país pertenecientes al sistema nacional de salud. Este sistema genera información homogénea de los diferentes niveles de atención, la información se adentra en la ocurrencia, distribución en el tiempo, lugar y persona, factores de riesgo y consecuencia de las enfermedades que afectan la salud de la población, esta es registrada en una clasificación de acuerdo al nivel local, estatal y nacional, para orientar y apoyar la toma de decisiones para el diseño y aplicación de planes y programas de salud en todo el país.

Para mantener la vigilancia constante en la operación hospitalaria dentro de los hospitales generales y de especialidades se ha diseñado el sistema RHOVE (La Red Hospitalaria para la Vigilancia Epidemiológica), establecida para solventar la gestión en la información acerca de enfermedades de notificación obligatoria y de infecciones graves, operando dentro de la red de institutos nacionales de salud dentro de la república mexicana. La vigilancia de las IAAS (Las Infecciones Asociadas a la Atención de la Salud), en México está a cargo de RHOVE, este sistema de vigilancia centinela es

responsable de unificar los criterios para la recopilación dinámica, sistemática y continua de la información generada por cada unidad de atención medica hospitalaria afiliada mediante la NOM-045-SSA2-2005, para la vigilancia, prevención y control de las Infecciones Nosocomiales, y el manual de procedimientos estandarizados para la vigilancia epidemiológica, él envió de la información se realiza bajo un sistema informático en línea, permitiendo la conexión remota a la base de datos, proporcionando información en tiempo real, facilitando la gestión de los datos recopilados. [5, p. 10]

Las actividades hospitalarias dan carta abierta a la demanda de nuevas tecnologías que analicen de manera precisa como es el comportamiento en los niveles de atención en la población, para esto se han creado modelos de investigación, que simula como es el ejercicio clínico en unidades hospitalarias dentro de las áreas de emergencia. La gestión hospitalaria influye en los estándares de calidad y el uso de tecnologías que permiten evaluar de forma objetiva los servicios para dar apertura a metodologías de optimización de los recursos, estos modelos se han utilizado en los procesos de manufactura industrial de armadoras, y se ha aplicado al sistema de urgencias para la recopilación de datos con una metodología de corte transversal. Utilizando el software Arena, se simulo el flujo de trabajo de un mes de operación, esto genero estadísticas y realizando un proceso de animación en 30 días, generando tres distintos escenarios que identificaban al cambio del personal para optimizar el servicio, y se localizaron cuellos de botella dentro de las áreas de consulta externa y al área de cajas, dicha simulación permitió demostrar que con datos reales recopilados en el flujo de trabajo hospitalario se atendió en el área de administración a 3.7 pacientes por hora, del área de enfermería a 3.7 pacientes por hora, en las consultas médicas a 6 pacientes por hora, dejando un promedio de espera de los pacientes de 8 minutos para la atención y 18 minutos en el proceso de pago. [6].

La simulación realizada permitió generar puntos esenciales en la mejora continua del servicio de atención y dar certeza del tiempo en el que se efectúa la atención para los pacientes entrantes al área de emergencia, el análisis de los datos introducidos genera un mejor nivel de confianza para poder analizar con este método otras áreas de intervención como seria el área de pediatría, cirugía como área de rehabilitación y estancia hospitalaria para pacientes en recuperación, la implementación de la tecnología ha abierto un campo amplio en el análisis de datos y en la recopilación y creación de modelos estadísticos que prevengan problemas futuros.

Dentro de los estudios clínicos y modelos epidemiológicos se han creado herramientas relevantes que permiten el análisis minuciosos de grandes cantidades de datos tomados en gran escala, es por ello que la epidemiología digital ha propuesto modelos computacionales que han incrementado el estudio furtivo a nuevas fuentes de datos. Gracias a la interconexión, los modelos generan comportamientos sociales clave para entender el desenlace epidemiológico de ciertos sectores, dentro del análisis la Big Data ha permitido la refinación de modelos generando mapeos en el registro de la historia general de la enfermedad, optimizando los recursos y en propagación de campañas preventivas, esto a su vez ha generado la realización de protocolos, planes y políticas de acción institucionales dentro del sector salud público y privado, de acuerdo al estudio en base a el seguimiento del estudio epidemiológico, un elemento clave de los modelos epidemiológicos más sofisticados es introducir la complejidad de los contactos entre personas en forma de redes. El comportamiento de algunas enfermedades es sistemático y por tanto su propagación depende de la estructura del sustrato en el que se propagan, el estudio a escala de las redes sociales es útil para predecir la proporción de enfermedades y actuar en consecuencia. Las redes permiten modelar el componente de comportamiento de la transmisibilidad de las enfermedades mediante la caracterización de la topología de red y también su dinámica. [7, p. 4],

Este estudio propone como se debe de recolectar la información y como procesarla para el análisis posterior, para el entendimiento en la dinámica de la propagación de enfermedades, gracias al flujo de interacción entre los individuos mediante el uso de herramientas tecnológicas, datos como los de geolocalización y monitorización de desplazamiento de los dispositivos dentro del mapeo de áreas y regiones de mayor concentración poblacional, se generan comportamientos que pueden abrir camino en la difusión de concentraciones vulnerables, justificando que la movilidad tiene un impacto directo en la propagación de enfermedades de transmisión aérea a través de vectores que influyen en el ambiente que los rodea, incluso este comportamiento de altas concentraciones de individuos puede compararse en nuevas formas de movilidad y transmisión vistas en el turismo, eventos sociales, empresas y en el ejercicio de la educación.

Los modelos plantean una respuesta temprana y rápida a una epidemia, permitiendo controlar y mitigar su impacto, la predicción de estos modelos se basa fundamentalmente en las dimensiones geográficas, temporal y transmisión, las variables utilizadas para caracterizar los modelos son fundamentalmente demográficos en lo que se incluye la densidad de población, edad y sexo, limitando en rendimiento, las epidemias y la habilidad de los modelos para predecir la misma

propagación y su impacto. Variables como la vulnerabilidad, la desigualdad socioeconómica o las infraestructuras y saneamiento son clave para tener modelos más eficaces. [7, p. 8]

El uso de la información es crucial para un análisis más profundo en la toma de decisiones y planes de acción dentro del entorno de salud público y privado, ambos modelos parten de la misma interacción con la comunidad, poblaciones y factores epidemiológicos para implementar acciones que eviten que los sistemas de salud colapsen en caso de emergencia, la gestión hospitalaria ejerce una presión en medida de los casos que atienden dentro de la población vulnerable, para ello se retomará un punto crucial en el tercer nivel de atención hospitalaria, dentro de este ejercicio se cuentan con espacios adaptados para la prestación de servicios de alta especialidad, consulta interna y salas de operación y post operatoria, así como un área dedicada al manejo de los recursos humanos y recursos materiales, como una planeación logística que en vanguardia con las actividades diarias dentro de las instalaciones de salud se coordinan para llevar las tareas de distribución y manejo de instrumental, insumos clínicos y farmacia como equipos de gabinete y laboratorio, que ayudan a proporcionar datos esenciales para el diagnóstico y tratamiento de los pacientes.

Es por ello que la gestión hospitalaria debe de ejercer un control que brinde la seguridad en la atención en las solicitudes y demandas de áreas con flujo concurrente, para ser atendidas de manera óptima y segura para los tratamientos, seguimientos post operatorios y autorizaciones que salvaguarden la integridad y estabilidad de los pacientes en estado crítico. La administración de las actividades es vital para un ejercicio clínico eficaz, es por esta razón que el diseño de la aplicación se centra en la validación de actividades que necesiten una evaluación exhaustiva por la junta médica que evalúe de manera correcta la situación de pacientes que necesiten trasplantes, intervenciones de alto riesgo, traslados, tratamientos experimentales, como un control en caso de epidemias, endemias o pandemias, que afecten a un gran número de individuos en el rango de atención del sector poblacional al que se le presta servicio.

La aplicación móvil para la gestión en la Dirección Hospitalaria, está dirigida para coordinar una vigilancia continua de las peticiones de los departamentos que integra la estructura organizacional para el Hospital Privilege Care, esta organización permite establecer un límite entre el control de actividades como el flujo de permisos ejercidos dentro de un periodo establecido de tiempo en el cual recopila datos estadísticos para un análisis posterior en el flujo de las actividades ejercidas.

En ella se ejerce un control en las actividades entrantes, para un manejo en la transparencia para los ejercicios de auditoría y normativas de control en los tratamientos clínicos, enumerando de manera única cada petición registrada, para su validación los profesionales de salud solicitan los procedimientos y acciones a autorizar para que el comité responsable de legue la autorización de dichas solicitudes, dentro de la estructura interna del hospital se manejan eventos especiales a los que se les debe de dar prioridad los cuales se ejecutan de acuerdo a lo establecido dentro de la norma oficial mexicana NOM-027-SSA3-2013 encargada de la regulación de los servicios de salud, estableciendo criterios fundamentales en la atención en los servicios de urgencias de los establecimientos para la atención médica. Los cuales deben de ser atendidos directamente por las áreas de recepción y tratamiento oportuno, esto da prioridad para salvaguardar la vida de los pacientes en estado crítico.

Las actividades ejecutadas dentro de las instalaciones hospitalarias requieren de una coordinación constante con su personal de salud, es por ello que dentro de la gestión hospitalaria la aplicación móvil permitirá a los directivos de esta área implementar nuevos espacios de manera temporal y permanente para desahogar el flujo de actividades clínicas que estén en un pico alto de atención hacia sus pacientes entrantes, esto no solo evitará que el sistema colapse sino que permitirá a los clínicos dar una atención de calidad a sus pacientes, además de que optimizará los recursos disponibles del hospital a disposición de sus derecho habientes.

La organización y la gestión de peticiones serán monitorizadas dentro de la aplicación generando modelos acumulados de datos en gráficos que muestren si las actividades y acciones, como la toma de decisiones han permitido generar mejoras o deficiencias en la atención del sector poblacional, permitiendo generar estadísticas y pronósticos más precisos que sirvan como evidencia en la búsqueda de una mejora en la atención de salud ejercida dentro de las instalaciones hospitalarias.

Dentro del ejercicio clínico la información generada y almacenada dentro de la bitácora de actividades, no solo como actividades de transparencia sino como medida en el registro de las practicas medicas realizadas, es por ello que cada una de las peticiones realizadas es registrada dentro de la base de datos interna y será consultada para su estudio posterior dentro del análisis en las auditorías y comisiones encargadas dedicadas en la mejora del sector salud.

A diferencia de los modelos y sistemas mencionados con anterioridad, ejecutados para el servicio público y privado, la aplicación móvil de gestión hospitalaria para la supervisión de actividades de

validación y autorización de solicitudes, muestra datos en tiempo real y la información reflejada para su análisis es visible de manera inmediata, por lo que se pueden hacer cortes de supervisión los cuales permitan auditar las actividades y el flujo de las demandas realizadas en las operaciones clínicas hospitalarias, generando un ecosistema fiable y relevante para la toma de decisiones que anticipen escenarios de colapso institucional en el ejercicio clínico, como anticipar suministros de los departamentos con mayor demanda y la recolocación y reestructuración de áreas con mayor afluencia de pacientes entrantes.

El manejo de los datos adquiridos puede llevar a una evaluación más precisa en la administración de los departamentos como de la atención de escenarios catastróficos que podrían llevar al colapso en la atención de la salud pública.

Objetivo General

Desarrollar una aplicación móvil que facilite la gestión eficaz y la administración de la Dirección General del “Hospital PrivilegeCare”, utilizando herramientas de TIC para optimizar procesos, mejorar la gestión de solicitudes hospitalarias.

Objetivos Específicos

1. Fomentar la colaboración interdepartamental: Desarrollar una APP del módulo de dirección general que potencie la comunicación y colaboración entre departamentos del hospital, facilitando la gestión de solicitudes de manera eficiente.
2. Optimizar el proceso de envío de solicitudes: Desarrollar el módulo de dirección general que permita facilitar a los especialistas de todas las áreas hospitalarias enviar solicitudes de manera rápida y efectiva para su evaluación por parte de la Dirección General.
3. Priorizar solicitudes de manera automatizada: Desarrollar mecanismos de gestión de priorización basados en la urgencia y el tipo de solicitud, garantizando una atención inmediata a los casos críticos.
4. Aumentar la eficiencia en el flujo de solicitudes: Optimizar el flujo de solicitudes ajustándose a la demanda y las prioridades, minimizando cuellos de botella y asegurando una respuesta ágil a los casos urgentes.
5. Asegurar tiempos de respuesta efectivos: Desarrollar un mecanismo de alerta que garantice tiempos de respuesta eficientes para todas las solicitudes, asegurando que ninguna quede sin atender en un plazo razonable y mejorando la calidad del servicio.
6. Mejorar la coordinación de respuestas: Desarrollar un mecanismo de respuesta que garantice respuestas inmediatas y adecuadas a las solicitudes, asegurando que sean dirigidas correctamente a las áreas responsables.
7. Fortalecer el control y trazabilidad de solicitudes: Desarrollar un sistema de control que asegure que todas las solicitudes sean gestionadas por personal capacitado, incrementando la eficiencia y efectividad en cada intervención.
8. Realizar un análisis exhaustivo de datos: Implementar un sistema de recopilación de datos sobre solicitudes y respuestas, permitiendo un análisis posterior para identificar oportunidades de mejora y ajustar procesos que optimicen la gestión.

9. Realizar el control de historial de actividad de usuarios: Implementar un registro de actividad del usuario que permita visualizar las acciones realizadas dentro de la aplicación, mejorando la trazabilidad y transparencia.

10. Realizar el módulo con tecnologías: Desarrollar el proyecto mediante el uso de Frameworks de Desarrollo y tecnologías para el despliegue de Frontend y Backend tales como: Flutter, Android Studio, Python - Fast Api, BD SQL y NoSQL, Amazon Amplify, Aiven – Para la construcción de BD remota, Render – Gestión remota para despliegue para la BD remoto, Git Hub – Para monitorear el control de actividades del proyecto, Jira – Para gestión de incidencias.

Planteamiento del problema

La gestión hospitalaria en el sector privado enfrenta múltiples desafíos para mantener altos estándares de calidad en el servicio. Este esfuerzo se ve limitado por la capacidad de pago de los clientes, ya que los servicios médicos de calidad tienden a estar reservados para quienes pueden costearlos. En contraste, el sector público experimenta una gran demanda, lo que exige una atención masiva y de calidad, aunque a menudo se ve comprometida por procesos burocráticos extensos y una insuficiente capacitación del personal. Esta situación resulta en largos tiempos de espera, una capacidad de respuesta limitada ante emergencias y un tratamiento deficiente de enfermedades crónicas y degenerativas, afectando la satisfacción y bienestar del paciente.

A nivel operativo, los hospitales públicos como privados presentan un desequilibrio en la administración de sus recursos. En el sector público, la falta de un control centralizado y eficiente sobre la distribución de insumos y equipos médicos suele dar lugar a escasez de suministros o a compras innecesarias. En situaciones críticas, los médicos incluso se ven obligados a adquirir materiales con recursos propios, lo cual genera costos personales y deja expuesta la falta de un sistema robusto que gestione de forma precisa y oportuna los insumos y las solicitudes hospitalarias. En el sector privado, aunque los recursos están más disponibles, la ausencia de una gestión eficiente en tiempo real impide que la Dirección General del hospital tenga una visión clara y directa de las operaciones, lo cual limita su capacidad de respuesta y la optimización de sus servicios.

Otro reto significativo es la falta de integración y centralización en los sistemas de información hospitalarios. La ausencia de una plataforma digital centralizada en ambos sectores genera un entorno donde la información se encuentra dispersa y poco accesible, dificultando la toma de decisiones y afectando la coordinación entre departamentos.

Esto limita la capacidad de la Dirección General para analizar indicadores clave de desempeño y gestionar la operación hospitalaria de manera estratégica y basada en datos. Además, el no contar con reportes unificados y actualizados sobre la disponibilidad de camas, la productividad del personal y el estado de los equipos y suministros médicos crea cuellos de botella en el flujo de trabajo, afectando negativamente tanto al personal como a los pacientes.

A nivel de seguridad, existe una preocupación en cuanto a la privacidad de los datos médicos y el acceso seguro a la información. Tanto en el sector público como en el privado, la implementación

de un sistema robusto de protección de datos es fundamental, ya que el cumplimiento de las normativas de salud requiere no solo una gestión eficiente, sino también la garantía de que la información confidencial de los pacientes esté debidamente protegida y accesible solo para el personal autorizado. Sin un sistema adecuado, se aumenta el riesgo de violaciones de seguridad y de filtración de datos, poniendo en peligro tanto la privacidad de los pacientes como la reputación del hospital.

Ante estos problemas, es esencial implementar un sistema de gestión hospitalaria centralizado que permita conectar en tiempo real las áreas operativas y administrativas del hospital. Este sistema debe contar con funciones avanzadas de monitoreo de recursos, personal y equipos médicos, así como con herramientas de generación de reportes personalizados para el análisis y optimización continua.

Además, debe incluir mecanismos de comunicación efectiva entre los departamentos, asegurando que la Dirección General pueda supervisar y coordinar eficientemente todas las operaciones desde una única plataforma. Con una infraestructura digital que integre seguridad, eficiencia operativa y una gestión precisa de la información, el sistema permitirá reducir los tiempos de respuesta, mejorar la atención al paciente y elevar la calidad del servicio hospitalario, en beneficio de ambos sectores.

Hipótesis:

1. La aplicación móvil de gestión hospitalaria permitirá reducir considerablemente el tiempo de respuesta a las solicitudes y optimizar la asignación de recursos. Al integrar una supervisión en tiempo real, la Dirección General podrá tener una visión clara y actualizada de las operaciones diarias, asegurando una coordinación efectiva entre las distintas áreas del hospital.
2. La función para generar reportes personalizados en formatos PDF y Excel facilitará el análisis rápido y detallado de indicadores de rendimiento en cada área del hospital. Esto no solo permitirá ajustar operaciones con mayor precisión, sino que mejorará la toma de decisiones basadas en información clara y detallada.
3. Al incluir herramientas de comunicación entre áreas, la app ayudará a resolver uno de los grandes problemas de la administración hospitalaria: la fragmentación de procesos. Este módulo en Dirección General facilitará la recepción y el análisis inmediato de solicitudes interdepartamentales, agilizando aprobaciones y la gestión de recursos, y reduciendo así los tiempos de espera.
4. La aplicación permitirá que tanto los departamentos como la Dirección General vean en tiempo real el estatus de las solicitudes, agilizando los procesos de atención y garantizando que la información esté siempre disponible para quien la necesite.
5. La confiabilidad del sistema en cuanto a la carga de datos asegurará que el personal encargado gestione permisos y solicitudes sin errores ni retrasos, manteniendo un flujo de trabajo ordenado y claro, con registros actualizados en tiempo real.
6. La posibilidad de monitorear la productividad en cada área hospitalaria y hacer un registro quincenal permitirá a la Dirección General ajustar la distribución de recursos de manera ágil, con base en las necesidades reales de cada área y sin esperar a que surjan problemas de capacidad.

7. El análisis de las solicitudes que realiza la Dirección General permitirá anticiparse a posibles problemas de saturación, ayudando a optimizar los procesos antes de que lleguen a ser un obstáculo para la calidad del servicio al paciente.
8. La clasificación de solicitudes por prioridad en el módulo de aprobaciones asegurará que la Dirección General pueda asignar los recursos de manera eficiente, atendiendo primero a las solicitudes críticas y manteniendo el hospital operando sin retrasos innecesarios.
9. Las notificaciones y alertas integradas en la app permitirán que el personal del hospital esté al tanto de cualquier cambio o necesidad de manera inmediata, lo cual reducirá significativamente el tiempo de reacción ante emergencias o ajustes en la operación.
10. Al ser compatible con dispositivos móviles de gama media en sistemas operativos Android y iOS, la app estará disponible para todo el personal sin importar qué dispositivos utilicen. Esto hará que la plataforma sea accesible y fácil de utilizar para todos.
11. La inclusión de un sistema de seguridad con acceso controlado y encriptación de datos garantizará que toda la información de los pacientes esté protegida y sea accesible únicamente por el personal autorizado, manteniendo un alto estándar de privacidad.
12. La automatización en la aprobación de solicitudes críticas optimizará la carga de trabajo en Dirección General, facilitando que se puedan concentrar en decisiones estratégicas y logrando que los procesos más urgentes se ejecuten con mayor rapidez.

Justificación

La implementación de un sistema de gestión hospitalaria digital para el Hospital PrivilegeCare se justifica plenamente al considerar los retos críticos y las oportunidades de mejora en el sector salud, tanto en el ámbito privado como en el público. En un entorno hospitalario, la necesidad de optimizar los procesos de atención médica y administrativos resulta fundamental para elevar la calidad del servicio y responder a la creciente demanda de los pacientes. La falta de un sistema de gestión centralizado afecta la capacidad del hospital para responder de manera eficiente a las necesidades clínicas, impactando tanto en la experiencia del paciente como en la productividad del personal. A través de una plataforma digital, será posible coordinar y monitorear en tiempo real los recursos, solicitudes y actividades de cada departamento, brindando una visión integral de la operación hospitalaria.

En el sector público, los hospitales suelen estar limitados por el poder adquisitivo de los pacientes, lo que restringe el acceso a servicios de alta calidad. Sin embargo, con una gestión optimizada, PrivilegeCare puede diferenciarse al ofrecer un servicio médico de excelencia, maximizando sus recursos y fortaleciendo la fidelidad de sus pacientes. La implementación de esta plataforma también permitirá a los directivos del hospital ajustar sus operaciones de acuerdo con indicadores clave y gestionar de forma eficiente los recursos humanos, equipos médicos y suministros. Esta capacidad de reacción en tiempo real no solo mejorará la experiencia de los pacientes, sino que también incrementará la eficiencia del hospital al evitar pérdidas de recursos y garantizar la disponibilidad de los mismos en momentos críticos.

Además, la integración de un sistema digital que cumpla con las normativas de seguridad y privacidad de los datos permitirá una protección efectiva de la información médica de los pacientes. En un contexto donde la privacidad y la confidencialidad son cada vez más relevantes, esta plataforma brindará la seguridad necesaria para que solo el personal autorizado acceda a datos sensibles. Esto no solo garantiza el cumplimiento normativo, sino que también protege al hospital de posibles sanciones y el resguardo de su reputación frente al sector social.

La creación de una app móvil dedicada para la Dirección General permitirá agilizar la supervisión de indicadores de desempeño, mejorando la capacidad de los responsables del hospital para tomar decisiones estratégicas basadas en datos confiables y actualizados. Esta herramienta les permitirá atender y priorizar solicitudes, monitorear el flujo de trabajo y evaluar la calidad de los servicios, lo

cual se traduce en una mejora en la administración de recursos y en una atención médica más efectiva. Además, al optimizar la gestión de inventarios y la disponibilidad de equipos médicos, el hospital estará en mejores condiciones para ofrecer respuestas inmediatas en situaciones de emergencia, donde la precisión y la rapidez son vitales para la atención de los pacientes.

El desarrollo de una plataforma de gestión centralizada con un módulo específico para la Dirección General es una solución innovadora y necesaria para el Hospital PrivilegeCare. No solo contribuye a resolver problemas operativos y de gestión de recursos, sino que también posiciona al hospital en la vanguardia de la innovación en salud. Este sistema, al permitir una administración eficiente y una mejora en la experiencia del paciente, facilitará la competitividad y la sostenibilidad del hospital en el mercado, consolidando su compromiso con la excelencia en la atención y en el bienestar de sus pacientes.

Marco Teórico

Para el análisis de requerimientos se utilizaron las técnicas para ello la ingeniería de requerimientos comprende las actividades de obtención (captura, descubrimiento y adquisición), análisis (negociación), especificación y validación de requerimientos.

Para la obtención de la información con más precisión se aplicaron las técnicas de Investigación, para ello se explicará a grandes rasgos que son estas técnicas:

Las Técnicas de Investigación

son el conjunto de herramientas, procedimientos e instrumentos utilizados para obtener información y conocimiento. Se utilizan de acuerdo a los protocolos establecidos en una metodología de investigación determinada.

Las técnicas de investigación son los recursos disponibles para un investigador, que le permiten obtener datos e información. Las técnicas no garantizan por sí mismas que la interpretación o las conclusiones obtenidas sean correctas. Para esto último es necesario que la técnica se aplique en el marco de un método.

El empleo de técnicas de investigación es fundamental en todo proceso de investigación científica, ya que:

Permite organizar las diversas etapas de la investigación, desde la recolección de datos hasta su análisis e interpretación.

Facilita el control de la cantidad y la calidad de la información obtenida a lo largo de la investigación.

Constituye una guía sobre la validez de la hipótesis de la investigación.

En todas las áreas del saber, desde la biología y la física hasta la psicología y la historia, se emplean técnicas de investigación. Algunas técnicas, como la entrevista, son comunes a muchas disciplinas. Otras, en cambio, son propias de algún área en particular. Es el caso de, por ejemplo, las técnicas de cultivos de células, utilizadas en investigaciones bioquímicas.

Una Definición de Técnica de Investigación

La técnica de investigación científica es un procedimiento típico, validado por la práctica, orientado generalmente —aunque no exclusivamente— a obtener y transformar información útil para la solución de problemas de conocimiento en las disciplinas científicas. Toda técnica prevé el uso de un instrumento de aplicación; así, el instrumento de la técnica de Encuesta es el cuestionario; de la técnica de Entrevista es la Guía de tópicos de entrevista.

Tipos de Técnicas

Generalmente se distinguen dos tipos de técnicas: cualitativas y cuantitativas, y se han clasificado así en atención del predominio interno de las características para aplicarlas y procesar la información que se obtiene; ya contengan en ello números o no; aunque la clasificación es endeble porque en ambos tipos de técnicas es frecuente que se utilicen números y elementos discursivos e interpretativos. Así, hay que tomar esa clasificación con algo de humor. Una clasificación más responde a criterios de aplicación; así, se habla de técnicas de investigación documental, que se orientan a trabajo con información contenida en soportes documentales como impresos, grabaciones o relacionada con computadoras y redes virtuales; o técnicas de campo, cuando se trabaja con personas, con lugares o con aparatos.

Algunas de las técnicas cualitativas más utilizadas son las siguientes:

- Técnicas de investigación documental (procedimientos orientados a la aproximación a, procesamiento y recuperación de información contenida en documentos, independientemente del soporte documental en que se hallen) Ejemplos de estas técnicas son el aparato crítico (conjunto de apoyos al texto principal de un documento científico, según normas internacionales), las técnicas de lectura (aproximación al documento) y la reseña, el resumen, etc. (recuperación de la información)
- Técnicas para la obtención de información en campo (procedimientos del tipo sujeto-objeto como la observación directa, o sujeto-sujeto como la entrevista) que se aplican a procesos sociales u objetos.
- Técnicas de procesamiento de información (procedimientos que se aplican a la información obtenida en campo y cuya base es la interpretación, por ejemplo, la TKJ o la Delphi).

Diseño de la Técnica

El Diseño de una técnica es un documento, con aparato crítico, que respalda y explica cada uno de los elementos que concluyen en el instrumento de aplicación de esa técnica y en la manera en que se validó dicho instrumento; prevé las condiciones que deben cumplirse para la aplicación correcta de ese instrumento y para procesar la información que se obtiene al aplicar el instrumento. El documento debería acompañar a cada proceso de investigación de campo; en fin, el diseño da cuenta de:

- Las causas que llevaron a la elección de esa técnica entre otras posibilidades técnicas (justificación) atendiendo a razones de concordancia con las dimensiones (teórica, por ejemplo) de estudio o en el concierto de otras posibilidades técnicas.
- La definición conceptual de la técnica que explica la versión elegida, citadas las referencias disponibles.
- La población objeto a la que se pretende aplicar el instrumento y la descripción del contexto.
- Un ejercicio heurístico a manera de hipotetizarían acerca de la información que se espera encontrar con la aplicación de cada uno de los ítems del instrumento, con ejemplos concretos justamente como un Modelo.
- El instrumento de la técnica (que ha “traducido” a lenguaje ordinario los conceptos provenientes de la teoría), con explicaciones acerca de la congruencia teórica o dimensional de los ítems.
- Previsiones acerca de la aplicación del instrumento, con instrucciones precisas para ella.
- La previsión acerca de la manera en que se procesará la información obtenida mediante la aplicación del instrumento (atendiendo, por ejemplo, a un modelo estadístico o teórico; congruencia).
- Explicación del proceso de validación que llevó a la versión definitiva del instrumento.

Validación de Instrumentos Técnicos

El proceso de diseño de una técnica de investigación implica obligadamente la validación del instrumento.

La validación consiste en un procedimiento que somete a prueba un instrumento mediante un par de tácticas: la consulta y prueba con expertos (que generalmente son las mismas personas que lo

aplicarán o investigadores con experiencia) que calificarán el instrumento y recomendarán modificaciones al diseño; y, en segundo lugar, una prueba piloto del instrumento, que se aplica a personas que se asemejan a la población objeto o forman parte de ella. Al término del procedimiento de validación se cuenta ya con un instrumento técnico que presenta la mayor posibilidad de lograr los resultados previstos por los investigadores.

Técnicas de Investigación Documental

En general, las fuentes de información utilizadas en la investigación se denominan genéricamente Unidades Conservatorias de Información, y se trata de personas, instituciones, documentos, cosas, bibliografías, publicaciones, Estados del Arte, Estados del Conocimiento, Tesis, Bases de datos, fuentes electrónicas situadas en la red web, etc. cuya función es la de almacenar o contener información.

En particular, un documento está constituido por dos partes: el soporte documental, que es el material físico en que se presenta el documento; y, por otro lado, la información contenida en el documento [8, pp. 278,282].

Las Técnicas de investigación documental se aplican a algunos tipos de documentos (generalmente a textos: libros, artículos en revistas, reportes de entrevistas, notas de clase, registros de observación directa) Pueden ser clasificadas de la siguiente manera:

- Técnicas de investigación documental de Aproximación.
- Técnicas de lectura, como PQRST, Diagonal, etc.
- Tomas exploratorias de testimonios, entrevistas exploratorias, visitas, vistas previas de documentos.
- Técnicas de investigación documental de Procesamiento.
- Aparato crítico, subrayado, notas al margen de texto, notas de sesión.
- Consulta a fuentes alternativas como Diccionarios, lecturas complementarias
- Técnicas de investigación documental de Recuperación:
- Interpretación hermenéutica
- Mapas mentales, conceptuales, cuadros sinópticos
- Fichas bibliográficas, de trabajo, reseñas, resúmenes, controles de lectura
- Clippings, registros de observación, reportes de entrevista

Diagramas UML

UML (Unified Modeling Language) es un lenguaje que permite modelar, construir y documentar los elementos que forman un sistema software orientado a objetos. Se ha convertido en el estándar de facto de la industria, debido a que ha sido concebido por los autores de los tres métodos más usados de orientación a objetos: Grady Booch, Ivar Jacobson y Jim Rumbaugh. Estos autores fueron contratados por la empresa Rational Software Co. para crear una notación unificada en la que basar la construcción de sus herramientas CASE. En el proceso de creación de UML han participado, no obstante, otras empresas de gran peso en la industria como Microsoft, Hewlett-Packard, Oracle o IBM, así como grupos de analistas y desarrolladores.

Esta notación ha sido ampliamente aceptada debido al prestigio de sus creadores y debido a que incorpora las principales ventajas de cada uno de los métodos particulares en los que se basa: Booch, OMT y OOSE. UML ha puesto fin a las llamadas “guerras de métodos” que se han mantenido a lo largo de los 90, en las que los principales métodos sacaban nuevas versiones que incorporaban las técnicas de los demás. Con UML se fusiona la notación de estas técnicas para formar una herramienta compartida entre todos los ingenieros software que trabajan en el desarrollo orientado a objetos.

El objetivo principal cuando se empezó a gestar UML era posibilitar el intercambio de modelos entre las distintas herramientas CASE orientadas a objetos del mercado. Para ello era necesario definir una notación y semántica común.

Un modelo representa a un sistema software desde una perspectiva específica. Al igual que la planta y el alzado de una figura en dibujo técnico nos muestran la misma figura vista desde distintos ángulos, cada modelo nos permite fijarnos en un aspecto distinto del sistema. Los modelos de UML que se tratan en esta parte son los siguientes: ·

- Diagrama de Estructura Estática.
- Diagrama de Casos de Uso.
- Diagrama de Secuencia.
- Diagrama de Colaboración.
- Diagrama de Estados.

Dependencias

La relación de dependencia entre dos elementos de un diagrama significa que un cambio en el elemento destino puede implicar un cambio en el elemento origen (por tanto, si cambia el elemento destino habría que revisar el elemento origen). Una dependencia se representa por medio de una línea de trazo discontinuo entre los dos elementos con una flecha en su extremo. El elemento dependiente es el origen de la flecha y el elemento del que depende es el destino (junto a él está la flecha).

Diagramas de Estructura Estático

Con el nombre de Diagramas de Estructura Estática se engloba tanto al Modelo Conceptual de la fase de Análisis como al Diagrama de Clases de la fase de Diseño. Ambos son distintos conceptualmente, mientras el primero modela elementos del dominio el segundo presenta los elementos de la solución software. Sin embargo, ambos comparten la misma notación para los elementos que los forman (clases y objetos) y las relaciones que existen entre los mismos (asociaciones).

Clases

Una clase se representa mediante una caja subdividida en tres partes: En la superior se muestra el nombre de la clase, en la media los atributos y en la inferior las operaciones. Una clase puede representarse de forma esquemática (plegada), con los detalles como atributos y operaciones suprimidos, siendo entonces tan solo un rectángulo con el nombre de la clase. En la Figura 5 se ve cómo una misma clase puede representarse a distinto nivel de detalle según interese, y según la fase en la que se esté.

Objetos

Un objeto se representa de la misma forma que una clase. En el compartimento superior aparecen el nombre del objeto junto con el nombre de la clase subrayados.

Asociaciones

Las asociaciones entre dos clases se representan mediante una línea que las une. La línea puede tener una serie de elementos gráficos que expresan características particulares de la asociación.

Nombre de la Asociación y Dirección

El nombre de la asociación es opcional y se muestra como un texto que está próximo a la línea. Se puede añadir un pequeño triángulo negro sólido que indique la dirección en la cual leer el nombre de la asociación.

Los nombres de las asociaciones normalmente se incluyen en los modelos para aumentar la legibilidad. Sin embargo, en ocasiones pueden hacer demasiado abundante la información que se presenta, con el consiguiente riesgo de saturación. En ese caso se puede suprimir el nombre de las asociaciones consideradas como suficientemente conocidas.

Multiplicidad

La multiplicidad es una restricción que se pone a una asociación, que limita el número de instancias de una clase que pueden tener esa asociación con una instancia de la otra clase.

Roles

Para indicar el papel que juega una clase en una asociación se puede especificar un nombre de rol. Se representa en el extremo de la asociación junto a la clase que desempeña dicho rol.

Agregación

El símbolo de agregación es un diamante colocado en el extremo en el que está la clase que representa el “todo”.

Clases Asociación

Cuando una asociación tiene propiedades propias se representa como una clase unida a la línea de la asociación por medio de una línea a trazos. Tanto la línea como el rectángulo de clase representan el mismo elemento conceptual: la asociación. Por tanto, ambos tienen el mismo nombre, el de la asociación.

Asociaciones N-Arias

En el caso de una asociación en la que participan más de dos clases, las clases se unen con una línea a un diamante central. Si se muestra multiplicidad en un rol, representa el número potencial de tuplas de instancias en la asociación cuando el resto de los N-1 valores están fijos.

Diagrama de Casos de Uso

Un Diagrama de Casos de Uso muestra la relación entre los actores y los casos de uso del sistema. Representa la funcionalidad que ofrece el sistema en lo que se refiere a su interacción externa.

Elementos

Los elementos que pueden aparecer en un Diagrama de Casos de Uso son: actores, casos de uso y relaciones entre casos de uso.

Actores

Un actor es una entidad externa al sistema que realiza algún tipo de interacción con el mismo. Se representa mediante una figura humana dibujada con palotes. Esta representación sirve tanto para actores que son personas como para otro tipo de actores (otros sistemas, sensores, etc.).

Casos de Uso

Un caso de uso es una descripción de la secuencia de interacciones que se producen entre un actor y el sistema, cuando el actor usa el sistema para llevar a cabo una tarea específica. Expresa una unidad coherente de funcionalidad, y se representa en el Diagrama de Casos de Uso mediante una elipse con el nombre del caso de uso en su interior. El nombre del caso de uso debe reflejar la tarea específica que el actor desea llevar a cabo usando el sistema.

Relaciones entre Casos de Uso

Entre dos casos de uso puede haber las siguientes relaciones:

- Extiende: Cuando un caso de uso especializa a otro extendiendo su funcionalidad.
- Usa: Cuando un caso de uso utiliza a otro. Se representan como una línea que une a los dos casos de uso relacionados, con una flecha en forma de triángulo y con una etiqueta <> o <> según sea el tipo de relación.

En el diagrama de casos de uso se representa también el sistema como una caja rectangular con el nombre en su interior. Los casos de uso están en el interior de la caja del sistema, y los actores fuera, y cada actor está unido a los casos de uso en los que participa mediante una línea.

Diagrama de Estados

Un Diagrama de Estados muestra la secuencia de estados por los que pasa un caso de uso o un objeto a lo largo de su vida, indicando qué eventos hacen que se pase de un estado a otro y cuáles son las respuestas y acciones que genera. En cuanto a la representación, un diagrama de estados

es un grafo cuyos nodos son estados y cuyos arcos dirigidos son transiciones etiquetadas con los nombres de los eventos. Un estado se representa como una caja redondeada con el nombre del estado en su interior. Una transición se representa como una flecha desde el estado origen al estado destino. La caja de un estado puede tener 1 o 2 compartimentos. En el primer compartimento aparece el nombre del estado. El segundo compartimento es opcional, y en él pueden aparecer acciones de entrada, de salida y acciones internas. Una acción de entrada aparece en la forma (entrada/acción asociada) donde (acción asociada) es el nombre de la acción que se realiza al entrar en ese estado. Cada vez que se entra al estado por medio de una transición la acción de entrada se ejecuta. Una acción de salida aparece en la forma (salida/acción asociada). Cada vez que se sale del estado por una transición de salida la acción de salida se ejecuta. Una acción interna es una acción que se ejecuta cuando se recibe un determinado evento en ese estado, pero que no causa una transición a otro estado. Se indica en la forma nombre de (evento/acción asociada).

Un diagrama de estados puede representar ciclos continuos o bien una vida finita, en la que hay un estado inicial de creación y un estado final de destrucción (del caso de uso o del objeto). El estado inicial se muestra como un círculo sólido y el estado final como un círculo sólido rodeado de otro círculo. En realidad, los estados inicial y final son pseudo estados, pues uno objeto no puede “estar” en esos estados, pero nos sirven para saber cuáles son las transiciones inicial y final(es) [9].

Modelo Entidad Relación

El modelo entidad-relación es el modelo conceptual más utilizado para el diseño conceptual de bases de datos. Fue introducido por Peter Chan en 1976. El modelo entidad-relación está formado por un conjunto de conceptos que permiten describir la realidad mediante un conjunto de representaciones gráficas y lingüísticas. Originalmente, el modelo entidad-relación sólo incluía los conceptos de entidad, relación y atributo. Más tarde, se añadieron otros conceptos, como los atributos compuestos y las jerarquías de generalización, en lo que se ha denominado modelo entidad-relación extendido.

Entidad

Cualquier tipo de objeto o concepto sobre el que se recoge información: cosa, persona, concepto abstracto o suceso. Por ejemplo: coches, casas, empleados, clientes, empresas, oficios, diseños de productos, conciertos, excursiones, etc. Las entidades se representan gráficamente mediante

rectángulos y su nombre aparece en el interior. Un nombre de entidad sólo puede aparecer una vez en el esquema conceptual.

Hay dos tipos de entidades: fuertes y débiles. Una entidad débil es una entidad cuya existencia depende de la existencia de otra entidad. Una entidad fuerte es una entidad que no es débil.

Conjunto de Entidades

Es una colección de entidades que comparten los mismos atributos o características:

Ejemplos:

- Todos los atletas que participan en los Juegos Olímpicos, comparten sus atributos: nombre, número de identificación, edad, peso, categoría...
- Todos los países del mundo, comparten las características: nombre, continente, área, lengua principal, lengua secundaria, moneda, etc.

Relación (interrelación)

Es una correspondencia o asociación entre dos o más entidades. Cada relación tiene un nombre que describe su función. Las relaciones se representan gráficamente mediante rombos y su nombre aparece en el interior.

Las entidades que están involucradas en una determinada relación se denominan entidades participantes. El número de participantes en una relación es lo que se denomina grado de la relación.

Por lo tanto, una relación en la que participan dos entidades es una relación binaria; si son tres las entidades participantes, la relación es ternaria; etc.

Una relación recursiva

Es una relación donde la misma entidad participa más de una vez en la relación con distintos papeles. El nombre de estos papeles es importante para determinar la función de cada participación.

La cardinalidad

Es la que participa en una relación especifica el número mínimo y el número máximo de correspondencias en las que puede tomar parte cada ocurrencia de dicha entidad. La participación de una entidad en una relación es obligatoria (total) si la existencia de cada una de sus ocurrencias requiere la existencia de, al menos, una ocurrencia de la otra entidad participante. Si no, la

participación es opcional (parcial). Las reglas que definen la cordialidad de las relaciones son las reglas de negocio.

Una de las trampas que pueden encontrarse ocurre cuando el esquema representa una relación entre entidades, pero el camino entre algunas de sus ocurrencias es ambiguo. El modo de resolverla es reestructurando el esquema para representar la asociación entre las entidades correctamente.

Conjunto de relaciones

Consiste en una colección de relaciones de la misma naturaleza.

Ejemplo:

- Dados los conjuntos de entidades "Habitación" y "Huésped", todas las relaciones de la forma habitación-huésped, permiten obtener la información de los huéspedes y sus respectivas habitaciones.
- La dependencia o asociación entre los conjuntos de entidades es llamada participación. En el ejemplo anterior los conjuntos de entidades "Habitación" y "Huésped" participan en el conjunto de relaciones habitación-huésped

Atributo

Es una característica de interés o un hecho sobre una entidad o sobre una relación. Los atributos representan las propiedades básicas de las entidades y de las relaciones. Toda la información extensiva es portada por los atributos. Gráficamente, se representan mediante bolitas que cuelgan de las entidades o relaciones a las que pertenecen.

Cada atributo tiene un conjunto de valores asociados denominado dominio. El dominio define todos los valores posibles que puede tomar un atributo. Puede haber varios atributos definidos sobre un mismo dominio.

Los atributos pueden ser simples o compuestos. Un atributo simple es un atributo que tiene un solo componente, que no se puede dividir en partes más pequeñas que tengan un significado propio. Un atributo compuesto es un atributo con varios componentes, cada uno con un significado por sí mismo. Un grupo de atributos se representa mediante un atributo compuesto cuando tienen afinidad en cuanto a su significado, o en cuanto a su uso. Un atributo compuesto se representa gráficamente mediante un óvalo.

Los atributos también pueden clasificarse en monovalentes o polivalentes. Un atributo monovalente es aquel que tiene un solo valor para cada ocurrencia de la entidad o relación a la que pertenece. Un atributo polivalente es aquel que tiene varios valores para cada ocurrencia de la entidad o relación a la que pertenece. A estos atributos también se les denomina multivaluados, y pueden tener un número máximo y un número mínimo de valores. La cardinalidad de un atributo indica el número mínimo y el número máximo de valores que puede tomar para cada ocurrencia de la entidad o relación a la que pertenece. El valor por omisión es (1,1).

Identificador

Un identificador de una entidad es un atributo o conjunto de atributos que determina de modo único cada ocurrencia de esa entidad. Un identificador de una entidad debe cumplir dos condiciones:

- No pueden existir dos ocurrencias de la entidad con el mismo valor del identificador.
- Si se omite cualquier atributo del identificador, la condición anterior deja de cumplirse.

Toda entidad tiene al menos un identificador y puede tener varios identificadores alternativos. Las relaciones no tienen identificadores.

Jerarquía de Generalización

Una entidad E es una generalización de un grupo de entidades E^1, E^2, E^n , si cada ocurrencia de cada una de esas entidades es también una ocurrencia de E . Todas las propiedades de la entidad genérica E son heredadas por las subentidades.

Cada jerarquía es total o parcial, y exclusiva o superpuesta. Una jerarquía es total si cada ocurrencia de la entidad genérica corresponde al menos con una ocurrencia de alguna subentidad. Es parcial si existe alguna ocurrencia de la entidad genérica que no corresponde con ninguna ocurrencia de ninguna subentidad. Una jerarquía es exclusiva si cada ocurrencia de la entidad genérica corresponde, como mucho, con una ocurrencia de una sola de las subentidades. Es superpuesta si existe alguna ocurrencia de la entidad genérica que corresponde a ocurrencias de dos o más subentidades diferentes.

Un subconjunto es un caso particular de generalización con una sola entidad como subentidad. Un subconjunto siempre es una jerarquía parcial y exclusiva.

Atributos en Relaciones

Las relaciones también pueden tener atributos asociados. Se representan igual que los atributos de las entidades. Un ejemplo típico son las relaciones de tipo "histórico" donde debe constar una fecha o una hora. Por ejemplo, supongamos que es necesario hacer constar la fecha de emisión de una factura a un cliente, y que es posible emitir duplicados de la factura (con distinta fecha). En tal caso, el atributo "Fecha de emisión" de la factura debería colocarse en la relación "se emite".

Herencia

La herencia es un intento de adaptación de estos diagramas al paradigma orientado a objetos. La herencia es un tipo de relación entre una entidad "padre" y una entidad "hijo". La entidad "hijo" hereda todos los atributos y relaciones de la entidad "padre". Por tanto, no necesitan ser representadas dos veces en el diagrama. La relación de herencia se representa mediante un triángulo interconectado por líneas a las entidades. La entidad conectada por el vértice superior del triángulo es la entidad "padre". Solamente puede existir una entidad "padre" (herencia simple). Las entidades "hijo" se conectan por la base del triángulo.

Entidades Fuertes y Débiles

Cuando una entidad participa en una relación puede adquirir un papel fuerte o débil. Una entidad débil es aquella que no puede existir sin participar en la relación, es decir, aquella que no puede ser unívocamente identificada solamente por sus atributos. Una entidad fuerte (también conocida como entidad regular) es aquella que sí puede ser identificada unívocamente. En los casos en que se requiera, se puede dar que una entidad fuerte "preste" algunos de sus atributos a una entidad débil para que, esta última, se pueda identificar.

Las entidades débiles se representan mediante un doble rectángulo, es decir, un rectángulo con doble línea.

Cardinalidad de las Relaciones

El tipo de cardinalidad se representa mediante una etiqueta en el exterior de la relación, respectivamente: "1:1", "1: N" y "N:M", aunque la notación depende del lenguaje utilizado, la que más se usa actualmente es el unificado. Otra forma de expresar la cardinalidad es situando un símbolo cerca de la línea que conecta una entidad con una relación:

- "0" si cada instancia de la entidad no está obligada a participar en la relación.
- "1" si toda instancia de la entidad está obligada a participar en la relación y, además, solamente participa una vez.
- "N", "M", o "*" si cada instancia de la entidad no está obligada a participar en la relación y puede hacerlo cualquier número de veces.

Ejemplos de relaciones que expresan cardinalidad:

- Cada esposo (entidad) está casado (relación) con una única esposa (entidad) y viceversa. Es una relación 1:1.
- Una factura (entidad) se emite (relación) a una persona (entidad) y sólo una, pero una persona puede tener varias facturas emitidas a su nombre. Todas las facturas se emiten a nombre de alguien. Es una relación 1: N.
- Un cliente (entidad) puede comprar (relación) varios artículos (entidad) y un artículo puede ser comprado por varios clientes distintos. Es una relación N: M.

Restricciones

Son reglas que deben mantener los datos almacenados en la base de datos.

Correspondencia de Cardinalidades

Dado un conjunto de relaciones en el que participan dos o más conjuntos de entidades, la correspondencia de cardinalidad indica el número de entidades con las que puede estar relacionada una entidad dada.

Dado un conjunto de relaciones binarias y los conjuntos de entidades A y B, la correspondencia de cardinalidades puede ser:

- Uno a uno: Una entidad de A se relaciona únicamente con una entidad en B y viceversa.
- Uno a varios: Una entidad en A se relaciona con cero o muchas entidades en B. Pero una entidad en B se relaciona con una única entidad en A.
- Varios a uno: Una entidad en A se relaciona exclusivamente con una entidad en B. Pero una entidad en B se puede relacionar con 0 o muchas entidades en A.
- Varios a varios: Una entidad en A se puede relacionar con 0 o muchas entidades en B y viceversa.

Restricciones de participación

Dado un conjunto de relaciones R en el cual participa un conjunto de entidades A , dicha participación puede ser de dos tipos:

- Total: Cuando cada entidad en A participa en al menos una relación de R .
- Parcial: Cuando al menos una entidad en A NO participa en alguna relación de R .

Claves

Es un subconjunto del conjunto de atributos comunes en una colección de entidades, que permite identificar unívocamente cada una de las entidades pertenecientes a dicha colección. Asimismo, permiten distinguir entre sí las relaciones de un conjunto de relaciones.

Dentro de los conjuntos de entidades existen los siguientes tipos de claves:

- Superclase: Es un subconjunto de atributos que permite distinguir unívocamente cada una de las entidades de un conjunto de entidades. Si otro atributo unido al anterior subconjunto, el resultado seguirá siendo una superclase.
- Clave candidata: Dada una superclase, si ésta deja de serlo removiendo únicamente uno de los atributos que la componen, entonces ésta es una clave candidata.
- Clave primaria: Es una clave candidata, elegida por el diseñador de la base de datos, para identificar unívocamente las entidades en un conjunto de entidades.

Los valores de los atributos de una clave, no pueden ser todos iguales para dos o más entidades.

Para poder distinguir unívocamente las relaciones en un conjunto de relaciones R , se deben considerar dos casos:

- R NO tiene atributos asociados: En este caso, se usa como clave primaria de R la unión de las claves primarias de todos los conjuntos de entidades participantes.
- R tiene atributos asociados: En este caso, se usa como clave primaria de R la unión de los atributos asociados y las claves primarias de todos los conjuntos de entidades participantes.

Si el conjunto de relaciones, R , sobre las que se pretende determinar la clave primaria está compuesto de relaciones binarias, con los conjuntos de entidades participantes A y B , se consideran los siguientes casos, según sus cardinalidades:

- R es de muchos a uno de A a B entonces sólo se toma la clave primaria de A , como clave primaria de R .
- R es de uno a muchos de A a B entonces se toma sólo la clave primaria de B , como clave primaria de R .
- R es de uno a uno de A a B entonces se toma cualquiera de las dos claves primarias, como clave primaria de R .

Representaciones Gráficas

El primer paso en el diseño de una base de datos es la producción del esquema conceptual. Normalmente, se construyen varios esquemas conceptuales, cada uno para representar las distintas visiones que los usuarios tienen de la información. Cada una de estas visiones suelen corresponder a las diferentes áreas funcionales de la empresa como, por ejemplo, producción, ventas, recursos humanos, etc.

Estas visiones de la información, denominadas vistas, se pueden identificar de varias formas. Una opción consiste en examinar los diagramas de flujo de datos, que se pueden haber producido previamente, para identificar cada una de las áreas funcionales. La otra opción consiste en entrevistar a los usuarios, examinar los procedimientos, los informes y los formularios, y también observar el funcionamiento de la empresa.

A los esquemas conceptuales correspondientes a cada vista de usuario se les denomina esquemas conceptuales locales. Cada uno de estos esquemas se compone de entidades, relaciones, atributos, dominios de atributos e identificadores. El esquema conceptual también tendrá una documentación, que se irá produciendo durante su desarrollo. Las tareas a realizar en el diseño conceptual son las siguientes:

1. Identificar las entidades.
2. Identificar las relaciones.
3. Identificar los atributos y asociarlos a entidades y relaciones.

4. Determinar los dominios de los atributos.
5. Determinar los identificadores.
6. Determinar las jerarquías de generalización (si las hay).
7. Dibujar el diagrama entidad-relación. 8. Revisar el esquema conceptual local con el usuario.

Paso del modelo E-R al diseño de la base de datos

- Las entidades entre las que hay una interrelación uno a uno se debe fusionar en una sola entidad.
- Una vez hecho esto, cada una de las entidades que quedan se convierte en una tabla con una clave primaria y una serie de atributos, de los cuales algunos pueden ser claves secundarias.
- Las interrelaciones uno a muchos se transforma en atributo y clave secundaria de la tabla que representa a la entidad situada del lado de la interrelación correspondiente a muchos
- Las interrelaciones muchos a muchos entre dos entidades pasan a ser una tercera tabla con claves secundarias procedentes de ambas entidades. Estas claves secundarias deberán formar parte de la clave primaria de la tabla en la que se convierte la interrelación, cuando corresponda.
- Hay una serie de herramientas disponibles en el mercado que pueden automatizar el proceso de conversión de un modelo E/R en un esquema de base de datos [10].

Microsoft Visio

Es una herramienta especialmente diseñada para la creación de diagramas que ofrece una gran cantidad de funciones y utilidades al respecto. Sorprendentemente, esta aplicación tan útil de Microsoft no se incluye en ninguna de las versiones de Office y debe ser adquirida de forma independiente al famoso paquete ofimático de la empresa.

Microsoft Visio va mucho más allá de las utilidades que incorpora Word a la hora de crear diagramas y esquemas, con una interfaz especialmente diseñada para este fin y con múltiples herramientas y plantillas que facilitan la creación y personalización de cualquier tipo de diagrama. Microsoft Visio es una aplicación que se puede utilizar en cualquier sector para facilitar la creación de esquemas, gráficos y diagramas personalizados.

Los principales usos de esta solución gráfica de Microsoft son:

- Creación de diagramas de flujo (por ejemplo, para representar el flujo de trabajo en un departamento o empresa).
- Confeccionar organigramas.
- Construir los planos de una vivienda, oficina, local comercial...
- Realizar mapas en tres dimensiones.
- Establecer una escala de tiempos para eventos, tareas o procesos.
- Modelado de procesos de negocio.
- Crear diseños de ingeniería o mecánica.

Veamos cuáles son las características más destacadas de esta aplicación para creación de diagramas profesionales.

- Vista previa. La vista previa de Visio permite ver en tiempo real las distintas formas disponibles.
- Herramientas para trabajo colaborativo. Permiten que varios usuarios puedan trabajar de forma simultánea en el mismo proyecto.
- Integración con fuente de datos. Se puede integrar con otras aplicaciones para obtener datos, como bases de datos SQL, Access o con libros de Excel.
- Plantillas incorporadas. Cuenta con una serie de plantillas profesionales prediseñadas para poder acelerar la creación y personalización de toda clase de diagramas y gráficos.
- Compatibilidad con Office 365. Se puede conectar con las aplicaciones que contiene Office 365 (Word, Excel, PowerPoint...).
- Power BI. Microsoft Visio también es compatible con esta herramienta de inteligencia de negocios, ayudando a crear esquemas, diagramas y gráficos con un mayor nivel de profesionalidad y flexibilidad.

A la hora de crear gráficos y diagramas, Microsoft Visio es una de las herramientas más potentes y versátiles del mercado, tanto en proyectos profesionales, educativos como particulares.

Los beneficios más destacados que ofrece el uso de esta aplicación de diseño de gráficos y diagramas son:

Facilidad de Uso

Como la mayoría de aplicaciones de Microsoft y Office, Visio es una herramienta muy intuitiva que utiliza sistemas similares a los de la familia Microsoft. Por lo tanto, la curva de aprendizaje es muy suave, pudiendo crear gráficos y diagramas de gran calidad con apenas unas horas de experiencia utilizando la aplicación.

Muchas Opciones de Diseño

Con Visio hay disponible una amplia gama de opciones y utilidades para personalizar diagramas y gráficos según las necesidades de cada proyecto. La cantidad de gráficos, formas y otros elementos que incorpora Visio es mucho mayor que los de la gran mayoría de aplicaciones similares del mercado.

Integración con Múltiples Herramientas

Uno de los puntos fuertes de Visio se encuentra, sin duda, en su integración con múltiples herramientas para poder mejorar informes, paneles, presentaciones o flujos de trabajo, por ejemplo.

Las principales herramientas con las que se integra Visio son:

- Office y sus aplicaciones.
- Power BI (business intelligence).
- Bases de datos SQL.

Acelera el Proceso de Creación

Con Visio se incluyen muchas plantillas profesionales que sirven como base o guía para crear cualquier tipo de diagrama o gráfico. Utilizar estas plantillas prediseñadas acelera el proceso de creación, facilitando el trabajo de los diseñadores al contener ya la estructura y elementos principales.

Trabajo Colaborativo y en la Nube

Con Visio es posible trabajar en la nube, lo que posibilita el trabajo colaborativo a distancia. Pueden conectarse al mismo proyecto de Visio varios usuarios para poder trabajar de forma simultánea y conjunta, evitando así el uso de programas de mensajería, videoconferencia, correo electrónico o de envío de archivos.

Si necesitas crear gráficos, esquemas y diagramas con un alto grado de personalización, apostar por Microsoft Visio es una gran elección. Esta herramienta de Microsoft ofrece todos los que necesitas para construir diagramas de todo tipo de forma rápida y sencilla, pudiéndolos integrar con otro tipo de documentos como textos de Word, hojas de cálculo de Excel, presentaciones de PowerPoint o incluso, paneles de mando e informes de Power BI.

Algunas Ediciones de Visio no Tienen Algunas o Todas las Características de Modelado de Base de Datos:

Visio Standard no incluye la plantilla Diagrama de modelo de base de datos.

Las ediciones de Visio Professional y Premium son compatibles con las características de ingeniería inversa de la plantilla de diagrama de modelo de base de datos (es decir, usar una base de datos existente para crear un modelo en Visio), pero no admiten la ingeniería directa (es decir, el uso de un modelo de base de datos de Visio para generar código SQL).

El conjunto completo de características de modelado de base de datos, incluidas las técnicas de ingeniería inversa y directa, está en Visio para arquitectos profesionales. Visio for Enterprise Architects se incluye en MSDN Premium Subscription, disponible en las ediciones basadas en roles Visual Studio Professional y Visual Studio Team System [11] [12].

Microsoft Word

Microsoft Word es un procesador de textos desarrollado por Microsoft y lanzado inicialmente en 1983. Como parte del paquete de Microsoft Office, Word es una herramienta ampliamente utilizada para la creación, edición, y formato de documentos de texto. Word permite a los usuarios crear documentos profesionales y cuenta con una amplia gama de herramientas para estructurar y personalizar el contenido, haciéndolo ideal para fines académicos, comerciales, y administrativos.

Características Clave de Microsoft Word:

- **Edición y Formato de Texto:** Word permite dar formato avanzado al texto, incluyendo opciones de fuente, tamaño, color, alineación, interlineado, y estilos.
- **Herramientas de Diseño y Maquetación:** Ofrece funcionalidades de diseño de página como encabezados, pies de página, numeración, y tablas de contenido, lo cual facilita la creación de documentos estructurados.

- Elementos Gráficos e Imágenes: Word permite insertar y editar gráficos, imágenes, tablas, y diagramas, lo que mejora la presentación visual del documento.
- Funciones de Revisión y Colaboración: Con herramientas como comentarios, control de cambios y revisión ortográfica, Word facilita la colaboración en documentos y garantiza un control de calidad, Cita: [11].

Función de Microsoft Word en el Proyecto

En el contexto del proyecto de gestión hospitalaria, Microsoft Word se utiliza principalmente para la creación y organización de la documentación del proyecto. Esta documentación es esencial para organizar la información, facilitar la comunicación con los stakeholders, y garantizar que todos los detalles del sistema estén bien registrados. A continuación, se detallan algunas de las aplicaciones específicas de Microsoft Word en este proyecto:

1. Documentación del Marco Teórico y Conceptual:
 - Estructura y Redacción de Contenidos Técnicos: Word permite redactar y organizar el marco teórico y conceptual del proyecto, detallando las tecnologías, metodologías y fundamentos detrás del sistema de gestión hospitalaria.
 - Tablas de Contenido y Referencias: Word facilita la creación de tablas de contenido, índices y secciones de referencias, lo cual permite una navegación rápida dentro del documento y mejora la organización del contenido.
2. Creación de Reportes y Documentación de Avances:
 - Reportes de Avance del Proyecto: Word se utiliza para documentar los progresos y resultados de cada fase del proyecto, permitiendo un seguimiento detallado de las actividades, logros y posibles desafíos enfrentados.
 - Reportes Financieros y de Recursos: A través de Word, el equipo puede crear reportes que detallen el uso de recursos (humanos, tecnológicos y económicos), permitiendo al equipo y a los gestores hospitalarios revisar y evaluar el rendimiento del proyecto.
3. Manual de Usuario y Guías de Operación:
 - Guía de Usuario Final: Word es ideal para crear un manual de usuario que explique de manera detallada y organizada cómo utilizar la aplicación, especialmente para el personal médico y administrativo que usará el sistema de gestión hospitalaria.

- Documentación Técnica para Soporte: Word permite detallar los procesos técnicos y configuraciones del sistema, de modo que los equipos de soporte tengan la información necesaria para resolver problemas técnicos.
4. Colaboración y Revisión de Documentos:
- Control de Cambios: Word permite realizar un seguimiento de todas las modificaciones en el documento a través de la función de control de cambios, lo cual es útil para trabajar colaborativamente en la redacción y revisión de informes.
 - Comentarios y Notas: Los usuarios pueden insertar comentarios y notas en el documento para señalar áreas de mejora, preguntas o instrucciones, facilitando la revisión colaborativa entre los miembros del equipo.
5. Elementos Gráficos y Diagramas:
- Inserción de Tablas y Gráficos: Word permite insertar tablas y gráficos que resumen datos importantes del sistema, como estadísticas de uso, diagramas de flujo y métricas de rendimiento. el diseño del sistema y sus componentes.

Relevancia de Microsoft Word en el Proyecto

Microsoft Word es fundamental en el proyecto por varias razones:

1. Creación de Documentación Estructurada y Profesional: Word facilita la creación de documentación profesional, que es esencial para la comunicación formal del proyecto con el personal administrativo, el equipo de desarrollo, y otros stakeholders. Su capacidad para estructurar contenido y proporcionar un formato profesional asegura que la documentación sea clara y fácil de entender.
2. Facilidad para Revisión y Colaboración: Word permite que varios miembros del equipo participen en la edición y revisión de documentos de manera simultánea, gracias a herramientas como el control de cambios y los comentarios. Esto asegura que todos los miembros estén alineados y permite un proceso de revisión eficiente y efectivo.
3. Estandarización de Documentos Técnicos y Administrativos: Word permite crear plantillas y documentos estandarizados, lo cual ayuda a mantener una coherencia en el estilo y la presentación de los documentos del proyecto. Esta estandarización es importante para asegurar que todos los documentos del sistema hospitalario sigan una estructura y formato unificados.

4. Organización de Información Crítica del Proyecto: La capacidad de Word para manejar documentos extensos y complejos lo hace ideal para consolidar toda la información relevante del proyecto en un solo archivo. Esto facilita la consulta y referencia de información en el futuro, asegurando que el equipo tenga acceso a una fuente de información integral sobre el sistema.
5. Accesibilidad y Compatibilidad: Microsoft Word es ampliamente utilizado y accesible, lo cual asegura que cualquier documento creado en esta plataforma pueda ser leído y editado por otros miembros del equipo o stakeholders que utilicen diferentes dispositivos y plataformas. Esto asegura que la documentación sea compatible y fácil de distribuir.

Figma

Es un interesante e innovador editor de gráficos vectoriales diseñado para páginas web y cuyo uso ha ido creciendo sin parar durante los últimos años. A día de hoy, es una de las plataformas más utilizadas por diseñadores UX/UI, agencias de comunicación y empresas de todo el mundo.

Figma es una plataforma de edición gráfica y diseño de interfaces. Además, es una plataforma online y colaborativa. Con Figma se puede hacer un poco de todo a nivel de diseño gráfico, desde diseñar páginas web e interfaces gráficas de aplicaciones, o crear publicaciones para redes sociales, hasta la posibilidad de poder crear presentaciones. Por este motivo, es una de las herramientas más valoradas por empresas y estudios de diseño gráfico.

Una de las características más innovadoras de Figma es, sin duda, el hecho de que es una herramienta colaborativa. En un mismo archivo, pueden trabajar todos los distintos miembros de un equipo y, además, en tiempo real: esto significa que los diseñadores gráficos pueden empezar con la creación de un diseño, pero que los desarrolladores y los copywriters ya están involucrados en el proyecto desde las primeras fases del mismo.

Todos los progresos se guardan en la nube y en tiempo real, evitando problemas varios como el de trabajar en archivos que han quedado obsoletos o el de perder mucho tiempo intercambiando archivos muy pesados. Evidentemente, para utilizar Figma hay que estar prácticamente siempre online y tener una conexión a Internet buena y estable.

Lluvias de ideas, evaluaciones de distintas opciones gráficas, prototipado y recopilación e implementación de comentarios: Figma se puede usar de forma muy eficaz en todas estas fases.

Por ejemplo, en Figma, un equipo de diseñadores puede crear colaborativamente el wireframe de una página web (es decir, el esqueleto de un diseño sin colores o estilos), recopilar los primeros comentarios del resto de miembros del equipo o votar la mejor opción. También pueden hacer el prototipo de la interfaz de una aplicación: es decir, simular en las primeras fases del proyecto la navegación y la interacción en un diseño.

Crear una Cuenta

Para crear una cuenta, tan solo tienes que registrarte de forma gratuita en figma.com. Una vez que te hayas registrado, puedes crear equipos, proyectos y archivos (files). Además, puedes invitar a otros usuarios del equipo para darles acceso a los proyectos en los que vais a trabajar.

El Espacio de Trabajo

El espacio de trabajo de Figma es enorme (un cuadrado de hasta 65 000 píxeles) y dentro puedes añadir las distintas pantallas del diseño en el que estés trabajando.

Para configurar las pantallas, utiliza la herramienta de marcos (frames): puedes elegir entre muchos tamaños predefinidos, como por ejemplo el del iPhone 14, el del Apple Watch 41 mm o el de una publicación de Facebook. De esta forma, por ejemplo, puedes recopilar en un solo archivo las distintas páginas de una página web o mostrar cómo se visualizará una misma página en dispositivos diferentes.

Bibliotecas y Comunidad

Otra herramienta interesante de Figma son las bibliotecas de componentes. Si no quieres perder tiempo creando de cero los elementos, puedes acelerar la creación de un diseño (o de un boceto) utilizando los elementos gráficos más dispares que ya estén disponibles en las bibliotecas. En las bibliotecas puedes encontrar cosas como los iconos o botones más comunes.

Navega por la comunidad de Figma (uno de los apartados más interesantes y útiles de la plataforma) para encontrar estos y otros elementos. En ella puedes encontrar inspiración, materiales para hacer una lluvia de ideas y los denominados sistemas de diseño, es decir, un conjunto de reglas, directrices y elementos gráficos que ayudan a dar coherencia a tu proyecto gráfico.

Prototipado

En la sección Prototype de Figma, puedes vincular entre sí distintas pantallas o elementos gráficos mediante la identificación de nodos: por ejemplo, puedes colocar un nodo en el elemento «Acerca de» de un menú y este llevará al usuario a la página correspondiente. O también puedes hacerlo en el botón «Suscríbete a la newsletter». Si pones en marcha el prototipo, puedes simular el funcionamiento de una página web o de una interfaz de una aplicación incluso en las primeras fases del proyecto. Así puedes saber qué funciona y qué hay que mejorar.

Recopilación de Comentarios

Una vez que hayas compartido el archivo con el resto del equipo, se puede recibir y hacer comentarios a través de notas de colores. Estas notas se utilizan para votar, para informar de problemas o para dar consejos a través de comentarios más estructurados. Por ejemplo, puedes presentar a tu equipo tres opciones diferentes de diseño y saber, en tiempo real, lo que piensan tus compañeros.

¿Qué Ventajas Ofrece Figma?

Una de las principales ventajas de Figma es su accesibilidad y facilidad de uso. Al ser una herramienta basada en la web, no requiere instalación ni configuración compleja, lo que significa que puedes comenzar a trabajar de inmediato desde cualquier navegador. Esto lo hace especialmente útil para equipos distribuidos o para aquellos que necesitan acceso rápido y sencillo a sus proyectos de diseño.

Figma también se distingue por su enfoque en la colaboración. A diferencia de otros programas de diseño que limitan la colaboración a la visualización o comentarios posteriores, Figma permite una verdadera colaboración en tiempo real. Esto no solo agiliza el flujo de trabajo, sino que también permite una mejor comunicación y coordinación entre los miembros del equipo, lo que resulta en un producto final más cohesivo y bien diseñado.

Además, Figma es una herramienta muy flexible y adaptable a diferentes flujos de trabajo. Su sistema de diseño basado en componentes y estilos globales facilita la creación de sistemas de diseño escalables, lo que resulta en una mayor consistencia y eficiencia en proyectos grandes.

Por último, Figma es una opción económicamente accesible para equipos de todos los tamaños. Ofrece una versión gratuita con funciones completas y opciones de pago asequibles para aquellos

que necesitan características avanzadas o mayores niveles de colaboración. Esto hace que Figma sea una excelente opción tanto para freelancers como para grandes equipos corporativos.

Figma ha demostrado ser una herramienta innovadora que ha cambiado la manera en que los equipos de diseño y desarrollo trabajan juntos.

Si estás interesado en aprender más sobre Figma y otras herramientas esenciales para el diseño web, no dudes en explorar nuestros cursos de diseño web [13] [14].

Visual Studio Code

Visual Studio Code es un editor de código fuente avanzado, desarrollado por Microsoft y lanzado en 2015. No es solo un editor de texto, sino una herramienta completa de desarrollo diseñada para soportar múltiples lenguajes de programación y tareas de desarrollo en diferentes plataformas (Windows, MacOS y Linux). Al ser open-source, permite a la comunidad de desarrolladores contribuir con extensiones y mejoras, lo que lo hace una de las herramientas más versátiles y personalizables en el mercado de desarrollo.

- **Ligereza y Velocidad:** VS Code es conocido por su velocidad y eficiencia, ya que utiliza el motor de rendering de Electrón. Esto lo hace menos pesado que otros entornos de desarrollo integrados (IDEs), permitiendo que corra con fluidez en máquinas con pocos recursos.

Características Clave de VS Code

1. Extensiones y Personalización

- **Extensiones de Marketplace:** El Marketplace de VS Code permite a los usuarios instalar extensiones para casi cualquier lenguaje de programación o tecnología, desde Python y JavaScript hasta herramientas de DevOps como Docker y Kubernetes. Esto transforma a VS Code de un editor de texto básico a un IDE completo según las necesidades de cada desarrollador.
- **Configuración Personalizable:** Los usuarios pueden personalizar casi todos los aspectos de VS Code mediante archivos de configuración, como settings.json, launch.json, y tasks.json. Esto incluye desde atajos de teclado hasta configuraciones avanzadas de compilación y depuración.

2. Integración con Sistemas de Control de Versiones

- **Git Integrado:** VS Code tiene integración nativa con Git, permitiendo a los desarrolladores realizar commits, ver cambios y administrar ramas directamente desde el editor. La interfaz es intuitiva y muestra de manera visual los cambios que se están realizando en el código.
- **Compatibilidad con GitHub y Azure DevOps:** Ofrece integración adicional con plataformas de desarrollo colaborativo en la nube como GitHub y Azure DevOps, permitiendo a los equipos de trabajo compartir y sincronizar sus cambios en tiempo real.

3. Terminal Integrada

- **Terminal Integrada:** VS Code permite abrir una terminal dentro del editor, lo que facilita la ejecución de comandos de línea en el contexto del proyecto actual. Esto es útil para ejecutar scripts, instalar dependencias y manipular archivos sin tener que salir del editor.
- **Compatibilidad Multilenguaje:** La terminal puede configurarse para soportar múltiples lenguajes de shell (como Bash, PowerShell, o cmd), adaptándose a las preferencias del desarrollador.

4. Depuración Avanzada

- **Soporte de Depuración Multi-Lenguaje:** VS Code es compatible con varios depuradores, lo cual permite identificar errores en el código de forma rápida. Soporta depuración en lenguajes como Python, JavaScript, y C++ de manera nativa o mediante extensiones.
- **Breakpoints y Monitoreo de Variables:** Los desarrolladores pueden establecer puntos de ruptura, monitorear variables y ver el flujo de ejecución paso a paso en el código, lo que facilita la resolución de errores complejos.
- **Depuración en Tiempo Real con Live Share:** Esta extensión permite que otros desarrolladores se unan a la sesión de depuración en tiempo real, facilitando el trabajo colaborativo en la resolución de errores.

5. Funciones de Colaboración

- **Live Share:** Esta característica de colaboración permite a los desarrolladores trabajar simultáneamente en el mismo código, compartiendo su sesión de edición, depuración e incluso terminales y servidores locales. Live Share es especialmente útil para trabajo en equipo remoto, permitiendo que otros desarrolladores puedan ver o editar el código en tiempo real.

- Visual Studio Code Spaces: Microsoft también ofrece una versión en la nube de VS Code que permite a los desarrolladores trabajar desde cualquier dispositivo. Esto es ideal para trabajar en equipo, ya que proporciona entornos de desarrollo aislados y configurables, accesibles desde navegadores Cita: [15].

Aplicaciones y Relevancia de VS Code en el Proyecto

En el contexto del desarrollo de la aplicación móvil para la Dirección General del Hospital, Visual Studio Code es una herramienta esencial por varias razones:

1. Desarrollo Multilenguaje: VS Code permite manejar varios lenguajes y tecnologías al mismo tiempo. En este proyecto, facilita el desarrollo de Backend en Python y Frontend en Flutter, y permite integrarse con bases de datos SQL y NoSQL.
2. Integración de Control de Versiones: La integración con Git facilita la colaboración en equipo, permitiendo el control de versiones, que es crucial en un proyecto hospitalario donde la integridad y el historial de cambios en el sistema son críticos para la seguridad de los datos.
3. Productividad con Extensiones: Las extensiones para Python, Dart (usado en Flutter), y SQL agilizan el flujo de trabajo, permitiendo que el equipo se enfoque en el desarrollo de la lógica hospitalaria sin preocuparse por configuraciones complicadas.
4. Depuración y Pruebas en Tiempo Real: Gracias a las funcionalidades de depuración, los desarrolladores pueden identificar y resolver errores en tiempo real. La posibilidad de compartir sesiones a través de Live Share ayuda a los equipos a colaborar y resolver problemas de forma más rápida y eficiente.
5. Automatización del Flujo de Trabajo: La terminal integrada y la personalización de tareas permite automatizar procesos repetitivos, como la compilación y ejecución de pruebas, lo cual es ideal para mantener la eficiencia y la calidad en un proyecto que puede necesitar pruebas constantes debido a su naturaleza crítica en el ámbito hospitalario.

La versatilidad y personalización que Visual Studio Code ofrece lo convierten en una herramienta fundamental para el desarrollo de aplicaciones modernas, especialmente en entornos de alta demanda como los sistemas hospitalarios. Su integración con sistemas de control de versiones, capacidades de depuración avanzada, y soporte para múltiples lenguajes y tecnologías permiten que el proyecto se desarrolle con mayor agilidad y eficiencia, mientras que la documentación y el

soporte de la comunidad aseguran que el equipo tenga acceso a las mejores prácticas en todo momento [15].

Python

Python es un lenguaje de programación de alto nivel, interpretado, y de propósito general que fue creado por Guido van Rossum y lanzado en 1991. Desde sus inicios, Python ha ganado popularidad debido a su simplicidad, legibilidad y la versatilidad de sus aplicaciones. Está diseñado para promover la claridad y la productividad, lo que permite a los desarrolladores escribir menos código en comparación con otros lenguajes de programación [16].

Características Clave de Python:

- **Lenguaje de Tipado Dinámico:** Python es de tipado dinámico, lo que significa que no se requiere especificar el tipo de variable (como `Int` o `String`). Esto facilita la escritura de código y permite una mayor flexibilidad.
- **Multiparadigma:** Python soporta la programación orientada a objetos, la programación funcional y la programación estructurada, lo cual permite adaptarlo a distintos estilos y necesidades.
- **Gran Biblioteca Estándar:** Python ofrece una biblioteca estándar muy amplia, conocida como "batteries included", que cubre una gran variedad de tareas desde manejo de archivos y bases de datos hasta web scraping y computación científica.
- **Multiplataforma:** Python es compatible con múltiples sistemas operativos (Windows, macOS, Linux), y su código puede ejecutarse sin modificaciones en diferentes plataformas, Cita: [16].

Funciones y Aplicaciones de Python en el Proyecto

Python ofrece una variedad de funciones y capacidades que son altamente beneficiosas para el desarrollo de la aplicación móvil para la Dirección General Hospitalaria. Estas capacidades se pueden clasificar en varias áreas clave:

1. Desarrollo de API y Backend con FastAPI:

- **Framework FastAPI:** Python se utiliza junto con FastAPI, un framework moderno y de alto rendimiento, diseñado específicamente para el desarrollo de APIs basadas en Python.

FastAPI permite a los desarrolladores construir APIs RESTful y basadas en microservicios de manera eficiente, utilizando anotaciones de tipo para una validación de datos rápida.

- **Capacidad Asíncrona:** Una de las ventajas más importantes de FastAPI es su soporte para asincronía, lo que permite manejar múltiples solicitudes simultáneamente sin bloquear el sistema. Esto es fundamental para el proyecto, dado que un sistema hospitalario puede requerir el procesamiento de múltiples solicitudes de manera concurrente.
- **Documentación Automática:** FastAPI genera automáticamente documentación de la API en formato OpenAPI y Swagger, facilitando la comprensión y el uso de la API por parte de otros desarrolladores o aplicaciones que necesiten interactuar con el sistema.

2. Gestión de Base de Datos y Operaciones CRUD:

- **SQLAlchemy y PyMySQL:** Python se integra fácilmente con bases de datos SQL mediante bibliotecas como SQLAlchemy y PyMySQL. SQLAlchemy es un ORM (Object Relational Mapping) que permite mapear objetos Python a tablas en bases de datos, facilitando la realización de operaciones CRUD (Create, Read, Update, Delete) de manera segura y estructurada.
- **Interacción con Bases de Datos SQL y NoSQL:** Python también soporta bases de datos NoSQL (como MongoDB) con bibliotecas como PyMongo, lo que lo hace ideal para aplicaciones que necesitan combinar diferentes tipos de almacenamiento de datos.
- **Consultas y Optimización:** La flexibilidad de Python permite realizar consultas SQL complejas y optimizadas, lo que garantiza que la aplicación pueda manejar grandes volúmenes de datos y obtener información en tiempo real.

3. Automatización de Procesos y Tareas Programadas:

- **Automatización de Tareas con Cron y Celery:** Python permite la automatización de tareas utilizando Celery para la ejecución de tareas en segundo plano y cron jobs para programación de tareas periódicas. Estas herramientas son cruciales para tareas como el envío de notificaciones automáticas, la limpieza de datos o la generación de reportes programados.
- **Scripts de Mantenimiento:** Python facilita la creación de scripts que pueden realizar tareas rutinarias de mantenimiento del sistema, como respaldos de bases de datos, verificación de integridad de datos y limpieza de registros antiguos.

4. Generación de Reportes y Análisis de Datos:

- **Bibliotecas de Análisis:** Python es ampliamente utilizado en análisis de datos, con bibliotecas como Pandas, NumPy y Matplotlib. Estas bibliotecas permiten la manipulación de grandes conjuntos de datos, su análisis y la generación de gráficos, lo cual es importante para el sistema hospitalario al necesitar reportes detallados y visualización de datos.
- **Visualización de Datos:** A través de Matplotlib y Seaborn, Python permite crear gráficos que ayudan a visualizar indicadores clave, como el rendimiento del sistema, estadísticas de solicitudes, y datos relevantes para la toma de decisiones.

5. Desarrollo de Algoritmos de Seguridad:

- **Manejo de Encriptación y Seguridad:** Python permite implementar medidas de seguridad avanzadas mediante bibliotecas como cryptography para el cifrado de datos sensibles y hashlib para la generación de hashes seguros. Estas capacidades aseguran que la información confidencial de los pacientes y el hospital esté protegida, cumpliendo con las normativas de privacidad.

6. Soporte para Machine Learning y Análisis Predictivo:

- **Inteligencia Artificial:** Con bibliotecas de aprendizaje automático como TensorFlow y Scikit-Learn, Python puede implementar modelos predictivos que optimicen la gestión de recursos y permitan análisis de patrones en los datos de los pacientes. Esto es especialmente útil para prever necesidades en inventarios, planificación de personal o incluso en diagnósticos automatizados.

Relevancia de Python en el Proyecto:

Python es una elección fundamental en este proyecto debido a sus ventajas en cuanto a simplicidad, flexibilidad y rendimiento. A continuación, se detallan las razones por las cuales Python es crítico para el desarrollo de este sistema:

1. **Facilidad de Desarrollo y Mantenimiento:** Python tiene una sintaxis simple y clara, lo que reduce la cantidad de tiempo necesario para desarrollar el sistema y facilita el mantenimiento del código a largo plazo. Esto es especialmente importante en un sistema hospitalario, donde el tiempo de implementación es crucial y las actualizaciones deben realizarse rápidamente.

2. Escalabilidad y Flexibilidad: Python permite una escalabilidad sin problemas, ya que se adapta fácilmente a arquitecturas distribuidas y al trabajo con microservicios. Esto permite que el sistema crezca y se expanda en función de las necesidades del hospital, sin tener que modificar significativamente la estructura del proyecto.
3. Ecosistema de Herramientas para DevOps y CI/CD: Python es compatible con herramientas de integración continua y despliegue continuo (CI/CD) como Jenkins y Docker, facilitando el desarrollo en un entorno DevOps. Esto es vital para un sistema hospitalario, donde los cambios deben implementarse de manera segura y rápida sin interrupciones en el servicio.
4. Adaptabilidad a Normativas y Cumplimiento de Seguridad: Python permite la integración de bibliotecas de seguridad y el cumplimiento de normas como GDPR o HIPAA (normativas de privacidad en el manejo de datos de salud). Esto asegura que el sistema cumpla con las políticas de protección de datos y que toda la información sea manejada de forma ética y segura.
5. Soporte de la Comunidad y Extensa Documentación: La comunidad de Python es una de las más grandes y activas del mundo, ofreciendo soporte y recursos abundantes. Esto permite al equipo de desarrollo resolver problemas rápidamente, acceder a una amplia variedad de bibliotecas y seguir las mejores prácticas para asegurar la calidad del proyecto.

SQL

SQL (Structured Query Language) es un lenguaje estándar de manipulación y consulta de datos en bases de datos relacionales. Fue desarrollado en la década de 1970 en los laboratorios de IBM y se ha convertido en el estándar para trabajar con bases de datos, utilizado ampliamente en sistemas de administración de datos empresariales, científicos, financieros, y muchos otros. SQL permite a los desarrolladores y administradores de bases de datos crear, modificar, y consultar datos almacenados en bases de datos de manera estructurada y organizada.

Características Clave de SQL:

- Lenguaje Declarativo: SQL es un lenguaje declarativo, lo que significa que permite a los usuarios especificar qué datos quieren obtener sin tener que detallar cómo lograrlos, haciendo que las consultas sean claras y legibles.

- Estándares y Compatibilidad: SQL es un lenguaje estandarizado por ANSI/ISO, lo que garantiza que las consultas SQL básicas funcionen en la mayoría de los sistemas de bases de datos relacionales como MySQL, PostgreSQL, Oracle, SQL Server, y otros.
- Control de Acceso y Seguridad: SQL soporta mecanismos avanzados de seguridad, incluyendo permisos, roles, y auditoría, que son fundamentales en sistemas que manejan datos confidenciales.

Funciones y Aplicaciones de SQL en el Proyecto

En este proyecto de gestión hospitalaria, SQL se utiliza para gestionar y operar con los datos críticos del sistema, como información de pacientes, solicitudes de recursos médicos, y otros datos relevantes. SQL es crucial para mantener la integridad, seguridad y eficiencia en el manejo de estos datos. A continuación, se detallan algunas de las principales aplicaciones y funcionalidades de SQL en este contexto:

1. Creación y Manipulación de Bases de Datos y Tablas:

- Data Definition Language (DDL): SQL permite crear y definir estructuras de datos en la base de datos, como tablas, índices y vistas. Mediante comandos como CREATE, ALTER, y DROP, SQL permite a los desarrolladores definir tablas específicas para gestionar la información del hospital, garantizando la organización y la accesibilidad de los datos.
- Índices y Optimizaciones: SQL permite crear índices en columnas de tablas que se consultan frecuentemente, optimizando la velocidad de búsqueda y recuperación de datos. Esto es crucial en el proyecto, ya que se trabaja con grandes volúmenes de datos y se necesita un acceso rápido y eficiente.

2. Operaciones CRUD (Create, Read, Update, Delete):

- Create (INSERT): SQL permite añadir nuevos registros en las tablas. Esto es útil para insertar nuevas solicitudes de recursos, registro de pacientes, y otras entradas en el sistema.
- Read (SELECT): Con el comando SELECT, SQL permite recuperar datos específicos o conjuntos de datos según criterios predefinidos. En el proyecto, esto permite la recuperación de información detallada de pacientes, reportes de solicitudes, y cualquier dato necesario para la toma de decisiones.

- **Update (UPDATE):** SQL permite modificar datos existentes en las tablas. Esto es crucial en el contexto hospitalario, ya que la información de los pacientes y los recursos debe actualizarse constantemente.
- **Delete (DELETE):** El comando DELETE permite eliminar registros específicos. En el proyecto, es útil para eliminar datos obsoletos o limpiar registros antiguos, manteniendo la base de datos organizada y sin información innecesaria.

3. Consultas Avanzadas y Filtrado de Datos:

- **Consultas de Filtrado Complejas:** SQL permite el uso de WHERE, AND, OR, y LIKE para aplicar filtros avanzados en los datos. Esto es útil para obtener información muy específica, como solicitudes de recursos en un rango de fechas o listas de pacientes según su diagnóstico.
- **Consultas de Agregación:** A través de funciones de agregación (SUM, COUNT, AVG, MAX, MIN), SQL permite calcular estadísticas, como el número total de solicitudes de recursos, el promedio de tiempos de espera de pacientes, o el uso de inventarios médicos.
- **Agrupación y Ordenación de Resultados:** Con el uso de GROUP BY y ORDER BY, SQL permite organizar datos de manera que sean fáciles de interpretar, lo cual es importante en los reportes generados para la gestión hospitalaria.

4. Uniones y Relaciones entre Tablas:

- **INNER JOIN, LEFT JOIN, RIGHT JOIN:** SQL permite combinar datos de varias tablas relacionadas, lo que facilita una visión más completa de la información. Por ejemplo, se puede combinar la información de pacientes con sus solicitudes y recursos asignados en una sola consulta.
- **Vistas:** SQL permite la creación de vistas, que son consultas predefinidas que pueden usarse como si fueran tablas. Las vistas son útiles en el proyecto para presentar datos específicos y restringidos, facilitando el acceso seguro a la información sin mostrar todos los detalles de la base de datos.

5. Gestión de Transacciones y Control de Concurrencia:

- **Transacciones:** SQL permite realizar múltiples operaciones en una sola transacción usando comandos como BEGIN, COMMIT y ROLLBACK. Esto asegura que las operaciones críticas

se completen completamente o se reviertan en caso de error, manteniendo la integridad de los datos en el sistema hospitalario.

- Bloqueo y Aislamiento de Transacciones: SQL soporta niveles de aislamiento y bloqueos, lo que garantiza que múltiples usuarios puedan acceder y modificar la base de datos sin conflictos, manteniendo la consistencia en un sistema de múltiples usuarios como el hospital.

6. Seguridad y Control de Acceso:

- Roles y Permisos: SQL permite la creación de usuarios con permisos específicos para acceder o modificar la información. En el sistema de gestión hospitalaria, es esencial que solo personal autorizado tenga acceso a datos sensibles de los pacientes.
- Auditoría y Registro de Actividades: SQL permite habilitar auditoría para registrar actividades de usuarios en la base de datos. Esto es útil para rastrear quién accede o modifica información crítica, garantizando la seguridad y la conformidad con normativas de protección de datos, Cita: [10].

Relevancia de SQL en el Proyecto

SQL es fundamental en este proyecto por varias razones que aseguran una administración de datos eficaz, segura y escalable para la aplicación de gestión hospitalaria:

1. Eficiencia y Velocidad en la Manipulación de Datos: SQL permite realizar consultas optimizadas que mejoran la velocidad en la recuperación y manipulación de datos, lo cual es esencial para una aplicación hospitalaria que maneja grandes volúmenes de datos y necesita ofrecer respuestas rápidas para garantizar la eficiencia en la atención de los pacientes.
2. Organización y Estructura en el Almacenamiento de Información: Con SQL, es posible estructurar y normalizar la base de datos, reduciendo la redundancia de datos y mejorando la organización de la información. En el contexto hospitalario, donde los datos de pacientes y solicitudes son sensibles y numerosos, la estructura clara de SQL facilita su gestión.
3. Escalabilidad y Mantenimiento Sencillo: Al ser un estándar de la industria, SQL permite que el sistema se expanda fácilmente si aumenta la demanda de almacenamiento y procesamiento de datos. Esto permite al sistema crecer con el hospital, garantizando que la base de datos se pueda adaptar y mantener de manera eficiente.

4. **Integridad de los Datos y Cumplimiento de Normativas:** Con sus mecanismos de control de transacciones, SQL garantiza que los datos permanezcan consistentes incluso en caso de fallos. Además, la capacidad de implementar roles y permisos asegura que el acceso a la información se maneje conforme a las políticas de privacidad y protección de datos, como el GDPR o las normativas de HIPAA en salud.
5. **Facilidad de Integración con Herramientas y Frameworks de Desarrollo:** SQL se integra bien con múltiples lenguajes de programación, como Python, Java, y frameworks de desarrollo de backend como FastAPI. Esto hace que SQL sea ideal para un sistema de gestión hospitalaria, permitiendo que el equipo de desarrollo trabaje sin problemas en un entorno centralizado y compatible.

MySQL

MySQL es un sistema de gestión de bases de datos relacional (RDBMS) de código abierto desarrollado originalmente por la empresa sueca MySQL AB en 1995 y posteriormente adquirido por Sun Microsystems y, más tarde, por Oracle Corporation. Es uno de los sistemas de bases de datos más populares en el mundo debido a su estabilidad, escalabilidad, y rendimiento. MySQL utiliza el lenguaje SQL (Structured Query Language) como estándar para la manipulación de datos, lo cual permite realizar consultas de manera estructurada y eficiente.

Características Clave de MySQL:

- **Código Abierto y Comunidad Activa:** MySQL es de código abierto, permitiendo a los desarrolladores acceder, modificar y distribuir el código base según sus necesidades. Existen también versiones comerciales con soporte adicional, ofrecidas por Oracle.
- **Arquitectura Cliente-Servidor:** MySQL opera bajo una arquitectura cliente-servidor, permitiendo a múltiples usuarios acceder a la base de datos simultáneamente desde diferentes ubicaciones.
- **Alto Rendimiento y Velocidad:** MySQL es conocido por su capacidad de manejar grandes volúmenes de datos de manera rápida y eficiente, siendo una elección común para aplicaciones que requieren tiempos de respuesta rápidos.
- **Compatibilidad Multiplataforma:** MySQL se ejecuta en varios sistemas operativos, incluyendo Windows, macOS, Linux, y Unix, lo que lo hace adecuado para diversos entornos de desarrollo, Cita: [17].

Funciones y Aplicaciones de MySQL en el Proyecto

En el contexto del proyecto de gestión hospitalaria, MySQL desempeña un papel fundamental en la administración de datos sensibles y en la ejecución de operaciones de almacenamiento, consulta, y modificación de datos. Algunas de las principales funciones y aplicaciones de MySQL en este proyecto son:

1. Almacenamiento y Gestión de Datos Estructurados:

- **Creación de Esquemas de Bases de Datos:** MySQL permite crear esquemas de bases de datos personalizados con tablas, vistas e índices que representan la estructura de los datos hospitalarios. Esto permite organizar la información de pacientes, personal médico, inventario de recursos médicos y demás elementos de forma sistemática.
- **Integridad de Datos:** MySQL permite establecer restricciones en las tablas (como PRIMARY KEY, FOREIGN KEY, UNIQUE, y CHECK), asegurando que los datos ingresados cumplan con los requisitos de integridad necesarios para mantener una base de datos precisa y confiable.

2. Consultas y Manipulación de Datos:

- **Consultas SQL Avanzadas:** Con MySQL, es posible realizar consultas avanzadas utilizando JOIN, GROUP BY, ORDER BY, y subconsultas. Estas consultas son esenciales para recuperar y analizar datos específicos, como el número de solicitudes médicas atendidas en un período determinado o la disponibilidad de ciertos recursos en el inventario hospitalario.
- **Soporte de Operaciones CRUD:** MySQL permite realizar operaciones CRUD (Create, Read, Update, Delete), que son fundamentales en cualquier sistema de gestión de datos. En el sistema hospitalario, esto incluye agregar nuevos registros de pacientes, actualizar la información de solicitudes, consultar datos de inventario y eliminar registros obsoletos.
- **Optimización de Consultas con Índices:** MySQL permite crear índices en columnas clave para mejorar el rendimiento de las consultas, acelerando significativamente el tiempo de respuesta en la recuperación de datos críticos en el entorno hospitalario.

3. Seguridad y Control de Acceso:

- **Gestión de Usuarios y Permisos:** MySQL proporciona una robusta gestión de usuarios que permite definir permisos específicos para cada usuario o grupo de usuarios, asegurando que solo el personal autorizado pueda acceder y modificar datos sensibles. Esto es fundamental en un entorno hospitalario para proteger la privacidad de los pacientes y el cumplimiento de normativas.
- **Encriptación y Seguridad de Datos:** MySQL permite encriptar conexiones mediante SSL/TLS, garantizando la seguridad de los datos durante su transmisión y evitando accesos no autorizados.
- **Registro y Auditoría de Actividades:** MySQL soporta el registro de actividad mediante logs de consultas y auditoría, permitiendo rastrear cualquier cambio en los datos y mejorar la transparencia y seguridad en el manejo de la información.

4. Alta Disponibilidad y Recuperación ante Fallos:

- **Replicación de Datos:** MySQL ofrece múltiples opciones de replicación (como replicación maestro-esclavo y replicación circular), lo cual permite crear copias de la base de datos en tiempo real y garantiza que los datos estén disponibles incluso si ocurre una falla en el servidor principal.
- **Clustering y MySQL InnoDB Cluster:** A través de su arquitectura de clúster, MySQL permite configurar bases de datos distribuidas y tolerantes a fallos. Esto asegura que el sistema de gestión hospitalaria siga operando sin interrupciones, incluso si falla un nodo en el clúster.
- **Copias de Seguridad (Backups):** MySQL permite realizar respaldos incrementales y totales de la base de datos, lo cual es fundamental para la recuperación ante desastres y la prevención de pérdida de datos críticos en el sistema hospitalario.

5. Escalabilidad y Adaptabilidad:

- **Particionamiento de Datos:** MySQL permite dividir grandes tablas en particiones más pequeñas, facilitando la administración y acelerando las consultas. Esto es útil en el contexto hospitalario, donde es probable que algunos datos (como registros de pacientes) crezcan considerablemente con el tiempo.

- Integración con Servicios en la Nube: MySQL es compatible con plataformas en la nube como Amazon RDS, Google Cloud SQL y Azure Database for MySQL. Esto permite almacenar los datos en la nube, asegurando accesibilidad, alta disponibilidad y escalabilidad para satisfacer las demandas de la infraestructura hospitalaria.

6. Soporte para Integración con Lenguajes de Programación:

- Compatibilidad con Python, Java, y otros Lenguajes: MySQL puede integrarse con varios lenguajes de programación a través de conectores específicos. En este proyecto, se usa el conector MySQL-Python, que permite realizar consultas SQL desde el backend en Python (usando FastAPI) de manera rápida y segura.
- Automatización de Procesos y Tareas: Con la ayuda de scripts SQL y herramientas de programación, MySQL permite automatizar procesos rutinarios, como el mantenimiento de la base de datos y la limpieza de registros antiguos, lo cual es fundamental en un entorno de datos críticos y en constante cambio como el hospitalario, Cita: [17].

Relevancia de MySQL en el Proyecto

MySQL es fundamental en este proyecto de gestión hospitalaria debido a sus características y ventajas que lo hacen ideal para manejar datos críticos y asegurar la eficiencia, seguridad y disponibilidad de la información:

1. Eficiencia en el Manejo de Datos en Tiempo Real: La capacidad de MySQL para realizar consultas de manera rápida permite recuperar datos en tiempo real, lo cual es crucial para un sistema hospitalario en el que la información sobre pacientes y recursos debe estar disponible inmediatamente para tomar decisiones médicas rápidas.
2. Confiabilidad y Seguridad de la Información: MySQL proporciona un alto nivel de seguridad y permite asignar permisos específicos a los usuarios, asegurando que solo el personal autorizado pueda acceder y modificar datos críticos, como el historial médico de los pacientes. Esto ayuda a cumplir con normativas de privacidad y seguridad, como HIPAA en los Estados Unidos.
3. Escalabilidad y Preparación para Crecimiento: MySQL es una base de datos escalable que permite manejar grandes volúmenes de datos a medida que el sistema crece. La posibilidad de realizar particionamiento y clustering garantiza que la base de datos pueda adaptarse a

un aumento de usuarios y transacciones, haciendo de MySQL una solución preparada para el crecimiento.

4. Soporte para Alta Disponibilidad: Gracias a sus opciones de replicación y clustering, MySQL garantiza que los datos estén siempre disponibles y protegidos contra fallos del sistema. Esta alta disponibilidad es esencial en un sistema de gestión hospitalaria donde el tiempo de inactividad debe minimizarse para no afectar el flujo de trabajo del personal de salud.
5. Facilidad de Integración y Compatibilidad: MySQL se integra fácilmente con diferentes herramientas y lenguajes de programación, permitiendo un desarrollo ágil en combinación con otras tecnologías como Python y FastAPI. Esta capacidad de integración permite que MySQL se adapte de manera efectiva a los requerimientos específicos de este proyecto hospitalario.

Framework

Un framework es una estructura o conjunto de herramientas y bibliotecas que proporcionan un soporte estándar para el desarrollo de software. En términos simples, un framework establece una estructura y patrones de desarrollo que los programadores pueden seguir, proporcionando componentes predefinidos que simplifican el desarrollo de aplicaciones complejas. Al usar un framework, los desarrolladores pueden enfocarse en construir la funcionalidad específica de la aplicación sin tener que desarrollar cada función desde cero.

Características Clave de un Framework:

- Modularidad y Reutilización: Un framework permite que los desarrolladores reutilicen componentes de software ya probados, asegurando modularidad y reduciendo errores.
- Estandarización: Proporciona una arquitectura y un flujo de trabajo prediseñados, lo cual ayuda a estandarizar los procesos de desarrollo y mantener la coherencia del código.
- Abstracción de Complejidades: Los frameworks ofrecen una capa de abstracción que facilita tareas complejas como la gestión de bases de datos, la autenticación, el enrutamiento y el manejo de sesiones.
- Escalabilidad: La mayoría de los frameworks modernos están diseñados para ser escalables, lo cual es ideal para proyectos que podrían crecer en complejidad o tamaño, Cita: [18].

Función General de un Framework en el Proyecto

En el contexto de este proyecto, los frameworks son esenciales para el desarrollo rápido, seguro y escalable de la aplicación. Al utilizar frameworks tanto para el backend como para el frontend, el equipo de desarrollo puede implementar funcionalidades específicas del sistema de gestión hospitalaria de manera eficiente. Los frameworks permiten:

- Facilitar la conexión y gestión de datos en la base de datos.
- Establecer estructuras y patrones de desarrollo claros, lo que reduce errores y facilita el mantenimiento.
- Optimizar la carga de trabajo al permitir la reutilización de componentes y la implementación rápida de funcionalidades críticas.

Frameworks Utilizados en el Proyecto

Para este proyecto de gestión hospitalaria, se han seleccionado frameworks específicos tanto para el desarrollo de la API y backend como para la interfaz de usuario del sistema. A continuación, se describen los frameworks clave utilizados en el proyecto y sus beneficios:

FastAPI (Backend Framework)

FastAPI es un framework web de alto rendimiento basado en Python, diseñado para construir APIs RESTful. Es rápido, moderno y orientado a manejar aplicaciones de alto rendimiento gracias a su arquitectura asíncrona. FastAPI utiliza el tipado de Python y Pydantic para validar datos, lo cual mejora la seguridad y confiabilidad de la API.

Características de FastAPI:

- Alta Velocidad y Eficiencia: FastAPI se basa en Starlette y Pydantic, lo que permite una ejecución rápida y manejo de múltiples solicitudes simultáneamente.
- Generación Automática de Documentación: FastAPI genera documentación automática en formato OpenAPI y Swagger, facilitando la integración con otros sistemas y la comprensión de los endpoints.
- Validación de Datos: Utiliza anotaciones de tipo de Python y validación de datos mediante Pydantic, lo que minimiza errores y asegura que los datos ingresados cumplan con los requisitos.

Relevancia en el Proyecto:

FastAPI es crucial para el backend del sistema hospitalario, permitiendo la creación de una API eficiente y escalable que maneje las solicitudes de usuarios, la consulta de datos de pacientes y el procesamiento de operaciones de la base de datos. Su capacidad para gestionar tareas asíncronas permite manejar múltiples solicitudes de forma simultánea, lo cual es importante en un sistema hospitalario que podría recibir numerosas solicitudes en tiempo real, Cita: [19].

Flutter (Frontend Framework)

Flutter es un framework de desarrollo de interfaces de usuario (UI) creado por Google, que permite el desarrollo de aplicaciones multiplataforma desde una sola base de código. Utiliza Dart como lenguaje de programación y un motor gráfico propio para renderizar interfaces nativas de alta calidad en Android, iOS, y web.

Características de Flutter:

- **Desarrollo Multiplataforma:** Con una sola base de código, Flutter permite crear aplicaciones para diferentes plataformas, lo que reduce el tiempo de desarrollo.
- **Rendimiento Nativo:** Flutter utiliza su propio motor gráfico, Skia, que permite que las aplicaciones se ejecuten con una apariencia y rendimiento nativos.
- **Hot Reload:** Permite ver los cambios en el código de manera instantánea, lo que acelera el proceso de desarrollo y facilita la depuración.
- **Widgets Personalizables:** Flutter ofrece una amplia variedad de widgets prediseñados y personalizables, lo que permite crear interfaces visuales atractivas y adaptadas a las necesidades del proyecto, Cita: [20].

Relevancia en el Proyecto:

Flutter es fundamental para el desarrollo de la interfaz de usuario de la aplicación hospitalaria, ya que permite crear una experiencia de usuario coherente en múltiples plataformas, incluyendo dispositivos móviles y la web. Esto asegura que los usuarios (personal médico y administrativo) puedan acceder a la aplicación desde distintos dispositivos con una experiencia de usuario homogénea y rápida.

¿Qué es SQLAlchemy?

SQLAlchemy es un ORM (Object Relational Mapping) para Python, que facilita la interacción entre el código Python y las bases de datos relacionales, como MySQL. Con SQLAlchemy, los desarrolladores pueden trabajar con objetos Python en lugar de escribir consultas SQL complejas, lo que simplifica el manejo de datos.

Características de SQLAlchemy:

- **Mapeo de Objetos a Tablas:** SQLAlchemy permite mapear clases Python a tablas de bases de datos, lo que facilita la manipulación de datos desde el código.
- **Consulta de Datos Simplificada:** Con SQLAlchemy, es posible realizar consultas complejas de manera sencilla, utilizando métodos Python en lugar de escribir SQL.
- **Compatibilidad con Múltiples Bases de Datos:** SQLAlchemy es compatible con varios sistemas de bases de datos, lo cual permite flexibilidad en el desarrollo.

Cita: [21]

Relevancia en el Proyecto:

SQLAlchemy se utiliza en el backend del proyecto para facilitar el manejo de datos en MySQL desde el código Python. Permite que los desarrolladores realicen operaciones CRUD de manera eficiente, reduciendo el tiempo de desarrollo y minimizando el riesgo de errores en las consultas SQL.

Los frameworks seleccionados (FastAPI, Flutter, SQLAlchemy) ofrecen una combinación de herramientas que facilitan el desarrollo, escalabilidad, y la eficiencia del sistema de gestión hospitalaria. Estos frameworks garantizan una estructura bien definida, mejoran la seguridad de la aplicación y aseguran que el sistema pueda crecer y adaptarse a las necesidades cambiantes del hospital.

Overview (Monitoreo de Bases de Datos)

Overview es una herramienta avanzada de monitoreo de bases de datos que proporciona visibilidad en tiempo real del rendimiento, la salud y el uso de recursos de las bases de datos en funcionamiento. Esta herramienta permite a los administradores de sistemas y a los desarrolladores identificar problemas de rendimiento, optimizar consultas y mejorar la eficiencia general del sistema.

de bases de datos. Overview es utilizada en distintos entornos donde la disponibilidad y la rapidez de respuesta de la base de datos son esenciales, lo cual incluye aplicaciones de gran escala en áreas como comercio electrónico, finanzas, y, en este caso, sistemas de gestión hospitalaria.

Overview se centra en facilitar la administración de bases de datos mediante el análisis profundo y la monitorización de métricas clave. Esto permite que las empresas y organizaciones se aseguren de que sus bases de datos están funcionando a su máxima capacidad, minimizando problemas que podrían afectar la experiencia del usuario final.

Características Clave de Overview:

1. Análisis de Rendimiento en Tiempo Real:

Overview permite una monitorización continua y en tiempo real del rendimiento de la base de datos, brindando visibilidad sobre:

- **Carga de Trabajo:** Muestra cómo se distribuye la carga de trabajo en diferentes momentos, lo cual permite a los administradores ajustar recursos según la demanda.
- **Consultas Lentas:** Identifica las consultas que tardan demasiado en ejecutarse, lo cual ayuda a localizar áreas problemáticas que requieren optimización.
- **Consumo de Recursos:** Supervisa el uso de CPU, memoria, disco y otros recursos de hardware, ayudando a identificar si el rendimiento bajo se debe a un problema de recursos o a la estructura de la base de datos.

2. Optimización de Consultas y Ajuste de Índices:

Overview facilita la optimización de consultas, proporcionando datos y recomendaciones específicas sobre consultas que pueden ralentizar el sistema. Entre las funcionalidades de optimización se incluyen:

- **Recomendaciones de Índices:** Sugiere la creación o ajuste de índices para mejorar la velocidad de recuperación de datos en tablas de gran tamaño.
- **Análisis de Costo de Consultas:** Calcula el costo de cada consulta en términos de tiempo de CPU y otros recursos, lo cual permite a los desarrolladores optimizar el código SQL.
- **Identificación de Bloqueos y Contención:** Localiza problemas de contención en los datos, como bloqueos entre transacciones que puedan estar afectando el rendimiento de consultas críticas.

3. Alertas y Notificaciones Inteligentes:

Overview permite configurar alertas personalizadas para notificar automáticamente a los administradores sobre problemas de rendimiento antes de que estos afecten la experiencia del usuario final. Las alertas se pueden personalizar para:

- Desempeño: Alertar sobre consultas que superen cierto umbral de tiempo o que tengan un alto impacto en el rendimiento.
- Recursos: Notificar sobre el uso elevado de CPU, memoria o espacio en disco, lo cual permite tomar acciones antes de que ocurra una falla.
- Eventos Anómalos: Detectar y alertar sobre patrones de uso inusuales, como picos inesperados en la carga de trabajo, los cuales pueden indicar problemas potenciales o intentos de acceso no autorizado.

4. Historial y Análisis de Tendencias:

Overview almacena datos históricos de rendimiento y consumo de recursos, lo cual permite:

- Identificar Tendencias: Detectar patrones de uso de la base de datos a lo largo del tiempo para planificar la capacidad futura.
- Análisis Comparativo: Comparar el rendimiento actual con el pasado para evaluar si las optimizaciones implementadas están teniendo el efecto deseado.
- Análisis de Picos de Demanda: Permite comprender en qué momentos o fechas la base de datos recibe mayor cantidad de solicitudes, lo cual es útil para planificar escalabilidad.

5. Seguridad y Auditoría:

Overview incluye herramientas de auditoría y seguridad para asegurar la protección de los datos y la integridad del sistema. Estas funciones permiten:

- Monitorear Accesos: Identificar quién está accediendo a la base de datos y qué operaciones están realizando.
- Rastrear Modificaciones en Configuraciones: Registrar cambios en la configuración de la base de datos que puedan afectar el rendimiento o la seguridad.
- Reportes de Conformidad: Generar reportes de auditoría que faciliten el cumplimiento de regulaciones de protección de datos, como HIPAA o GDPR, Cita: [22].

Aplicación de Overview en el Proyecto de Gestión Hospitalaria

La implementación de Overview en el sistema de gestión hospitalaria aporta beneficios esenciales para la administración y operación de una base de datos que maneja información crítica y sensible. A continuación, se describen las principales aplicaciones y beneficios específicos de Overview en este entorno:

1. Supervisión Constante del Rendimiento de la Base de Datos:

En un entorno hospitalario, la disponibilidad y rapidez de acceso a la información son fundamentales. Overview asegura que la base de datos esté optimizada para manejar múltiples solicitudes simultáneas de distintos usuarios (médicos, personal administrativo, etc.) sin retrasos significativos. Esto es especialmente importante en momentos de alta demanda, donde los tiempos de respuesta deben ser inmediatos para apoyar la toma de decisiones clínicas.

2. Optimización de Consultas para Acceso Rápido a Datos de Pacientes y Solicitudes:

Las consultas en un sistema hospitalario deben ser rápidas y eficientes. Overview permite identificar y optimizar consultas complejas o lentas, asegurando que los datos críticos, como el historial médico o las solicitudes de recursos, estén siempre disponibles en el menor tiempo posible. Esto mejora la experiencia del usuario final y garantiza que el personal médico tenga acceso a información relevante para proporcionar una atención de calidad.

3. Monitoreo de Salud del Sistema y Prevención de Fallas:

La herramienta Overview permite monitorear la salud de la base de datos en tiempo real, detectando problemas antes de que afecten el rendimiento del sistema. En el contexto de un hospital, donde el sistema debe estar disponible 24/7, Overview facilita la detección temprana de problemas que puedan derivar en fallas, asegurando una operación continua. Esto reduce el riesgo de interrupciones inesperadas que podrían afectar el acceso a la información de los pacientes.

4. Alertas Proactivas para Soporte Técnico:

Gracias a las alertas personalizables, Overview notifica al equipo de TI cuando se detectan problemas de rendimiento o de consumo de recursos, permitiendo que estos se resuelvan antes de

que impacten al usuario. En el entorno hospitalario, esto permite que el equipo de soporte pueda actuar rápidamente para prevenir demoras en la disponibilidad de los datos médicos.

5. Análisis de Tendencias para Escalabilidad y Planificación de Capacidad:

La recopilación de datos históricos por parte de Overview ayuda a identificar patrones de uso, como los momentos del día o del año en que la base de datos recibe más solicitudes. Esto permite planificar la escalabilidad del sistema y anticipar la necesidad de recursos adicionales, asegurando que el sistema esté preparado para soportar un aumento en la carga de trabajo, como durante emergencias o períodos de alta demanda.

6. Cumplimiento de Normativas y Auditoría de Acceso:

En un sistema que maneja información sensible de pacientes, el cumplimiento de normativas de protección de datos es esencial. Overview permite monitorear los accesos a la base de datos y registrar todas las operaciones realizadas, facilitando la generación de reportes de auditoría que aseguren el cumplimiento con normativas como HIPAA y GDPR. Esto es crucial en un entorno de gestión hospitalaria, donde la seguridad y confidencialidad de la información son prioritarias.

Amazon Amplify (Alternativa para Despliegue y Escalabilidad)

Amazon Amplify es una plataforma de servicios y herramientas de Amazon Web Services (AWS) que facilita el desarrollo y despliegue de aplicaciones full-stack en la nube. Amplify está diseñado para simplificar y automatizar tareas complejas como el manejo de autenticación, bases de datos, almacenamiento, y monitoreo, brindando una solución completa para construir aplicaciones modernas, escalables y seguras. Amplify permite a los desarrolladores implementar aplicaciones frontend y backend desde una única interfaz, con la ventaja de integrar recursos de AWS como S3, Cognito, DynamoDB, entre otros.

Características Clave de Amazon Amplify:

1. Soporte Full-Stack:

- Amplify ofrece herramientas tanto para el backend como para el frontend, proporcionando una experiencia de desarrollo integral. Permite a los desarrolladores gestionar APIs, bases de datos y almacenamiento en la nube, al mismo tiempo que ofrece herramientas de UI para crear interfaces de usuario optimizadas y personalizadas.

2. Integración Sencilla con Servicios de AWS:

- Amplify se integra de forma nativa con una variedad de servicios de AWS, como Cognito (para autenticación y autorización), S3 (para almacenamiento de archivos), DynamoDB (para bases de datos NoSQL) y Lambda (para funciones sin servidor). Esto permite que los desarrolladores añadan funcionalidades avanzadas a sus aplicaciones sin tener que configurar cada servicio por separado.

3. Autenticación y Seguridad Avanzada:

- Amplify facilita la implementación de sistemas de autenticación y autorización mediante AWS Cognito, lo cual asegura que solo usuarios autorizados puedan acceder a los datos. Esto es particularmente útil para aplicaciones que manejan información sensible, como los datos médicos de pacientes en un sistema hospitalario.

4. Escalabilidad Automática:

- Gracias a la infraestructura de AWS, Amazon Amplify proporciona una capacidad de escalabilidad automática que permite ajustar los recursos según la demanda. Esto asegura que la aplicación esté preparada para manejar aumentos inesperados en la carga de trabajo sin comprometer el rendimiento.

5. Capacidades de CI/CD Integradas:

- Amplify incluye un flujo de integración y entrega continua (CI/CD) que permite automatizar la implementación de cambios desde un repositorio Git, haciendo que los nuevos desarrollos o actualizaciones se desplieguen automáticamente y con mayor rapidez.

6. Almacenamiento y Sincronización en Tiempo Real:

- Amplify permite la sincronización de datos en tiempo real entre la aplicación y el backend, lo que es fundamental en aplicaciones que requieren actualizaciones constantes, como los sistemas de gestión hospitalaria. Con Amplify DataStore, los desarrolladores pueden gestionar la sincronización de datos tanto en modo offline como en línea, asegurando que la información esté siempre actualizada y disponible.

7. Analítica y Monitoreo de Usuarios:

- Amplify permite integrar analítica y monitoreo del comportamiento de los usuarios a través de Amazon Pinpoint, proporcionando métricas detalladas que ayudan a entender cómo los usuarios interactúan con la aplicación, así como el rendimiento general del sistema.

Cita: [23]

Aplicación de Amazon Amplify en el Proyecto de Gestión Hospitalaria

Aunque Render es la plataforma de despliegue principal en este proyecto, Amazon Amplify ofrece una alternativa robusta para gestionar aspectos de escalabilidad, seguridad y monitoreo en un sistema tan crítico como el de gestión hospitalaria. A continuación, se detallan las principales aplicaciones y beneficios de Amazon Amplify en el contexto del proyecto:

1. Escalabilidad y Alta Disponibilidad para el Sistema Hospitalario:

- Con su capacidad de escalabilidad automática, Amplify permite que la aplicación hospitalaria esté preparada para manejar altos volúmenes de solicitudes, asegurando que el sistema se mantenga estable y rápido. Esto es crucial en un entorno hospitalario, donde los tiempos de respuesta deben ser mínimos para permitir que el personal médico y administrativo pueda acceder rápidamente a los datos necesarios.

2. Autenticación y Control de Acceso Seguro:

Amazon Amplify, a través de AWS Cognito, permite implementar autenticación multifactorial y acceso basado en roles, lo cual es esencial en el sistema hospitalario para garantizar que solo el personal autorizado pueda acceder a la información confidencial de los pacientes. Además, esta capa de seguridad facilita el cumplimiento con normativas de protección de datos como HIPAA o GDPR.

3. Gestión de Datos y Sincronización en Tiempo Real:

Con Amplify DataStore, el sistema de gestión hospitalaria puede actualizarse en tiempo real para reflejar cambios críticos en los datos de pacientes y recursos médicos, asegurando que toda la información esté sincronizada entre el backend y el frontend. Esto permite, por ejemplo, que los médicos accedan a los registros médicos de los pacientes con información actualizada sin retrasos, mejorando la precisión en el diagnóstico y tratamiento.

4. Automatización de Despliegues y Actualizaciones con CI/CD:

La integración continua y la entrega continua (CI/CD) de Amplify permite que las actualizaciones de la aplicación hospitalaria se implementen automáticamente después de probarse, lo cual minimiza los errores y acelera el tiempo de entrega de nuevas funcionalidades o correcciones. Este flujo es

especialmente útil en un entorno de alta demanda como el hospitalario, donde la disponibilidad del sistema es prioritaria.

5. Almacenamiento Seguro y Acceso a Archivos:

Con Amazon S3 integrado en Amplify, el sistema puede almacenar archivos de manera segura, como documentos médicos, análisis, imágenes de radiografías, y otros datos relevantes. Amplify también permite controlar el acceso a estos archivos según permisos específicos, garantizando que solo el personal médico autorizado pueda acceder a ellos.

6. Analítica de Uso y Monitoreo de la Aplicación:

A través de Amazon Pinpoint, Amplify permite al equipo de desarrollo y administración monitorear el uso del sistema y analizar el comportamiento del usuario. Esto facilita la identificación de posibles áreas de mejora y permite realizar ajustes que mejoren la experiencia del usuario. En el caso de un sistema hospitalario, esta capacidad de monitoreo también ayuda a evaluar el rendimiento del sistema y la satisfacción de los usuarios internos.

Comparativa con Render

Aunque Render es la plataforma principal de despliegue, Amplify ofrece ventajas adicionales que pueden complementar el proyecto, especialmente en términos de:

- **Escalabilidad:** Amplify está diseñado para aplicaciones con una base de usuarios que puede crecer significativamente, como podría ser el caso en un sistema de salud con acceso de múltiples usuarios.
- **Facilidad de Integración con Otros Servicios de AWS:** Amplify permite el acceso directo a otros servicios de AWS, lo cual es beneficioso si el proyecto requiere funcionalidades avanzadas como machine Learning, análisis de grandes volúmenes de datos, o integración con aplicaciones de terceros.

Render (Despliegue en la Nube)

Render es una plataforma de alojamiento en la nube que permite a los desarrolladores y empresas desplegar y escalar aplicaciones de manera rápida y sencilla. Con Render, es posible alojar aplicaciones web, APIs, bases de datos, y otros servicios en la nube sin la necesidad de configurar

infraestructura compleja, lo cual permite concentrarse en el desarrollo del producto en lugar de en la administración de servidores. Render está diseñado para ser altamente accesible y automatizado, facilitando el despliegue de aplicaciones con características avanzadas como escalabilidad automática, integración continua (CI/CD), y certificados SSL gratuitos.

Características Clave de Render:

1. Despliegue Simplificado y Automatización:

- Render facilita el despliegue de aplicaciones web y backend mediante integración directa con repositorios Git. Una vez conectado un repositorio, Render puede desplegar aplicaciones automáticamente cada vez que se realicen cambios en el código, simplificando el proceso de implementación y manteniendo la aplicación actualizada.

2. Escalabilidad Automática:

- Render permite configurar la escalabilidad automática para ajustarse a la demanda. Esto significa que la plataforma puede asignar recursos adicionales en momentos de alta demanda y reducirlos cuando la carga de trabajo disminuye, optimizando el uso de recursos y costos.

3. Certificados SSL y Seguridad Integrada:

- Render ofrece certificados SSL gratuitos para todas las aplicaciones, garantizando conexiones seguras entre el servidor y los usuarios. Además, Render incluye herramientas de configuración de roles y permisos de acceso, lo cual es esencial para proteger la información sensible que se maneja en aplicaciones como los sistemas hospitalarios.

4. Despliegue de Bases de Datos y Microservicios:

- Render permite desplegar bases de datos administradas, soportando opciones como PostgreSQL, que pueden integrarse directamente con las aplicaciones hospedadas. Además, Render es compatible con arquitecturas de microservicios, permitiendo dividir el sistema en componentes independientes que se ejecutan de manera separada, facilitando el mantenimiento y la escalabilidad del sistema.

5. Integración Continua y Entrega Continua (CI/CD):

- Render ofrece un flujo de CI/CD integrado que permite la entrega automática de cambios desde un repositorio Git (como GitHub, GitLab o Bitbucket), asegurando que las actualizaciones se implementen de manera rápida y segura. Esto garantiza que el equipo

de desarrollo pueda lanzar nuevas funcionalidades o corregir errores con eficiencia, minimizando interrupciones en el servicio.

6. Supervisión y Monitoreo de Rendimiento:

- Render proporciona herramientas de monitoreo y visualización de métricas para rastrear el rendimiento de las aplicaciones. Esto incluye el tiempo de respuesta de los servidores, el uso de recursos (como CPU y memoria), y alertas en tiempo real, permitiendo al equipo técnico identificar y solucionar problemas antes de que impacten al usuario final.

7. Flexibilidad en la Configuración y Administración:

- Render ofrece opciones de configuración personalizables, como dominios personalizados, entornos de prueba y producción, y control de versiones. Esta flexibilidad es importante en aplicaciones complejas como los sistemas de gestión hospitalaria, ya que permite crear entornos seguros y consistentes que apoyen el ciclo de vida del desarrollo.

Cita: [24]

Aplicación de Render en el Proyecto de Gestión Hospitalaria

Render es la plataforma de despliegue principal para el sistema de gestión hospitalaria debido a su capacidad para alojar aplicaciones de manera segura, escalable y con mínima intervención manual. A continuación, se detallan las aplicaciones y beneficios específicos de Render en este proyecto:

1. Despliegue Continuo y Actualizaciones Automatizadas:

- Con Render, el equipo de desarrollo puede integrar el sistema de control de versiones Git para implementar actualizaciones automáticamente cada vez que se realicen cambios en el código. Esto es clave en un sistema hospitalario, donde las mejoras y correcciones deben aplicarse rápidamente para mantener la eficiencia y seguridad del sistema.

2. Escalabilidad y Manejo de Carga de Trabajo Variable:

- La función de escalabilidad automática de Render permite que el sistema hospitalario se adapte a diferentes niveles de demanda sin intervención manual. Por ejemplo, en situaciones de alta demanda, como emergencias o picos de solicitudes, Render puede asignar recursos adicionales para asegurar un rendimiento óptimo. Esta flexibilidad es crucial para una aplicación hospitalaria, donde la rapidez y disponibilidad constante del sistema son esenciales para la atención médica.

3. Seguridad en la Transmisión de Datos y Cumplimiento de Normativas:

- Render asegura que todas las conexiones estén protegidas con SSL, lo cual es esencial para proteger la información confidencial de los pacientes. La plataforma también permite configurar permisos y roles de acceso, asegurando que solo el personal autorizado tenga acceso a datos sensibles, lo cual es crucial para cumplir con normativas como GDPR y HIPAA.

4. Soporte para Arquitectura de Microservicios:

- Render permite dividir la aplicación en microservicios, donde cada componente (por ejemplo, manejo de pacientes, gestión de inventarios, etc.) funciona de manera independiente. Esto facilita el mantenimiento y escalabilidad del sistema hospitalario, ya que cada microservicio puede actualizarse o escalarse de forma aislada sin afectar a los demás componentes.

5. Monitoreo en Tiempo Real para Detección Temprana de Problemas:

- Render ofrece monitoreo en tiempo real de métricas clave, como el tiempo de respuesta y el uso de recursos. En un sistema de gestión hospitalaria, esta función permite detectar y resolver problemas de rendimiento rápidamente, asegurando que el sistema esté siempre disponible y responda de manera óptima a las solicitudes.

6. Despliegue de Bases de Datos en la Nube:

- Render permite hospedar bases de datos en la nube, integrándolas con la aplicación principal de manera eficiente. Esto garantiza que los datos del sistema hospitalario sean accesibles y se mantengan seguros en la nube. La administración de bases de datos en Render incluye respaldo y restauración, lo cual es fundamental para proteger los datos críticos y asegurar la disponibilidad en caso de fallos.

Comparativa con Amazon Amplify

Aunque Amazon Amplify también es una plataforma de despliegue robusta, Render presenta ventajas específicas que lo hacen ideal para este proyecto de gestión hospitalaria:

- Despliegue Simplificado: Render ofrece una experiencia de despliegue más sencilla y menos dependiente de la configuración avanzada de AWS, facilitando el uso por parte del equipo de desarrollo.

- Enfoque en Microservicios y Aplicaciones Web: Render está optimizado para aplicaciones web y microservicios, mientras que Amplify es ideal para aplicaciones full-stack. Esto hace que Render sea más adecuado para el tipo de arquitectura modular que requiere el sistema hospitalario.
- Costo Eficiente para Proyectos de Tamaño Mediano: Render es una opción de alojamiento en la nube más económica para aplicaciones con requisitos específicos de escalabilidad y seguridad, mientras que Amplify podría ser útil en proyectos que requieren integración directa con un ecosistema completo de servicios AWS.

Marco Metodológico

Método o Metodologías

Metodología Scrum

La metodología Scrum para el desarrollo ágil de software es un marco de trabajo diseñado para lograr la colaboración eficaz de equipos en proyectos, que emplea un conjunto de reglas y artefactos y define roles que generan la estructura necesaria para su correcto funcionamiento.

Scrum utiliza un enfoque incremental que tiene como fundamento la teoría de control empírico de procesos. Esta teoría se fundamenta en transparencia, inspección y adaptación; la transparencia, que garantiza la visibilidad en el proceso de las cosas que pueden afectar el resultado; la inspección, que ayuda a detectar variaciones indeseables en el proceso; y la adaptación, que realiza los ajustes pertinentes para minimizar el impacto de las mismas.

Los llamados Equipos Scrum son autogestionados, multifuncionales y trabajan en iteraciones. Los integrantes del equipo tienen todos los conocimientos necesarios (por ser multifuncionales) para llevar a cabo el trabajo. La entrega del producto se hace en iteraciones; cada iteración crea nuevas funcionalidades o modifica las que el dueño del producto requiera.

El ciclo de vida de este marco de trabajo está compuesto de cuatro fases: planeación, puesta en escena, desarrollo y entrega. En la planeación se establece la visión, se fijan las expectativas y se asegura el financiamiento. En la puesta en escena se identifican más requerimientos y se priorizan para la primera iteración. En la implementación se desarrolla el sistema, y en la entrega se hace el despliegue operativo.

El proyecto del hospital se dividirá en tres principales Springs, siendo:

- **SP1 - La Restructuración de la base de datos y Creación de Login de la aplicación.**
- **SP2 - Procesos Automatizados, interfaces y roles.**
- **SP3 - Dashboard y Gráficos dinámicos.**

Las actividades de acuerdo a la metodología se gestionarán en la plataforma digital Jira, donde cada tarea que involucren la intervención del Spring será enlistadas y programadas en el calendario que junto a la configuración integrada del Diagrama de permitirá medir el tiempo que conlleve su realización.

DevOps

De acuerdo con la guía DevOps, “Una de las claves para garantizar la calidad en las entregas es reducir la intervención humana a la hora de instalar entornos, empaquetar, testear, entregar o incluso volver al estado anterior en caso de problemas.” [25]

Esta metodología permitirá establecer cuatro criterios principales:

La planificación permitirá definir las características y la funcionalidad de la aplicación móvil, avanzando de manera secuencial sin detener la producción del proyecto y especificando las tareas. La creación de un registro de trabajos pendientes, el seguimiento de los errores en el desarrollo del proyecto, la administración del desarrollo de software ágil con Scrum y la visualización del progreso permitirá que el equipo de desarrollo tenga una visión amplia de las etapas concluidas y de aquellas aún en preparación. Estos parámetros incluyen la reestructuración de la base de datos, la migración de la base de datos hacia el backend desarrollado en Django y FastAPI, y el modelado de las vistas y el diseño de la aplicación.

El desarrollo incluye todos los aspectos relacionados con la programación de la aplicación en Flutter, la gestión del funcionamiento de la base de datos tanto de manera local como desplegada a través de FastAPI y Swagger UI, y las pruebas de funcionamiento del API creado, incluyendo su conexión con la aplicación web y móvil. El equipo de DevOps sincronizará con rapidez y fluidez los procesos para que se gestionen de manera automatizada, sin sacrificar la calidad, estabilidad ni productividad.

El proceso de implementación permite que las funciones de la aplicación desarrollada se ejecuten correctamente en los dispositivos móviles. La fase de entrega incluye la implementación y configuración de la infraestructura básica, garantizando que la aplicación funcione sin errores, interrupciones o caídas del sistema. Además, se define el versionamiento del software y las automatizaciones correspondientes, lo que permite al equipo de desarrollo DevOps realizar entregas con facilidad, confianza y tranquilidad.

El uso implica supervisar el funcionamiento de la aplicación y establecer soluciones a problemas emergentes dentro del entorno de producción. Esto permite al equipo trabajar para asegurar la confiabilidad, la alta disponibilidad y el objetivo de eliminar el tiempo de inactividad del sistema, mientras se refuerzan la seguridad y la fiabilidad del mismo. Además, se implementan alertas que facilitan la toma de medidas correspondientes para mejorar el desempeño de la aplicación.

Presentación de la conexión metodológica

El desarrollo del proyecto, con la implementación de la metodología Scrum, ha permitido organizar la gestión de manera secuencial, realizando revisiones constantes y actualizando el análisis de los requisitos del proyecto. Gracias a esta gestión, se ha logrado reajustar el diseño de la base de datos, el uso de frameworks de diseño y maquetación, así como la migración y reestructuración del back-end, tanto de forma local como desplegada en servidores externos para su consulta, adaptándose a las necesidades requeridas en cada sprint.

La comunicación asertiva y la planificación de actividades, basadas en el análisis de las evidencias registradas durante la ejecución de cada proceso, han permitido que el productor Owen y el equipo de desarrollo estén en constante contacto. Esto ha facilitado la definición de prioridades, la demostración de secciones funcionales, la identificación de errores de ejecución y la implementación de cambios que mejoran la calidad del software desarrollado.

Una de las principales ventajas de la implementación de la metodología Scrum es que, al finalizar cada sprint, se entregan secciones funcionales del proyecto. Esto permite realizar pruebas internas de funcionamiento, identificando errores en dichas implementaciones, lo que amplía los criterios de calidad y brinda mayor fiabilidad y confianza al cliente o a los directivos del proyecto hacia el equipo encargado del desarrollo del sistema.

Contar con evidencias claras del funcionamiento de cada entregable en la gestión Scrum mantiene al equipo de desarrollo más motivado, al tener la certeza de que cada avance ha sido evaluado y aprobado por el equipo de dirección del proyecto. Esto, a su vez, facilita escalar configuraciones futuras y permite abrir un panorama hacia las demandas del mercado, destacando la adaptabilidad del sistema a nuevas necesidades.

Por esta razón, se han utilizado tableros para analizar las tecnologías que pueden integrarse al desarrollo realizado, garantizando su compatibilidad con múltiples dispositivos. Esto ha sido posible gracias al uso de frameworks de desarrollo tanto para el diseño del Front-End como del Back-End, además de la implementación de patrones estructurales que mejoran la experiencia del usuario.

Esto, a su vez, mejora la calidad del producto en desarrollo en cada revisión, reduciendo los riesgos y gastos, y optimizando la gestión de los patrones funcionales esperados de la aplicación. Junto con

el diseño web realizado anteriormente, el desarrollo de la aplicación ha permitido organizar el flujo de información de la API construida, que, en conjunto con la aplicación web y móvil, se unifica para conectarlas mediante pruebas exhaustivas de funcionalidad. Esto garantiza que ambos sistemas sigan los mismos principios de calidad esperada.

Este principio amplio se centra en la aplicación de DevOps (Desarrollo y Operaciones). Con la implementación de la estructura del back-end en el diseño de la aplicación web, se gestionó su despliegue en un servidor externo para que la funcionalidad de la API estuviera disponible en la web. A su vez, el trabajo realizado con Rest-API, y posteriormente reestructurado en FastAPI, permitió que el despliegue de la aplicación se gestionara con automatización. Esta automatización evalúa si cada componente del CRUD es funcional y si la configuración asignada respeta los principios establecidos en el diseño de la base de datos. Además, genera modelos estructurados en el API que pueden ser interpretados como MySQL, permitiendo analizar y verificar, de manera remota, las inserciones, actualizaciones, lecturas y eliminaciones desde el gestor Swagger UI de FastAPI Django, desplegado en la nube.

Esta sincronía entre el entorno de la aplicación web y móvil permitirá que el flujo de peticiones hacia el back-end siga el flujo de información esperado, ya que el proceso de compilación de solicitudes se ha automatizado y ya no requiere una fuente local para su funcionamiento.

Al realizar entregas de peticiones al back-end, se mejora la calidad del proceso, permitiendo implementar filtros de seguridad y alertas de errores con especificaciones claras sobre lo que está ocurriendo en el sistema. Esto facilita al equipo de desarrollo resolver los conflictos de manera más rápida, al poder localizar la fuente del error con mayor eficacia.

Tanto el equipo de operaciones como el de desarrollo están en constante comunicación. Cuando se presenten errores que involucren a ambas partes, esta comunicación permitirá que la organización resuelva el conflicto de manera breve y eficaz, lo que contribuirá a mejorar la estabilidad y el rendimiento del sistema.

Al tener un panorama claro de comunicación y atención a los problemas, el tiempo de rediseño se reduce, e incluso puede no ser necesario detener todos los procesos. Utilizando una arquitectura de componentes, se puede desactivar únicamente aquellos que presenten conflictos, sin afectar a los demás, lo que garantiza la confiabilidad del sistema.

Esta gestión de componentes permite otorgar escalabilidad y flexibilidad a la configuración del sistema, ajustando rápidamente la infraestructura y las aplicaciones a la demanda, en cada despliegue de versión del back-end.

La aplicación web también implementa este principio de arquitectura por componentes o microservicios. En cada versión de despliegue, la aplicación es funcional en los aspectos adaptativos a la infraestructura realizada en el back-end. Ambos sistemas están aislados, pero conectados mediante el API. Sin embargo, al ser servicios independientes, la caída de uno de estos no afecta el funcionamiento del otro. De igual manera, la aplicación web responde a este principio: está conectada al API del Back-End, es completamente independiente para ejecutarse en los dispositivos móviles, pero requiere la información del API del back-end para procesar peticiones y solicitudes.

La implementación del API del back-end ha permitido optimizar los recursos en el desarrollo, al ser la fuente principal de información que procesan tanto la aplicación web como la móvil. El mantenimiento y la recuperación del sistema permiten resolver problemas antes de que impacten al usuario final, minimizando el tiempo de inactividad y asegurando la disponibilidad.

Al estar desplegada, la API también permite automatizar tareas repetitivas, lo que libera tiempo para que los equipos de desarrollo se concentren en actividades estratégicas de alto impacto, como actualizaciones y mejoras futuras, así como en implementaciones estructurales de la aplicación web y móvil.

La aplicación de DevOps en el despliegue del API del back-end permite filtrar errores durante la operación de despliegue. Esta automatización analiza los paquetes y las dependencias que podrían causar conflictos, así como las alertas de seguridad en la implementación, lo que asegura que el sistema de calidad no tenga fallos en el rendimiento y ejecución, y señala lo que debe corregirse antes de realizar nuevamente el despliegue de la configuración del API.

Esto permite generar métricas y alertas constantes que ayudan a mejorar el rendimiento del sistema y ajustar los procesos con datos precisos, lo que facilita entregas más rápidas y un software de mayor calidad, con tiempos de respuesta más eficientes para el cliente.

Presentación de los Instrumentos de elaboración

Antecedentes Epidemiológicos y Estructuración de los Requerimientos Funcionales

Para entender el enfoque de atención médica las funciones esenciales en la atención se centran en atender la problemática y los componentes que se utilizarán como fuente de recursos necesarios y esenciales en la aplicación de medidas de atención pública que permita a las instituciones de salud llevar a cabo sus tareas, para ello se han identificado los siguientes enfoques en la estructuración de atención:



Fuente: SIDSS - DGPLADES - SAODIR (2019).

ILUSTRACIÓN 1. COMPARACIÓN ENTRE LA PROBLEMÁTICA Y EL COMPONENTE FUNCIONAL EN LAS OPERACIONES DE ATENCIÓN MÉDICA.

El gráfico anterior se menciona en el artículo de Administración y Organización de los Distritos de Salud [26], que identifica la problemática y la funcionalidad de respuesta a la que hace referencia, haciendo una interacción continua con su área respectiva.

1.- Requisito funcional

Tomando como referencia la problemática y las funcionalidades para el desarrollo de la aplicación móvil se necesita implementar la respuesta apropiada a las solicitudes que correspondan a las peticiones de los sectores de salud críticos con las áreas de especialización, es por ello que la organización debe de establecer parámetros que prioricen la atención oportuna con un tiempo de respuesta eficaz.

Coordinación sectorial e Intersectorial

Para dar cumplimiento con la protección de la salud se necesita salvar los principios éticos y morales en la ejecución médica, es por ello que se debe de garantizar la atención oportuna que garantice la calidad del servicio médico, esto puede llevar a dificultades, ya que la demanda de atención y tiempo de respuesta puede variar, dependiendo de la región donde se estén dando el servicio médico, tomando la descripción del artículo [26] que retoma los objetivos en la atención clínica menciona:

- Aumento de los puntos de acceso a los servicios sin importar el tipo de condición laboral.
- **Mejorar el acceso a servicios de mayor complejidad;**
- Uso racional y eficiente de los recursos disponibles;
- **Diagnósticos oportunos y tratamientos precoces para mejorar la satisfacción de las personas con los servicios recibidos y para disminuir el tiempo de la enfermedad.**
- **Atención médica continúa desde la detección, diagnóstico, tratamiento, seguimiento, control y rehabilitación que las personas requieren para la protección de la salud;**
- Rendición de cuentas con la participación de la población;
- Efectividad, eficacia y equidad en los servicios de salud.

2.- Requisito Funcional

Con base en esta información se incluye que las peticiones realizadas además de ser entendidas en tiempo y forma esta debe de llevar un control y seguimiento, aperturado el seguimiento de los casos con situación de gravedad a canalizarlos a área de atención con mayor complejidad. Si el caso lo requiere redirigir a los pacientes a otros hospitales que cuenten con el equipo necesario para su atención oportuna.

3. Requisito Funcional

Dado que cada solicitud aprobada debe ser ejecutada en un tiempo específico, es necesario llevar un control pos - atención el cual requiere necesariamente de las observaciones del profesional clínico, es por ello que en cada solicitud se requerirá de las observaciones y anotaciones del tratamiento, servicio, intervención, traslado, etc. de quien está solicitando el servicio, como los datos del paciente que recibirá la atención o intervención , aplicando el objetivo de Atención médica

continua desde la detección, diagnóstico, tratamiento, seguimiento, control y rehabilitación que las personas requieren para la protección de la salud.

4. Requisito Funcional

Dentro de las actividades de gestión clínica y autorización de los protocolos de seguimiento en la autorización de los servicios de atención, se hace un registro con la información recapitulada de cada procedimiento el cual permite generar un historial clínico, estos registros permiten que los diagnósticos y tratamientos efectuados mejoren la calidad de vida de los pacientes, todos estos registrados en una bitácora de actividades, estos registros son de vital importancia para la medición de los índices de atención y las incidencias de morbilidad, causalidad y mortalidad de cierto grupo de individuos atendidos dentro de las instalaciones hospitalarias.

Ruta de atención Medica Continua

De acuerdo al artículo [26], la RAMC, se entiende como la trayectoria que recorre la persona a fin de obtener alguna o varias acciones de la atención médica, de acuerdo a su condición de salud, ya sea preventivas, curativas, de rehabilitación o paliativas a través de los diferentes niveles y sitios de atención del sistema de salud. Este es un instrumento de control y monitoreo de los recursos de salud que se efectúan dentro de las instituciones prestadoras de servicio de salud, esto permite dar certeza en la atención integral y continua a las personas.

5.- Requisito Funcional

Tomando en cuenta las acciones necesarias en la Ruta de Atención Medica Continua (RAMC), dentro de la aplicación móvil se establecen los criterios esenciales que identifica la especificación de los datos y la solicitudes que se abordarán como la toma de decisiones y planes de acción como protocolos a seguir para garantizar una respuesta inmediata en la atención de la salud, dentro de la gestión de las solicitudes se especificara con iconografías que permitan identificar las áreas de atención, servicio médico prestado, especialidad que atendió, laboratorio solicitante o laboratorio que ha prescrito, estudios de gabinete solicitados, personal de recursos humanos solicitante si es el caso, equipo médico o profesional de salud que expide la orden, registro de medicamentos administrados o solicitados, e insumos clínicos solicitados o para su redistribución dentro de las instalaciones clínicas.



ILUSTRACIÓN 2. SERVICIO QUE INCORPORA LAS OPERACIONES DE FUNCIONALIDAD HOSPITALARIA.

6.- Requisito Funcional

Para mejora en la calidad de experiencia de usuario y marcar indicativos dentro de las iconografías que resumen en su totalidad la descripción de las acciones a realizar, dentro de las sección de servicios, especialidad, medicamentos e insumos clínicos, se activarán a alertas de puntos críticos de atención para el área de Dirección General, dichas alertas permitirán la señalización de priorizar el nivel de solicitud evitando problemas futuros por una deficiencia en el control de actividades atendidas y pendientes, lo que evitara que se generen demoras y pérdidas de vidas humanas por falta de atención oportuna.

Logística y Operación

Dentro de la institución hospitalaria se considera la gestión de todas las acciones necesarias que garanticen la continuidad de sus operaciones y la existencia de reservas de insumos clínicos, como de tecnologías que permitan llevar a cabo las actividades de operación de los departamentos de atención médica. Las actividades logísticas conforman un subsistema que será enlace entre las unidades médicas en contacto con la persona usuario, las unidades administrativas y los proveedores, ya que todos los elementos se encuentran separados por el tiempo y la distancia. Por medio de la administración logística y la cadena de suministros se priorizará o suspenderán las actividades a realizar por las áreas de gestión hospitalaria.

7.- Requerimiento Funcional

Es importante especificar dentro del diseño de las alertas de la Aplicación por los departamentos de suministros clínicos, equipo médico como de especialista, se notifique dentro de la aprobación si existe el personal necesario para realizar una intervención quirúrgica, como el análisis de especialistas en cirugía que atiendan los inconvenientes de gravedad surgidas en las intervenciones, así de que se debe de agregar la las notificaciones de que existe el suficiente instrumental adecuado para la práctica de dicha intervención, al validarse la existencia de los elementos necesarios de las solicitud de intervención quirúrgica, el área de dirección general dará

el visto bueno y pasara al área de quirófanos para su programación, al no cumplirse los criterios esenciales de la solicitud esta será invalidada y tendrá que ser solicitada de nuevo.

8.- Requerimiento funcional

Para la validación de los procesos de administración en la cadena de suministros de material quirúrgico como de medicamentos el área de dirección general solicitará al área de distribución los lotes necesarios para el periodo de operación solicitado anticipadamente, esto resguarda la perdida en las actividades y el tiempo de respuesta a emergencias, por lo que su aprobación estará supervisada con antelación ante cualquier emergencia suscitada en la población.

9.- Requisitos Funcionales

Dentro de la aplicación de la Dirección General hospitalaria se marcarán dentro de las solicitudes con prioridad los mínimos existentes y los mínimos máximos como alertas que señalen una anticipada deficiencia por loque dicha alerta permitirá mantener una comunicación constante con los departamentos de Farmacia intra Hospitalaria como la de insumos médicos en base a sus necesidades y así poder reabastecer suministros sin afectar la cadena de distribución dentro de las operaciones del hospital.

Conservación Y Mantenimiento

Las actividades relacionadas con la conservación y mantenimiento de infraestructura física en salud de acuerdo a la artículo [26], se entiende como el conjunto de actividades técnicas y administrativas a desarrollar en inmuebles, sistemas, instalaciones y equipos (médicos y no médicos), con la finalidad de que estos recuperen, mantengan y ofrezcan los productos y servicios para los cuales fueron diseñados originalmente, dentro de un marco de seguridad, continuidad, eficiencia y economía.

10.- Requerimiento Funcional

Dentro de las solicitudes con prioridad como consecuencia de la importancia que adquiere el enfoque de calidad en la atención de la salud, para proporcionar una prestación ininterrumpida de servicios o minimizar los paros no programados, preservar y optimizar la inversión en infraestructura en condiciones óptimas de operación. El área de dirección General Hospitalaria debe de marcar los tiempos programados para una revisión de la funcionalidad de cada sector, con la Aplicación Móvil se pretende acelerar la organización, ya que garantizará que todos los sectores involucrados estén

reportando sus actividades dentro de la bitácora general de actividades en función de ello se puede verificar si existen alteraciones o señalamientos que indique una deficiencia en la prestación de servicios del hospital [26].

Innovaciones en gestión hospitalaria en México.

Gestión y monitoreo de control de personal

En el artículo de la Secretaría de Salud (2020), se enfatiza la importancia del factor humano en la implementación de un hospital, destacando interrogantes clave como el perfil del personal necesario, la estructura organizativa óptima y el diseño de un organigrama efectivo para la coordinación de actividades. Además, se aborda la necesidad de definir procesos de reclutamiento y selección, así como la consideración de la contratación externa de recursos humanos para mejorar la atención. El artículo subraya que un cálculo preciso de la plantilla es esencial para el éxito institucional, alineando la contratación con la ocupación de camas y los servicios ofrecidos (pág. 47 - 52).

11.- Requerimiento Funcional (Recursos Humanos)

La aplicación permitirá a la Dirección General supervisar el control de recursos humanos. Deberá incluir alertas que notifiquen sobrecargas laborales o escasez de personal en áreas críticas, y facilitará que el personal encargado envíe solicitudes a la Dirección General para su análisis y aprobación.

12. Requerimiento Funcional (Monitoreo y Control de Personal)

El sistema deberá contar con una funcionalidad que permita a los departamentos enviar solicitudes a la Dirección General. Esta permitirá que se analicen y aprueben peticiones para asignar personal disponible a las áreas que lo requieran, en función de la demanda de atención médica en tiempo real.

Infraestructura y Sistemas de Información

Integración con Sistemas Hospitalarios:

- En el informe de la [26], se destaca la importancia de la integración de los sistemas de información hospitalarios para facilitar la toma de decisiones en la gestión de recursos, suministros y ocupación de camas. Este proceso es clave para asegurar que la Dirección

General tenga acceso inmediato a la información crítica necesaria para la operación del hospital (pág. 53 - 58).

13.- Requerimiento Funcional (Integración con Sistemas Hospitalarios):

La aplicación deberá integrarse con los sistemas de información existentes del hospital, como el ERP, gestión de pacientes y sistemas de inventarios, para que la Dirección General pueda monitorear la disponibilidad de camas, equipos médicos y suministros en tiempo real a través de los departamentos a cargo (forma indirecta).

Gestión Centralizada de Información:

- El informe de la [26], también señala la necesidad de centralizar la gestión de información en el hospital, con un enfoque en paneles centralizados que permitan a la Dirección General supervisar el estado de los recursos y tomar decisiones basadas en datos precisos (pág. 59 - 64).

14.- Requerimiento Funcional (Gestión Centralizada de Información):

La aplicación proporcionará un panel centralizado para visualizar datos críticos como la ocupación de camas, el uso de recursos y el rendimiento de las áreas clave a través de solicitudes que hagan los departamentos encargados para su aprobación. También incluirá gráficos y reportes interactivos para facilitar la toma de decisiones.

Monitoreo en Tiempo Real de Indicadores Clave:

- Según el análisis presentado en la [26], es esencial el monitoreo constante de los indicadores clave del hospital para evaluar el desempeño y los niveles de atención médica en las diversas áreas del hospital (pág. 65 - 68).

15.- Requerimiento Funcional (Monitoreo en Tiempo Real de Indicadores Clave):

La aplicación deberá permitir el monitoreo en tiempo real de indicadores clave como el uso de equipos médicos, el rendimiento por área y los tiempos de espera de los pacientes a través de solicitudes que creen los departamentos para su análisis y a aprobación dirigida a Dirección General.

Coordinación entre Departamentos:

El estudio de la [26], subraya la importancia de una comunicación fluida entre departamentos críticos, sobre todo en situaciones de emergencia, para garantizar una respuesta oportuna y coordinada (pág. 73 - 78).

16.- Requerimiento Funcional (Coordinación entre Departamentos):

La aplicación facilitará la coordinación entre departamentos mediante notificaciones y alertas instantáneas, así como un sistema de mensajería interna para mejorar la comunicación en situaciones de emergencia.

Toma de Decisiones Basada en Datos:

- El informe destaca que las decisiones informadas y basadas en datos en tiempo real pueden marcar la diferencia en la eficiencia operativa del hospital (pág. 79 - 82).

17.- Requerimiento Funcional (Toma de Decisiones Basada en Datos):

La aplicación permitirá que la Dirección General tome decisiones basadas en datos en tiempo real, redistribuyendo los recursos y el personal según sea necesario.

Monitoreo de la Calidad de los Servicios:

- El control de calidad en los servicios hospitalarios es esencial para garantizar una atención eficiente y oportuna a los pacientes, como lo señala el estudio de la [26] (pág. 83 - 87).

18.- Requerimiento Funcional (Monitoreo de la Calidad de los Servicios):

La aplicación incluirá módulos que permitirán monitorear la calidad de los servicios, con indicadores como tiempos de espera y satisfacción del paciente, para optimizar la atención médica.

Cumplimiento Normativo:

- Cumplir con las normativas de salud es una prioridad para cualquier institución hospitalaria, según el informe de la [26] (pág. 88 - 90).

19.-Requerimiento Funcional (Cumplimiento Normativo): La aplicación deberá generar informes automáticos que verifiquen el cumplimiento de las normativas de salud y estarán disponibles para auditorías por parte de entidades reguladoras.

12. Mantenimiento Preventivo de Equipos Médicos:

- El mantenimiento preventivo de los equipos médicos es clave para asegurar que estos se encuentren en correcto funcionamiento, según se menciona en el informe de la [26] (pág. 96 - 99).

20.- Requerimiento Funcional (Mantenimiento Preventivo de Equipos Médicos): La aplicación monitoreará el estado de los equipos médicos y generará notificaciones para realizar el mantenimiento preventivo y así sea fácil informar a dirección general para que haga las acciones correspondientes, garantizando su disponibilidad cuando se necesiten.

Requerimientos no funcionales

Infraestructura Tecnológica para el Desarrollo de la Aplicación:

En el artículo de la Secretaría de Salud, se subraya la importancia de contar con una infraestructura tecnológica robusta que permita la integración eficiente entre sistemas hospitalarios y aplicaciones externas. Este enfoque facilita la interoperabilidad y asegura la integridad de los datos compartidos:

1. Requerimiento No Funcional (Base de Datos)

La base de datos será desarrollada siguiendo los modelos compatibles con la estructura de FastAPI, garantizando la migración de datos configurados y evitando la pérdida de información. Además, se mantendrá la consistencia en las tablas a través de procesos de validación automatizados.

2. Requerimiento No Funcional (Seguridad de la Información)

Toda la información consultada desde la base de datos será gestionada de forma segura utilizando credenciales cifradas mediante MySQL y Navicat. Esto permitirá garantizar la integridad y confidencialidad de los datos almacenados.

3. Requerimiento No Funcional (Rutas de API)

El sistema contará con un API desarrollado en FastAPI que enlazará los datos almacenados en la base de datos. Las rutas estarán organizadas por módulos específicos, documentadas adecuadamente, y configuradas para cumplir con los estándares de rendimiento y seguridad.

4. Autenticación y Control de Accesos:

El informe de la Secretaría de Salud, menciona que la autenticación de usuarios es un componente esencial en la gestión hospitalaria, ya que asegura que los sistemas sean utilizados únicamente por personal autorizado, evitando brechas de seguridad.

5. Requerimiento No Funcional (Autenticación de Usuarios)

La aplicación implementará autenticación mediante OAuth 2.0 para validar credenciales de acceso. Se utilizará el API de Google Developer para gestionar las credenciales de inicio de sesión y garantizar un acceso seguro al sistema.

6. Requerimiento No Funcional (Gestión de Permisos)

El acceso al API estará limitado a usuarios con privilegios específicos. Las acciones como consultas, modificaciones o eliminaciones de datos serán realizadas únicamente por personal autorizado, según los permisos asignados.

7. Gestión de Estilos y Experiencia de Usuario:

El diseño de interfaces accesibles y uniformes es clave para garantizar una experiencia de usuario eficiente, como lo destaca el análisis de la Secretaría de Salud. Este enfoque asegura que las plataformas tecnológicas sean intuitivas y fáciles de usar para todos los usuarios.

8. Requerimiento No Funcional (Diseño de Estilos)

El diseño visual de la aplicación será gestionado con Bootstrap y Tailwind CSS, siguiendo una paleta de colores institucional y un diseño centrado en la accesibilidad.

9. Requerimiento No Funcional (Buscador de Datos)

El sistema contará con un buscador integrado en el menú principal, el cual permitirá filtrar parámetros específicos y devolver resultados relevantes a las consultas realizadas.

Gestión Centralizada de Datos y Versiones:

En el informe de la Secretaría de Salud, se menciona que una gestión centralizada de datos y un control estricto de versiones son esenciales para evitar errores en la operación de sistemas hospitalarios.

10. Requerimiento No Funcional (Control de Versiones)

El desarrollo del sistema incluirá documentación detallada de los procedimientos implementados, con un control de versiones para facilitar su mantenimiento y evolución a futuro.

11. Requerimiento No Funcional (Variables de Entorno)

Las variables de entorno utilizadas en el desarrollo estarán protegidas y seguirán estándares de seguridad para evitar exposiciones de credenciales relacionadas con bases de datos, rutas API y almacenamiento en la nube.

Diseño del Proyecto

Diagramas de caos de uso

Este informe detalla el flujo de trabajo hospitalario centrado en la Dirección General, que coordina diversos actores y procesos críticos dentro de la institución. El diagrama describe cómo se relacionan estos actores en los distintos casos de uso, destacando el papel del Comité de

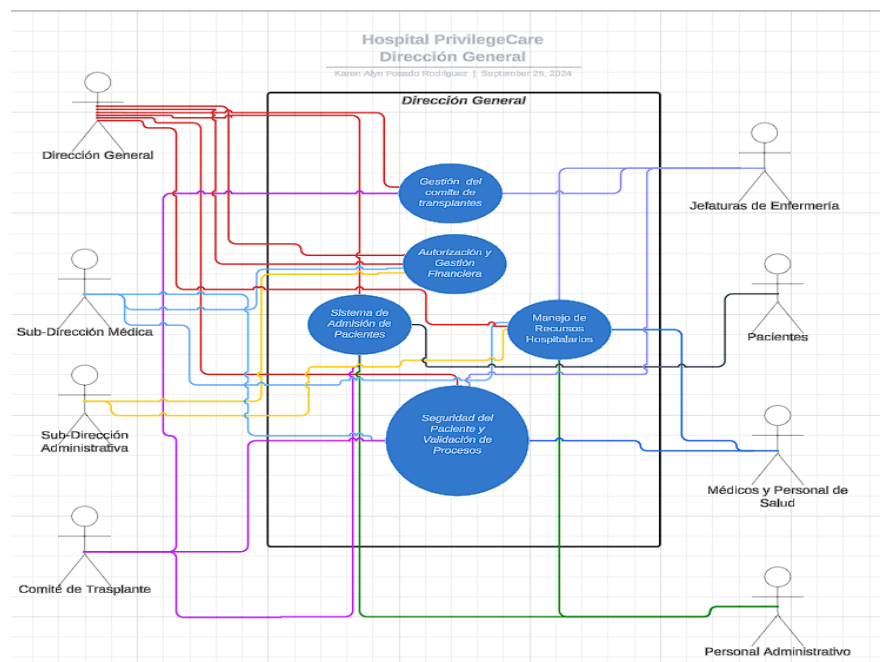


ILUSTRACIÓN 3. DIAGRAMA DE CASO DE USO DE DIRECCIÓN GENERAL.

Trasplantes, la gestión de recursos, la seguridad del paciente, y la administración financiera.

Explicación de los componentes que vemos en el diagrama:

1. Sistema: Es el conjunto de funcionalidades que ofrece el hospital (o la aplicación). Se representa dentro del rectángulo grande que encierra los casos de uso.

2. Actores: Son los usuarios o sistemas que interactúan con el sistema. En este caso, hay dos actores representados por figuras humanas fuera del sistema. Los actores pueden ser, por ejemplo:

- Dirección General: Supervisa el funcionamiento general del hospital y los casos de uso más críticos, tomando decisiones estratégicas.
- Sub-Dirección Médica: Se encarga de la coordinación médica y los servicios de apoyo.
- Sub-Dirección Administrativa: Maneja los recursos financieros y materiales necesarios para el funcionamiento hospitalario.
- Comité de Trasplantes: Encargado de la evaluación y programación de los trasplantes de órganos.
- Jefaturas de Enfermería: Supervisan y coordinan los servicios de enfermería relacionados con los pacientes en diferentes fases del tratamiento.
- Pacientes: Los receptores de los servicios médicos, tanto en procedimientos quirúrgicos como en atención postoperatoria.
- Médicos y Personal de Salud: Incluye a los médicos especialistas y otros profesionales de la salud que participan en los procedimientos médicos.
- Personal Administrativo: Encargado de la gestión operativa del hospital, incluyendo la gestión de pacientes, recursos y procesos.

3. Casos de Uso Principales:

Gestión del Comité de Trasplantes:

- Descripción: El Comité de Trasplantes coordina todas las actividades relacionadas con los trasplantes de órganos. Este proceso incluye la evaluación de donantes y receptores, programación de cirugías, y la coordinación con las diferentes áreas médicas y administrativas.
- Relación con actores:
 - La Dirección General supervisa y aprueba las decisiones críticas del Comité.
 - El Personal Médico ofrece evaluaciones clínicas necesarias para determinar la viabilidad de los trasplantes.
 - El Personal Administrativo apoya en la programación y asignación de recursos.

Autorización y Gestión Financiera:

- Descripción: La Dirección General, junto con la Sub-Dirección Administrativa, se encarga de autorizar los recursos financieros para los trasplantes. Esto incluye la validación de seguros y la gestión de costos hospitalarios.
- Relación con actores:
 - Dirección General toma decisiones finales sobre la asignación de fondos.
 - El Personal Administrativo gestiona los costos y asegura la viabilidad financiera del proceso.
 - Los Pacientes interactúan con este proceso para asegurar que su trasplante esté cubierto financieramente.

Sistema de Admisión de Pacientes:

- Descripción: Este sistema gestiona el ingreso de pacientes al hospital, ya sea para procedimientos de trasplante o para otros tratamientos. Incluye la recopilación de datos médicos y administrativos.
- Relación con actores:
 - La Sub-Dirección Médica coordina el ingreso y los informes médicos de los pacientes.
 - El Personal Administrativo gestiona la información relacionada con el historial médico y los datos de los pacientes.
 - Los Médicos y Enfermeras colaboran para dar seguimiento a los pacientes ingresados.

Manejo de Recursos Hospitalarios:

- Descripción: La Sub-Dirección Administrativa y el Comité de Trasplantes coordinan la disponibilidad de quirófanos, medicamentos y otros recursos críticos para asegurar que los procedimientos quirúrgicos se realicen a tiempo.
- Relación con actores:
 - La Sub-Dirección Administrativa gestiona los recursos materiales (quirófanos, insumos médicos).
 - El Comité de Trasplantes se asegura de que los recursos estén disponibles para las cirugías.
 - Enfermería y Médicos utilizan estos recursos durante los procedimientos quirúrgicos.

Seguridad del Paciente y Validación de Procesos:

1. Descripción: Este proceso asegura que todas las medidas de seguridad se cumplan durante las operaciones, así como el seguimiento de los pacientes postoperatoriamente. Involucra auditorías internas y controles de calidad.
2. Relación con actores:
 - La Dirección General y la Sub-Dirección Médica aseguran el cumplimiento de los estándares de seguridad.
 - El Personal Médico es responsable de implementar las medidas de seguridad durante las cirugías.
 - Enfermería asegura que los pacientes estén estables y bien cuidados en el proceso postoperatorio.

Flujo de Trabajo:

1. Planificación Inicial:
 - La Dirección General establece las políticas y directrices para los trasplantes, coordinando con las Sub-Direcciones Médica y Administrativa.
2. Evaluación y Preparación:
 - El Comité de Trasplantes evalúa a los donantes y receptores en colaboración con los Médicos Especialistas.
 - Se asegura que los recursos estén disponibles mediante la Sub-Dirección Administrativa.
3. Autorización y Ejecución:
 - La Dirección General autoriza los recursos financieros y operativos necesarios.
 - El Personal Médico realiza el procedimiento quirúrgico, apoyado por Enfermería y otros Servicios de Apoyo.
4. Seguimiento y Seguridad:
 - Tras la cirugía, el Personal de Enfermería asegura el seguimiento postoperatorio de los pacientes.
 - Se realizan validaciones de los procesos por parte del Departamento de Calidad para garantizar la seguridad del paciente.

Conclusión:

El diagrama de caso de uso refleja un hospital bien organizado, donde la Dirección General juega un papel clave en la supervisión, mientras que los actores médicos y administrativos colaboran estrechamente en la ejecución de trasplantes y la seguridad del paciente. La interrelación entre los actores asegura que se mantenga un flujo eficiente de trabajo, desde la evaluación hasta la ejecución de los trasplantes.

Contexto del negocio

La gestión de un hospital abarca múltiples sectores de la población, y el enfoque en servicios de alta calidad suele estar reservado para los hospitales privados. En este sector, a través de cuotas trimestrales, seguros o planes anuales, los hospitales están obligados a ofrecer servicios con los más altos estándares de calidad. Sin embargo, como se mencionó previamente, el mercado objetivo de estos servicios está limitado por el poder adquisitivo de sus clientes. En consecuencia, la demanda de servicios en hospitales privados no es tan elevada como en el sector público, donde la atención se brinda a un mayor volumen de pacientes.

En el sector público, el nivel de atención está determinado por la capacidad del hospital para atender a un gran número de pacientes, ofreciendo una gama de servicios amplia. Esto incluye la atención a derechohabientes inscritos en el Seguro Social Nacional (SSN), así como la atención de consultas internas y externas. Además, los hospitales públicos tienen el compromiso de atender a aquellos que son canalizados desde otras unidades médicas conectadas a la misma red del seguro social. No obstante, la atención oportuna puede complicarse por la existencia de documentación excesiva y procesos burocráticos que retrasan la atención de pacientes con enfermedades crónicas y degenerativas. Además, la falta de capacitación adecuada del personal puede llevar a la negligencia en el servicio y el manejo de la información de los pacientes.

Los hospitales públicos también suelen estar involucrados en campañas gubernamentales y comunitarias, promoviendo visitas a la comunidad a través de iniciativas como campañas de vacunación, donación de sangre, prevención de enfermedades crónicas, y concienciación sobre enfermedades de transmisión sexual. El sector privado participa en estas campañas al donar material médico y aliarse con las autoridades para mejorar la calidad de vida de las personas más vulnerables. Sin embargo, atender a toda la población vulnerable puede ser un desafío, debido a la gran demanda de servicios médicos de calidad.

En cuanto a los hospitales públicos, estos son administrados por el gobierno, que se encarga de suministrar el material clínico y quirúrgico necesario para la operación y atención de pacientes. No obstante, a menudo se generan desequilibrios en la gestión de los recursos debido a una falta de control adecuado, lo que afecta la eficiencia del servicio. En algunos casos, los médicos, en un esfuerzo por brindar atención de calidad, recurren a comprar insumos por su cuenta. Este problema se deriva de la mala administración de los recursos y la falta de un sistema eficiente que gestione las solicitudes y los suministros hospitalarios.

El sector privado, por otro lado, es financiado por aseguradoras y empresas bancarias, que promueven la contratación de servicios médicos mediante seguros y planes de cobertura. Estas entidades mantienen una relación de "ganar-ganar" con sus clientes, utilizando medios masivos de comunicación para promocionar sus servicios. En algunos casos, realizan actos de responsabilidad social, creando fundaciones para captar donativos y apoyar a personas vulnerables. Estas fundaciones a menudo se especializan en áreas como el tratamiento de enfermedades crónicas (por ejemplo, el cáncer) o de enfermedades de transmisión sexual (como el VIH), buscando mejorar la relación entre el hospital y la comunidad.

En el sector público, las reglas son más estrictas. La operación de los hospitales está sujeta a planeación estratégica basada en estudios epidemiológicos, que identifican las áreas de atención prioritarias. Los pacientes con enfermedades crónicas, adicciones u otros problemas de salud graves son canalizados a áreas especializadas para recibir tratamiento. Sin embargo, debido a la alta demanda, los tiempos de espera pueden ser prolongados. Las autoridades estatales y federales revisan constantemente el funcionamiento del sistema y la gestión de sus procesos.

El cumplimiento de normas estrictas, como las NOM 035 SSA3 2012 y NOM 027 SSA3 2013, es fundamental para la operación de los hospitales públicos. Estas normas establecen los procedimientos para la recolección, procesamiento y divulgación de información en salud, así como los criterios para la atención en servicios de urgencias.

El financiamiento en el sector público está vinculado al presupuesto federal anual, lo que significa que las actualizaciones en infraestructura y servicios dependen de asignaciones presupuestarias anuales o bimestrales. Sin embargo, estas actualizaciones suelen ser insuficientes para satisfacer la demanda creciente. Por ello, es necesario optimizar los procesos de gestión, tales como el manejo de inventarios, documentación y expedientes médicos, gestión de personal y administración de

medicamentos. Un sistema digital eficiente que conecte en tiempo real a todas las áreas del hospital podría mejorar significativamente la atención al paciente, asegurando que la información esté debidamente protegida y disponible solo para el personal autorizado.

Finalmente, es crucial implementar medidas robustas de seguridad informática, protegiendo los datos confidenciales de los pacientes mediante encriptación y asegurando que solo el personal autorizado tenga acceso a la información. La implementación de este nivel de seguridad dependerá de las políticas y regulaciones vigentes, pero es esencial para garantizar la privacidad y protección de los datos sensibles.

Organización de la Empresa

Cada una de las unidades clínicas que dan servicio están sujetas a una estricta organización que las sujeta a brindar un servicio eficiente, es por ello que el modelo que se seguirá está en base al sistema de gestión de servicio público de tercer nivel de atención médica, la cual aborda una amplia gama de servicios de atención.

Todas las instituciones públicas como privadas cuentan con una dirección general que administra y supervisa el ejercicio dentro del trabajo diario de sus centros de atención, es por ello que la organización está establecida para otorgar y organizar los servicios integrales que ejerce cada departamento a su cargo. Siendo la cabeza, la Dirección General se encargará de supervisar y atender las necesidades de los departamentos que integra la unidad médica, como primera instancia se encuentra la junta de gobierno, encargada de mantener y ejecutar la idealización de una atención oportuna para los derechohabiente poniendo en primer lugar buscar las mejores soluciones que puedan salvaguardar la vida preservando la salud como prevención oportuna, por lo que considera de ejecutar y organizar los planes de acción para la mejora de la comunidad, aquí se establecen las políticas y las normas, además de integrar la misión y la visión que la unidad medica proyecta para el futuro.

Dentro de la estructura Organizacional de la dirección general se toman todas las decisiones que ayuden a mejorar la atención médica, a qui se integra la dirección de cada uno de los departamentos, creando una comitiva estructurada para la toma de decisiones de alto impacto para toda la unidad médica, a nivel institucional su estructura es la que organiza en cierta medida como se solventarán las demandas correspondientes a cada departamento.

Ya que las decisiones de mayor peso son una gran responsabilidad en un hospital, se integra un comité de trasplantes que junto a la dirección de alta especialidad como comité legal realizan y priorizan la candidatura a personas que necesitan de estos tratamientos delicados, por lo que se maneja una responsabilidad enorme dentro de este comité.

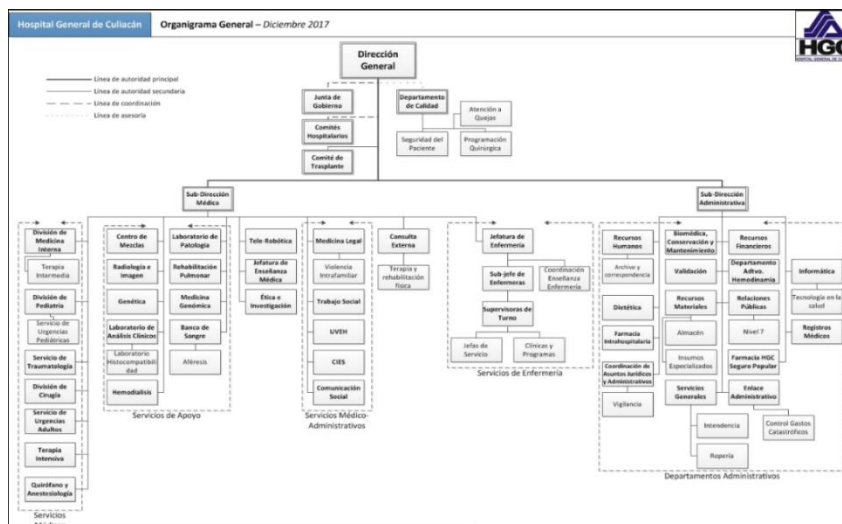


ILUSTRACIÓN 4. ORGANIGRAMA DEL HOSPITAL GENERAL

Áreas Hospitalarias Involucradas



ILUSTRACIÓN 5. ESTRUCTURA DE DIRECCIÓN GENERAL.

Dentro de lo que se destaca de la organización en la estructura de la dirección general se tiene el departamento de seguimiento de calidad que vigila cada uno de los procesos llevados a cabo como la atención a quejas y demandas, salva guardado de datos confidenciales de los pacientes como de archivos clínicos, y la programación y aprobación de intervenciones quirúrgicas.

Dado a la descripción anterior esta es la cabecera del hospital, de aquí se divide en dos ramales principales que son la Sub dirección Médica y Sub Dirección Administrativa. La primera engloba todos los servicios que se ejercen para la práctica médica profesional, mientras la segunda se encarga de administrar los procesos que se llevan a cabo dentro del hospital tanto como internos como externos.

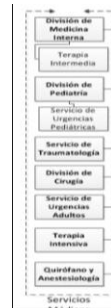


ILUSTRACIÓN 6. SERVICIOS MÉDICOS DEL HOSPITAL GENERAL DE CULIACÁN.

Siendo más preciso, las áreas medicas están subdivididas en servicios integrales, la primera línea se enfoca en los servicios médicos de primer nivel de atención a los derecho habientes, entre ellos se encuentra la división de medicina interna, conectada con la terapia intermediaria como subnivel de atención de este departamento, como segundo ramal se cuenta a atención pediátrica que se enfoca a la atención de niños y adolescente con un ramal de servicio de urgencias pediátricas de especialidad, como tercer ramal se encuentra la atención de traumatología y servicios de radiología, aquí se incluye también dentro de los ramales la división de la organización de cirugías rutinarias, junto a la atención de urgencias geriátricas, y tratamiento a pacientes de terapia intensiva, como salas de operaciones y departamentos de intervenciones quirúrgicas.



ILUSTRACIÓN 7. SERVICIOS DE APOYO EN EL HOSPITAL GENERAL DE CULIACÁN.

El siguiente modulo administra los servicios de apoyo que son departamentos que se encargan del análisis exhaustivo de estudios y pruebas esenciales para un mejor pronostico clínico en el diagnostico oportuno como lo son departamentos de análisis patológico, estudios genómicos, banco de sangre, laboratorio de estudio de histocompatibilidad, estudios hemáticos y hemodiálisis, laboratorio de estudios generales de análisis clínicos, como la toma de estudios de radiología e imagenología clínica.



ILUSTRACIÓN 8. SERVICIOS DE RELACIÓN EN EL HOSPITAL GENERAL DE CULIACÁN.

Dentro de la sub dirección Médica existen departamentos que conectan directamente a esta sección, siendo el área de Tele-robotica para la investigación e implementación de procedimientos clínicos como la rama de investigadores del área médica que engloban protocolos de investigación y participación en la mejora de la rama de salud, como también la enseñanza clínica a estudiantes del área médica para realizar prácticas profesionales, la subdirección es la encargada directamente de canalizar el flujo de la atención de los pacientes ambulatorios para la consulta médica como canalizar los hacia otra áreas de atención medica que necesiten de una consulta externa como terapias adicionales que mejoren su calidad de vida.



ILUSTRACIÓN 9. SERVICIOS MÉDICOS DEL HOSPITAL GENERAL DE CULIACÁN.

Para un mejor orden de la legalidad del servicio y casos de estudio clínico se encuentra el área de Servicios Médicos Administrativos que canaliza los casos de acuerdo a la asignación legal correspondiente, tratando de buscar las mejores soluciones que ayuden a mejorar la calidad de vida de quien está involucrado en actos de legalidad de acuerdo al rubro de gravedad de su situación física. Integrando en primer plano el área de medicina legal, conectada directamente a la atención de víctimas de violencia intrafamiliar, esta dependencia canaliza los casos con el área de trabajo social para la ejecución de las acciones legales, también se encuentra la unidad de vigilancia Epidemiológica hospitalaria (UVEH), que implementará los planes de acción en caso de requerir una intervención mayor y mejora de casos especiales o de índole delicada que perjudique la salud poblacional. Junto a las instituciones de investigación clínica el Centro de Investigación Epidemiológica de esta unidad, se encargarán de seguir los protocolos de seguridad en caso de contingencia sanitaria. También dentro de este departamento se encuentra el área de comunicación social, que se encargará de publicar campañas y servicios a implementar dentro de la unidad médica.



ILUSTRACIÓN 10. SERVICIOS DE ENFERMERÍA DEL HOSPITAL GENERAL DE CULIACÁN.

La Subdirección Medica también respalda el área de servicios de Enfermería coordinando su personal interno dentro de las áreas de atención de especialización clínica organizando su plan de trabajo, la Jefatura de Enfermería organizará su subdepartamentos en cabezas de mando al igual a la capacitación de personal de enfermería en formación, se encargara de turnar a su personal mediante roles específicos dando respuesta oportuna a las demandas de los pacientes y como apoyo altamente calificado al personal médico.

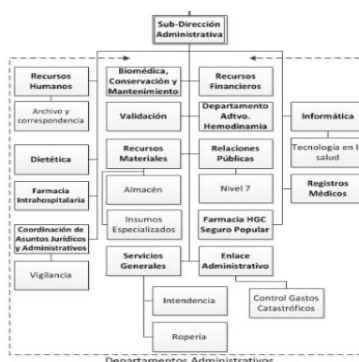


ILUSTRACIÓN 11. DEPARTAMENTO ADMINISTRATIVOS DEL HOSPITAL GENERAL DE CULIACÁN

La subdirección de Administración es un ramal con una organización que administra los recursos y actividades que giran en torno al hospital, dividida en departamentos documenta los procesos y protocolos a seguir para su ejercicio, involucrando principalmente el área de recursos humanos encargada de la administración del personal que labora en todas las áreas del hospital llevando y registrando en el área de almacenamiento de información útil para los datos históricos de sus empleados. Se encuentra el área de dietética que lleva los registros de nutrición de pacientes y los tratamientos administrados para la mejora de su condición. Se Encuentra el área de Farmacia que administra y suministra los medicamentos para las áreas requeridas y a los pacientes que lo necesitan. Se encuentra el área de asuntos legales y jurídicos que supervisan las actividades realizadas de acuerdo a las normativas y legalidades de los procedimientos médicos ejercidos dentro de la instalación hospitalaria junto al departamento de vigilancia y seguridad supervisan el orden y correcta ejecución de las actividades clínicas.

La colaboración de departamentos es esencial para una operación optimizada, es por ello que la organización dependerá la aprobación de ciertos sectores de acuerdo a la prioridad de las demandas y las necesidades a atender para una mejora del servicio, tal es el caso de la supervisión de los recursos financieros que equipará a los sectores con mayor prioridad, junto al área de recursos materiales y la de validación operarán en conjunto, y se dará mantenimiento a las instalaciones de acuerdo a los periodos establecidos por las normativas por el estado, todos los

recursos disponibles y surtidos deben de ser inventariados y almacenados en perfecto estado para su uso en el hospital, así se asegura que se brinda servicio de calidad con un equipamiento de tercer nivel de atención médica.

Las relaciones públicas con los sectores involucrados permitirán tanto la activación de la materialización del hospital como la entrada de nuevas tecnologías de uso médico, y relaciones públicas con personal de alta especialización con otros hospitales del país, lo que abrirá al sector público y privado una seguridad de que los procedimientos médicos cumplen con los estándares de calidad y servicio.

Para la operación del hospital es imprescindible el departamento de intendencia y ropería, siendo aplicando las metodologías de asepsia calificada para las actividades clínicas y quirúrgicas que son esenciales en la operación del hospital.

El área administrativa cuenta con el área de informática y aplicación tecnológica en la salud operando en manera conjunta para brindar un servicio médico de calidad para los usuarios del hospital. Dentro de la administración del hospital es fundamental llevar un registro de todos sus derechos habientes e intervenciones y procedimientos que se han llevado, por lo que cada paciente tendrá un registro de las actividades clínicas y procedimientos que se han administrado para mejorar la calidad de vida.

Proceso De Negocio

Anterior mente se ha comentado de manera general como está organizada la operación del servicio médico, al entender que cada sector lleva un mando de responsabilidad y actividades específicas, es por ello que se abordará solo en las actividades operativas que se ejecutan dentro de la dirección general del hospital, siendo la cabeza central que administra en conjunto todos los departamentos que involucra las actividades clínicas ejercidas, se especializa en generar datos que aprueben y avalen todos los procedimientos que se ejecutan en las instalaciones.

Dentro de los parámetros operacionales del hospital se mencionarán algunos procedimientos que permiten hacer una correcta dirección.

Maquetación y Bocetos

Vista de Carga

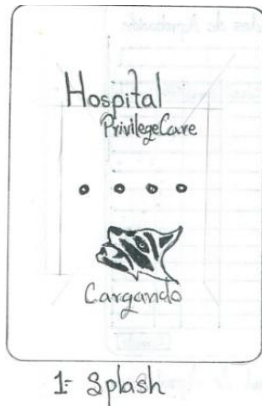


ILUSTRACIÓN 14. A) VISTA DE CARGA (SKETCH).

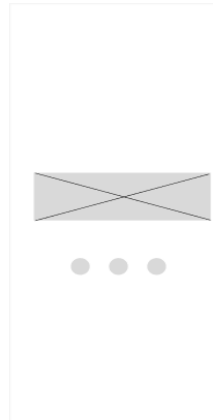


ILUSTRACIÓN 13. B) VISTA DE CARGA (WIREFRAME)



ILUSTRACIÓN 12. C) VISTA DE CARGA (MOCKUP).

- A. La propuesta en el diseño de la aplicación ejemplifica una carga previa momentánea, esta se activa inmediatamente al dar apertura a la aplicación permitiendo cargar todos los elementos necesarios para la funcionalidad de la aplicación.
- B. El diseño inicial del wireframe para la pantalla de carga se centra en mantener la simplicidad y resaltar el logotipo del hospital y el de los creadores en una posición central. Incluye un texto breve de bienvenida justo debajo del logotipo y un indicador de carga en la parte inferior para informar al usuario sobre el progreso. Aunque el wireframe no utiliza colores finales, se emplea un fondo neutro (gris claro) para marcar las áreas de diseño clave. Los elementos importantes, como el logotipo y el indicador de carga, están delineados en líneas negras.
- C. El diseño final para la pantalla de carga incluye un fondo blanco con el logotipo del hospital en tonos azul y verde pastel, colores que representan la confianza y la salud. El indicador de carga se transforma en una animación sutil en azul oscuro, mostrando profesionalismo y dinamismo.

Colores del Mockup:

- Fondo: Blanco puro para transmitir limpieza y profesionalismo.
- Logotipo: Azul celeste y verde pastel.
- Indicador de carga: Azul oscuro, con animación en tonos degradados

Vista de Inicio de Sesión

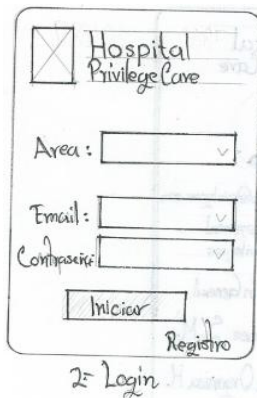


ILUSTRACIÓN 15. A) VISTA DE INICIO DE SESIÓN (SKETCH).

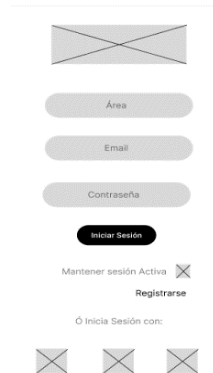


ILUSTRACIÓN 16. B) VISTA DE INICIO DE SESIÓN (WIREFRAME).

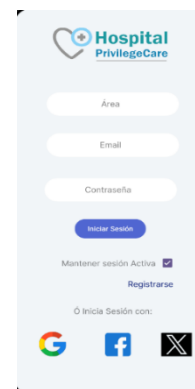


ILUSTRACIÓN 17. C) VISTA DE INICIO DE SESIÓN (MOCKUP).

- A. La vista de inicio secundaria permite realizar el logue correspondiente del usuario, sin embargo, se abordará en el diseño la discusión si se le sugiere al usuario dejar su inicio de sesión activa o que introduzca datos de validación en la apertura de las funciones del sistema.
- B. El wireframe para la pantalla de inicio de sesión está diseñado para ser funcional y directo. Contiene campos para usuario y contraseña, un botón para iniciar sesión y enlaces adicionales como "¿Olvidaste tu contraseña?" o "Registrarse". La disposición de los elementos prioriza la facilidad de uso y la accesibilidad. Se emplean bloques grises para representar campos de entrada y botones, mientras que las líneas negras delimitan las áreas de interacción.
- C. El mockup muestra un diseño minimalista con campos de entrada definidos y un botón llamativo para iniciar sesión. Se añaden iconos discretos junto a cada campo, indicando su función, tecnologías con las cuales podrás logearte, y se usa un estilo visual que refuerza la confianza del usuario.

Colores del Mockup:

- Fondo: Degradado azul cielo a blanco.
- Campos de texto: Fondo blanco con bordes grises claros.
- Botones: Azul oscuro con texto blanco.

Vista de Registro de Personas

ILUSTRACIÓN 19. B) VISTA DE REGISTRO DE PERSONAS (SKETCH).

ILUSTRACIÓN 20. B) VISTA DE REGISTRO DE PERSONAS (WIREFRAME).

ILUSTRACIÓN 18. C) VISTA DE REGISTRO DE PERSONA (MOCKUP)

- A. Se necesita un registro previo existente para dar apertura a los permisos por lo que el usuario podrá registrarse a la plataforma con el llenado de sus datos personales, los cuales se resguardarán por el sistema.
- B. El wireframe para el registro muestra un formulario básico dividido en secciones con indicadores de progreso. Se añaden iconos representativos junto a los campos importantes para mejorar la comprensión del usuario. Se utiliza un fondo blanco con divisores grises y líneas negras para estructurar los campos. Los botones están delineados en negro para destacarlos visualmente.
- C. El diseño del mockup presenta un formulario limpio y estructurado. Se incluyen indicadores visuales para procesos, como una barra en color azul. Los iconos tienen un diseño moderno que facilita la interacción visual del usuario.

Colores del Mockup:

- Fondo: Blanco.
- Bordes: Gris claro para los campos de texto.
- Botones: azul con texto blanco.
-

Vista Principal Conforme al Rol del Usuario

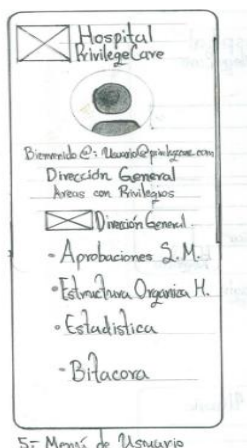


ILUSTRACIÓN 23. A) VISTA PRINCIPAL
CONFORME AL ROL DEL USUARIO
(SKETCH).



ILUSTRACIÓN 22. B) VISTA PRINCIPAL
CONFORME AL ROL DEL USUARIO
(WIREFRAME).



ILUSTRACIÓN 21. C) LISTA PRINCIPAL
CONFORME AL ROL DEL USUARIO
(MOCKUP)

- A. Al existir un registro válido el sistema apertura la información disponible por parte del usuario asignado, en esta vista aparecerá la imagen del usuario con privilegios como las áreas correspondientes para su interacción con los servicios disponibles.
- B. El diseño inicial organiza los accesos principales del usuario en función de su rol. Incluye un menú superior con nombre y rol del usuario, accesos directos en forma de tarjetas o botones principales, y un menú inferior para cerrar sesión.

Colores del Wireframe:

- El wireframe emplea tonos grises neutros para los botones y divisores, con líneas negras para estructurar las áreas clave.

- C. El diseño final utiliza un esquema limpio y jerarquizado, con un fondo en degradado azul cielo a blanco. Los accesos principales están resaltados en tarjetas con bordes suaves y colores diferenciados para cada sección.

Colores del Mockup:

- Fondo: Degradado azul cielo a blanco.
- Botones: Azul oscuro con texto blanco.
- Tarjetas: Tonos pastel según la función (azul y verde).

Vista de Solicitudes de Aprobación



ILUSTRACIÓN 26. A) VISTA DE SOLICITUDES DE APROBACIÓN (SKETCH).

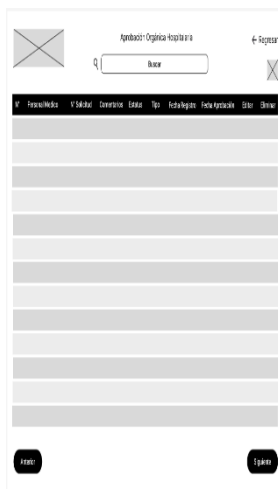


ILUSTRACIÓN 25. B) VISTA DE SOLICITUDES DE APROBACIÓN (WIREFRAME).

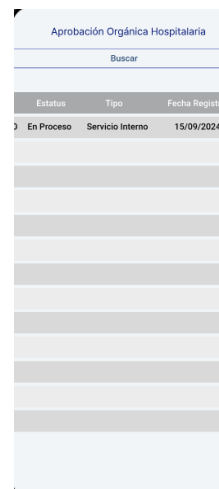


ILUSTRACIÓN 24. C) VISTA DE SOLICITUDES DE APROBACIÓN (MOCKUP).

- A. La vista de solicitudes registrará toda la información necesaria que se requiere para que se asignen los permisos necesarios para la resolución de cada solicitud por los departamentos aledaños a la institución.
- B. Se muestran las solicitudes pendientes en formato de lista, cada una acompañada de un ícono de estado pudiendo ahí mismo aprobar o rechazar. Los detalles de cada solicitud se despliegan al seleccionarla.

Colores del Wireframe:

- Tonos neutros para las solicitudes y divisores grises para separar los elementos de la lista.

- C. Incluye íconos en colores claros para cada estado de la solicitud. La lista desplegable tiene la acción (aprobar/rechazar) están destacados con iconos que se pueden distinguir respectivamente.

Colores del Mockup:

- Fondo: Blanco.
- Texto: Negro.
- Botones: Verde para aprobar y rojo para rechazar.

Vista de Monitorización de Actividades



ILUSTRACIÓN 28. A) VISTA DE MONITORIZACIÓN DE ACTIVIDADES (SKETCH).



ILUSTRACIÓN 29. B) VISTA DE MONITORIZACIÓN DE ACTIVIDADES (WIREFRAME).



ILUSTRACIÓN 27. C) VISTA DE SOLICITUDES DE ACTIVIDADES (MOCKUP).

- A. La estructura orgánica organiza la distribución de los departamentos hospitalarios y monitoriza permanentemente sus actividades por lo que la dirección general podrá hacer los cambios correspondientes de acuerdo a los criterios que su gestión requiera.
- B. Presenta una tabla con información sobre las actividades en curso, incluyendo columnas como fecha, responsable y estado. Se incluye un filtro de búsqueda en la parte superior para buscar actividades específicas.

Colores del Wireframe:

- Fondo blanco con divisores grises claros para las filas de la tabla.

- C. La tabla es organizada y visualmente clara, con filas alternadas en gris claro y blanco. Los filtros están en la parte superior en cajas resaltadas.

Colores del Mockup:

- Fondo: Blanco con filas alternadas en gris claro.
- Texto: Negro con iconos en azul para detalles.

Vista de Información Estadística Representado en Graficas

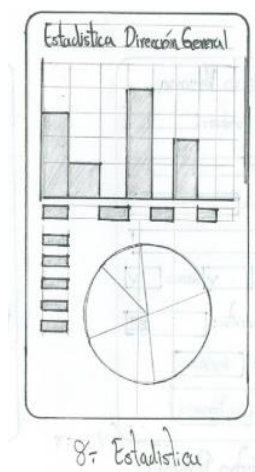


ILUSTRACIÓN 32. A) VISTA DE INFORMACIÓN
ESTADÍSTICA REPRESENTADA EN GRÁFICAS
(SKETCH).

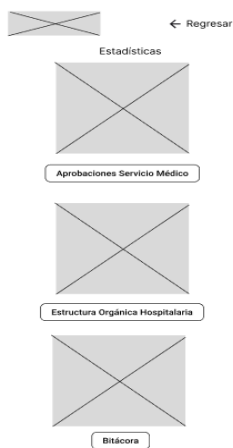


ILUSTRACIÓN 31. B) VISTA DE
INFORMACIÓN ESTADÍSTICA REPRESENTADA
EN GRÁFICAS (WIREFRAME).



ILUSTRACIÓN 30. C) VISTA DE
INFORMACIÓN ESTADÍSTICA
REPRESENTADA EN GRÁFICAS (MOCKUP).

A. La información recabada del área de dirección general hospitalaria se plasmará con datos históricos en gráficos que permitan evaluar los movimientos realizados en periodos asignados para la modificación y toma de decisiones para la mejora de los departamentos, estos datos permitirán dar las observaciones necesarias para la mejora de servicios hospitalarios.

B. Muestra gráficos básicos en barras, líneas y pastel. Incluye filtros para seleccionar el período de análisis y un resumen numérico de los datos principales.

Colores del Wireframe:

- Se emplean líneas negras para dividir las secciones y cuadros grises para representar los gráficos.

C. Los gráficos son dinámicos, con colores vibrantes que representan diferentes áreas o actividades. Se incluye un título superior y una leyenda en el centro.

Colores del Mockup:

- Gráficos: Azul, verde y naranja.
- Fondo: Blanco.

Vista de Registros de Ejecución

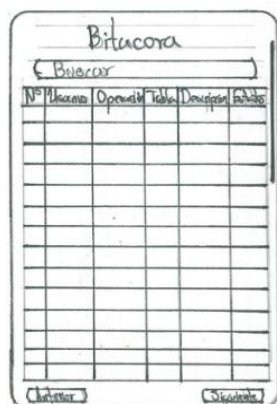


ILUSTRACIÓN 35. A) VISTA DE REGISTROS DE EJECUCIÓN (SKETCH).



ILUSTRACIÓN 34. B) VISTA DE REGISTROS DE EJECUCIÓN (WIREFRAME).



ILUSTRACIÓN 33. C) VISTA DE REGISTROS DE EJECUCIÓN (MOCKUP).

A. Cada uno de los registros realizados en la ejecución del sistema se anotará en la bitácora de actividades como control en la asignación de tareas y permisos correspondidos a las áreas de solicitud, evitando conflictos administrativos futuros.

B. Organiza los registros en formato de tabla, con filtros para buscar por fecha, estado o departamento. Cada registro tiene una opción para consultar detalles.

Colores del Wireframe:

- Neutros en toda la estructura, con líneas claras para dividir las secciones.

C. Diseño del Mockup:

El diseño incluye una tabla clara con bordes definidos y filtros accesibles. Los botones de detalles están en azul para destacarse.

Colores del Mockup:

- Fondo: Azul cielo con divisores grises.
- Texto: Negro.

Vista de Términos y Condiciones



ILUSTRACIÓN 37. A) VISTA DE TÉRMINOS Y CONDICIONES (SKETCH).

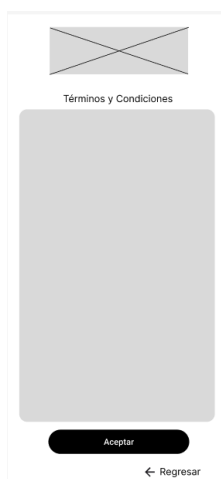


ILUSTRACIÓN 38. B) VISTA DE TÉRMINOS Y CONDICIONES (WIREFRAME).

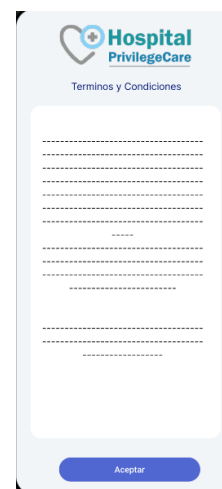


ILUSTRACIÓN 36. C) VISTA DE TÉRMINOS Y CONDICIONES (MOCKUP).

- A. Se incluye dentro del desarrollo de la aplicación las normativas legales para poder ejecutar el sistema reasignando responsabilidad y compromiso en la privacidad de la información contenida y la resguarda de datos personales en base a la ley vigente.
- B. Se presenta un área de texto con scroll para los términos y condiciones, con un botón "Aceptar" fijo en la parte inferior.

Colores del Wireframe:

- Fondo blanco y texto gris oscuro para mayor claridad.

- C. El texto está presentado en una fuente clara y legible, con scroll habilitado. El botón "Aceptar" está en azul para resaltar la acción principal.

Colores del Mockup:

- Fondo: Blanco.
- Texto: Gris oscuro.
- Botón: Azul con texto blanco.

Vista de acerca de nosotros

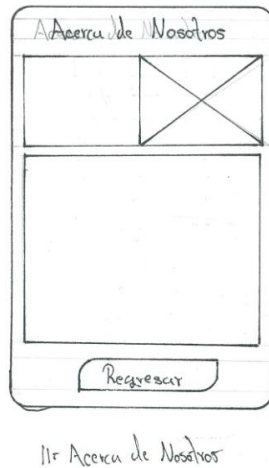


ILUSTRACIÓN 39. A) VISTA DE
ACERCA DE NOSOTROS
(SKETCH).

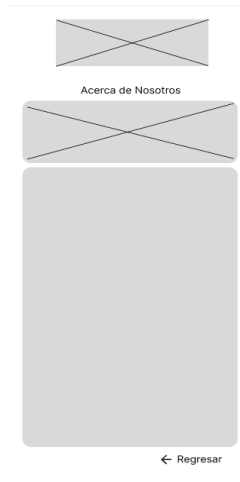


ILUSTRACIÓN 40. B) VISTA DE
ACERCA DE NOSOTROS
(WIREFRAME).

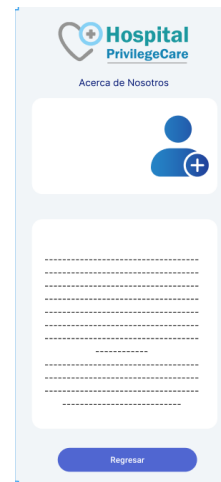


ILUSTRACIÓN 41. C) VISTA DE
ACERCA DE NOSOTROS
(MOCKUP)

A. Se anexa una vista dedicada a los desarrolladores del proyecto donde se especifica de manera breve la versión y el cuerpo de desarrolladores, como los derechos de autor e involucrados en el proyecto.

B. Incluye una descripción general del hospital, su misión y visión, con imágenes representativas en la parte superior y un botón de contacto en la parte inferior.

Colores del Wireframe:

- Fondo blanco con áreas grises para las imágenes y divisores.

C. El diseño final incluye imágenes de alta calidad en la parte superior, texto centrado con una fuente elegante y un botón resaltado.

Colores del Mockup:

- Fondo: Blanco.
- Texto: Blanco.
- Botón: Azul oscuro con texto blanco.

Desarrollo

Dentro de un hospital la organización puede ser compleja dependiendo de la estructura y distribución diseñada para la atención de primer nivel, segundo nivel, y tercer nivel. Su estructura dependerá de la atención especializada y servicios que presta a sus derecho habientes para mejorar la calidad de vida. Es por ello que la organización dependerá directamente de una planeación previa que constituya las funciones y obligaciones que cada departamento atenderá, de acuerdo a las normativas y leyes establecidas, estas estrategias de planeación permitirán mitigar y predecir cambios irregulares y crisis sanitarias que no perjudiquen la atención de los pacientes más vulnerables.

La dirección general respalda todas las acciones planteadas para una mejora continua del sistema sanitario y se encarga también de facilitar todas las herramientas necesarias que el personal de atención medica necesita para la atención oportuna de sus derechohabientes.

Para una mejora continua del sistema hospitalario, este se involucra en inspecciones continuas generando estadísticas que ayuden a predecir y mejorar la atención a sus pacientes, gracias a su trabajo se generan informes anuales en donde se justifican los procedimientos y medidas necesarias para mitigar los problemas que afectan a las áreas de atención primaria, secundaria y terciaria.

Es importante mencionar que la dirección general lleva la batuta en la toma de decisiones importantes, es por ello que se organizan comités de alto nivel que canalizan, organizan y dictaminan las situaciones más relevantes en los procedimientos médicos.

Dentro de las funciones en las que se responsabiliza es el respaldo de todos sus médicos y personal de atención, sin embargo, las organizaciones hospitalarias dependen directamente de sus áreas de recursos humanos que aligeran la responsabilidad del manejo de personal, permitiéndoles centrarse en los puntos más relevantes del ejercicio de la Medicina.

Incluye una organización y comité de ética Médica que imparte en todos sus departamentos, por lo que está en constante vigilancia del cumplimiento de las obligaciones de su personal médico y la supervisión de que los pacientes reciban una atención medica de calidad y un trato digno.

Identificación de Atributos de Cada Entidad

En este paso se realiza un análisis detallado de cada característica (atributo) de las Entidades detectadas en el paso anterior, es importante considerar el contexto del mundo real de lo abstraído, así como el posible uso de los datos.

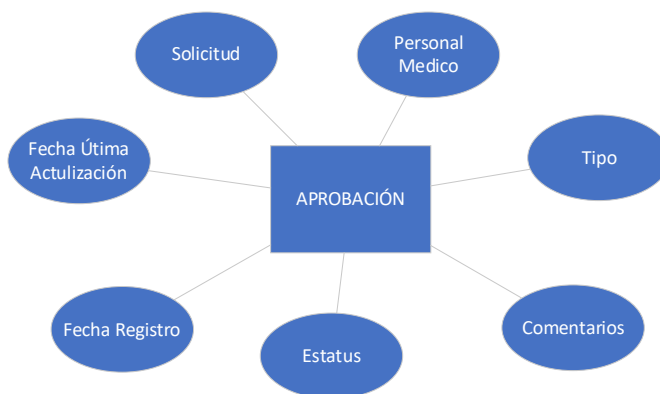


ILUSTRACIÓN 42. MODELO ER – APROBACIÓN – IDENTIFICACIÓN DE ATRIBUTOS

Descripción: Conjunto de datos que aprueban una solicitud de algún servicio hospitalario, con la finalidad de que las áreas directivas den el consentimiento y se puedan obtener estadísticas de los servicios proporcionados por las áreas médicas. Autorización realizada para el personal médico o directivo respecto a las solicitudes de los servicios médicos proporcionados por el hospital.

Tipo Naturaleza: Conceptual **Tipo Nivel Dependencia:** Fuerte **Tipo Clasificación:** Genética

Clasificación de los Atributos

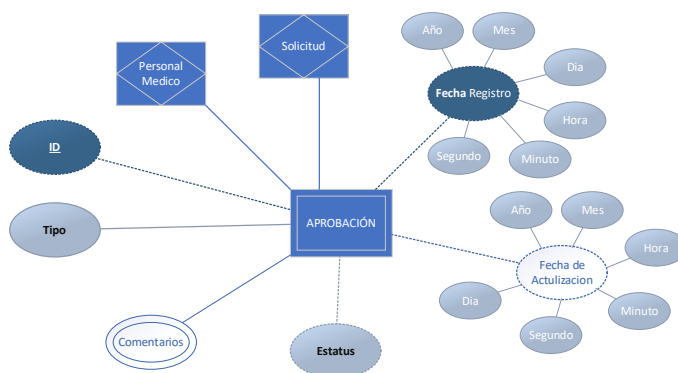


ILUSTRACIÓN 43. CLASIFICACIÓN DE ATRIBUTOS.

El objetivo es determinar la naturaleza de los atributos en base a su obligatoriedad, composición, tipo de almacenamiento y correspondencia de valores.

Descripción: Conjunto de datos que aprueban una solicitud de algún servicio hospitalario, con la finalidad de que las áreas directivas den el consentimiento y se puedan obtener estadísticas de los servicios proporcionados por las áreas médicas. Autorización realizada para el personal médico o directivo respecto a las solicitudes de los servicios médicos proporcionados por el hospital.

Identificación de Relaciones Directas Entre Entidades

En este paso se realiza análisis individual de cada ENTIDAD contra las restantes con la finalidad de identificar si correspondencia, es decir su relación.

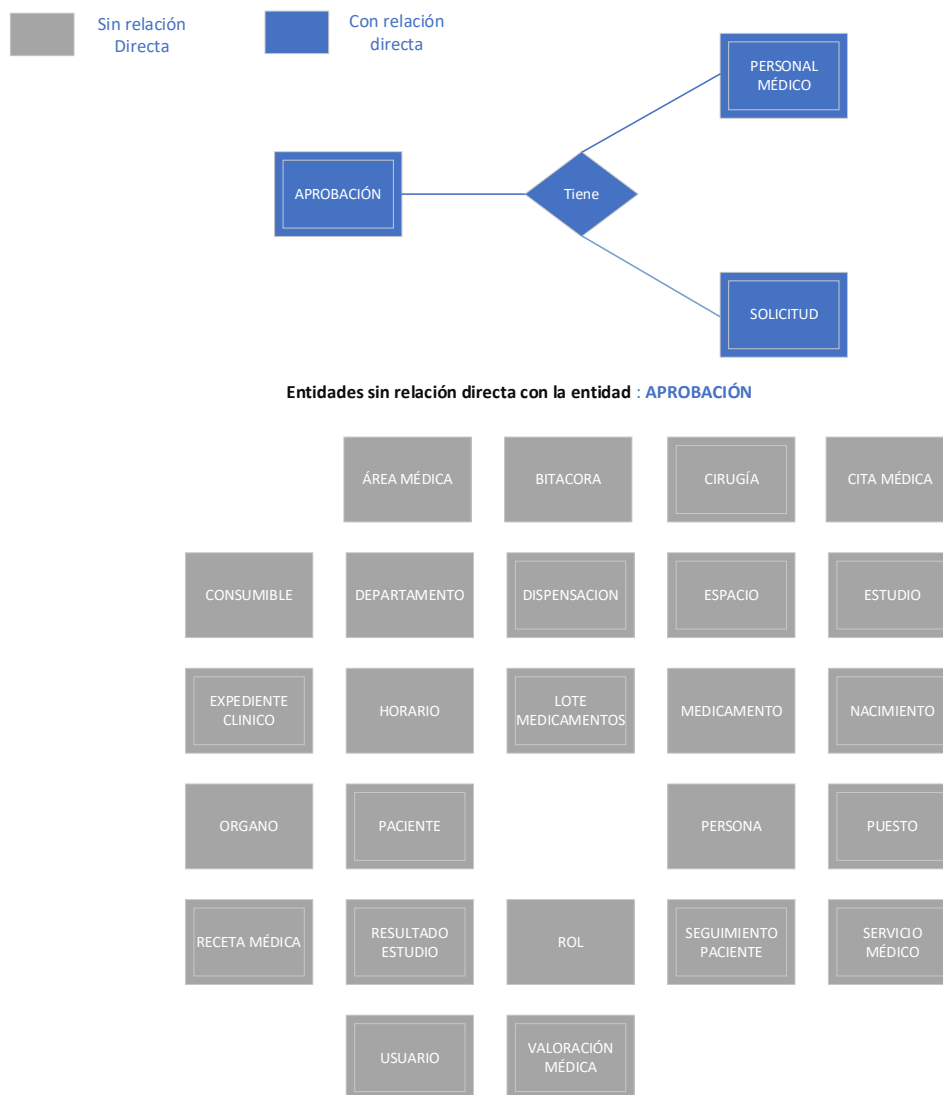


ILUSTRACIÓN 44. IDENTIFICACIÓN CON LAS RELACIONES DIRECTAS A OTRAS TABLAS EXISTENTES DENTRO DE LA BASE DE DATOS.

Construcción de la Tabla SQL

```
1 • CREATE TABLE "tbb_aprobaciones" (
2     "id" int NOT NULL AUTO_INCREMENT,
3     "Personal_Medico_ID" int NOT NULL,
4     "Solicitud_id" int NOT NULL,
5     "Comentario" varchar(500) DEFAULT NULL,
6     "Estatus" enum('Registrada','Programada','Cancelada','Reprogramada','En_Proceso','Realizada','Aprobado') DEFAULT NULL,
7     "Tipo" enum('Servicio_Interno','Traslados','Subrogado','Administrativo') DEFAULT NULL,
8     "Fecha_Registro" datetime DEFAULT NULL,
9     "Fecha_Actualizacion" datetime DEFAULT NULL,
10    PRIMARY KEY ("id"),
11    KEY "Personal_Medico_ID" ("Personal_Medico_ID"),
12    KEY "Solicitud_id" ("Solicitud_id"),
13    KEY "ix_tbb_aprobaciones_id" ("id"),
14    CONSTRAINT "tbb_aprobaciones_ibfk_1" FOREIGN KEY ("Personal_Medico_ID") REFERENCES "tbb_personal_medico" ("Persona_ID"),
15    CONSTRAINT "tbb_aprobaciones_ibfk_2" FOREIGN KEY ("Solicitud_id") REFERENCES "tbd_solicitudes" ("ID")
16 )
```

ILUSTRACIÓN 45. ESTRUCTURA DE LA TABLA DE APROBACIONES EN MYSQL.

La tabla Aprobaciones es de tipo Tabla Base, esta permitirá aprobar la solicitud de procedimientos clínicos, internos, traslados y Administrativos, dichas solicitudes serán vinculadas con el personal médico, que hace dicha solicitud.

TRIGGERS

La tabla Aprobaciones integra cuatro Triggers que vinculan la información insertada en la tabla hacia la tabla Bitácora, para su consulta se utiliza la siguiente orden SQL:

```
253 • SHOW TRIGGERS LIKE 'tbb_aprobaciones';
```

254

Trigger	Event	Table	Statement	Timing	Created	sql_mode	Definer	character_set_client	collation_connection	Database Collation
tbb_aprobacion_AFTER_INSERT	INSERT	tbb_aprobaciones	BEGIN -- Insertar en la tabla tbi_bitacor...	AFTER	2024-06-06 22:10:52.00	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900_ai_ci
tbd_usuarios_roles_BEFORE_UPDATE	UPDATE	tbb_aprobaciones	BEGIN set new.fecha_actualizacion = current_b...	BEFORE	2024-06-06 21:41:26.09	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900_ai_ci
tbb_aprobacion_AFTER_UPDATE	UPDATE	tbb_aprobaciones	BEGIN -- Insertar en la tabla tbi_bitacora I...	AFTER	2024-06-06 22:26:11.98	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900_ai_ci
tbb_aprobacion_AFTER_DELETE	DELETE	tbb_aprobaciones	BEGIN -- Insertar en la tabla tbi_bitacora I...	AFTER	2024-06-06 22:31:00.10	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900_ai_ci

ILUSTRACIÓN 46. INVOCACIÓN DE LOS TRIGGERS INCLUIDOS EN LA TABLA TBB_APROBACIONES

Trigger After Insert

Permite la inserción de la información en los campos, siendo su estructura:

```
1 • CREATE DEFINER='avnadmin'@'%' TRIGGER "tbb_aprobaciones_BEFORE_INSERT" BEFORE INSERT ON "tbb_aprobaciones" FOR EACH ROW BEGIN
2     -- Declaración de variables
3     DECLARE v_estatus_descripcion VARCHAR(20) DEFAULT 'En_Proceso';
4     DECLARE v_tipo_solicitud VARCHAR(20) DEFAULT 'Servicio_Interno';
5     DECLARE personal_medico VARCHAR(200) DEFAULT 'No Aplica';
6     DECLARE v_personal_medico_id INT;
7     DECLARE v_solicitud_id INT;
8     DECLARE solicitud VARCHAR(200) DEFAULT 'Sin datos de Solicitud';
```

ILUSTRACIÓN 47. PRIMERA PARTE DEL TRIGGER AFTER INSERT

```

9
10 -- Asignar el id del personal médico
11 SET v_personal_medico_id = NEW.personal_medico_id;
12
13 -- Asignación de la solicitud
14 SET v_solicitud_id = NEW.solicitud_id;
15
16 -- Intentar obtener el nombre del personal médico con su rol
17 BEGIN
18 DECLARE CONTINUE HANDLER FOR NOT FOUND SET personal_medico = 'No Aplica - Sin Rol';
19 SELECT CONCAT(p.Titulo_Cortesía, ' ', p.Nombre, ' ', p.Primer_Apellido, ' ', COALESCE(p.Segundo_Apellido, ''), ' - ', COALESCE(r.nombre, 'Sin Rol'))
20 INTO personal_medico
21 FROM tbb_personas p
22 LEFT JOIN tbc_rols r ON p.id = r.id
23 WHERE p.id = v_personal_medico_id AND p.Titulo_Cortesía IN ('Dr.', 'Dra.', 'Lic.', 'Ing.', 'Tec.', 'Q.F.C.');
```

24 END;

25

26 -- Intentar obtener la descripción de la solicitud

27 BEGIN

28 DECLARE CONTINUE HANDLER FOR NOT FOUND SET solicitud = 'Sin datos de Solicitud';

29 SELECT CONCAT('Prioridad: ', Prioridad, ' - ', 'Estatus: ', Estatus, ' ')

30 INTO solicitud

31 FROM tbd_solicitudes

32 WHERE id = v_solicitud_id;

33 END;

34

35 -- 'Estatus' enum('Registrada','Programada','Cancelada','Reprogramada','En Proceso','Aprobada') NOT NULL,

36 -- Validación del estatus del registro

37 -- 'Registrada','Programada','Cancelada','Reprogramada','En Proceso','Realizada','Aprobado'

38 CASE NEW.Estatus

39 WHEN 'Registrada' THEN SET v_estatus_descripcion = 'Registrada';

40 WHEN 'Programada' THEN SET v_estatus_descripcion = 'Programada';

41 WHEN 'Cancelada' THEN SET v_estatus_descripcion = 'Cancelada';

42 WHEN 'Reprogramada' THEN SET v_estatus_descripcion = 'Reprogramada';

43 WHEN 'En Proceso' THEN SET v_estatus_descripcion = 'En Proceso';

44 WHEN 'Realizada' THEN SET v_estatus_descripcion = 'Realizada';

45 WHEN 'Aprobado' THEN SET v_estatus_descripcion = 'Aprobado';

46 END CASE;

47

48 -- Validación del tipo de solicitud

49 CASE NEW.tipo

50 WHEN 'Servicio_Interno' THEN SET v_tipo_solicitud = 'Servicio Interno';

51 WHEN 'Traslados' THEN SET v_tipo_solicitud = 'Traslados';

52 WHEN 'Subrogado' THEN SET v_tipo_solicitud = 'Subrogado';

53 WHEN 'Administrativo' THEN SET v_tipo_solicitud = 'Administrativo';

54 END CASE;

55

56 -- Inserción en la tabla tbi_bitacora

57 INSERT INTO tbi_bitacora (

58 Usuario,

59 Operacion,

60 Tabla,

61 Descripción,

62 Estatus,

63 Fecha_Registro

64) VALUES (

65 CURRENT_USER(),

66 'Create',

67 'tbb_aprobaciones',

68 CONCAT(

69 'Se ha registrado una nueva aprobación con los siguientes datos:', '\n',

70 'Personal Médico: ', personal_medico, '\n',

71 'Datos de Solicitud: ', solicitud, '\n',

72 'Comentario: ', COALESCE(NEW.comentario, 'Sin Comentarios'), '\n',

73 'Estatus: ', v_estatus_descripcion, '\n',

74 'Tipo: ', v_tipo_solicitud, '\n',

75 'Fecha de Registro: ', COALESCE(NEW.fecha_registro, 'N/A'), '\n',

76 'Fecha de Actualización: ', COALESCE(NEW.fecha_actualizacion, 'N/A')

77),

78 default,

79 NOW()

80);

81 END

ILUSTRACIÓN 48. SEGUNDA PARTE DEL TRIGGER AFTER INSERT.

Trigger Before Update

Permite la actualización de la fecha cuando se realicen cambios en algún elemento de la solicitud agregada a la tabla Aprobaciones, su estructura es la siguiente:

```
1 • CREATE DEFINER="avnadmin"@"%" TRIGGER "tbb_aprobaciones_BEFORE_UPDATE" BEFORE UPDATE ON "tbb_aprobaciones" FOR EACH ROW BEGIN
2     SET NEW.fecha_actualizacion = CURRENT_TIMESTAMP();
3     END
```

ILUSTRACIÓN 49. ESTRUCTURA DEL TRIGGER BEFORE UPDATE.

Trigger After Update

Permite la actualización de todos los campos que se encuentran en la tabla, principalmente para el Tipo de Solicitud, Estado de Solicitud y Comentario, su estructura es la siguiente:

```
1 • CREATE DEFINER="avnadmin"@"%" TRIGGER "tbb_aprobaciones_AFTER_UPDATE" AFTER UPDATE ON "tbb_aprobaciones" FOR EACH ROW BEGIN
2     -- Declaración de variables
3     DECLARE v_estatus_descripcion VARCHAR(20) DEFAULT 'En_Proceso';
4     DECLARE v_tipo_solicitud VARCHAR(20) DEFAULT 'Servicio_Interno';
5     DECLARE personal_medico VARCHAR(200) DEFAULT 'No Aplica';
6     DECLARE v_personal_medico_id INT;
7     DECLARE v_solicitud_id INT;
8     DECLARE solicitud VARCHAR(200) DEFAULT 'Sin datos de Solicitud';
9
10    -- Asignar el id del personal médico
11    SET v_personal_medico_id = NEW.personal_medico_id;
12
13    -- Asignación de la solicitud
14    SET v_solicitud_id = NEW.solicitud_id;
15
16    -- Intentar obtener el nombre del personal médico con su rol
17    BEGIN
18        DECLARE CONTINUE HANDLER FOR NOT FOUND SET personal_medico = 'No Aplica - Sin Rol';
19        SELECT CONCAT(p.Titulo_Cortesia, ' ', p.Nombre, ' ', p.Primer_Apellido, ' ', COALESCE(p.Segundo_Apellido, ''), ' - ', COALESCE(r.nombre, 'Sin Rol'))
20        INTO personal_medico
21        FROM tbb_personas p
22        LEFT JOIN tbc_rols r ON p.id = r.id
23        WHERE p.id = v_personal_medico_id AND p.Titulo_Cortesia IN ('Dr.', 'Dra.', 'Lic.', 'Ing.', 'Tec.', 'Q.F.C.');
```

ILUSTRACIÓN 50. PRIMERA SECCIÓN DEL TRIGGER AFTER UPDATE.

```

47     WHEN 'Registrada' THEN SET v_estatus_descripcion = 'Registrada';
48     WHEN 'Programada' THEN SET v_estatus_descripcion = 'Programada';
49     WHEN 'Cancelada' THEN SET v_estatus_descripcion = 'Cancelada';
50     WHEN 'Reprogramada' THEN SET v_estatus_descripcion = 'Reprogramada';
51     WHEN 'En_Proceso' THEN SET v_estatus_descripcion = 'En_Proceso';
52     WHEN 'Realizada' THEN SET v_estatus_descripcion = 'Realizada';
53     WHEN 'Aprobado' THEN SET v_estatus_descripcion = 'Aprobado';
54 END CASE;
55
56 -- Validación del tipo de solicitud
57 CASE NEW.tipo
58     WHEN 'Servicio_Interno' THEN SET v_tipo_solicitud = 'Servicio_Interno';
59     WHEN 'Traslados' THEN SET v_tipo_solicitud = 'Traslados';
60     WHEN 'Subrogado' THEN SET v_tipo_solicitud = 'Subrogado';
61     WHEN 'Administrativo' THEN SET v_tipo_solicitud = 'Administrativo';
62 END CASE;
63
64 CASE OLD.tipo
65     WHEN 'Servicio_Interno' THEN SET v_tipo_solicitud = 'Servicio_Interno';
66     WHEN 'Traslados' THEN SET v_tipo_solicitud = 'Traslados';
67     WHEN 'Subrogado' THEN SET v_tipo_solicitud = 'Subrogado';
68     WHEN 'Administrativo' THEN SET v_tipo_solicitud = 'Administrativo';
69 END CASE;
70
71 -- Inserción en la tabla tbi_bitacora
72 INSERT INTO tbi_bitacora (
73     Usuario,
74     Operacion,
75     Tabla,
76     Descripcion,
77     Estatus,
78     Fecha_Registro
79 ) VALUES (
80     CURRENT_USER(),
81     'Update',
82     'tbb_aprobaciones',
83     CONCAT(
84         'Se ha registrado una nueva aprobación con los siguientes datos:', '\n',
85         'Personal Médico: ', personal_medico, '\n',
86         'Solicitud: ', solicitud, '\n',
87         'Comentario: ', COALESCE(CONCAT('- Se Complementó: ', NEW.comentario), COALESCE(OLD.comentario, 'Sin Nuevos Comentarios')), '\n',
88         'Estatus Inicial: ', v_estatus_descripcion, '\n',
89         'Estatus: ', COALESCE(CONCAT('- Actualizado: ', NEW.Estatus), COALESCE(OLD.Estatus, 'Sin Cambio de Estado')), '\n',
90         'Tipo: ', v_tipo_solicitud, '\n',
91         'Tipo: ', COALESCE(CONCAT('- Se Actualizo: ', NEW.Tipo), COALESCE(OLD.Tipo, 'Sin Cambio de Tipo')), '\n',
92         'Fecha de Registro: ', COALESCE(NEW.fecha_registro, 'N/A'), '\n',
93         'Fecha de Actualización: ', COALESCE(NEW.fecha_actualizacion, 'N/A')
94     ),
95     default,
96     NOW()
97 );
98
99 -----
100 -- Actualiza el estatus en la tabla tbd_solicitudes
101 UPDATE tbd_solicitudes SET estatus = NEW.estatus WHERE id = NEW.solicitud_id;
102
103 -- Si el estatus es 'Aprobada', también cambia estatus_aprobacion a 1
104 IF NEW.estatus = 'Aprobada' THEN UPDATE tbd_solicitudes SET estatus_aprobacion = true WHERE id = NEW.solicitud_id;
105 END IF;
106 -- `Estatus` enum('Registrada','Programada','Cancelada','Reprogramada','En Proceso','Aprobada') NOT NULL,

```

ILUSTRACIÓN 51. SEGUNDA SECCIÓN DEL TRIGGER AFTER UPDATE.

```

107 IF NEW.estatus = 'Cancelada' THEN UPDATE tbd_solicitudes SET estatus_aprobacion = false WHERE id = NEW.solicitud_id;
108 END IF;
109
110 IF NEW.estatus = 'Registrada' THEN UPDATE tbd_solicitudes SET estatus_aprobacion = false WHERE id = NEW.solicitud_id;
111 END IF;
112
113 IF NEW.estatus = 'Programada' THEN UPDATE tbd_solicitudes SET estatus_aprobacion = false WHERE id = NEW.solicitud_id;
114 END IF;
115
116 IF NEW.estatus = 'Reprogramada' THEN UPDATE tbd_solicitudes SET estatus_aprobacion = false WHERE id = NEW.solicitud_id;
117 END IF;
118
119 IF NEW.estatus = 'En Proceso' THEN UPDATE tbd_solicitudes SET estatus_aprobacion = false WHERE id = NEW.solicitud_id;
120 END IF;
121
122 -----
123
124 END

```

ILUSTRACIÓN 52. TERCERA SECCIÓN DEL TRIGGER AFTER UPDATE.

Trigger After Delete

Permite la eliminación de solicitudes dentro de la tabla aprobaciones, dejando la descripción lo que se ha eliminado:

```

1 CREATE DEFINER="avnadmin"@"%" TRIGGER "tbb_aprobaciones_AFTER_DELETE" AFTER DELETE ON "tbb_aprobaciones" FOR EACH ROW BEGIN
2     -- Declaración de variables
3     DECLARE v_estatus_descripcion VARCHAR(20) DEFAULT 'En_Proceso';
4     DECLARE v_tipo_solicitud VARCHAR(20) DEFAULT 'Servicio_Interno';
5     DECLARE personal_medico VARCHAR(200) DEFAULT 'No Aplica';
6     DECLARE v_personal_medico_id INT;
7     DECLARE v_solicitud_id INT;
8     DECLARE solicitud VARCHAR(200) DEFAULT 'Sin datos de Solicitud';
9
10    -- Restringir titulo
11    -- DECLARE v_titulo VARCHAR(20);
12
13    -- Asignar el id del personal médico
14    SET v_personal_medico_id = old.personal_medico_id;
15
16    -- Asignación de la solicitud
17    SET v_solicitud_id = old.solicitud_id;
18    -----
19    -- Verificar mediante una condicion si el Titulo es permitido
20    /*
21    -----
22    -- Intentar obtener el nombre del personal médico con su rol
23    BEGIN
24        DECLARE CONTINUE HANDLER FOR NOT FOUND SET personal_medico = 'No Aplica - Sin Rol';
25        SELECT CONCAT(p.Titulo_Cortesía, ' ', p.Nombre, ' ', p.Primer_Apellido, ' ', COALESCE(p.Segundo_Apellido, ''), ' - ', COALESCE(r.nombre, 'Sin Rol'))
26        INTO personal_medico
27        FROM tbb_personas p
28        LEFT JOIN tbc_rols r ON p.id = r.id
29        WHERE p.id = v_personal_medico_id AND p.Titulo_Cortesía IN ('Dr.', 'Dra.', 'Lic.', 'Ing.', 'Tec.', 'Q.F.C.');

```

ILUSTRACIÓN 53. PRIMERA SECCIÓN DE TRIGGER AFTER DELETE.

```

50      -- Validación del estatus del registro
51      CASE old.Estatus
52          WHEN 'Registrada' THEN SET v_estatus_descripcion = 'Registrada';
53          WHEN 'Programada' THEN SET v_estatus_descripcion = 'Programada';
54          WHEN 'Cancelada' THEN SET v_estatus_descripcion = 'Cancelada';
55          WHEN 'Reprogramada' THEN SET v_estatus_descripcion = 'Reprogramada';
56          WHEN 'En_Proceso' THEN SET v_estatus_descripcion = 'En_Proceso';
57          WHEN 'Realizada' THEN SET v_estatus_descripcion = 'Realizada';
58          WHEN 'Aprobado' THEN SET v_estatus_descripcion = 'Aprobado';
59      END CASE;
60
61      -- Validación del tipo de solicitud
62      CASE old.tipo
63          WHEN 'Servicio_Interno' THEN SET v_tipo_solicitud = 'Servicio_Interno';
64          WHEN 'Traslados' THEN SET v_tipo_solicitud = 'Traslados';
65          WHEN 'Subrogado' THEN SET v_tipo_solicitud = 'Subrogado';
66          WHEN 'Administrativo' THEN SET v_tipo_solicitud = 'Administrativo';
67      END CASE;
68
69      -- Inserción en la tabla tbi_bitacora
70      INSERT INTO tbi_bitacora (
71          Usuario,
72          Operación,
73          Tabla,
74          Descripción,
75          Estatus,
76          Fecha_Registro
77      ) VALUES (
78          CURRENT_USER(),
79          'Delete',
80          'tbb_aprobaciones',
81          CONCAT(
82              'Se ha Eliminado un Registro con los Sigüientes Datos:', '\n',
83              'Personal Médico: ', personal_medico, '\n',
84              'Solicitud: ', solicitud, '\n',
85              'Comentario: ', COALESCE(old.comentario, 'Sin Comentarios'), '\n',
86              'Estatus: ', v_estatus_descripcion, '\n',
87              'Tipo: ', v_tipo_solicitud, '\n',
88              'Fecha de Registro: ', COALESCE(old.fecha_registro, 'N/A'), '\n',
89              'Fecha de Actualización: ', COALESCE(old.fecha_actualizacion, 'N/A')
90          ),
91          default,
92          NOW()
93      );
94      END

```

ILUSTRACIÓN 54. SEGUNDA SECCIÓN DE TRIGGER AFTER DELETE.

Los Triggers, también llamados disparadores, son procedimientos que se ejecutan de manera automatizada dentro de la configuración de la base de datos. La codificación ilustrativa de cada uno de ellos se integra en la tabla tbb_aprobaciones. Estos se activan automáticamente al registrarse los eventos de INSERT, UPDATE o DELETE, y cada uno implementa una lógica distinta. Esto permite registrar actividades y facilita el manejo de grandes volúmenes de datos, considerándolos como elementos validadores que habilitan determinadas acciones que pueden añadirse como restricciones en la tabla tbb_aprobaciones.

Tabla Bitácora

```
create table tbi_bitacora(
  Id int unsigned auto_increment primary key,
  Usuario varchar(50) not null,
  Operacion enum('Create','Read','Update','Delete') not null,
  Tabla Varchar(50),
  Descripcion text not null,
  Estatus bit default b'1',
  Fecha_Registro Datetime not null default current_timestamp
);
```

ILUSTRACIÓN 55. ESTRUCTURACIÓN DE LA TABLA BITÁCORA.

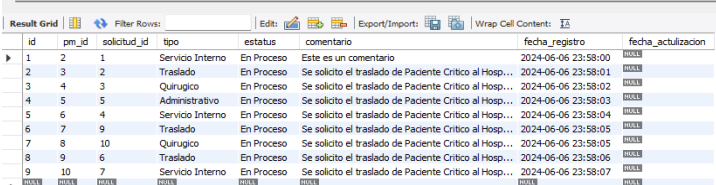
Esta tabla recibe todos los movimientos registrados en la Inserción, Actualización y Eliminación de los datos que se almacenan en la tabla Aprobaciones, esta tabla está integrada dentro de los **AFTER INSERT, AFTER UPDATE,**

After Delete.

La determinación de pruebas es crucial para identificar que el flujo de información es correcto y se almacena en la tabla correspondiente a Bitácora, la activación de los Triggers en la inserción de información permite generar registros de las actividades realizadas en cada inserción y eliminación de información, como la actualización de parámetros, para esta tarea se especifican los parámetros a insertar de acuerdo a la estructura de tipos de datos aceptados para cada atributo, como se mencionó anteriormente la [ilustración 6] de esta sección.

Inserción de Datos de Prueba

```
256 • INSERT INTO tbb_aprobaciones (pm_id, solicitud_id, tipo, comentario, fecha_registro) VALUES (2, 1, 'Servicio Interno','Este es un comentario', now());
257 • INSERT INTO tbb_aprobaciones (pm_id, solicitud_id, tipo, comentario, fecha_registro) VALUES (3, 2, 'Traslado','Se solicito el traslado de Paciente Critico al Hospital de Infantil de la CDMEH', now());
258 • INSERT INTO tbb_aprobaciones (pm_id, solicitud_id, tipo, comentario, fecha_registro) VALUES (4, 3, 'Quirurgico','Se solicito el traslado de Paciente Critico al Hospital de Infantil de la CDMEH', now());
259 • INSERT INTO tbb_aprobaciones (pm_id, solicitud_id, tipo, comentario, fecha_registro) VALUES (5, 5, 'Administrativo','Se solicito el traslado de Paciente Critico al Hospital de Infantil de la CDMEH', now());
260 • INSERT INTO tbb_aprobaciones (pm_id, solicitud_id, tipo, comentario, fecha_registro) VALUES (6, 4, 'Servicio Interno','Se solicito el traslado de Paciente Critico al Hospital de Infantil de la CDMEH', now());
261 • INSERT INTO tbb_aprobaciones (pm_id, solicitud_id, tipo, comentario, fecha_registro) VALUES (7, 9, 'Traslado','Se solicito el traslado de Paciente Critico al Hospital de Infantil de la CDMEH', now());
262 • INSERT INTO tbb_aprobaciones (pm_id, solicitud_id, tipo, comentario, fecha_registro) VALUES (8, 10, 'Quirurgico','Se solicito el traslado de Paciente Critico al Hospital de Infantil de la CDMEH', now());
263 • INSERT INTO tbb_aprobaciones (pm_id, solicitud_id, tipo, comentario, fecha_registro) VALUES (9, 6, 'Traslado','Se solicito el traslado de Paciente Critico al Hospital de Infantil de la CDMEH', now());
264 • INSERT INTO tbb_aprobaciones (pm_id, solicitud_id, tipo, comentario, fecha_registro) VALUES (10, 7, 'Servicio Interno','Se solicito el traslado de Paciente Critico al Hospital de Infantil de la CDMEH', now());
265
```



id	pm_id	solicitud_id	tipo	estatus	comentario	fecha_registro	fecha_actualizacion
1	2	1	Servicio Interno	En Proceso	Este es un comentario	2024-06-06 23:58:00	NULL
2	3	2	Traslado	En Proceso	Se solicito el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:01	NULL
3	4	3	Quirurgico	En Proceso	Se solicito el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:02	NULL
4	5	5	Administrativo	En Proceso	Se solicito el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:03	NULL
5	6	4	Servicio Interno	En Proceso	Se solicito el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:04	NULL
6	7	9	Traslado	En Proceso	Se solicito el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:05	NULL
7	8	10	Quirurgico	En Proceso	Se solicito el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:05	NULL
8	9	6	Traslado	En Proceso	Se solicito el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:06	NULL
9	10	7	Servicio Interno	En Proceso	Se solicito el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:07	NULL

ILUSTRACIÓN 56. INSERCIÓN DE REGISTROS DE PRUEBA A LA TABLA TBB_APROBACIONES.

Los datos insertados ejemplifican el proceso de ejecución de solicitudes en la tabla **tbb_aprobaciones**, que posteriormente se integrará mediante la conexión de llaves foráneas con la tabla base (**tbb_solicitudes**) y la tabla catálogo (**tbc_personal_medico**). Esta integración se

realizará de acuerdo al ID de la solicitud y a la asignación de la solicitud según su departamento de especialización. Al insertarse cada registro, se configurará de manera temporal con un estatus de "En Proceso", lo que indica que la solicitud está siendo atendida. Además, se asignará el tipo de solicitud, que involucra tanto al departamento solicitante como a la gravedad de la solicitud, lo que servirá como un indicativo de atención prioritaria.

Actualización de datos de prueba

```
269 • UPDATE tbb_aprobaciones SET tipo = 'Traslado' WHERE tipo = 'Servicio Interno' and id = 5;
270 • UPDATE tbb_aprobaciones SET Estatus = 'Aprobado' WHERE Estatus = 'En Proceso' and id = 3;
271 • UPDATE tbb_aprobaciones SET Comentario = 'Solicitud de traslado a la UTI' WHERE Comentario = 'Este es un comentario' and id = 1;
```

Result Grid							
Filter Rows:							
Edit: Export/Import: Wrap Cell Content:							
	id	pm_id	solicitud_id	tipo	estatus	comentario	fecha_registro
1	2	1		Servicio Interno	En Proceso	Solicitud de traslado a la UTI	2024-06-06 23:58:00
2	3	2		Traslado	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:01
3	4	3		Quirugico	Aprobado	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:02
4	5	5		Administrativo	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:03
5	6	4		Traslado	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:04
6	7	9		Traslado	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:05
7	8	10		Quirugico	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:05
8	9	6		Traslado	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:06
9	10	7		Servicio Interno	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:07
	NULL	NULL	NULL	NULL	NULL	NULL	NULL

ILUSTRACIÓN 57. ACTUALIZACIÓN DE DATOS DE MANERA MANUAL EN MySQL.

La automatización de los registros permite que estos se procesen de manera correcta al insertar el tipo de dato asignado para el parámetro a actualizar. Sin embargo, esto también puede convertirse en una variante importante en la adecuación de los tipos de datos asignados para los modelos diseñados en FastAPI, lo que puede provocar cambios en su sintaxis. Además, es crucial garantizar que los modelos en FastAPI estén sincronizados con los esquemas de la base de datos para evitar errores en la validación de datos. Esto incluye la configuración adecuada de validaciones en Pydantic y ajustes necesarios en la estructura de las tablas para manejar eficientemente la relación entre los parámetros y los datos procesados, asegurando así la integridad y consistencia de la información.

Eliminación de datos de prueba

```
273 • delete from tbb_aprobaciones where id = 1;
```

Result Grid							
Filter Rows:							
Edit: Export/Import: Wrap Cell Content:							
	id	pm_id	solicitud_id	tipo	estatus	comentario	fecha_registro
2	3	2		Traslado	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:01
3	4	3		Quirugico	Aprobado	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:02
4	5	5		Administrativo	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:03
5	6	4		Traslado	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:04
6	7	9		Traslado	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:05
7	8	10		Quirugico	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:05
8	9	6		Traslado	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:06
9	10	7		Servicio Interno	En Proceso	Se solicita el traslado de Paciente Critico al Hosp...	2024-06-06 23:58:07
	NULL	NULL	NULL	NULL	NULL	NULL	NULL

ILUSTRACIÓN 58. ELIMINACIÓN DE PRUEBA DENTRO DE LA TABLA TBB_APROBACIONES.

La eliminación de registros se realizará identificando el ID correspondiente. Sin embargo, su implementación requerirá una dependencia directa con las tablas **tbc_personal_medico** y **tbb_solicitudes**, ya que estas relaciones son esenciales para procesar correctamente la eliminación en la implementación final de la tabla. Esto asegura que la operación respete las restricciones de integridad referencial y que no se produzcan inconsistencias en los datos relacionados.

Resultados en tabla Bitácora

```
303 • select * from tbi_bitacora;
304
305
```

Id	Usuario	Operacion	Tabla	Descripcion	Estatus	Fecha_Registro
1	root@localhost	Create	tbb_aprobacion	Se ha generado una nueva solicitud de Aprobación con los siguientes datos: Personal Médico: 2 Nú...	1	2024-06-06 22:11:00
2	root@localhost	Update	tbb_aprobacion	Se ha Actualizado la solicitud solicitud de Aprobación con los siguientes datos: Personal Médico: 2 Nú...	1	2024-06-06 22:23:04
3	root@localhost	Update	tbb_aprobacion	Se ha Actualizado la solicitud solicitud de Aprobación con los siguientes datos: Personal Médico: 2 Nú...	1	2024-06-06 22:23:52
4	root@localhost	Update	tbb_aprobacion	Se ha Actualizado la solicitud solicitud de Aprobación con los siguientes datos: Personal Médico: 2 Nú...	1	2024-06-06 22:28:10
5	root@localhost	Create	tbb_aprobacion	Se ha generado una nueva solicitud de Aprobación con los siguientes datos: Personal Médico: 3 Nú...	1	2024-06-06 22:34:05
6	root@localhost	Delete	tbb_aprobacion	Se ha Eliminado una Solicitud de Aprobación con los siguientes datos: Personal Médico: 3 Número de ...	1	2024-06-06 22:35:42
7	root@localhost	Create	tbb_aprobacion	Se ha generado una nueva solicitud de Aprobación con los siguientes datos: Personal Médico: 3 Nú...	1	2024-06-06 23:56:16
8	root@localhost	Create	tbb_aprobacion	Se ha generado una nueva solicitud de Aprobación con los siguientes datos: Personal Médico: 4 Nú...	1	2024-06-06 23:56:17
9	root@localhost	Create	tbb_aprobacion	Se ha generado una nueva solicitud de Aprobación con los siguientes datos: Personal Médico: 5 Nú...	1	2024-06-06 23:56:18
10	root@localhost	Create	tbb_aprobacion	Se ha generado una nueva solicitud de Aprobación con los siguientes datos: Personal Médico: 6 Nú...	1	2024-06-06 23:56:19

ILUSTRACIÓN 59. REGISTRO DE MOVIMIENTOS REALIZADOS EN LA TABLA BITÁCORA.

Como se ha mencionado, todos los registros y movimientos realizados en la tabla **tbb_aprobaciones** son documentados de manera específica, detallando puntualmente el tipo de operación y la tabla donde se llevó a cabo dicha transacción. El registro de cada trigger se refleja en esta tabla, verificando que su función sea correcta y eficiente. Se espera que en esta tabla se registren todas las tablas integradas en la base de datos, las cuales también incluirán sus propias reglas de negocio específicas para cada movimiento registrado.

Asimismo, la configuración del usuario estará determinada de acuerdo con los permisos asignados, como lectura, escritura, actualización y eliminación. Estos permisos serán gestionados cuidadosamente para garantizar la seguridad y el control de acceso en la base de datos. Esto permitirá establecer políticas que limiten las operaciones de los usuarios según su rol, evitando modificaciones no autorizadas y asegurando el cumplimiento de los lineamientos del sistema. Además, la documentación detallada de los movimientos facilitará la auditoría y el mantenimiento de la base de datos, asegurando la integridad y la trazabilidad de los datos.

De acuerdo con lo mencionado anteriormente, la estructura valida el flujo de datos incorporado en el modelo reverso de la base de datos. Este flujo describe cómo las peticiones realizadas interactúan

con otros sectores dentro del sistema. Cada petición es única y está asociada a identificadores que controlan los atributos de las tablas. Al estar conectadas entre sí, estas tablas establecen un criterio que permite vincular las funciones del modelo con el flujo de datos, influyendo directamente en el comportamiento de la capa de Front-End.

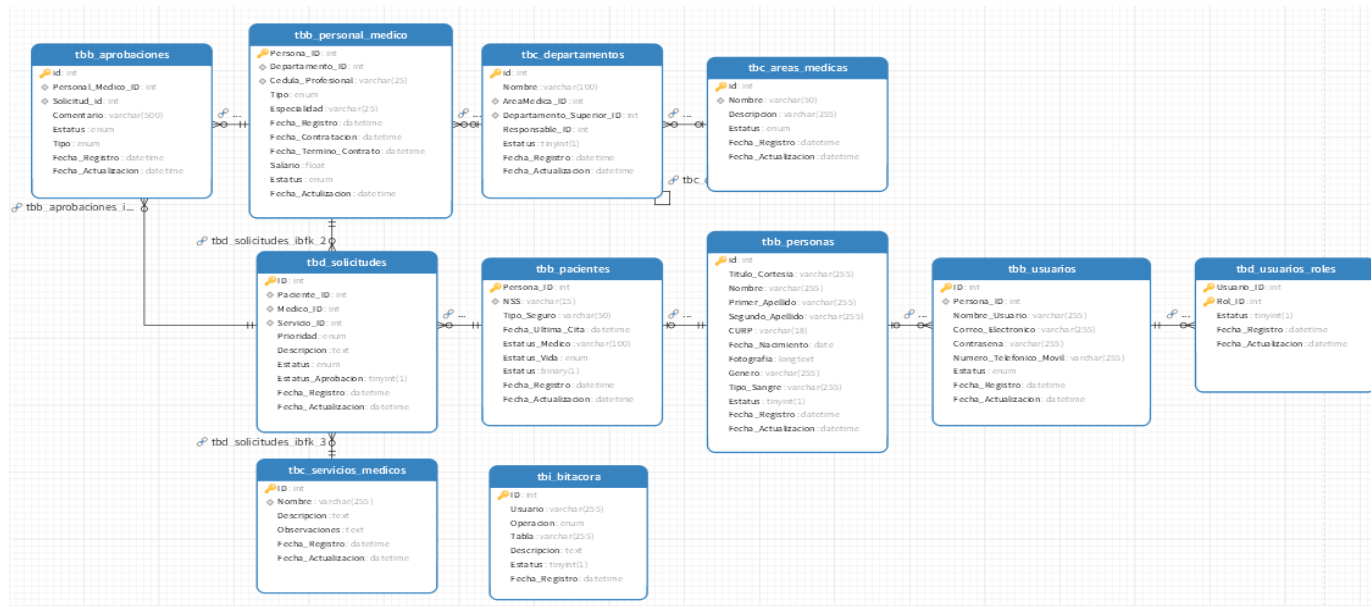


ILUSTRACIÓN 60. MODELO REVERSO DE LOS MÓDULOS QUE INTERACTUARÁN CON LA TABLA TBB_APROBACIONES.

La tabla `tbb_aprobaciones` (tbc: tabla catálogo) se vincula directamente con dos tablas principales. La primera relación es con la tabla `tbb_personal_medico` (tbb: tabla base), que incorpora todos los atributos necesarios para establecer la conexión mediante su identificador ID. Además, esta tabla forma parte de una jerarquía que se extiende hacia la tabla `tbc_departamentos` (tbc: tabla catálogo), la cual está vinculada a un departamento específico dentro de la tabla `tbc_areas_medicas` (tbc: tabla catálogo).

La segunda relación de `tbb_aprobaciones` es con la tabla `tbd_solicitudes`, que a su vez se conecta directamente con las tablas `tbc_servicios_medicos` y `tbb_pacientes` mediante su identificador ID. De estas, la tabla `tbb_pacientes` incluye dependencias adicionales, como `tbb_persona`, que contiene todos los atributos necesarios para detallar los datos del paciente; `tbb_usuarios`, que registra a las personas en el sistema; y `tbd_usuarios_roles`, que define la función y las actividades asignadas a las personas dentro del hospital.

La construcción del modelo inverso es importante, y permite visualizar el flujo de conexiones de llaves foráneas y dependencias directas, lo que permite un control en las peticiones.

Construcción del Back-End

El Back-End ha sido desarrollado utilizando el framework **FastAPI**, reconocido por su alta performance y simplicidad en el desarrollo de APIs REST. Este entorno permite la integración eficiente con bases de datos relacionales, así como la gestión de dependencias, autenticación y validación de datos de manera robusta, manteniendo un diseño limpio y modular.

El proyecto requiere una **estructura limpia y modular** que facilite la implementación de la lógica siguiendo buenas prácticas de diseño. Este enfoque permite al desarrollador mantener el código organizado y escalable, facilitando futuras modificaciones y mejoras. El **archivo main.py** actúa como el punto central de configuración y arranque del proyecto, integrando componentes clave como la gestión de rutas, la configuración de CORS (Cross-Origin Resource Sharing), la conexión con modelos de datos y la configuración de servicios esenciales.

La elección del archivo main.py obedece a la necesidad de contar con un núcleo que no solo inicie el proyecto, sino que también permita gestionar permisos de entrada y salida a través de peticiones HTTP estándar: **GET**, **POST**, **PUT**, y **DELETE**. Además, el proyecto incluye la configuración de CORS para habilitar solicitudes de origen cruzado, lo que asegura que aplicaciones externas puedan interactuar con el sistema de manera controlada. En este caso, se ha configurado para permitir todos los orígenes, métodos y encabezados, aunque esta configuración puede ajustarse a necesidades específicas de seguridad.

```
app = FastAPI(  
    title="HOSPITAL PrivilegeCare"  
)
```

ILUSTRACIÓN 61. IDENTIFICACIÓN DEL PROYECTO EN FASTAPI.

Instancia de la Aplicación FastAPI:

Se crea una instancia de FastAPI con el título "HOSPITAL PrivilegeCare", que da identidad al proyecto.

Configuración de Middleware CORS:

Esto permite solicitudes de diferentes dominios, habilitando las credenciales y métodos HTTP necesarios para el correcto funcionamiento del sistema.

```
# Configurar CORS  
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["*"], # Permite todas las solicitudes de origen cruzado. Puedes especificar una lista de dominios permitidos.  
    allow_credentials=True,  
    allow_methods=["*"], # Permite todos los métodos HTTP (GET, POST, etc.)  
    allow_headers=["*"], # Permite todos los encabezados  
)
```

ILUSTRACIÓN 62. CONFIGURACIÓN DE LOS CORS DEL PROYECTO.

Inclusión de Rutas

El archivo incluye routers de diversas funcionalidades, como usuarios, roles, áreas médicas, estudios, pacientes y personal médico, asegurando una estructura modular donde cada componente maneja su lógica de forma independiente.

Cada una de las rutas se importa de manera jerárquica, haciendo referencia de abajo hacia arriba aquellas que depende directamente de otras para poder funcionar, la ruta que pertenece a la tabla de `tbb_aprobaciones`, es un ejemplo claro de dependencia de las demás rutas.

```
from routes.persons import person
from routes.users import user
from routes.usuarios import usuario
from routes.roles import roles
from routes.tbb_usuarios import tbb_usuarios_router
from routes.usuario_roles import usuario_roles_router
from routes.cirugia import cirugia_router
from routes.horarios import horarios
from routes.espacios import espacio
from routes areas_medicas import area_medica
from routes.bitacora import bitacora
from routes.consumibles import consumible
from routes.dispensaciones import dispensacion
from routes.estudios import estudios
from routes.resultados_estudios import resultados_estudios
from routes.lotes import lote
from routes.medicamentos import medicamento
from routes.puestos import puesto
from routes.puestos_departamentos import puesto_departamento
from routes.tbc_servicios_medicos import servicios_medicos
from routes.tbc_departamentos import departamentos
from routes.personal_medico import personal_medico
from routes.tbb_pacientes import tbb_pacientes
from routes.solicitudes import request
from routes.tbb_aprobaciones import tbb_aprobaciones
from routes.tbc_organos import tbc_organos
from routes.Pediatria.nacimientos import baby
from routes.Pediatria.viewCiudad import view1
from routes.Pediatria.viewGenero import view2
```

ILUSTRACIÓN 63. RUTAS ESTABLECIDAS QUE SE CONECTAN CON EL ROUTES.

Gestión de Archivos Estáticos:

Se monta un directorio para almacenar y servir archivos estáticos como imágenes o documentos, configurado en la ruta `/uploads`.

```
# Configurar el directorio de archivos estáticos
app.mount("/uploads", StaticFiles(directory="uploads", html=False), name="uploads")
```

ILUSTRACIÓN 64. CARPETA ASIGNADA PARA EL ALOJAMIENTO DE LOS ARCHIVOS ESTÁTICOS.

Registro de Logs:

Se configura un logger para registrar mensajes importantes, incluyendo un mensaje de bienvenida que indica que el sistema está listo para usarse.

```
# Mensaje de bienvenida usando logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
logger.info(" Bienvenido al Back-End del Hospital PrivilegeCare")
```

ILUSTRACIÓN 65. MENSAJE QUE SE MOSTRARA EN EL ENTORNO DE SWAGGER UI.

Los pintos señalados en la configuración, main.py no solo funciona como el núcleo del sistema, sino que también establece la base para la interacción segura y eficiente entre las diferentes capas del proyecto, desde los modelos de datos hasta los endpoints de la API. Esto garantiza que el desarrollo y la implementación sigan un flujo lógico y organizado, manteniendo la calidad del código y facilitando la colaboración entre desarrolladores.

Estructura del Proyecto

La construcción del proyecto incorpora una distribución estructurada que facilita la identificación lógica de sus componentes, promoviendo buenas prácticas en el desarrollo. Este diseño garantiza un entorno organizado, modular y mantenible, asegurando la escalabilidad y eficiencia del proyecto. A continuación, se detallan las carpetas y elementos principales que conforman la arquitectura del proyecto:

El proyecto utiliza un entorno virtual denominado EntornoVirtualApi, que proporciona un espacio aislado y seguro para ejecutar el proyecto y gestionar sus dependencias. Al activar este entorno, se garantiza que todas las bibliotecas y configuraciones estén adecuadamente contenidas, evitando conflictos con otros proyectos y asegurando un despliegue confiable.

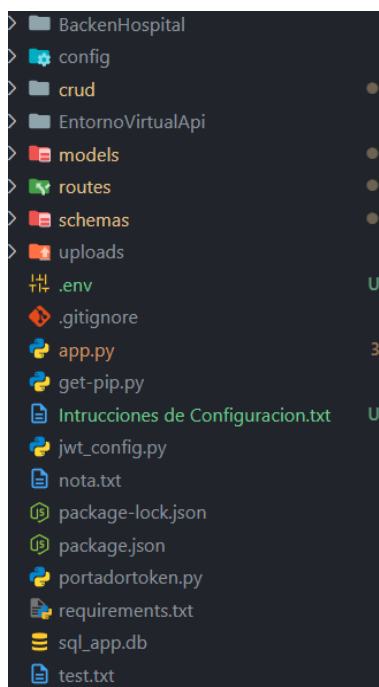


ILUSTRACIÓN 66. DISTRIBUCIÓN DE CARPETAS EN EL ENTORNO DE DESARROLLO DEL BACK-END.

Carpeta Config

Esta carpeta contiene la configuración necesaria para la vinculación de la base de datos, incluyendo las conexiones a bases de datos de desarrollo, pruebas y producción. Este enfoque permite que el proyecto sea flexible y esté preparado para diferentes entornos, asegurando una transición fluida desde el desarrollo local hasta el despliegue en producción.

Carpeta Models

En esta carpeta se definen los modelos de datos que estructuran las tablas de la base de datos MySQL. Cada modelo está diseñado siguiendo principios de claridad y eficiencia, lo que facilita:

- La creación de la base de datos.
- La validación y reconstrucción de esquemas.
- La incorporación de datos tanto en entornos de prueba como en producción.

Esto asegura que los modelos sean reutilizables y puedan evolucionar sin comprometer el rendimiento del sistema.

Carpeta Routes

Aquí se encuentran todas las rutas de la API, responsables de conectar los modelos, esquemas y la configuración del CRUD. Cada ruta está organizada de manera modular y bien documentada, lo que facilita su despliegue y visualización en Swagger UI. Esto proporciona una interfaz interactiva para probar y validar las funcionalidades de la API.

Carpeta Schemas

Esta carpeta define los esquemas de validación de datos utilizando Pydantic. Los esquemas especifican los tipos de datos y restricciones que se esperan para cada atributo, asegurando que solo datos válidos ingresen al sistema. Además, los esquemas vinculan los IDs y nombres de las clases necesarias para su correcto funcionamiento en las rutas.

Carpeta CRUD

Los archivos en esta carpeta implementan las operaciones CRUD: GET, POST, PUT y DELETE. Estas operaciones están diseñadas para interactuar con la base de datos a través de los modelos, garantizando una capa de abstracción clara entre la lógica de negocio y las interacciones con los datos. Esta división permite gestionar las peticiones de manera eficiente y centralizada, evitando duplicación de código.

Dado que cada componente del proyecto tiene una asignación claramente definida, esta organización permite una distribución eficiente que facilita tanto el análisis como la modificación en caso de conflictos. La estructura modular favorece la localización y aislamiento de problemas, asegurando que los cambios necesarios se realicen de manera controlada sin afectar el funcionamiento correcto de otros componentes.

Además, esta configuración promueve la escalabilidad y el crecimiento del proyecto, permitiendo la incorporación de nuevas funcionalidades sin interferir con las ya implementadas. Este enfoque modular no solo mejora la eficiencia en la resolución de problemas, sino que también asegura un mantenimiento constante y ágil, optimizando la colaboración entre los equipos de desarrollo Back-End y Front-End. Esto fomenta un flujo de trabajo más claro y organizado, facilitando la distribución de la aplicación de manera cohesiva.

Estructura del modelo para la tabla `tbb_aprobaciones`

El modelo de datos en Python que representa la tabla `tbb_aprobaciones` en una base de datos. Utiliza SQLAlchemy, un ORM (Object-Relational Mapping), para facilitar la interacción con la base de datos mediante objetos Python en lugar de escribir consultas SQL manuales.

```
models > tbb_aprobaciones.py > Estatus
1 from sqlalchemy import Column, Integer, String, ForeignKey, DateTime, Enum # type: ignore
2 from config.db import Base
3 from sqlalchemy.orm import relationship # type: ignore
4 import enum
```

ILUSTRACIÓN 67. IMPORTACIÓN DE MÓDULOS Y CLASES.

Se importan los módulos necesarios de SQLAlchemy para definir las columnas, tipos de datos, claves foráneas y relaciones. Se importa Base desde config.db, que es el punto de partida para la creación de modelos en SQLAlchemy. enum se utiliza para definir enumeraciones personalizadas, representando valores limitados para ciertos campos (por ejemplo, Estatus y Tipo).

```
6 class Estatus(str, enum.Enum):
7     Registrada = "Registrada"
8     Programada = "Programada"
9     Cancelada = "Cancelada"
10    Reprogramada = "Reprogramada"
11    En_Proceso = "En Proceso"
12    Realizada = "Realizada"
13    Aprobado = "Aprobado"
```

ILUSTRACIÓN 68. DEFINICIÓN DE ENUMERACIONES.

Representa los posibles estados de una aprobación, como "Registrada", "Programada", etc. La clase Estatus asegura que solo estos valores específicos puedan ser asignados a la columna Estatus.

```
15 class Tipo(str, enum.Enum):
16     Servicio_Interno = "Servicio Interno"
17     Traslados = "Traslados"
18     Subrogado = "Subrogado"
19     Administrativo = "Administrativo"
```

ILUSTRACIÓN 69. DEFINE TIPOS ESPECÍFICOS DE APROBACIONES, COMO "SERVICIO INTERNO" O "TRASLADOS".

Define tipos específicos de aprobaciones, como "Servicio Interno", "Traslados", Subrogado y Administrativo.

```
21 class Aprobaciones(Base):
22     __tablename__ = "tbb_aprobaciones"
23     id = Column(Integer, primary_key=True, index=True)
24     Personal_Medico_ID = Column(Integer, ForeignKey("tbb_personal_medico.Persona_ID"), nullable=False)
25     # Personal_Medico_ID = Column(Integer)
26     Solicitud_id = Column(Integer, ForeignKey("tbd_solicitudes.ID"), nullable=False)
27     # Solicitud_id = Column(Integer)
28     Comentario = Column(String(500))
29     Estatus = Column(Enum(Estatus))
30     Tipo = Column(Enum(Tipo))
31     Fecha_Registro = Column(DateTime)
32     Fecha_Actualizacion = Column(DateTime, nullable=True, default=None)
33
34     personal_medico = relationship("PersonalMedico", back_populates="aprobaciones")
35     solicitudes = relationship("Solicitud", back_populates="aprobaciones")
```

ILUSTRACIÓN 70. DEFINICIÓN DE LA CLASE APROBACIONES.

Representa la tabla tbb_aprobaciones en la base de datos. La clase hereda de Base, lo que permite que SQLAlchemy registre este modelo como una tabla.

- **ID:** Es la clave primaria de la tabla, que identifica de manera única cada registro.
- **Personal Médico:** Representa una relación con el modelo tbb_personal_medico. Cada aprobación está asociada a un miembro del personal médico, cuyo identificador (Persona_ID) es una clave foránea.
- **Solicitud:** Define una relación con el modelo tbd_solicitudes. Cada aprobación está vinculada a una solicitud específica.
- **Comentario:** Almacena comentarios relacionados con la aprobación, con una longitud máxima de 500 caracteres.

- **Estatus y Tipo:** Restringen los valores de estas columnas a los definidos en las enumeraciones Estatus y Tipo.
- **Fechas:**
 1. **Fecha_Registro:** Almacena la fecha de creación del registro.
 2. **Fecha_Actualizacion:** Almacena la fecha de la última actualización, si aplica.
- **Relaciones:**
 1. **personal_medico:** Define una relación bidireccional con el modelo Personal Médico, lo que permite acceder a las aprobaciones asociadas desde el personal médico.
 2. **solicitudes:** Define una relación bidireccional con el modelo Solicitud, lo que permite acceder a las aprobaciones asociadas desde una solicitud.

Estructura del schemas para la tabla tbb_aprobaciones

Estos esquemas están diseñados para la gestión de aprobaciones en un sistema y trabajan en conjunto con el modelo de SQLAlchemy para garantizar integridad y consistencia de los datos en las operaciones de una API.

```
schemas > tbb_aprobaciones.py > Tipo
1 from pydantic import BaseModel
2 from datetime import datetime
3 from typing import Optional
4 import enum
```

ILUSTRACIÓN 71. IMPORTACIONES DEL SCHEMAS.

BaseModel: Clase base de Pydantic que permite la creación de modelos de datos con validación automática. **datetime:** Se utiliza para trabajar con fechas y horas. **Optional:** Indica que un campo puede ser opcional (puede tener valor o ser None). **enum:** Permite definir conjuntos de valores constantes para campos específicos.

Al igual que el modelo se establecen los mismo campos que define los posibles estados para una aprobación, restringiendo los valores que puede tomar el campo Estatus. De igual forma se establece los mismos tipos utilizados para enum que se incorpora dentro del modelo.

```
22 class TbbAprobacionesBase(BaseModel):
23     Personal_Medico_ID: int
24     Solicitud_id: int
25     Comentario: str
26     Estatus: Estatus
27     Tipo: Tipo
28     Fecha_Registro: datetime
29     Fecha_Actualizacion: Optional[datetime]
```

ILUSTRACIÓN 72. ESQUEMA BASE (TBBAPROBACIONESBASE).

Personal_Medico_ID: Identificador del personal médico asociado; **Solicitud_id:** Identificador de la solicitud asociada; **Comentario:** Texto para observaciones o notas relacionadas con la aprobación; **Estatus:** Estado actual de la aprobación (valores definidos por la enumeración Estatus); **Tipo:** Tipo de aprobación (valores definidos por la enumeración Tipo); **Fecha_Registro:** Fecha en que se creó el registro; **Fecha_Actualizacion:** Fecha de la última actualización del registro (opcional).

```

31 class TbbAprobacionesCreate(TbbAprobacionesBase):
32     pass
33
34 class TbbAprobacionesUpdate(TbbAprobacionesBase):
35     pass
36
37     class Config:
38         from_attributes = True
39
40 class TbbAprobaciones(TbbAprobacionesBase):
41     id: int
42
43     class Config:
44         from_attributes = True

```

ILUSTRACIÓN 73. ESQUEMAS ESPECÍFICOS.

TbbAprobacionesCreate: Hereda de TbbAprobacionesBase, Representa el esquema para la creación de una nueva aprobación. No requiere campos adicionales más allá de los definidos en TbbAprobacionesBase.

TbbAprobacionesUpdate: También hereda de TbbAprobacionesBase, Representa el esquema para la actualización de aprobaciones existentes, La configuración `from_attributes = True` permite que Pydantic trabaje con modelos de SQLAlchemy directamente, mapeando atributos de modelos a los esquemas.

TbbAprobaciones: Amplía TbbAprobacionesBase añadiendo el campo `id`, que representa el identificador único del registro, Es utilizado para representar aprobaciones completas, incluyendo su identificador único.

El esquema garantiza que los datos cumplan con las reglas definidas para tipos de datos, valores permitidos entre otros, antes de que estos sean procesados.

La separación del esquema base, creación y de actualización permiten una claridad y reutilización del código.

La configuración `from_attributes` implementada dentro del desarrollo con RestAPI, asegura una integración eficiente con lo modelos de SQLAlchemy dentro de este framework.

Estructura del routes para la tabla tbb_aprobaciones

Estas rutas permiten realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre el modelo tbb_aprobaciones.

```
routes > tbb_aprobaciones.py > read_aprobacion
1 from fastapi import APIRouter, HTTPException, Depends # type: ignore
2 from sqlalchemy.orm import Session # type: ignore
3 import crud.tbb_aprobaciones, config.db, schemas.tbb_aprobaciones, models.tbb_aprobaciones
4 from typing import List
```

ILUSTRACIÓN 74. IMPORTACIONES QUE SE VINCULAN CON EL CRUD, SCHEMAS Y MODELOS PARA LA TABLA TBB_APROBACIONES.

APIRouter: Permite agrupar rutas relacionadas en un módulo independiente; **HTTPException:** Se utiliza para devolver errores HTTP personalizados; **Depends:** Gestiona dependencias, como la conexión a la base de datos; **Session:** Permite interactuar con la base de datos mediante SQLAlchemy; **List:** Tipo de dato para listas, usado en las respuestas de las rutas.

```
6 tbb_aprobaciones = APIRouter()
7
8 models.tbb_aprobaciones.Base.metadata.create_all(bind=config.db.engine)
9
10 def get_db():
11     db = config.db.SessionLocal()
12     try:
13         yield db
14     finally:
15         db.close()
```

ILUSTRACIÓN 75. CONFIGURACIÓN INICIAL Y DEPENDENCIAS DE BASE DE DATOS.

APIRouter: Crea un grupo de rutas relacionadas con tbb_aprobaciones; **metadata.create_all:** Genera las tablas en la base de datos si aún no existen, basándose en los modelos definidos; **get_db:** Configura una conexión a la base de datos utilizando una sesión SQLAlchemy. Se asegura de cerrar la conexión después de usarla.

```
17 @tbb_aprobaciones.get("/tbb_aprobaciones/", response_model=List[schemas.tbb_aprobaciones.TbbAprobaciones], tags=["Tbb_Aprobaciones"])
18 def read_aprobaciones(db: Session = Depends(get_db)):
19     db_aprobaciones = crud.tbb_aprobaciones.get_aprobaciones(db=db)
20     return db_aprobaciones
```

ILUSTRACIÓN 76. LEER TODAS LAS APROBACIONES.

Se utiliza el Método GET que devuelve todas las aprobaciones que se establecen en la base de datos y están almacenadas en la tabla tbb_aprobaciones, en base a la estructura que se establecio en el modelo y el schemas, response_model: Indica que la respuesta será una lista de objetos del esquema TbbAprobaciones. get_aprobaciones: Llama a la función CRUD para obtener los datos de la base de datos.

```
22 @tbb_aprobaciones.get("/tbb_aprobaciones/{id}", response_model=schemas.tbb_aprobaciones.TbbAprobaciones, tags=["Tbb_Aprobaciones"])
23 def read_aprobacion(id: int, db: Session = Depends(get_db)):
24     db_aprobacion = crud.tbb_aprobaciones.get_aprobacion(db=db, id=id)
25     if db_aprobacion is None:
26         raise HTTPException(status_code=404, detail="Aprobación no encontrada")
27     return db_aprobacion
```

ILUSTRACIÓN 77. LEER UNA APROBACIÓN ESPECÍFICA.

Nuevamente el Método **GET** devuelve una aprobación específica vinculándola con su ID como identificador único, esto permitirá guiar la búsqueda que se estableció en el esquema para los patrones UPDATE, Si no se encuentra, lanza una excepción 404 con un mensaje de error, como manejo de control de errores, notificando que la aprobación no se encontró.

```
29 @tbb_aprobaciones.post("/tbb_aprobaciones/", response_model=schemas.tbb_aprobaciones.TbbAprobaciones, tags=["Tbb_Aprobaciones"])
30 def create_tbb_aprobacion(tbb_aprobacion: schemas.tbb_aprobaciones.TbbAprobacionesCreate, db: Session = Depends(get_db)):
31     return crud.tbb_aprobaciones.create_aprobacion(db=db, aprobacion=tbb_aprobacion)
```

ILUSTRACIÓN 78. CREAR UNA NUEVA APROBACIÓN.

Se utiliza el Método **POST** para crear una nueva aprobación, esto recibe un objeto basado en el esquema TbbAprobacionesCreate y lo pasa a la función CRUD para guardarlo en la base de datos.

```
33 @tbb_aprobaciones.put("/tbb_aprobaciones/{id}", response_model=schemas.tbb_aprobaciones.TbbAprobaciones, tags=["Tbb_Aprobaciones"])
34 def update_tbb_aprobacion(id: int, tbb_aprobacion: schemas.tbb_aprobaciones.TbbAprobacionesUpdate, db: Session = Depends(get_db)):
35     db_tbb_aprobacion = crud.tbb_aprobaciones.update_aprobacion(db=db, id=id, aprobacion=tbb_aprobacion)
36     if db_tbb_aprobacion is None:
37         raise HTTPException(status_code=404, detail="Aprobación no encontrada, no se pudo actualizar")
38     return db_tbb_aprobacion
```

ILUSTRACIÓN 79. ACTUALIZAR UNA APROBACIÓN.

Nuevamente la aplicación del Método **PUT** permite actualizar una aprobación existente, si no se encuentra el registro, lanza una excepción 404, esto para el control de errores envía una notificación como indicador e identificador de donde se está generando el error y la leyenda correspondiente.

```
40 @tbb_aprobaciones.delete("/tbb_aprobaciones/{id}", response_model=schemas.tbb_aprobaciones.TbbAprobaciones, tags=["Tbb_Aprobaciones"])
41 def delete_aprobacion(id: int, db: Session = Depends(get_db)):
42     db_tbb_aprobacion = crud.tbb_aprobaciones.delete_aprobacion(db=db, id=id)
43     if db_tbb_aprobacion is None:
44         raise HTTPException(status_code=404, detail="Aprobación no encontrada, no se pudo eliminar")
45     return db_tbb_aprobacion
```

ILUSTRACIÓN 80. ELIMINAR UNA APROBACIÓN.

El método **DELETE** elimina una aprobación específica mediante su **ID**. Esta identificación garantiza que la aprobación vinculada a otras tablas relacionadas se gestione adecuadamente, permitiendo la validación de la transacción. Una vez completada, el método devuelve el objeto eliminado. Si no se encuentra el registro, se genera una excepción con el código **404**, indicando que la aprobación no fue encontrada.

Estructura del CRUD para la tabla **tbb_aprobaciones**

La estructura CRUD (Create, Read, Update, Delete) implementada proporciona las operaciones básicas para manejar la tabla **tbb_aprobaciones** en la base de datos mediante SQLAlchemy. Al igual que las estructuras anteriores se importan el modelo y el schemas, previamente configurados que ayudaran a la lógica de construcción del CRUD.

```

6  def get_aprobaciones(db: Session):
7      try:
8          return db.query(models.tbb_aprobaciones.Aprobaciones).all()
9      except SQLAlchemyError as e:
10         print(f"Error al obtener aprobaciones: {e}")
11         return []
12
13  def get_aprobacion(db: Session, id: int):
14      try:
15          return db.query(models.tbb_aprobaciones.Aprobaciones).filter(models.tbb_aprobaciones.Aprobaciones.id == id).first()
16      except SQLAlchemyError as e:
17         print(f"Error al obtener aprobacion con ID {id}: {e}")
18         return None

```

ILUSTRACIÓN 81. CONFIGURACIÓN GET_APROBACIONES.

get_aprobaciones: Recuperar todas las aprobaciones almacenadas en la tabla, Utiliza db.query para realizar una consulta que devuelve todas las filas de la tabla Aprobaciones, Si ocurre algún error durante la consulta, captura la excepción y devuelve una lista vacía como medida de seguridad.

get_aprobacion: Recuperar una aprobación específica mediante su ID, Filtra las filas de la tabla Aprobaciones usando el atributo id y devuelve la primera coincidencia encontrada, Si no encuentra un registro o ocurre un error, devuelve None.

```

20  def create_aprobacion(db: Session, aprobacion: schemas.tbb_aprobaciones.TbbAprobacionesCreate):
21      try:
22         db_aprobacion = models.tbb_aprobaciones.Aprobaciones(
23             Personal_Medico_ID=aprobacion.Personal_Medico_ID,
24             Solicitud_id=aprobacion.Solicitud_id,
25             Comentario=aprobacion.Comentario,
26             Estatus=aprobacion.Estatus,
27             Tipo=aprobacion.Tipo,
28             Fecha_Registro=aprobacion.Fecha_Registro,
29             Fecha_Actualizacion=aprobacion.Fecha_Actualizacion,
30         )
31         db.add(db_aprobacion)
32         db.commit()
33         db.refresh(db_aprobacion)
34         return db_aprobacion
35     except SQLAlchemyError as e:
36         print(f"Error al crear aprobacion: {e}")
37         db.rollback()
38         return None

```

ILUSTRACIÓN 82. CONFIGURACIÓN DE CREACIÓN DE APROBACIÓN.

create_aprobacion: Crear una nueva aprobación en la base de datos, Recibe un objeto del esquema TbbAprobacionesCreate como entrada, Crea una instancia del modelo Aprobaciones con los datos proporcionados, agrega la instancia a la sesión (db.add), guarda los cambios (db.commit) y actualiza el estado del objeto (db.refresh), Si ocurre algún error, revierte los cambios (db.rollback) y devuelve None.

```
40 def update_aprobacion(db: Session, id: int, aprobacion: schemas.tbb_aprobaciones.TbbAprobacionesUpdate):
41     try:
42         db_aprobacion = db.query(models.tbb_aprobaciones.Aprobaciones).filter(models.tbb_aprobaciones.Aprobaciones.id == id).first()
43         if db_aprobacion:
44             db_aprobacion.Personal_Medico_ID = aprobacion.Personal_Medico_ID
45             db_aprobacion.Solicitud_id = aprobacion.Solicitud_id
46             db_aprobacion.Comentario = aprobacion.Comentario
47             db_aprobacion.Estatus = aprobacion.Estatus
48             db_aprobacion.Tipo = aprobacion.Tipo
49             db_aprobacion.Fecha_Registro = aprobacion.Fecha_Registro
50             db_aprobacion.Fecha_Actualizacion = aprobacion.Fecha_Actualizacion
51             db.commit()
52             db.refresh(db_aprobacion)
53             return db_aprobacion
54         else:
55             return None
56     except SQLAlchemyError as e:
57         print(f"Error al actualizar aprobacion con ID {id}: {e}")
58         db.rollback()
59         return None
```

ILUSTRACIÓN 83. ACTUALIZACIÓN DE LA APROBACIÓN.

update_aprobacion: Actualizar una aprobación existente basada en su ID, Busca la aprobación utilizando su ID con filter y first, Si encuentra el registro, actualiza sus atributos con los datos recibidos, Guarda los cambios en la base de datos y devuelve la instancia actualizada, Si el registro no existe, devuelve None, En caso de error, revierte los cambios y devuelve None.

```
61 def delete_aprobacion(db: Session, id: int):
62     try:
63         db_aprobacion = db.query(models.tbb_aprobaciones.Aprobaciones).filter(models.tbb_aprobaciones.Aprobaciones.id == id).first()
64         if db_aprobacion:
65             db.delete(db_aprobacion)
66             db.commit()
67             return db_aprobacion
68         else:
69             return None
70     except SQLAlchemyError as e:
71         print(f"Error al eliminar aprobacion con ID {id}: {e}")
72         db.rollback()
73         return None
```

ILUSTRACIÓN 84. ELIMINACIÓN DE APROBACIÓN.

delete_aprobacion: Eliminar una aprobación específica por su ID, Busca el registro usando el ID proporcionado, Si lo encuentra, lo elimina con db.delete y guarda los cambios con db.commit, devuelve la instancia eliminada como confirmación, Si no se encuentra o hay un error, revierte la transacción y devuelve None.

Gestión de Back-End dentro del controlador Swagger

Swagger UI es una herramienta que facilita la documentación y prueba de las APIs. En FastAPI, esta funcionalidad está integrada de manera nativa, proporcionando automáticamente una interfaz gráfica interactiva basada en la especificación OpenAPI.

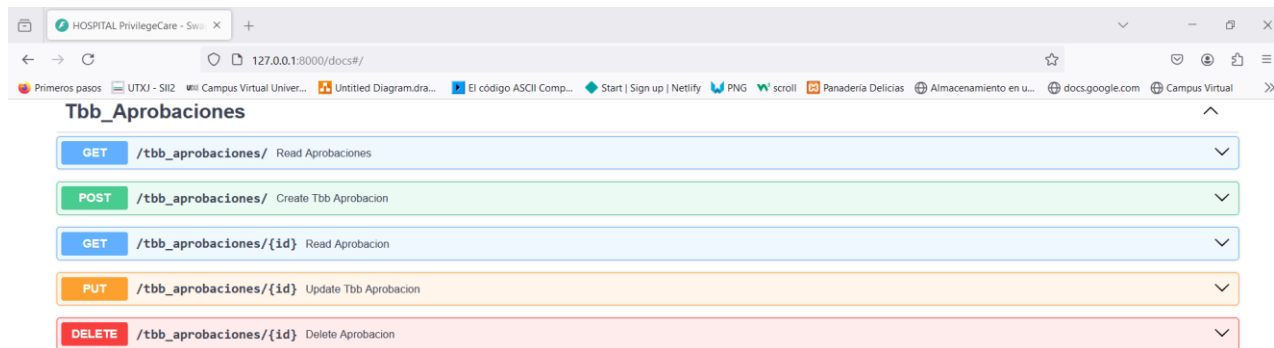


ILUSTRACIÓN 85. VISUALIZACIÓN DE LA TABLA TBB_APROBACIONES DENTRO DEL ENTORNO SWAGGER UI.

Cada endpoints definido en el archivo del controlador (tbb_aprobaciones) aparecerá automáticamente en Swagger UI, junto con su descripción y parámetros.

GET /tbb_aprobaciones/: Obtener todas las aprobaciones, Muestra los datos retornados en un formato JSON, facilitando las pruebas.

GET /tbb_aprobaciones/{id}: Recuperar una aprobación específica mediante su ID, solicita ingresar el parámetro id directamente en la interfaz, permite probar fácilmente si un ID válido retorna datos correctos.

POST /tbb_aprobaciones/: Crear una nueva aprobación, Proporciona un formulario para llenar los datos necesarios (basados en el esquema TbbAprobacionesCreate). Genera automáticamente ejemplos de entrada válidos.

PUT /tbb_aprobaciones/{id}: Actualizar una aprobación existente, Combina la entrada del id con los datos a modificar, verifica en tiempo real si los cambios son válidos antes de enviarlos.

DELETE /tbb_aprobaciones/{id}: Eliminar una aprobación, solicita el ID y ejecuta la operación, muestra el objeto eliminado o el error correspondiente.

Endpoints creados en Swagger

A continuación, se muestra la estructura Principal del API que permitirá enrutar mediante la generación de Endpoints o rutas HTTP para poder conectar el Back-End con el Front-End:

Ruta del Back-End: <https://back-end-hospital2-0.onrender.com/docs>

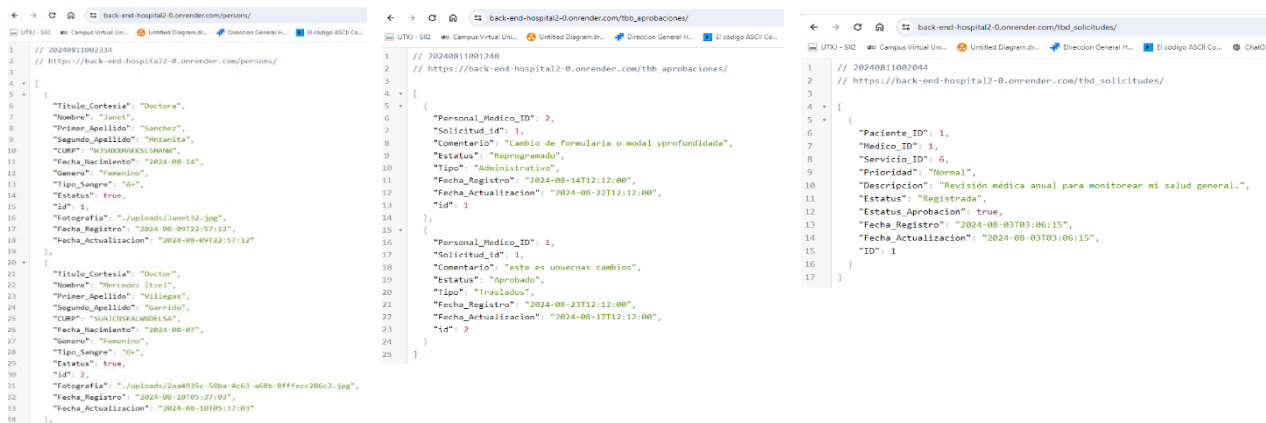
Endpoints principales involucrados en Aprobaciones:

https://back-end-hospital2-0.onrender.com/tbb_aprobaciones/?skip=0&limit=10

https://back-end-hospital2-0.onrender.com/tbd_solicitudes/?skip=0&limit=10

https://back-end-hospital2-0.onrender.com/usuario_rol/?skip=0&limit=10

Una vez completado el desarrollo del Back-End, el siguiente paso es integrar los endpoints creados con la estructura de la vista en el Front-End. Para que el Front-End sea funcional, se utiliza la API para interactuar con los endpoints, que almacenan y recuperan información en formato JSON. La base de datos MySQL define la arquitectura de las tablas encargadas de guardar los datos, mientras que la REST API facilita la gestión dinámica de la información, transformándola en archivos JSON que pueden ser consumidos por el Front-End.



```

1 // 20240811002334
2 // https://back-end-hospital2-0.onrender.com/persons/
3
4 {
5   "Titulo_Cortesia": "Doctora",
6   "Nombre": "Janet",
7   "Primer_Apellido": "Sancho",
8   "Segundo_Apellido": "Reza",
9   "CURP": "HJ0808080808080808",
10  "Fecha_Nacimiento": "2024-08-14",
11  "Genero": "Femenino",
12  "Tipo_Sangre": "A+",
13  "Estatus": true,
14  "id": 1,
15  "Fotografia": "/uploads/janet32.jpg",
16  "Fecha_Registro": "2024-08-09T12:07:12",
17  "Fecha_Actualizacion": "2024-08-09T12:07:12"
18 }
19
20 {
21   "Titulo_Cortesia": "Doctor",
22   "Nombre": "Herodes Itzel",
23   "Primer_Apellido": "Villagosa",
24   "Segundo_Apellido": "Serrano",
25   "CURP": "HJ0808080808080808",
26   "Fecha_Nacimiento": "2024-08-07",
27   "Genero": "Masculino",
28   "Tipo_Sangre": "O+",
29   "Estatus": true,
30   "id": 2,
31   "Fotografia": "/uploads/20240811002334-50ba-4c03-a0b0-8ff0cc286c3.jpg",
32   "Fecha_Registro": "2024-08-10T05:27:03",
33   "Fecha_Actualizacion": "2024-08-10T05:27:03"
34 }
35
36 }
  
```

```

1 // 20240811002448
2 // https://back-end-hospital2-0.onrender.com/tbb_aprobaciones/
3
4 {
5   "Personal_Medico_ID": 2,
6   "Solicitud_ID": 1,
7   "Comentario": "Cambio de formulario o modal y profundidad",
8   "Estatus": "Reprogramado",
9   "Tipo": "Administrativo",
10  "Fecha_Registro": "2024-08-14T12:12:00",
11  "Fecha_Actualizacion": "2024-08-23T12:12:00",
12  "id": 1
13 }
14
15 {
16   "Personal_Medico_ID": 1,
17   "Solicitud_ID": 1,
18   "Comentario": "este es un nuevo cambio",
19   "Estatus": "Aprobado",
20   "Tipo": "Traslados",
21   "Fecha_Registro": "2024-08-23T12:12:00",
22   "Fecha_Actualizacion": "2024-08-23T12:12:00",
23   "id": 2
24 }
25
26 }
  
```

```

1 // 20240811002044
2 // https://back-end-hospital2-0.onrender.com/tbd_solicitudes/
3
4 {
5   "Paciente_ID": 1,
6   "Medico_ID": 1,
7   "Servicio_ID": 6,
8   "Prioridad": "Normal",
9   "Descripcion": "Revisión médica anual para monitorear mi salud general.",
10  "Estatus": "Registrada",
11  "Estatus_Aprobacion": true,
12  "Fecha_Registro": "2024-08-03T03:06:15",
13  "Fecha_Actualizacion": "2024-08-03T03:06:15",
14  "ID": 1
15 }
16
17 }
  
```

ILUSTRACIÓN 86. ENDPOINTS DE TBB_PERSONAS, TBB_APROBACIONES Y TBD_SOLICITUDES.

Para la gestión del despliegue, como parte del modelo de implementación DevOps, se vinculó el funcionamiento del proyecto a un repositorio en GitHub, lo que permite almacenar todas las funciones directamente en la nube. Además, la relación con el repositorio y las implementaciones necesarias identifica de manera única la estructura del proyecto mediante el uso de tokens de autenticación, así como la gestión de claves y entornos virtuales. Esto facilita el control desde la nube, abarcando desde el servidor hasta la conexión de red.

Configuración de parámetros de la base de datos desplegada en Aiven

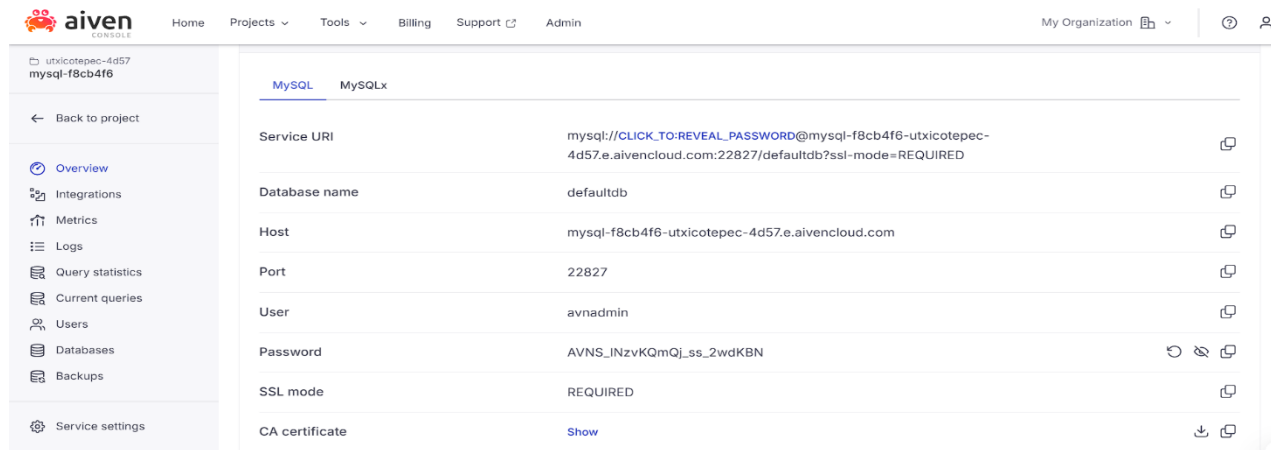


ILUSTRACIÓN 87. CONFIGURACIÓN DE PERMISOS, CLAVES Y USUARIO DENTRO DE AIVEN.

Con los parámetros de configuración del entorno externo, se establecen dentro del archivo de conexión a la base de datos, en la configuración de la API REST. Es fundamental tomar medidas de precaución, ya que las credenciales del proyecto podrían verse comprometidas si su visualización no está debidamente protegida. Este aspecto resulta crucial para la correcta configuración de las variables de entorno de la aplicación.

Para aplicar dicha configuración se estructura el archivo `bd.py` dentro del proyecto:

```
config > db.py > ...
1 from sqlalchemy import create_engine # type: ignore
2 from sqlalchemy.ext.declarative import declarative_base
3 from sqlalchemy.orm import sessionmaker # type: ignore
4 from dotenv import load_dotenv # type: ignore
5 import os
```

ILUSTRACIÓN 88. IMPORTACIÓN DE MÓDULOS.

sqlalchemy: Biblioteca que permite interactuar con bases de datos SQL de manera programática; **create_engine**: Crea una conexión con la base de datos; **declarative_base**: Se utiliza para definir modelos (tablas de la base de datos); **sessionmaker**: Crea sesiones para interactuar con la base de datos; **dotenv**: Carga variables de entorno desde un archivo `.env` para evitar que datos sensibles como contraseñas se encuentren directamente en el código; **os**: Permite acceder a las variables de entorno del sistema.

```
7 # Cargar las variables de entorno desde el archivo .env
8 load_dotenv()
9 #se esta entrando en la base de datos con
10 # usuario: root y contraseña: 1234, en el puerto de conexion de MySQL 3306
11 SQLALCHEMY_DATABASE_URL = os.getenv("SQLALCHEMY_DATABASE_URL")
12 # Crear el motor de conexión
13 engine = create_engine(SQLALCHEMY_DATABASE_URL)
```

ILUSTRACIÓN 89. CARGA DE LAS VARIABLES DE ENTORNO.

load_dotenv(): Carga las variables definidas en el archivo .env en las variables de entorno del sistema; **os.getenv("SQLALCHEMY_DATABASE_URL")**: Obtiene la URL de conexión de la base de datos desde una variable de entorno llamada SQLALCHEMY_DATABASE_URL. **create_engine**: Configura la conexión a la base de datos utilizando la URL especificada en SQLALCHEMY_DATABASE_URL.

```
15 #Probar la conexión
16 try:
17     connection = engine.connect()
18     print("Conexión exitosa a la base de datos Remota")
19     connection.close()
20 except Exception as e:
21     print(f"Error al conectar a la base de datos: {e}")
22
23 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
24 Base = declarative_base()
25
```

ILUSTRACIÓN 90. PRUEBA DE CONEXIÓN.

Intenta conectarse a la base de datos utilizando `engine.connect()`. Si la conexión es exitosa: Muestra un mensaje: "Conexión exitosa a la base de datos Remota", si no es así Cierra la conexión con `connection.close()`. Si ocurre un error, lo captura y lo imprime.

Configuración de la sesión local: **sessionmaker** se utiliza para crear sesiones que interactúan con la base de datos, **autocommit=False** indica que las transacciones no se confirman automáticamente, **autoflush=False** evita que los cambios pendientes se envíen automáticamente a la base de datos, **bind=engine** vincula el motor de la base de datos con las sesiones creadas, **declarative_base** permite definir las tablas de la base de datos como clases de Python.

Configuración del entorno de despliegue del back-end con Servicio web desde la plataforma Render

Render Dashboard es la interfaz web de la plataforma Render, que permite a los desarrolladores gestionar y supervisar sus aplicaciones, bases de datos, servicios y despliegues en la nube. Render es una plataforma de alojamiento de aplicaciones y servicios en la nube que simplifica el proceso de despliegue e implementación de aplicaciones, APIs, sitios web estáticos y bases de datos.

Render y DevOps están relacionados en el contexto de la implementación y gestión de aplicaciones en la nube. Mientras Render es una plataforma que simplifica el proceso de despliegue y alojamiento de aplicaciones, DevOps es una metodología que combina desarrollo y operaciones para entregar software de forma rápida, eficiente y confiable.

Para gestionar la vinculación de render, es necesario que el proyecto este contenido dentro de un repositorio de GitHub, permitiendo que render pueda detectar la estructura del proyecto y permita instalarlo en la nube.

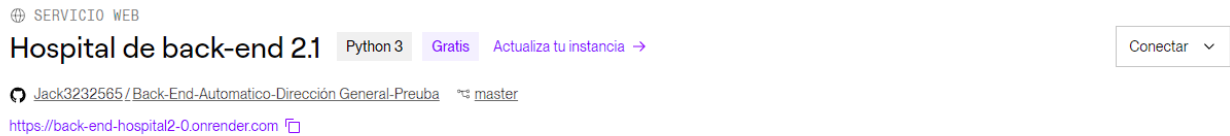


ILUSTRACIÓN 91. VINCULACIÓN DEL PROYECTO CON EL REPOSITORIO ALMACENADO EN GITHUB.

Render implementa medidas de seguridad esenciales al ocultar cualquier vinculación con accesos restringidos y la obtención de credenciales que puedan comprometer la integridad del proyecto. La configuración requiere insertar el vínculo `SQLALCHEMY_DATABASE_URL`, que establece una conexión directa con la base de datos remota proporcionada por AIVEN, y permite construir el proyecto de Python de manera segura y eficiente. En Render, las variables de entorno se integran como elementos de solo lectura, sirviendo exclusivamente para procesar el servicio. Esto asegura la validación adecuada del proyecto y facilita su despliegue en la nube.

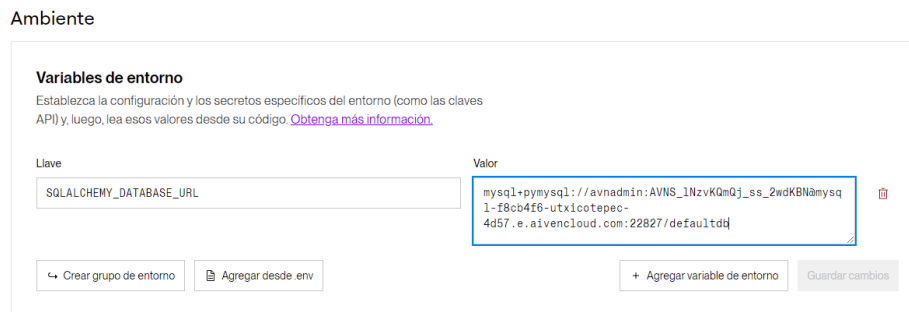


ILUSTRACIÓN 92. IMPLEMENTACIÓN DE VARIABLES DE ENTORNO PARA EL DESPLIEGUE DEL PROYECTO.

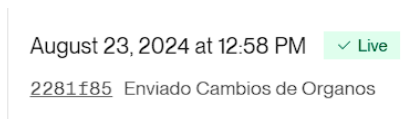


ILUSTRACIÓN 93. DESPLIEGUE EN LÍNEA.

La validación se efectúa cuando el entorno que se ha desplegado está en línea, listo para ser consultado, y como evidencia se muestra el ultimo commit realizado en el repositorio.

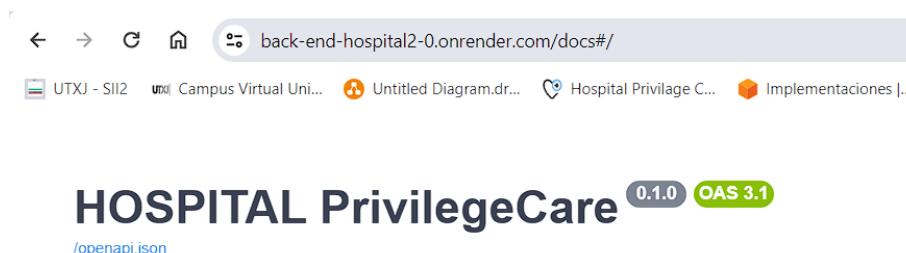


ILUSTRACIÓN 94. REPLIEGUE DE SWAGGER UI DE MANERA REMOTA.

El despliegue habilita el entorno previamente gestionado de forma local, permitiendo que Swagger UI sea accesible desde cualquier dispositivo para estructurar peticiones. Esto expone todas las configuraciones realizadas en el Back-End, haciendo que los endpoints estén disponibles mediante la vinculación a una URL específica. Cada endpoint se asocia con el nombre correspondiente a la solicitud de una tabla determinada, alojada de manera remota en AIVEN. Esta configuración permite consultar la información desde un editor de bases de datos, como MySQL, gracias a los permisos establecidos en AIVEN. Asimismo, facilita la visualización de esquemas de los modelos y su interacción con herramientas como Navicat.

Desarrollo de la Aplicación con el entorno Flutter

Flutter es un framework de desarrollo de aplicaciones de código abierto creado por Google. Permite a los desarrolladores construir aplicaciones nativas multiplataforma con un solo código base, lo que significa que el mismo código puede usarse para crear aplicaciones que se ejecuten en: Android, iOS, Web, Windows, macOS, Linux.

Flutter utiliza un lenguaje llamado Dart, diseñado por Google, para construir interfaces gráficas altamente dinámicas y fluidas. Las interfaces están compuestas por *widgets*, que son elementos reutilizables para construir la UI, como botones, listas, y diseños. Utiliza su propio motor de renderizado gráfico llamado Skia, que dibuja directamente en la pantalla, eliminando dependencias de componentes nativos y ofreciendo un rendimiento cercano al de aplicaciones nativas. Permite a los desarrolladores realizar cambios en el código y ver los resultados casi instantáneamente sin reiniciar la aplicación. Esto acelera el proceso de desarrollo y depuración.

Aunque es multiplataforma, Flutter ofrece una apariencia y comportamiento muy similares a los de aplicaciones completamente nativas en cada sistema operativo. Incluye una gran cantidad de bibliotecas y paquetes que permiten añadir funcionalidades como acceso a hardware, manejo de bases de datos, autenticación, entre otras. Flutter es de código abierto y cuenta con una comunidad global en constante crecimiento, lo que facilita el acceso a recursos, tutoriales y soporte.

Este entorno de desarrollo permite realizar, un entorno limpio y acelerado, permitiendo la utilización de una infraestructura adecuada para los dispositivos móviles, aplicando una distribución adaptativa para múltiples pantallas de dispositivos móviles.

El proyecto desarrollado en el entorno flutter se caracteriza por la construcción de sus carpetas aisladas que permitan dar una identificación mas clara de lo que se está contrayendo dando un contexto unificado a la hora de mostrar la información, es por ello que la distribución se ha considerado de la siguiente manera:

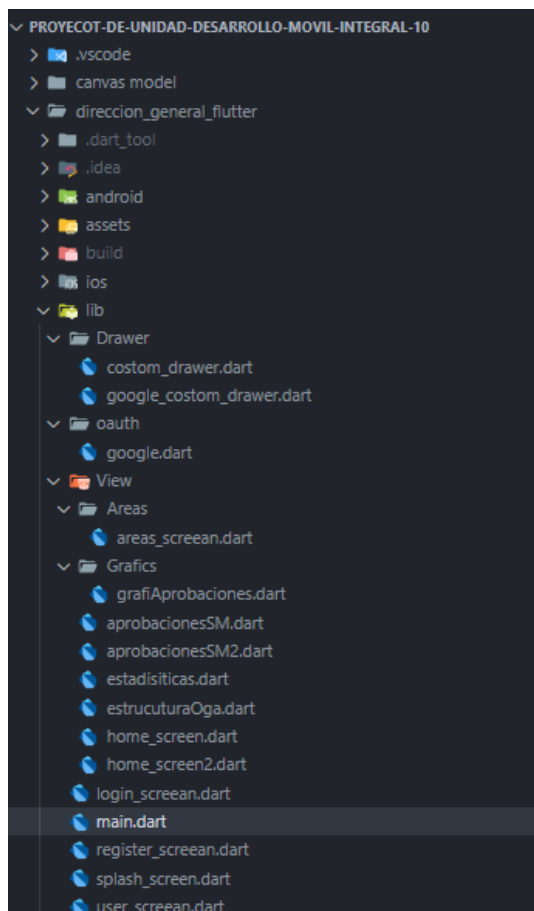


ILUSTRACIÓN 95. ESTRUCTURA DE CARPETAS DEL PROYECTO DE LA APP EN FLUTTER.

Toda la lógica que incorpora el comportamiento se encuentra dentro de la carpeta lib, la carpeta lib es el lugar principal donde se almacena el código fuente de tu aplicación. Su propósito principal es contener toda la lógica, widgets, y funciones necesarias para el desarrollo de la aplicación.

La carpeta lib es el lugar principal donde se almacena el código fuente de tu aplicación. Su propósito principal es contener toda la lógica, widgets, y funciones necesarias para el desarrollo de la aplicación.

Cada una de las capetas incorpora una estructura única, en la cual se integran estructuras de Widgets que maquetan la visualización de cada vista dentro de la aplicación, a continuación, se describen las principales estructuras involucradas en la aplicación.

Carpeta Drawer

```

custom_drawer.dart M X
direccion_general_flutter > lib > Drawer > custom_drawer.dart > fetchUserData
1 // custom_drawer.dart
2 import 'package:direccion_general_flutter/View/estadisticas.dart';
3 import 'package:flutter/material.dart';
4 import 'package:google_fonts/google_fonts.dart';
5 import '../View/home_screen.dart';
6 import 'package:direccion_general_flutter/View/aprobacionesSM.dart';
7 import 'package:http/http.dart' as http;
8 import 'dart:convert';
9
10 class CustomDrawer extends StatelessWidget {
11   final Future<Map<String, dynamic>> userDataFuture;
12   final String area;
13   final int personaId;
14   final Function logout;
15
16   const CustomDrawer({
17     super.key,
18     required this.userDataFuture,
19     required this.area,
20     required this.personaId,
21     required this.logout,
22   });
23
24   @override
25   Widget build(BuildContext context) { ...
26 }
27
28 Future<Map<String, dynamic>> fetchUserData(int personaId) async {
29   try {
30     final usuarioResponse = await http.get(
31       Uri.parse('https://back-end-hospital2-0.onrender.com/tbb_usuarios/'),
32     );
33     final personaResponse = await http.get(
34       Uri.parse('https://back-end-hospital2-0.onrender.com/persons/'),
35     );
36
37     // Verifica el estado de las respuestas
38     // print('Usuario Response Status: ${usuarioResponse.statusCode}');
39     // print('Persona Response Status: ${personaResponse.statusCode}');
40
41     if (usuarioResponse.statusCode == 200 && personaResponse.statusCode == 200) {
42       final List<dynamic> usuarios = jsonDecode(usuarioResponse.body);
43       final List<dynamic> personas = jsonDecode(personaResponse.body);
44
45       final userData = usuarios.firstWhere((user) => user['ID'] == personaId, orElse: () => null);
46       final personaData = personas.firstWhere((persona) => persona['id'] == personaId, orElse: () => null);
47
48       if (userData == null) {
49         throw Exception("User with ID $personaId not found");
50       }
51       if (personaData == null) {
52         throw Exception("Persona with ID $personaId not found");
53       }
54
55       return {
56         'correo': userData['Correo_Electronico'],
57         'fotografia': personaData['Fotografia'],
58       };
59     }
60
61     throw Exception("Failed to load user data");
62   } catch (e) {
63     // Manejo de excepciones
64     // print("Error fetching user data: $e");
65     throw Exception("Error fetching user data");
66   }
67 }

```

ILUSTRACIÓN 96. ARCHIVO CUSTOM_DRAWER.DART.

Custom_drawer.dart, define un widget personalizado en Flutter llamado CustomDrawer. Es un componente de menú lateral (drawer) que se utiliza para la navegación y para mostrar información del usuario.

El archivo comienza con las importaciones necesarias:

- Paquetes y bibliotecas esenciales: Incluye flutter/material.dart para la interfaz gráfica, google_fonts para fuentes personalizadas, y dart:convert para trabajar con datos JSON.
- Componentes personalizados: Importa vistas específicas como home_screen.dart, aprobacionesSM.dart, y estadísticas.dart, que representan distintas pantallas a las que se puede navegar desde el menú.
- HTTP: Usa la biblioteca http para realizar solicitudes a un backend remoto.

Clase CustomDrawer

Es una clase StatelessWidget que define un menú lateral. Este menú incluye:

Propiedades:

userDataFuture: Un Future que contiene los datos del usuario; area: Un texto que indica el área del usuario; personalId: Un identificador único para el usuario; logout: Una función que ejecuta la acción de cerrar sesión.

Constructor:

Se asegura de que todas las propiedades necesarias sean proporcionadas al crear una instancia del drawer.

Método build: Este método organiza los widgets dentro del menú: **Encabezado (DrawerHeader):** Usa un FutureBuilder para manejar datos asíncronos (userDataFuture). Dependiendo del estado del Future: **Cargando:** Muestra un indicador de carga (CircularProgressIndicator); **Error:** Muestra un mensaje de error; **Datos disponibles:** Renderiza información del usuario, incluyendo: Un logotipo, La fotografía del usuario (si está disponible). Si no, muestra un mensaje de texto, El correo electrónico y el área asignada.

Opciones de navegación (ListTile):

- **Inicio:** Navega a HomeScreen, pasándole el área y el ID del usuario. **Aprobaciones Servicio Médico:** Navega a AprobacionesScreen con parámetros similares.
- **Estadísticas:** Redirige a EstadisticaScreen.
- **Bitácora:** Actualmente no implementa acción específica.
- **Configuración:** Permite regresar al menú principal.
- **Salir:** Llama a la función de cierre de sesión proporcionada.

Pie de página:

Un texto que muestra los derechos de autor de la organización.

Función fetchUserData

Es una función asíncrona que recupera datos del usuario y su información personal desde un backend remoto:

Realiza solicitudes GET a dos endpoints:

- Uno para obtener datos de usuarios (tbb_usuarios).
- Otro para obtener datos personales (persons).

Procesa las respuestas:

- Decodifica los datos JSON.
- Busca el usuario y la persona correspondiente al personald.
- Si se encuentran, retorna un mapa con datos relevantes como el correo y la fotografía.
- Si no se encuentran o hay errores en las respuestas, lanza excepciones.

Manejo de errores:

- Los errores de red o inconsistencias en los datos se manejan con excepciones claras.

Características clave

Uso de FutureBuilder: Maneja el estado asíncrono de manera eficiente al renderizar distintos widgets dependiendo del estado del Future.

Personalización visual: Usa GoogleFonts para estilizar el texto y configura íconos personalizados con imágenes locales (assets).

Navegación entre pantallas: Implementa una navegación fluida entre diferentes vistas mediante Navigator.pushReplacement.

Integración backend: Conecta con un backend para obtener datos dinámicos del usuario.

Manejo de errores más descriptivo: Podría agregar mensajes de error específicos para que el usuario sepa qué está fallando.

```

google_custom_drawer.dart x
direccion_general_flutter > lib > Drawer > google_custom_drawer.dart > CustomDrawer2
1 import 'package:direccion_general_flutter/View/home_screen2.dart';
2 import 'package:flutter/material.dart';
3 import 'package:google_fonts/google_fonts.dart';
4 import 'package:google_sign_in/google_sign_in.dart';
5
6 class CustomDrawer2 extends StatelessWidget {
7   final String area;
8   final GoogleSignInAccount user;
9   final Function logout;
10
11   const CustomDrawer2({
12     super.key,
13     required this.area,
14     required this.user,
15     required this.logout,
16   });
17
18   @override
19   Widget build(BuildContext context) {
20     return Align(
21       alignment: Alignment.centerLeft,
22       child: FractionallySizedBox(
23         widthFactor: 0.7, // Drawer ocupará el 70% del ancho de la pantalla
24         child: Drawer(
25           child: Container(
26             color: Colors.white, // Fondo sólido blanco
27             child: Column(
28               children: [
29                 // Encabezado del Drawer
30                 DrawerHeader( // DrawerHeader ...
31
32                 // Opciones del Drawer
33                 Expanded( // Expanded ...
34
35                 // Pie de página
36                 Padding( // Padding ...
37
38               ],
39             ), // Column
40           ), // Container
41         ), // Drawer
42       ), // FractionallySizedBox
43     ); // Align
44   }
45
46   // Método para crear un ListTile estilizado
47   Widget _drawerTile(BuildContext context,
48     {required IconData icon, required String label, required VoidCallback onTap}) {
49     return ListTile(
50       leading: Icon(icon, color: const Color.fromARGB(255, 99, 105, 110)),
51       title: Text(
52         label,
53         style: GoogleFonts.comicNeue(fontSize: 16, color: Colors.black87),
54       ), // Text
55       onTap: onTap,
56     ); // ListTile
57   }
58 }

```

ILUSTRACIÓN 97. ARCHIVO GOOGLE_CUSTOM_DRAWER.DART.

Dentro del archivo se llama a la clase CustomDrawer2, que es un componente visual para Flutter encargado de construir un menú lateral personalizado. Este menú ofrece una interfaz intuitiva para la navegación de un usuario autenticado mediante Google Sign-In y se adapta para incluir detalles personales del usuario, como su foto y nombre, así como opciones de navegación esenciales. Vamos a desglosar su funcionamiento narrativamente:

Encabezado del Drawer

Cuando el componente se renderiza, la parte superior del menú, conocida como el "encabezado", muestra un diseño visual atractivo y organizado. Aquí encontramos lo siguiente:

- **Logo Institucional:** El encabezado comienza mostrando el logotipo de la organización (HPC-DG.png), representando la identidad visual de la app.
- **Foto del Usuario:** Si el usuario autenticado tiene una foto de perfil asociada a su cuenta de Google, esta se muestra dentro de un recorte circular. Si no hay foto disponible, se presenta una imagen predeterminada (default_avatar.png).
- **Bienvenida Personalizada:** Se saluda al usuario mostrando su nombre o un mensaje genérico ("Usuario") si no se dispone de su nombre.
- **Área del Usuario:** Se informa al usuario sobre la "área" en la que está asignado dentro del sistema, proporcionando contexto adicional.

Opciones del Menú

Bajo el encabezado, se encuentra una lista de opciones interactivas diseñadas como ListTile, cada una con su propósito y estilo distintivo:

- **Inicio:** Representado por un ícono de casa, avega al componente principal de la aplicación (HomeScreen2), que se personaliza con los datos del área y usuario autenticado, al hacer clic, el menú lateral se cierra automáticamente para regresar al contenido principal.
- **Configuración:** Utiliza un ícono de configuración, aunque no tiene una acción específica implementada en este momento, está diseñado para futuros ajustes o personalizaciones dentro de la aplicación.
- **Salir:** Se acompaña del ícono de cerrar sesión, su función principal es cerrar la sesión del usuario invocando un método proporcionado (logout), lo que asegura que el usuario sea desconectado de manera segura.

Pie de Página

Al final del menú, encontramos un pequeño mensaje institucional: "Dirección General © 2024". Este mensaje, colocado con un diseño minimalista y centrado, aporta un toque profesional y refuerza la marca de la organización.

Diseño y Responsividad

El diseño del CustomDrawer2 está cuidadosamente pensado para ser funcional y estéticamente agradable:

- **Ancho Ajustado:** El menú ocupa el 70% del ancho de la pantalla, asegurando que no domine toda la interfaz mientras permite un acceso cómodo a las opciones.
- **Colores y Tipografía:** Utiliza colores claros y una tipografía moderna (GoogleFonts.comicNeue) que refuerzan la accesibilidad y el atractivo visual.
- **Componentización:** Las opciones del menú (ListTile) están definidas mediante un método auxiliar `_drawerTile`, que simplifica la creación y reutilización de elementos estilizados.

Autenticación e Integración con Google

La clase interactúa directamente con la información proporcionada por la cuenta de Google del usuario autenticado. Esto incluye el manejo seguro de la foto del usuario y su nombre, asegurando una experiencia personalizada.

Este código implementa un menú lateral que no solo es funcional, sino que también se adapta al usuario autenticado. Está diseñado para integrarse perfectamente con la interfaz de Flutter y fomentar una experiencia de usuario intuitiva y profesional.

La clase CustomDrawer2 aprovecha la integración con Google Sign-In para extraer datos del usuario, como:

- Foto de perfil (`user.photoUrl`).
- Nombre del usuario (`user.displayName`).

Esto no solo mejora la personalización, sino que también simplifica la autenticación y el manejo de sesiones, haciendo que la experiencia sea más fluida y segura.

Privacidad: La clase está diseñada para evitar exponer información sensible del usuario, como su correo electrónico o ID, mostrando solo detalles básicos como el nombre y la foto.

Navegación Sencilla: Cada interacción con el drawer es intuitiva y eficiente, cerrando automáticamente el menú cuando se selecciona una opción, lo que asegura una experiencia continua para el usuario.

Carpeta Oauth

```

google.dart M X
direccion_general_flutter > lib > oauth > google.dart > ...
1 import 'dart:io'; // Para detectar si es Android o iOS
2 import 'package:flutter/foundation.dart'; // Para detectar si es web
3 import 'package:flutter/material.dart';
4 import 'package:google_fonts/google_fonts.dart';
5 import 'package:google_sign_in/google_sign_in.dart';
6 import 'package:direccion_general_flutter/View/home_screen2.dart';
7 import 'package:direccion_general_flutter/View/Areas/areas_screean.dart';
8
9 class GoogleScreen extends StatelessWidget {
10   const GoogleScreen({super.key});
11
12   @override
13   Widget build(BuildContext context) {
14     String? selectedArea;
15
16     return Scaffold(
17       appBar: AppBar(
18         title: const Text('Google Sign In'),
19       ), // AppBar
20       body: Center( // Center ...
21     ); // Scaffold
22   }
23 }
24
25 // Función para manejar la autenticación con Google
26 Future<void> _handleGoogleSignIn(BuildContext context, String selectedArea) async { The de
27   final user = await GoogleSignInApi.login();
28
29   if (user != null) { ...
30   } else {
31     // Maneja el caso de error o cancelación
32     ScaffoldMessenger.of(context).showSnackBar( Don't use 'BuildContext's across async gap
33       const SnackBar(content: Text('Inicio de sesión cancelado')),
34     );
35   }
36 }
37
38 class GoogleSignInButtonWeb extends StatelessWidget {
39   final VoidCallback onPressed;
40
41   const GoogleSignInButtonWeb({super.key, required this.onPressed});
42
43   @override
44   Widget build(BuildContext context) {
45     return ElevatedButton(
46       onPressed: onPressed,
47       child: const Text('Iniciar sesión con Google'),
48     ); // ElevatedButton
49   }
50 }
51
52 class GoogleSignInApi {
53   // Configura los client IDs para web y Android
54   static const _serverClientIdWeb =
55     '930523576022-5ndtgbclull8usogrloj4qlfqu8d8j.apps.googleusercontent.com';
56   static const _serverClientIdAndroid =
57     '510929384792-oql3avrc130ndcbvinfdd5bhm21fqsv7.apps.googleusercontent.com';
58
59   static GoogleSignIn get _googleSignIn { ...
60
61   static Future<GoogleSignInAccount?> login() async { ...
62
63   static Future<void> logout() => _googleSignIn.disconnect(); // Cierra la sesión de Google
64 }

```

ILUSTRACIÓN 98. ARCHIVO GOOGLE.DART.

La pantalla GoogleScreen es una interfaz simple pero poderosa, diseñada para manejar la autenticación con Google y la selección de un área específica dentro de una organización. Este diseño es ideal para aplicaciones empresariales donde el acceso depende tanto de la identidad del usuario como de su rol o departamento.

Características principales

Compatibilidad multiplataforma: Usa la biblioteca dart:io para diferenciar entre Android e iOS, Detecta si se ejecuta en la web mediante flutter/foundation.

Diseño del flujo de usuario: Selección de área: Obliga al usuario a elegir un área antes de iniciar sesión. Inicio de sesión con Google: Permite al usuario autenticarse mediante Google Sign-In, personalizando la experiencia basada en sus datos.

Navegación flexible: Una vez autenticado, el usuario es redirigido a HomeScreen2, pasando como argumentos el área seleccionada y los detalles del usuario autenticado.

Encabezado visual: Incluye un logo corporativo (HPC-DG.png) para reforzar la identidad de la organización, espaciado balanceado para un diseño limpio y profesional.

Selección de área: (AreaSelector) Un componente personalizable que permite seleccionar un área relevante. La selección es crucial, ya que define el contexto de las acciones del usuario dentro de la aplicación. Se asegura que el área esté seleccionada antes de proceder con el inicio de sesión.

Botón de Google Sign-In

- **Ícono:** Incluye un ícono de Google (icono_google.png) para mayor familiaridad.
- **Estilo:** Fondo blanco con texto negro, en línea con los lineamientos de diseño de Google, utiliza la fuente ComicNeue para un toque amigable.

Acción: Al presionar, se realiza la autenticación con Google y se valida que un área haya sido seleccionada.

Lógica de Google Sign-In

La autenticación con Google se maneja a través de la clase GoogleSignInApi, diseñada para adaptarse a diferentes plataformas:

Configuración de clientes: Web: Usa un client ID específico para autenticación en navegadores, **Android:** Usa un client ID adaptado para dispositivos móviles, **Otros:** Maneja plataformas adicionales de forma predeterminada.

Flujo de autenticación: Llama a `GoogleSignIn.signIn()` para iniciar sesión con la cuenta de Google, si la autenticación es exitosa, se devuelve un objeto `GoogleSignInAccount` que incluye detalles como nombre y foto de perfil, maneja errores y cancelaciones con mensajes amigables.

Cierre de sesión: Utiliza `disconnect` para cerrar la sesión del usuario, limpiando cualquier credencial almacenada localmente.

Navegación Condicional: Redirección a HomeScreen2, Una vez autenticado, se pasa el área seleccionada y los datos del usuario a la pantalla principal, si el usuario no selecciona un área, muestra un mensaje informativo para evitar un flujo incompleto.

Compatibilidad multiplataforma

El código está diseñado para ajustarse automáticamente según la plataforma:

- En la web, utiliza el client ID específico y permisos adicionales para manejar datos de usuario.
- En Android, sigue los estándares recomendados para autenticación en dispositivos móviles.
- Para otras plataformas, usa configuraciones genéricas.

Seguridad y Experiencia del Usuario

Mensajes informativos:

Utiliza `ScaffoldMessenger` para proporcionar retroalimentación inmediata al usuario, como la falta de selección de área o problemas de autenticación.

Estilo visual: La combinación de diseño minimalista con elementos visuales claros (logos e íconos) facilita la navegación, los colores y tipografía aseguran una experiencia moderna y atractiva.

Manejo de errores: Captura errores durante el inicio de sesión y proporciona detalles en el registro (`debugPrint`), lo que facilita la depuración.

Este diseño logra un equilibrio entre funcionalidad, seguridad y experiencia de usuario. Es flexible para ajustarse a diferentes plataformas y escalable para integrar más áreas o servicios en el futuro.

Entre las mejoras que espera en el desarrollo futuro es Permitir al usuario seleccionar varias áreas si su rol lo requiere, modificar `AreaSelector` para admitir listas o múltiples selecciones y enviar estas áreas a `HomeScreen2`.

Carpeta Áreas

```
areas_screan.dart X
direccion_general_flutter > lib > View > Areas > areas_screan.dart > ...
1 import 'package:flutter/material.dart';
2 import 'package:google_fonts/google_fonts.dart';
3 import 'package:http/http.dart' as http;
4 import 'dart:convert';
5
6 class AreaSelector extends StatefulWidget {
7   final Function(String?) onAreaSelected; // Callback para manejar el área seleccionada
8   const AreaSelector({super.key, required this.onAreaSelected});
9
10  @override
11  _AreaSelectorState createState() => _AreaSelectorState(); Invalid use of a private type in a public API.
12 }
13
14 class _AreaSelectorState extends State<AreaSelector> {
15   String? _selectedArea;
16   List<String> _areas = [];
17
18   @override
19   void initState() {
20     super.initState();
21     _fetchAreas();
22   }
23
24   Future<void> _fetchAreas() async {
25     try {
26       final response = await http.get(Uri.parse('https://back-end-hospital2-0.onrender.com/departamentos/'));
27       if (response.statusCode == 200) { ...
28         setState(() { ...
29       } else { ...
30     } catch (e) {
31       ScaffoldMessenger.of(context).showSnackBar( Don't use 'BuildContext's across async gaps. Try rewriting
32     }
33   }
34
35   @override
36   Widget build(BuildContext context) {
37     return DropdownButtonFormField<String>({
38       value: _selectedArea,
39       hint: Text('Seleccione un Área', style: GoogleFonts.comicNeue(fontSize: 16)),
40       decoration: const InputDecoration( // InputDecoration ...
41         items: _areas.map((String area) {
42           return DropdownMenuItem<String>( // DropdownMenuItem ...
43         }).toList(),
44       onChanged: (String? newValue) { ...
45     }); // DropdownButtonFormField
46   }
47 }
```

ILUSTRACIÓN 99. ARCHIVO AREAS_SCREEN.DART.

Este código implementa un **componente personalizado en Flutter** que permite al usuario seleccionar un área de una lista dinámica obtenida de una API REST. El widget está diseñado como un formulario desplegable (DropdownButtonFormField) y se actualiza automáticamente al recibir los datos desde un servidor.

El componente se llama AreaSelector y está diseñado como un StatefulWidget. Esto permite que el componente mantenga un estado mutable, lo que es crucial para gestionar la lista de áreas y la selección del usuario.

El propósito del widget es:

- Mostrar una lista de áreas obtenida desde una API remota.
- Notificar la selección del usuario mediante un callback proporcionado al componente.

Callback para Comunicación

El constructor de `AreaSelector` incluye un argumento `onAreaSelected`, que es una función (`Function(String?)`). Este callback es utilizado para comunicar el área seleccionada al componente padre. Es decir: Cuando el usuario selecciona un área del desplegable, esta función se ejecuta, enviando el valor seleccionado al contexto externo.

Inicialización y Obtención de Datos

Cuando se crea el componente, la lista de áreas aún no está disponible. Durante la inicialización (`initState`), se llama al método privado `_fetchAreas()`:

- **Petición HTTP:** Se realiza una solicitud GET a un endpoint remoto: `'https://back-end-hospital2-0.onrender.com/departamentos/'`. La respuesta es un JSON que contiene una lista de áreas.
- **Procesamiento de Datos:** Los datos recibidos se convierten en una lista de cadenas (`List<String>`), extrayendo el campo `'Nombre'` de cada objeto JSON, se utiliza `utf8.decode` para garantizar la compatibilidad con caracteres especiales.
- **Actualización del Estado:** Los valores procesados se asignan a `_areas` y se actualiza la interfaz del usuario mediante `setState`, si ocurre un error durante la petición o procesamiento, se muestra un mensaje de error usando un `SnackBar`.

Construcción de la Interfaz

Estilo y Decoración: Utiliza un cuadro desplegable (`DropDownButtonFormField`) con un borde resaltado y un ícono de prefijo (`Icons.area_chart`). Se usa la fuente personalizada `GoogleFonts.comicNeue` para los textos, lo que añade un toque estilístico único.

Opciones del Menú: La lista `_areas`, que contiene los nombres de las áreas, se mapea a una lista de elementos desplegables (`DropDownMenuItem`), cada elemento muestra el nombre de un área.

Gestión de Selección: Cuando el usuario selecciona un área, se actualiza el estado local (`_selectedArea`). El callback `widget.onAreaSelected` se ejecuta, notificando al componente padre de la selección realizada.

Manejo de Errores: El componente incluye manejo básico de errores: Si la API no responde correctamente (código de estado diferente de 200) o ocurre un error durante la petición, se muestra un mensaje en la interfaz mediante un `SnackBar`.

Carpeta Views

```

estadisticas.dart x
direccion_general_flutter > lib > View > estadisticas.dart > _EstadisticaScreenState
1 import 'package:direccion_general_flutter/Drawer/costom_drawer.dart';
2 import 'package:direccion_general_flutter/View/Grafics/GrafiAprobaciones.dart';
3 import 'package:direccion_general_flutter/login_screan.dart';
4 import 'package:flutter/material.dart';
5 import 'dart:convert';
6 import 'package:http/http.dart' as http;
7 import 'package:shared_preferences/shared_preferences.dart';
8 import 'package:google_fonts/google_fonts.dart';
9 import 'package:syncfusion_flutter_charts/charts.dart';
10
11 > class EstadisticaScreen extends StatefulWidget { ...
12
13 class _EstadisticaScreenState extends State<EstadisticaScreen> {
14   late Future<Map<String, dynamic>> userDataFuture;
15   Map<String, Map<String, int>> estatusTipoCount = { ...
16
17   List<RoleData> roleData = [];
18
19   @override
20   void initState() { ...
21
22   Future<void> fetchChartData() async {
23     try {
24       final response = await http.get(
25         Uri.parse('https://back-end-hospital2-0.onrender.com/tbb_aprobaciones/'),
26       );
27       final List<dynamic> data = jsonDecode(response.body);
28
29     } catch (e) {
30       print("Error fetching chart data: $e"); Don't invoke 'print' in production code. Try using a logging framework.
31     }
32   }
33
34   Future<void> fetchRoleData() async {
35     try {
36       final rolesResponse = await http.get(Uri.parse('https://back-end-hospital2-0.onrender.com/roles/'));
37       final userRolesResponse = await http.get(Uri.parse('https://back-end-hospital2-0.onrender.com/usuario_roles/'));
38
39       final List<dynamic> roles = jsonDecode(rolesResponse.body);
40       final List<dynamic> userRoles = jsonDecode(userRolesResponse.body);
41
42       Map<String, int> roleCounts = {};
43
44       for (var role in roles) { ...
45
46       setState(() {
47         roleData = roleCounts.entries.map((entry) => RoleData(entry.key, entry.value)).toList();
48       });
49     } catch (error) {
50       print("Error fetching role data: $error"); Don't invoke 'print' in production code. Try using a logging framework.
51     }
52   }
53
54   List<ChartData> generateChartData() { ...
55
56   Future<void> _logout() async { ...
57
58   @override
59   Widget build(BuildContext context) { ...
60
61   // Removed duplicate ChartData class definition
62
63   class RoleData {
64     final String role;
65     final int count;
66
67     RoleData(this.role, this.count);
68   }
69 }

```

ILUSTRACIÓN 100. ARCHIVO ESTADISTICAS.DART

El código presenta una pantalla en Flutter llamada **EstadisticaScreen**, diseñada para mostrar estadísticas relacionadas con aprobaciones y roles de usuario dentro de un área específica.

Importaciones

Paquetes básicos: Se importan paquetes como flutter/material.dart para widgets y dart:convert para trabajar con datos JSON.

Paquetes específicos:

- **http:** Realiza solicitudes HTTP para obtener datos del backend.
- **shared_preferences:** Gestiona datos locales como la sesión del usuario.
- **syncfusion_flutter_charts:** Proporciona gráficos (circular, barras, etc.).
- **google_fonts:** Permite usar fuentes personalizadas.

Archivos locales: Se importan componentes como CustomDrawer para el menú lateral y GraficoAprobaciones para gráficos de aprobaciones.

Clase Principal EstadisticaScreen: Es un widget con estado (StatefulWidget) que recibe dos parámetros:

- **area:** Nombre del área para mostrar estadísticas.
- **personalId:** ID del usuario que visualiza la pantalla.

Estado _EstadisticaScreenState: Contiene toda la lógica, desde la obtención de datos hasta la construcción de los gráficos.

a) Variables principales

- **userDataFuture:** Obtiene datos del usuario a partir del personalId.
- **estatusTipoCount:** Mapea estadísticas por estatus y tipo
- **roleData:** Lista para almacenar datos sobre roles de usuarios.

b) Métodos importantes

- **initState:** Llama a métodos para obtener datos de aprobaciones y roles al inicializar la pantalla.
- **fetchChartData:** Realiza una solicitud HTTP al endpoint de aprobaciones (/tbb_aprobaciones/). Actualiza el mapa estatusTipoCount con los datos obtenidos.
- **fetchRoleData:** Obtiene datos de roles (/roles/) y asignaciones de roles a usuarios (/usuario_rol/), calcula la cantidad de usuarios por rol y los guarda en roleData.
- **generateChartData:** Convierte el mapa estatusTipoCount en una lista de datos (ChartData) para los gráficos.

- **_logout:** Elimina los datos de la sesión local usando SharedPreferences y redirige al usuario a la pantalla de inicio de sesión.

c) Construcción de la interfaz gráfica

- **AppBar:** Muestra el título y un logo en la barra superior.
- **Drawer:** Incluye un menú lateral personalizado (CustomDrawer) con opciones y sesión del usuario.
- **Cuerpo (body):** Un Column muestra: Información del área, un gráfico de aprobaciones (GraficoAprobaciones), Un gráfico circular para distribución de usuarios por rol (SfCircularChart).
- Se encapsula en un SingleChildScrollView para permitir desplazamiento vertical si el contenido es extenso.

Clases Auxiliares

RoleData: Modela datos para el gráfico circular, con: role: Nombre del rol, count: Cantidad de usuarios con ese rol.

Flujo General

Al abrir la pantalla: Se cargan los datos del usuario, aprobaciones y roles, Los datos se procesan para actualizar las estructuras estatusTipoCount y roleData.

La interfaz gráfica: Muestra estadísticas del área en dos gráficos: uno de barras (aprobaciones) y otro circular (roles).

El usuario puede: Navegar en el menú lateral, Cerrar sesión, limpiando la memoria local y volviendo a la pantalla de inicio de sesión.

Consideraciones

Endpoints Backend: se asegura de que los endpoints (/tbb_aprobaciones/, /roles/, /usuario_rols/) devuelvan datos válidos, como se muestra en la [Ilustración 55].

Errores HTTP: Los errores se manejan en try-catch, y permiten mostrar mensajes más descriptivos al usuario. Esto permite la localización de la falla en su procesamiento, además en el desarrollo se han colocado banderas que identifique la lectura de las variables que integran los datos.

Archivo Principal en el Flujo de Datos de la Aplicación Parte 1

```

aprobacionesSM.dart M X
direccion_general_flutter > lib > View > aprobacionesSM.dart > ...
1 import 'package:direccion_general_flutter/login_screean.dart'; // Corrección en el nombre del archivo The file na
2 import 'package:direccion_general_flutter/View/home_screen.dart' as home; // Import the HomeScreen class with alias
3 import 'package:flutter/material.dart';
4 import 'dart:convert';
5 import 'package:http/http.dart' as http;
6 import 'package:shared_preferences/shared_preferences.dart';
7 import 'package:google_fonts/google_fonts.dart';
8 import 'package:intl/intl.dart';
9 import 'package:intl/intl.dart' as intl;
10 import 'package:direccion_general_flutter/Drawer/costom_drawer.dart';
11
12 // ----- Extraccion de datos de los medicos -----
13 > class Doctor { ...
14 //-----
15 // ----- Extraccion de datos de las solicitudes -----
16
17 > class Solicitud { ...
18 //-----
19
20 > class AprobacionesScreen extends StatefulWidget { ...
21
22 class _AprobacionesScreenState extends State<AprobacionesScreen> {
23
24   late Future<Map<String, dynamic>> userDataFuture; // Cambiado a late para inicializar en initState
25   List<dynamic> aprobaciones = []; // Cambiado a List<dynamic> para que coincida con el tipo de retorno
26   Map<int, Map<String, dynamic>> personalMedicoData = {};
27   // Cambiado a Map<int, Map<String, dynamic>> para que coincida con el tipo de retorno
28
29   List<Doctor> listaMedicos = []; // Cambiado a List<Doctor> para que coincida con el tipo de retorno
30
31   Map<int, Map<String, dynamic>> solicitudesRealizadas = {}; // Cambiado a map de tipo Solicitud
32
33   @override
34   > void initState() { ...
35
36   > Future<void> _logout() async { ...
37
38   > Future<void> fetchAprobaciones() async { ...
39
40   Future<void> fetchPersonalMedicoData() async {
41     try {
42       final response = await http.get(
43         Uri.parse('https://back-end-hospital2-0.onrender.com/persons/'),
44       );
45       if (response.statusCode == 200) { ...
46       } else { ...
47       } catch (e) {
48         print('Error: $e'); Don't invoke 'print' in production code. Try using a logging framework.
49       }
50     }
51
52     //-----
53     //Estraccion de los datos de los medicos
54
55     Future<List<Doctor>> fetchDoctors() async {
56       try {
57         final response = await http.get(Uri.parse('https://back-end-hospital2-0.onrender.com/persons/'));
58         if (response.statusCode == 200) { ...
59         } else { ...
60         } catch (e) {
61           print('Error fetching doctors: $e'); Don't invoke 'print' in production code. Try using a logging framework.
62           throw Exception('Error fetching doctors: $e');
63         }
64       }
65
66       //-----
67       //Estraccion de los datos de las solicitudes
68       // Función para obtener las solicitudes
69       // Función para obtener las solicitudes
70       > Future<void> fetchSolicitudes() async { ...
71       // -----
72
73       @override
74       > Widget build(BuildContext context) { ...
75
76       Widget _buildSolicitudesTable() {
77         return Expanded(
78           child: SingleChildScrollView(
79             scrollDirection: Axis.horizontal,
80

```

ILUSTRACIÓN 101.PRIMERA PARTE DEL ARCHIVO APROBACIONESSM.DART

La primera parte corresponde a los **importes** y la definición de las clases Doctor y Solicitud.

El código importa paquetes y archivos necesarios para el correcto funcionamiento de la aplicación.

Veamos para qué sirve cada uno:

1. **import 'package:direccion_general_flutter/login_screean.dart';** Importa el archivo que contiene la lógica o interfaz del inicio de sesión.
2. **import 'package:direccion_general_flutter/View/home_screen.dart' as home;** Importa la pantalla principal de la aplicación (HomeScreen) con un alias llamado home. Esto permite acceder a las funciones o clases dentro de este archivo utilizando el prefijo home.
3. **import 'package:flutter/material.dart';** Importa el paquete principal de Flutter para construir la interfaz gráfica, que incluye widgets como Scaffold, AppBar, y otros.
4. **import 'dart:convert';** Importa el paquete para trabajar con datos en formato JSON, lo cual es necesario para decodificar las respuestas de la API.
5. **import 'package:http/http.dart' as http;** Importa el paquete http para realizar solicitudes HTTP. Este paquete es útil para interactuar con APIs REST.
6. **import 'package:shared_preferences/shared_preferences.dart';** Importa el paquete para almacenar datos simples de manera local, como tokens de sesión o configuraciones.
7. **import 'package:google_fonts/google_fonts.dart';** Permite utilizar fuentes personalizadas en la aplicación, en este caso, las fuentes de Google Fonts.
8. **import 'package:intl/intl.dart';** y **import 'package:intl/intl.dart' as intl;** Este paquete se utiliza para manejar formatos de fecha y hora internacionales.
9. **import 'package:direccion_general_flutter/Drawer/costom_drawer.dart';** Importa un archivo que contiene el widget CustomDrawer, el cual parece ser un menú lateral personalizado.

Clase Doctor

Define la estructura y lógica para manejar la información de un médico. Esto incluye:

Propiedades:

- nombre: Nombre del médico.
- primerApellido: Primer apellido.
- segundoApellido: Segundo apellido.
- id: Identificador único del médico.

Constructor: El constructor requiere que todos los campos sean obligatorios (required).

factory Constructor: Convierte un objeto JSON en una instancia de Doctor. Es útil al manejar datos provenientes de una API

Clase Solicitud

Es similar a la clase Doctor, pero define la estructura para manejar datos de las solicitudes.

Propiedades:

- **solicitud:** Identificador único de la solicitud.
- **prioridad:** Nivel de prioridad de la solicitud.
- **descripción:** Descripción de la solicitud.
- **estado:** Estado actual (pendiente, aprobado, etc.).

Constructor: Todos los campos son obligatorios.

factory Constructor: Convierte un objeto JSON en una instancia de Solicitud.

El widget de pantalla en Flutter llamado AprobacionesScreen, manejar la lógica y la interfaz de usuario relacionada con la visualización y gestión de aprobaciones de solicitudes en un sistema médico, se compone de las siguientes estructuras:

Declaraciones Iniciales

Variables principales:

- **userDataFuture:** Almacena datos futuros del usuario actual mediante la función `fetchUserData`.
- **aprobaciones:** Lista dinámica que contiene aprobaciones obtenidas desde un endpoint.
- **personalMedicoData:** Mapa que asocia el ID de un personal médico con sus datos (nombre, apellidos, etc.).
- **listaMedicos:** Lista que contiene instancias del modelo Doctor, representando médicos.
- **solicitudesRealizadas:** Mapa que asocia IDs de solicitudes con los detalles de cada una.

Método initState()

Este método se ejecuta al inicializar el widget, Llama a funciones para obtener datos:

- **fetchUserData:** Recupera los datos del usuario.
- **fetchAprobaciones:** Obtiene aprobaciones desde un servidor y las almacena en aprobaciones.

- **fetchPersonalMedicoData:** Recupera los datos del personal médico y los almacena en `personalMedicoData`.
- **fetchDoctors:** Recupera y almacena médicos en la lista `listaMedicos`.
- **fetchSolicitudes:** Recupera solicitudes y las organiza en `solicitudesRealizadas`.

Lógica de Obtención de Datos

- **fetchAprobaciones():** Hace una solicitud HTTP para obtener aprobaciones desde un endpoint, Si la respuesta es exitosa (200), almacena las aprobaciones en el estado del widget (`aprobaciones`).
- **fetchPersonalMedicoData():** Solicita los datos de personas desde un endpoint, decodifica los datos usando `utf8` para soportar caracteres especiales, organiza los datos en un mapa (`personalMedicoData`) donde cada clave es un ID, y el valor contiene detalles personales.
- **fetchDoctors():** Solicita médicos desde un endpoint y mapea los datos para convertirlos en objetos del modelo `Doctor`.
- **fetchSolicitudes():** Solicita las solicitudes desde un endpoint, decodifica los datos y organiza un mapa donde la clave es el ID de la solicitud, y el valor es el detalle de esta.

Método `_logout()`

- Borra las credenciales de inicio de sesión almacenadas localmente (`personalId` y `selectedArea`) usando `SharedPreferences`, redirige al usuario a la pantalla de inicio de sesión.

Interfaz de usuario

- **AppBar:** Muestra el título de la pantalla, Incluye un logo que redirige a la pantalla principal.
- **Drawer:** Utiliza el widget `CustomDrawer` para mostrar un menú lateral que contiene, datos del usuario (`userDataFuture`), área seleccionada y función de cierre de sesión.
- **Cuerpo Principal:** Área actual (`widget.area`), título "Solicitudes Pendientes", botón para crear una aprobación (`_showCreateAprobacionModal`), Tabla de solicitudes construida con `_buildSolicitudesTable`.
- **Tabla de Solicitudes:** Construida con el widget `DataTable` con columnas: Número de solicitud, Personal médico, Solicitud asociada. Comentario, estado, tipo, fecha de registro, y fecha de aprobación.
- **Filas de la Tabla:** Las filas se generan dinámicamente con `_buildDataRows`, accediendo a aprobaciones y cruzando datos con `personalMedicoData` y `solicitudesRealizadas`.

Archivo Principal en el Flujo de Datos de la Aplicación Parte 2

```

286 child: SingleChildScrollView(
287   scrollDirection: Axis.vertical,
288   child: DataTable(
289     columns: const [
290       DataColumn(label: Text('N°')),
291       DataColumn(label: Text('Personal Médico')),
292       DataColumn(label: Text('Solicitud')), // Nueva columna
293       DataColumn(label: Text('Comentario')),
294       DataColumn(label: Text('Estatus')),
295       DataColumn(label: Text('Tipo')),
296       DataColumn(label: Text('Fecha de Registro')),
297       DataColumn(label: Text('Fecha de Aprobación')),
298     ],
299     rows: _buildDataRows(),
300   ), // DataTable
301 ), // SingleChildScrollView
302 ), // SingleChildScrollView
303 ); // Expanded
304 }
305
306
307 > List<DataRow> _buildDataRows() { // Cambiado a List<DataRow> para que coincida con el tipo de retorno...
308
309   // Función para mostrar el modal de opciones
310   void _showOptionsModal(BuildContext context, dynamic item, Map<String, dynamic>?
311 > medicosData, Map<String, dynamic>? solicitudData) { ...
312
313   // Función para mostrar el modal de confirmación de eliminación
314   void _showDeleteConfirmation(BuildContext context, int id) { ...
315
316   Future<void> _updateAprobacion(int id,
317     String comentario,
318     String estatus,
319     String tipo,
320     String fechaRegistro,
321     String fechaAprobacion,
322     int personalMedicoId,
323     int solicitudId) async {
324     try {
325       final response = await http.put(...
326       if (response.statusCode == 200) {
327         fetchAprobaciones(); // Recargar los datos después de la actualización
328         ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content: Text('Actualización exitosa'))); Don't use 'BuildContext'
329       } else {
330         print('Error en la respuesta: ${response.body}'); Don't invoke 'print' in production code. Try using a logging framework
331         throw Exception('Error al actualizar');
332       }
333     } catch (e) {
334       print('Error: $e'); Don't invoke 'print' in production code. Try using a logging framework.
335     }
336   }
337 }
338
339
340 > Future<void> _deleteAprobacion(int id) async { ...
341
342 //-----Creacion del la Aprobación-----
343 > Future<void> _createAprobacion( The declaration '_createAprobacion' isn't referenced. Try removing the declaration of '_create
344
345 //-----
346 // Opciones para el campo Tipo
347 > List<String> tipos = [ ...
348
349 // Opciones para el campo Estatus
350 > List<String> estatus = [ ...
351
352 // Opciones para el campo Prioridad
353 > final List<String> prioridades = [ ...
354
355 // -----Activacion del modal de creacion de aprobacion-----
356
357 // Función para abrir el modal de creación de aprobación
358 > Future<void> _createNewAprobacion( ...
359
360 // -----Activacion del modal de creacion de aprobacion-----
361 > void _showCreateAprobacionModal(BuildContext context) { ...
362
363 // Función auxiliar para crear TextField
364 > Widget _buildTextField({required TextEditingController controller, required String label}) {
365   return TextField(
366     controller: controller,
367     decoration: InputDecoration(labelText: label),
368   ); // TextField
369 }
370
371
372

```

ILUSTRACIÓN 102. SEGUNDA PARTE DEL ARCHIVO APROBACIONESSM.DART.

Método `_buildDataRows`: Este método genera una lista de filas (`DataRow`) que contienen celdas con datos dinámicos. El propósito es construir las filas de la tabla de acuerdo a los datos de las "aprobaciones". La lista de aprobaciones probablemente proviene de una fuente de datos externa, como una API o una base de datos.

Formato de la fecha: Utiliza el paquete `intl` para formatear fechas en el formato 'EEEE, d MMMM yyyy', lo que equivale a algo como "lunes, 1 enero 2024".

Maapeo de Aprobaciones

El código recorre la lista aprobaciones, utilizando `map<DataRow>((item) {...})`. Aquí se realiza la asignación de datos a las celdas, con un enfoque en obtener detalles específicos de otras colecciones como `personalMedicoData` y `solicitudesRealizadas`.

- **Datos del personal médico:**

`final medicoData = personalMedicoData[medicoid]`; recupera la información del personal médico asociada al ID que aparece en la aprobación actual.

- **Datos de la solicitud:**

Similarmente, el código obtiene datos de la solicitud correspondiente al `solicitudId`. Si no encuentra datos, muestra un mensaje de error.

Construcción de texto de solicitud: El texto que describe la solicitud se construye utilizando una función `decodeUtf8` para decodificar cualquier texto codificado en UTF-8 (para manejar caracteres especiales). Si los datos de la solicitud están disponibles, se crea un texto detallado con la información de la prioridad, descripción, estatus, etc.

Creación de cada `DataRow`: Cada `DataRow` tiene varias celdas (`DataCell`), que a su vez contienen un `GestureDetector` que permite detectar cuando se toca (tap) la celda. El propósito de esto es permitir que el usuario interactúe con las celdas de la tabla y se muestren más detalles en un modal (ventana emergente).

Columnas:

- **Número de solicitud:** Muestra el ID de la solicitud. Al tocarla, se abre un modal con más detalles.

- **Personal Médico:** Muestra el nombre completo del personal médico asociado a la solicitud, un mensaje de "Nombre no disponible" si no se encuentra la información.
- **Detalles de la Solicitud:** Muestra la descripción y otros datos relacionados con la solicitud.
- **Comentario:** Muestra un comentario asociado a la solicitud.
- **Estatus de la Solicitud:** Muestra el estatus actual de la solicitud.
- **Tipo de Solicitud:** Muestra el tipo de solicitud (por ejemplo, "Servicio Interno", "Traslados", etc.).
- **Fecha de Registro:** Muestra la fecha de registro de la solicitud, formateada.
- **Fecha de Aprobación:** Muestra la fecha en la que la solicitud fue aprobada o actualizada.

Modal de Opciones:

El método `_showOptionsModal` se utiliza para mostrar un modal (diálogo emergente) que permite al usuario editar detalles de la solicitud. Esto incluye:

- Comentarios, estatus, tipo de solicitud y fechas.
- El modal tiene campos de texto (`TextField`) y menús desplegables (`DropDownButtonFormField`) para elegir opciones de estatus y tipo de solicitud. Los valores de estos menús se pasan a través de las opciones correspondientes, como "Registrada", "Cancelada", etc.

Cuando el usuario hace tap sobre una celda (por ejemplo, en el "Estatus" o "Tipo de Solicitud"), se abre este modal y el usuario puede editar los datos.

Actualización de la Aprobación:

Una vez que el usuario haya realizado los cambios en el modal, puede presionar un botón para actualizar la solicitud. Esto probablemente envíe los datos actualizados a un servidor o base de datos, pero el código de actualización está truncado. La función `_updateAprobacion` parece encargarse de esta tarea, pasando los valores del formulario para actualizar los detalles de la aprobación.

Listas de opciones:

- **tipos:** Contiene los tipos de aprobación posibles, como 'Servicio Interno', 'Traslados', etc.

- **estatus:** Define los diferentes estados en los que puede estar una aprobación, como 'Registrada', 'Programada', etc.
- **prioridades:** Define las diferentes prioridades que puede tener la solicitud de la aprobación, como 'Urgente', 'Alta', 'Moderada', etc.

Función `_createNewAprobacion`: Esta función es la que se llama para hacer la solicitud HTTP y crear la aprobación en el servidor cuando el formulario está completo.

- **Parámetros:** Recibe varios parámetros que representan los datos que el usuario ingresa, como comentario, estatus, tipo, fechas, etc.
- **Validación:** Verifica que todos los campos estén completos antes de enviar la solicitud.
- **Solicitud HTTP:** Realiza una solicitud POST al servidor para crear la aprobación, enviando los datos en formato JSON.
- **Respuestas:** Si la creación es exitosa (código 200 o 201), se actualiza la pantalla y se muestra un mensaje de éxito. Si ocurre un error, se muestra un mensaje de error.

Función `_showCreateAprobacionModal`: Esta función abre un modal (AlertDialog) donde el usuario puede llenar el formulario para crear una nueva aprobación.

Campos de entrada:

- **Comentario:** Campo de texto donde el usuario ingresa un comentario.
- **Tipo y Estatus:** Campos desplegables (DropDownButtonFormField) que permiten seleccionar un tipo de aprobación y un estatus de entre las opciones definidas en las listas anteriores.
- **Fechas:** Los campos de fechas (Registro y Aprobación) utilizan un TextField junto con un GestureDetector que permite al usuario seleccionar la fecha a través de un DatePicker.
- **Médico seleccionado:** Un campo desplegable donde el usuario selecciona el médico que está asociado con la aprobación.
- **Solicitud seleccionada:** Un campo desplegable para seleccionar la solicitud que está asociada con la aprobación, mostrando su ID y prioridad.
- **Validación:** Antes de crear la aprobación, se asegura de que todos los campos estén completos. Si falta algún campo, muestra un mensaje de error.

Funciones auxiliares:

- **selectDate:** Muestra un selector de fecha para permitir al usuario elegir una fecha. Luego, la fecha seleccionada se formatea y se coloca en el campo correspondiente.
- **_buildTextField:** Función que crea un campo de texto para los comentarios o cualquier otro dato de entrada.

Mostrar el modal: El modal se muestra mediante un showDialog en Flutter, el cual contiene los campos mencionados y dos botones:

- **Cancelar:** Cierra el modal sin hacer nada.
- **Crear Aprobación:** Valida que todos los campos estén completos y luego llama a la función _createNewAprobacion para enviar la solicitud al servidor.

Estilo y UI:

- **DropdownMenuItems:** Los campos de tipo, estatus, médico y solicitud utilizan DropdownMenuItem para mostrar las opciones. Cada opción se representa con un texto y en algunos casos, como el campo de médicos, muestra el nombre completo del médico.
- **AbsorbPointer:** Se usa en los campos de fecha para deshabilitar la interacción directa con el TextField, de modo que solo pueda ser editado a través del selector de fecha.

La correspondencia y vinculación de los **endpoints** en el sistema permite que los datos sean procesados de manera eficiente y presentados adecuadamente en la interfaz, lo cual facilita su comprensión por parte del usuario. Este enfoque asegura que cada dato, desde la identificación del usuario registrado hasta las solicitudes, sea presentado de manera clara y precisa. Al modificar la estructura de los datos sin perder su contexto original, y mantener la coherencia en los mismos elementos, como el registro, la actualización y el manejo de fechas, se asegura que la información se procese con la mayor exactitud posible. Además, la correcta gestión de las fechas, tanto de registro como de actualización, es fundamental para garantizar que los datos sean procesados en el momento adecuado y reflejen la información más actualizada, lo que mejora la precisión del sistema y optimiza la toma de decisiones. Este enfoque integral no solo mejora la usabilidad, sino que también aumenta la fiabilidad y eficiencia del proceso en general, permitiendo una experiencia de usuario más fluida y consistente.

Inicio Principal de la App

```
home_screen.dart M X
direccion_general_flutter > lib > View > home_screen.dart > ...
1 import 'package:direccion_general_flutter/Drawer/coston_drawer.dart';
2 import 'package:direccion_general_flutter/View/aprobacionesSM.dart';
3 import 'package:direccion_general_flutter/login_screean.dart';
4 import 'package:flutter/material.dart';
5 import 'dart:convert';
6 import 'package:http/http.dart' as http;
7 import 'package:shared_preferences/shared_preferences.dart'; // Asegúrate de importar esto
8 import 'package:google_fonts/google_fonts.dart'; // Importa Google Fonts
9
10 class HomeScreen extends StatefulWidget {
11   final String area; // Agrega el parámetro de área
12   final int personaId;
13
14   const HomeScreen({super.key, required this.area, required this.personaId});
15
16   @override
17   _HomeScreenState createState() => _HomeScreenState(); Invalid use of a private type in a public API.
18 }
19
20
21 class _HomeScreenState extends State<HomeScreen> {
22   late Future<Map<String, dynamic>> userDataFuture;
23
24   @override
25   void initState() { ...
26
27     Future<void> _logout() async {
28       SharedPreferences prefs = await SharedPreferences.getInstance();
29       await prefs.remove('personaId');
30       await prefs.remove('selectedArea');
31
32       // Redirige a la pantalla de inicio de sesión
33       Navigator.pushReplacement(...
34     }
35
36   @override
37   Widget build(BuildContext context) { ...
38 }
39
40 //-----
41
42 Future<Map<String, dynamic>> fetchUserData(int personaId) async {
43   try {
44     final usuarioResponse = await http.get(
45       Uri.parse('https://back-end-hospital2-0.onrender.com/tbb_usuarios/'),
46     );
47     final personaResponse = await http.get(
48       Uri.parse('https://back-end-hospital2-0.onrender.com/persons/'),
49     );
50
51     // Verifica el estado de las respuestas
52     print('Usuario Response Status: ${usuarioResponse.statusCode}'); Don't invoke 'print' in production code.
53     print('Persona Response Status: ${personaResponse.statusCode}'); Don't invoke 'print' in production code.
54
55     if (usuarioResponse.statusCode == 200 && personaResponse.statusCode == 200) {
56       final List<dynamic> usuarios = jsonDecode(usuarioResponse.body);
57       final List<dynamic> personas = jsonDecode(personaResponse.body);
58
59       final userData = usuarios.firstWhere((user) => user['ID'] == personaId, orElse: () => null);
60       final personaData = personas.firstWhere((persona) => persona['id'] == personaId, orElse: () => null);
61
62       if (userData == null) {
63         throw Exception("User with ID $personaId not found");
64       }
65       if (personaData == null) {
66         throw Exception("Persona with ID $personaId not found");
67       }
68
69       return {
70         'correo': userData['Correo_Electronico'],
71         'fotografia': personaData['Fotografia'],
72       };
73     }
74
75     throw Exception("Failed to load user data");
76   } catch (e) {
77     // Manejo de excepciones
78     print("Error fetching user data: $e"); Don't invoke 'print' in production code. Try using a logging framework.
79     throw Exception("Error fetching user data");
80   }
81 }
82
83 }
```

ILUSTRACIÓN 103. ARCHIVO HOME_SCREEN.DART

El home Sreen inicia con serie de importaciones necesarias para utilizar distintas funcionalidades de Flutter:

- **Widgets:** Usados para construir la interfaz de usuario.
- **http:** Para realizar peticiones HTTP, que se utiliza para obtener los datos de usuario.
- **shared_preferences:** Para almacenar datos de manera persistente (por ejemplo, el ID de la persona).
- **google_fonts:** Para personalizar las fuentes de texto.

HomeScreen: Es un StatefulWidget, lo que significa que puede tener un estado mutable que puede cambiar durante el ciclo de vida del widget, area y personald se pasan como parámetros para personalizar la vista y la lógica según el área y la persona que estén interactuando.

_HomeScreenState: Es el estado asociado al widget HomeScreen. Aquí se maneja la lógica de la vista.

userDataFuture: Es una variable que almacena un Future, que representa los datos del usuario que se recuperan de un servidor a través de una petición HTTP.

initState: Es el método que se llama una vez cuando el widget es inicializado. Aquí se establece que userDataFuture se obtiene llamando a la función fetchUserData con el personald.

_logout: Este método elimina las preferencias almacenadas (como el ID de persona y el área seleccionada) usando SharedPreferences y redirige al usuario a la pantalla de inicio de sesión (LoginScreen).

build: Es el método principal que se ejecuta cada vez que se necesita construir la interfaz. El método devuelve un Scaffold, que es la estructura básica de la UI.

- **AppBar:** Contiene el título de la página. Muestra el logo y el área seleccionada.
- **Drawer:** Se utiliza un menú lateral (drawer), que muestra opciones adicionales. Aquí se pasa un CustomDrawer, que es un widget personalizado.
- **Body:** El cuerpo principal contiene un FutureBuilder que espera los datos del usuario.

FutureBuilder: Este widget se utiliza para construir la interfaz basada en los resultados de un Future.

- Si el Future aún está esperando, muestra un `CircularProgressIndicator`.
- Si ocurre un error, muestra el error.
- Si los datos están disponibles, muestra la información del usuario (como la foto, correo y área).

Se verifica si existe una fotografía del usuario. Si existe, se carga desde una URL utilizando `Image.network`, con un `ClipRRect` para hacer que la imagen tenga bordes redondeados. Si no se puede cargar la imagen, muestra un icono de error.

Botones de acción: Cada botón dentro de la columna está configurado con un icono y texto, y tiene una acción asociada que navega a una nueva pantalla cuando el usuario hace clic.

- **Botón Aprobaciones Servicio Médico:** Navega a `AprobacionesScreen`, que es otra pantalla de la app.
- **Botones adicionales:** Incluyen botones para "Estructura Orgánica Hospitalaria", "Estadística", y "Bitácora", aunque aún no tienen funcionalidades definidas en este código.

Navegación a otras pantallas: Los botones utilizan `Navigator.push` para navegar a otras pantallas (`AprobacionesScreen`, etc.)

fetchUserData: Esta función asíncrona se utiliza para obtener datos de un usuario y una persona específicos a partir de dos endpoints REST.

- La función devuelve un Future que contiene un `Map<String, dynamic>` (un mapa con claves de tipo String y valores de cualquier tipo).
- El parámetro `personalId` es el identificador único del usuario que se usará para buscar los datos específicos.

Llamadas HTTP

Se realizan dos solicitudes GET a los endpoints:

- `https://back-end-hospital2-0.onrender.com/tbb_usuarios/` para obtener la lista de usuarios.
- `https://back-end-hospital2-0.onrender.com/persons/` para obtener la lista de personas.

Se utilizan `await` para esperar a que las solicitudes terminen antes de continuar.

Verificación de estado de las respuestas

- Se imprimen los códigos de estado HTTP (200, 404, etc.) para depuración.
- Esto ayuda a verificar si las solicitudes fueron exitosas o si hubo algún error.

Comprobación de respuestas exitosas: Solo se procede si ambas solicitudes devolvieron un código de estado 200 (indica éxito).

Decodificación de JSON

- Las respuestas (response.body) están en formato JSON.
- Se decodifican a listas dinámicas (List<dynamic>) para manipular los datos.

Búsqueda de datos específicos

firstWhere busca el primer elemento en la lista que cumple con la condición

- En usuarios, busca un usuario donde el campo ID coincida con personalId.
- En personas, busca una persona donde el campo id coincida con personalId.

Si no se encuentra un elemento, devuelve null gracias al uso de orElse.

Verificación de resultados

Si no se encuentran los datos en las listas, se lanza una excepción indicando que no existe el usuario o persona con el ID especificado.

Retorno de datos

Se retorna un mapa con dos claves:

- correo: Extraído del campo Correo_Electronico de userData.
- fotografia: Extraído del campo Fotografia de personaData.

Manejo de errores

- Si ocurre un error durante las solicitudes, decodificación o búsqueda, se captura en el bloque catch.
- Se imprime el error para depuración y se lanza una excepción con un mensaje genérico, al igual que en los otros documentos el manejo de errores es importante para la correcta gestión de la lógica de la aplicación.

Configuración del Login de la Aplicación Primera Parte

```
login_screen.dart M X
direccion_general_flutter > lib > login_screen.dart > _LoginScreenState
1 import 'package:direccion_general_flutter/oauth/google.dart';
2 import 'package:direccion_general_flutter/register_screan.dart';
3 import 'package:flutter/material.dart';
4 import 'package:http/http.dart' as http;
5 import 'dart:convert';
6 import 'View/home_screen.dart'; // Importa la pantalla de inicio
7 import 'package:shared_preferences/shared_preferences.dart';
8 import 'package:google_fonts/google_fonts.dart';
9
10 class LoginScreen extends StatefulWidget {
11   const LoginScreen({super.key});
12
13   @override
14   _LoginScreenState createState() => _LoginScreenState(); Invalid use of a private type in a public API. Try making the pr
15 }
16
17 class _LoginScreenState extends State<LoginScreen> {
18   String? _selectedArea;
19   List<String> _areas = [];
20
21   final TextEditingController _usernameController = TextEditingController();
22   final TextEditingController _passwordController = TextEditingController();
23   bool _keepSessionOpen = false; // Variable para mantener la sesión abierta
24
25   @override
26   void initState() {
27     super.initState();
28     _fetchAreas();
29   }
30
31   Future<void> _fetchAreas() async {
32     final response = await http.get(Uri.parse('https://back-end-hospital2-0.onrender.com/departamentos/'));
33
34     if (response.statusCode == 200) {
35       List<dynamic> data = json.decode(utf8.decode(response.bodyBytes));
36       setState(() {
37         _areas = data.map((area) => area['Nombre'] as String).toList();
38       });
39     } else {
40       throw Exception('Error al cargar las áreas');
41     }
42   }
43
44   Future<void> _validateUser() async {
45     final username = _usernameController.text;
46     final password = _passwordController.text;
47
48     final response = await http.get(Uri.parse('https://back-end-hospital2-0.onrender.com/tbb_usuarios/'));
49
50     if (response.statusCode == 200) { ...
92     } else { ...
94   }
95 }
96
97
98 // Función para manejar la autenticación con Facebook...
100 void _handleFacebookSignIn() {
101   // Aquí iría la lógica para autenticación con Google
102   Navigator.pushReplacement(
103     context,
104     MaterialPageRoute(builder: (context) => HomeScreen(area: _selectedArea ?? 'Área no seleccionada', personaId: 0)),
105   );
106 }
107
108 //-----
```

ILUSTRACIÓN 104. ARCHIVO LOGIN_SCREEN.DART

Esta primera parte implementa la pantalla de inicio de sesión del hospital con opciones para validar usuarios, seleccionar un área, guardar preferencias y navegar a otra pantalla.

Se incluyen paquetes para Flutter (MaterialApp), solicitudes HTTP, decodificación JSON, navegación, y manejo de preferencias compartidas, en ella se importan componentes personalizados (google.dart, register_screean.dart, y home_screen.dart).

Clase LoginScreen: Define un *widget* con estado que representa la pantalla de inicio de sesión, super.key asegura que se gestione correctamente el estado del widget.

Clase State Variables que se incorporan:

- **_selectedArea:** Área seleccionada por el usuario.
- **_areas:** Lista de áreas obtenidas del servidor.
- **_usernameController y _passwordController:** Controladores para los campos de texto.
- **_keepSessionOpen:** Indica si se debe mantener la sesión abierta.

initState: Llama a `_fetchAreas` al inicializar la pantalla para cargar las áreas desde el servidor.

Función `_fetchAreas`: Realiza una solicitud GET para obtener las áreas desde el servidor, si el código de estado es 200, decodifica el JSON y convierte los nombres de las áreas en una lista de cadenas (List<String>), setState actualiza la lista `_areas` y actualiza la interfaz.

Función `_validateUser`: Obtiene el username y password de los controladores de texto, hace una solicitud GET para obtener los usuarios, busca un usuario cuyo nombre y contraseña coincidan con los ingresados:

Si encuentra el usuario:

- Guarda la preferencia de sesión (keepSessionOpen), personalId, y el área seleccionada.
- Muestra un mensaje de éxito.
- Navega a HomeScreen pasando el área seleccionada y el personalId.

Si no encuentra al usuario, muestra un mensaje de error.

Si la solicitud falla, lanza una excepción.

Función `_handleFacebookSignIn`: Simula la autenticación con Facebook, Navega directamente a HomeScreen con un personalId por defecto (0) y el área seleccionada.

Configuración del Login de la Aplicación Segunda Parte

```

108 //-----
109 @override
110 Widget build(BuildContext context) {
111   return Scaffold(
112     appBar: AppBar(
113       title: const Center(child: Text('Inicio de Sesión')),
114     ), // AppBar
115     body: SingleChildScrollView(
116       child: Center(
117         child: Padding(
118           padding: const EdgeInsets.symmetric(horizontal: 20.0),
119           child: Column(
120             mainAxisAlignment: MainAxisAlignment.center,
121             children: [
122               const SizedBox(height: 60),
123               Image.asset(
124                 'assets/HPC-DG.png',
125                 fit: BoxFit.contain,
126                 height: 90,
127               ), // Image.asset
128               const SizedBox(height: 60),
129               DropdownButtonFormField<String>(
130                 value: _selectedArea,
131                 hint: Text('Seleccione un Área', style: GoogleFonts.comicNeue(fontSize: 16)),
132                 decoration: const InputDecoration( // InputDecoration ...
133                   items: _areas.map((String area) { ...
134                 ),
135                 onChanged: (String? newValue) {
136                   setState(() {
137                     _selectedArea = newValue;
138                   });
139                 },
140               ), // DropdownButtonFormField
141               const SizedBox(height: 10),
142               TextField(
143                 controller: _usernameController,
144                 decoration: InputDecoration(
145                   labelText: 'Nombre de Usuario',
146                   labelStyle: GoogleFonts.comicNeue(fontSize: 16),
147                   hintStyle: GoogleFonts.comicNeue(fontSize: 16),
148                   border: const OutlineInputBorder(),
149                   prefixIcon: const Icon(Icons.person),
150                 ), // InputDecoration
151               ), // TextField
152               const SizedBox(height: 10),
153               TextField(
154                 controller: _passwordController,
155                 obscureText: true,
156                 decoration: InputDecoration(
157                   labelText: 'Contraseña',
158                   labelStyle: GoogleFonts.comicNeue(fontSize: 16),
159                   hintStyle: GoogleFonts.comicNeue(fontSize: 16),
160                   border: const OutlineInputBorder(),
161                   prefixIcon: const Icon(Icons.lock),
162                 ), // InputDecoration
163               ), // TextField
164               const SizedBox(height: 20),
165               Row(
166                 mainAxisAlignment: MainAxisAlignment.center,
167                 children: [
168                   ElevatedButton(
169                     onPressed: _validateUser,
170                     style: ElevatedButton.styleFrom(
171                       backgroundColor: const Color.fromARGB(255, 12, 76, 128),
172                       foregroundColor: Colors.white,
173                     ),
174                     child: Text('Iniciar Sesión', style: GoogleFonts.comicNeue(fontSize: 15)),
175                   ), // ElevatedButton
176                   const SizedBox(width: 16),
177                   ElevatedButton(
178                     onPressed: () {
179                       Navigator.push(
180                         context,
181                         MaterialPageRoute(builder: (context) => const RegisterScreen()),
182                       );
183                     },
184                   ),
185                 ],
186               ),
187             ],
188           ),
189         ),
190       ),
191     ),
192   );

```

ILUSTRACIÓN 104. SEGUNDA PARTE DEL LOGIN_SCREEN.DART.

Método build: Sobrescribe el método build para construir la interfaz de usuario de este widget. Usa un Scaffold, que proporciona la estructura básica para una pantalla de Flutter.

Scaffold:

- **AppBar:** Muestra una barra de herramientas en la parte superior de la pantalla con el título centrado "Inicio de Sesión".

Cuerpo con desplazamiento

- **SingleChildScrollView:** Permite que el contenido sea desplazable si es más grande que la pantalla.
- **Center:** Centra todo el contenido dentro del SingleChildScrollView.
- **Padding:** Agrega un margen horizontal de 20 píxeles alrededor del contenido.
- **Column:** Dispone los widgets hijos verticalmente.
 - **mainAxisAlignment:** MainAxisAlignment.center: Centra los widgets en el eje principal (vertical).

Espaciado y Logo

- **SizedBox:** Agrega espacio vertical antes y después del logo (60 píxeles).
- **Image.asset:** Muestra una imagen del logo ubicada en la carpeta assets con un ajuste proporcional (BoxFit.contain) y una altura de 90 píxeles.

Selector de Área

DropDownButtonFormField: Un selector desplegable que muestra las áreas disponibles.

- **value:** Valor actualmente seleccionado (_selectedArea).
- **hint:** Mensaje de guía cuando no se ha seleccionado un área.
- **decoration:** Estilo del cuadro, con un borde y un ícono de área (Icons.area_chart).
- **items:** Genera una lista de elementos (DropDownMenuItem) basados en _areas.
- **onChanged:** Actualiza el área seleccionada (_selectedArea) y refresca la interfaz con setState.

Campos de texto

TextField: Campo de entrada para el nombre de usuario y la contraseña.

- **controller:** Controlador para capturar el texto ingresado.

- **decoration:** Define el estilo del campo, con etiquetas (labelText), bordes, y un ícono (prefixIcon).
- **obscureText:** Oculta el texto ingresado en el campo de contraseña.

Botones

Row: Dispone los botones horizontalmente.

- **mainAxisAlignment:** MainAxisAlignment.center: Centra los botones.

ElevatedButton: Botón elevado con dos funcionalidades

Iniciar Sesión:

- Llama a `_validateUser` al presionarlo.
- Estilo personalizado con colores específicos.

Registrarse:

- Navega a la pantalla RegisterScreen mediante `Navigator.push`.

```

193         style: ElevatedButton.styleFrom(
194           backgroundColor: const Color.fromARGB(255, 2, 20, 35),
195           foregroundColor: Colors.white,
196         ),
197         child: Text('Registrar', style: GoogleFonts.comicNeue(fontSize: 15)),
198       ), // ElevatedButton
199     ],
200   ), // Row
201
202   const SizedBox(height: 20),
203
204   // Switch para mantener la sesión abierta
205   Row(
206     mainAxisAlignment: MainAxisAlignment.end, // Alinea los elementos al lado derecho
207     children: [
208       Text(
209         'Mantener sesión abierta',
210         style: GoogleFonts.comicNeue(fontSize: 14),
211       ), // Text
212       const SizedBox(width: 8), // Espacio entre el texto y el switch
213       Switch(
214         value: _keepSessionOpen,
215         onChanged: (bool value) {
216           setState(() {
217             _keepSessionOpen = value;
218           });
219         },
220         activeTrackColor: const Color.fromARGB(255, 12, 76, 128), // Color de la barra del switch
221       ), // Switch
222     ],
223   ), // Row
224
225   const SizedBox(height: 20),

```

ILUSTRACIÓN 105. TERCERA PARTE DEL ARCHIVO LOGIN_SCREEN.DART

ElevatedButton:

style: Configura el estilo visual del botón.

- **backgroundColor:** Color de fondo (un tono oscuro específico).
- **foregroundColor:** Color del texto (blanco).

child: Define el contenido del botón, en este caso un texto.

Texto: "Regístrate" estilizado con la fuente ComicNeue y tamaño de 15.

Función: Representa un botón de acción para llevar al usuario a la pantalla de registro. La funcionalidad completa no está visible en este fragmento, pero usualmente implicaría navegación o interacción con el sistema.

Switch para mantener la sesión abierta

mainAxisAlignment: MainAxisAlignment.end: Alinea los elementos al extremo derecho del contenedor.

Texto "Mantener sesión abierta": Etiqueta descriptiva para el Switch, estilizada con la fuente ComicNeue y tamaño 14.

Switch:

- **value:** Estado actual del switch, almacenado en `_keepSessionOpen`.
- **onChanged:** Callback que se ejecuta cuando el usuario interactúa con el switch, llama a `setState` para actualizar el valor de `_keepSessionOpen` y refrescar la interfaz.
- **activeTrackColor:** Define el color de la barra activa del switch.

Función: Permite al usuario activar o desactivar la opción de mantener la sesión abierta, Cambia dinámicamente el estado de `_keepSessionOpen`.

```

227 Row(
228   mainAxisAlignment: MainAxisAlignment.center,
229   children: [
230     // Botón para iniciar sesión con Google
231     ElevatedButton.icon(
232       onPressed: () {
233         Navigator.push(
234           context,
235           MaterialPageRoute(builder: (context) => const GoogleScreen()),
236         );
237       },
238       icon: Image.asset(
239         'assets/icono_google.png',
240         height: 24.0,
241         width: 24.0,
242       ), // Image.asset
243       label: Text('Google', style: GoogleFonts.comicNeue()),
244       style: ElevatedButton.styleFrom(
245         backgroundColor: Colors.white,
246         foregroundColor: Colors.black,
247         side: const BorderSide(color: Colors.black),

```

ILUSTRACIÓN 106. CUARTA PARTE DEL LOGIN_SCREEN.DART.

```

248     ),
249     ), // ElevatedButton.icon
250
251     const SizedBox(width: 16), // Espacio entre los botones
252
253     // Botón para iniciar sesión con Facebook
254     ElevatedButton.icon(
255       onPressed: _handleFacebookSignIn,
256       icon: Image.asset(
257         'assets/icono_facebook.png',
258         height: 24.0,
259         width: 24.0,
260       ), // Image.asset
261       label: Text('Facebook', style: GoogleFonts.comicNeue()),
262       style: ElevatedButton.styleFrom(
263         backgroundColor: Colors.white,
264         foregroundColor: Colors.black,
265         side: const BorderSide(color: Colors.black),
266       ),
267     ), // ElevatedButton.icon
268
269   ],
270 ), // Row
271 ],
272 ), // Column
273 ), // Padding
274 ), // Center
275 ), // SingleChildScrollView
276 ); // Scaffold
277 }
278 }
279

```

ILUSTRACIÓN 107. QUINTA PARTE DEL LOGIN_SCREEN.DART

SizedBox: Añade un espacio horizontal de 16 píxeles entre el botón previo y el nuevo botón que permite iniciar sesión con Facebook. Esto ayuda a separar visualmente ambos botones.

Botón de inicio de sesión con Facebook

- **ElevatedButton.icon:** Una variante de ElevatedButton que permite incluir tanto un ícono como un texto en el mismo botón.
- **onPressed:** Define la acción que se ejecuta al presionar el botón. En este caso, se llama a la función `_handleFacebookSignIn`, que probablemente maneja la lógica de autenticación con Facebook.

Image.asset:

- Carga una imagen local (`icono_facebook.png`) como ícono del botón.
- **Tamaño:** Configurado con una altura y anchura de 24 píxeles, asegurando que sea visible pero no ocupe demasiado espacio.

Etiqueta (Texto):

- Define el texto "Facebook" que aparece junto al ícono.
- Estilizado con la fuente `ComicNeue` para mantener coherencia con el diseño general.

Registro de Nuevos Usuarios

```

1  import 'dart:convert';
2  import 'dart:io';
3  import 'package:direccion_general_flutter/user_screean.dart';
4  import 'package:flutter/material.dart';
5  import 'package:image_picker/image_picker.dart'; // Importar image_picker
6  import 'package:http/http.dart' as http;
7  import 'package:intl/intl.dart'; // Para formatear la fecha
8  import 'package:google_fonts/google_fonts.dart'; // Importar google_fonts
9
10
11  class RegisterScreen extends StatefulWidget {
12    const RegisterScreen({super.key});
13
14    @override
15    _RegisterScreenState createState() => _RegisterScreenState(); Invalid use of a private ty
16  }
17
18  class _RegisterScreenState extends State<RegisterScreen> {
19    final _formKey = GlobalKey<FormState>();
20
21    // Controladores para campos
22    String? tituloCortesia;
23    String? nombre;
24    String? primerApellido;
25    String? segundoApellido;
26    String? curp;
27    String? genero;
28    String? tipoSangre;
29    String? fechaNacimiento;
30    File? fotografia;
31
32    String message = '';
33    bool success = false;
34
35    // Instancia de ImagePicker
36    final ImagePicker _picker = ImagePicker();
37
38    // Método para seleccionar una fecha
39  > Future<void> _selectDate(BuildContext context) async {...
40
41    // Método para tomar una foto con la cámara
42  > Future<void> takePhoto() async {...
43
44    // Método para seleccionar una imagen de la galería
45  > Future<void> pickImageFromGallery() async {...
46
47    // Método para validar si la persona ya existe
48    Future<bool> validatePerson(String curp) async {
49      try {
50        final response = await http.post(
51          Uri.parse('https://back-end-hospital2-0.onrender.com/persons/validate'),
52          headers: {'Content-Type': 'application/json'},
53          body: json.encode({'curp': curp}),
54        );
55        final data = json.decode(response.body);
56        return data['exists'] ?? false;
57      } catch (error) {
58        setState(() {
59          message = 'Error al validar la CURP.';
60          success = false;
61        });
62        return false;
63      }
64    }
65  }
66
67  }
68
69  }
70
71  }
72
73  }
74
75  }
76
77  }
78
79  }
80
81  }
82
83  }
84
85  }
86
87  }
88
89  }
90
91  }
92
93  }
94
95  }

```

ILUSTRACIÓN 108. PRIMERA PARTE DEL ARCHIVO REGISTER_SCREEN.DART

dart:convert: Permite trabajar con la codificación y decodificación de datos en formato JSON.

dart:io: Proporciona acceso a operaciones de entrada/salida, como trabajar con archivos (File).

Paquetes externos:

- **image_picker:** Para seleccionar imágenes desde la galería o tomar fotos con la cámara.
- **http:** Para realizar solicitudes HTTP.
- **intl:** Para manejar y formatear fechas.
- **google_fonts:** Permite usar fuentes de Google en los estilos de texto.

direccion_general_flutter/user_screean.dart: Archivo local (debes tenerlo en tu proyecto) que probablemente maneja la navegación u otra funcionalidad.

Clase RegisterScreen

RegisterScreen: Es un widget con estado (StatefulWidget) que permite cambios dinámicos en la interfaz (por ejemplo, al seleccionar una imagen).

_RegisterScreenState: Es el estado asociado al widget. Aquí se definen las variables y métodos para manejar la lógica y los datos del formulario de registro.

Variables y Controladores

formKey: Clave única para validar el formulario.

Variables para campos de entrada:

- **tituloCortesia, nombre, primerApellido, etc.:** Almacenan los valores ingresados por el usuario.
- **fotografia:** Guarda la imagen seleccionada o tomada con la cámara.

Mensajes y estados:

- **message:** Guarda un mensaje para el usuario (por ejemplo, si hubo un error).
- **success:** Indica si la operación fue exitosa.

picker: Instancia de ImagePicker para manejar la selección o captura de imágenes.

showDatePicker: Muestra un selector de fecha.

initialDate: Fecha inicial seleccionada (hoy por defecto).

firstDate y lastDate: Rango de fechas permitido (desde 1900 hasta hoy).

setState: Actualiza la variable fechaNacimiento con la fecha seleccionada, formateada como yyyy-MM-dd usando intl.

Tomar una foto con la cámara

- Usa `ImageSource.camera` para abrir la cámara y tomar una foto.
- Si la foto es válida (`photo != null`), guarda el archivo en la variable `fotografía`.

Seleccionar una imagen de la galería

- Usa `ImageSource.gallery` para abrir la galería y seleccionar una imagen.
- Similar al método anterior, guarda la imagen en la variable `fotografía`.

Método para validar si una persona ya existe

http.post: Envía una solicitud POST al servidor para validar la CURP, incluye:

- URL: `https://back-end-hospital2-0.onrender.com/persons/validate`.
- Cabeceras: Define que el contenido es JSON.
- Cuerpo: Contiene la CURP en formato JSON.

Respuesta:

- Decodifica la respuesta con `json.decode`.
- Retorna `true` si la persona existe (`data['exists']`), o `false` en caso contrario.

Manejo de errores:

- Si ocurre un error, actualiza el mensaje de error (`message`) y devuelve `false`.

```

97 // Método para registrar a la persona
98 Future<void> registerPerson() async {
99   if (!_formKey.currentState!.validate()) return;
100
101   final exists = await validatePerson(curp!);
102   if (exists) {
103     setState(() {
104       message = 'La persona ya está registrada.';
105       success = false;
106     });
107     return;
108   }
109
110   try {
111     final uri = Uri.parse('https://back-end-hospital2-0.onrender.com/person/');
112     final request = http.MultipartRequest('POST', uri);
113

```

ILUSTRACIÓN 109. SEGUNDA PARTE DEL ARCHIVO REGISTER_SCREEN.DART

El método **registerPerson**: Valida el formulario, verifica si la persona ya está registrada mediante su **CURP**, Si no está registrada, procede a enviar los datos a un endpoint utilizando una solicitud

HTTP MultipartRequest, que puede incluir tanto datos de texto como archivos (por ejemplo, la fotografía).

_formKey.currentState!.validate():

- Verifica si todos los campos del formulario cumplen con las validaciones definidas (por ejemplo, que no estén vacíos, que el CURP tenga el formato correcto, etc.).
- Si no se validan, el método se detiene con return.

validatePerson(curp!):

- Llama al método previamente explicado para verificar si la persona ya existe en el sistema (consultando al servidor con el CURP).
- Si exists es true, actualiza el estado (message y success) para informar al usuario que la persona ya está registrada y termina la ejecución.

Uri.parse(...):

- Crea una URI (Uniform Resource Identifier) a partir de la URL del endpoint de la API REST.
- En este caso, la URL es <https://back-end-hospital2-0.onrender.com/person/>.

http.MultipartRequest:

- Se utiliza para enviar datos en formato multipart/form-data, necesario para combinar texto y archivos en una sola solicitud.
- Configuración inicial: Método HTTP: POST, URL: uri.

```

114 // Adjuntar datos
115 request.fields['Titulo_Cortesía'] = tituloCortesía;
116 request.fields['Nombre'] = nombre;
117 request.fields['Primer_Apellido'] = primerApellido;
118 request.fields['Segundo_Apellido'] = segundoApellido;
119 request.fields['CURP'] = curp;
120 request.fields['Genero'] = genero;
121 request.fields['Tipo_Sangre'] = tipoSangre;
122 request.fields['Fecha_Nacimiento'] = fechaNacimiento;
123 request.fields['Estatus'] = 'true';
124 request.fields['Fecha_Registro'] = DateTime.now().toIso8601String();
125 request.fields['Fecha_Actualización'] = DateTime.now().toIso8601String();
126
127 // Adjuntar imagen
128 > if (fotografía != null) { ...
129 }
130
131
132 final response = await request.send(); // Enviar la solicitud
133
134 if (response.statusCode == 200) {
135 >   setState(() { ...
136 > } else { ...
137 > }
138 > } catch (error) {
139 >   setState(() { ...
140 > }
141 }

```

ILUSTRACIÓN 110. TERCERA PARTE DEL ARCHIVO REGISTER_SCREEN.DART.

request.fields: Se utiliza para agregar datos en formato clave-valor (textos) al cuerpo de la solicitud HTTP.

Cada línea asigna un valor ingresado en el formulario a un campo específico del cuerpo de la solicitud.

- **Titulo_Cortesía:** Título de cortesía de la persona (Sr., Sra., Dr., etc.).
- **Nombre, Primer_Apellido, Segundo_Apellido:** Datos personales básicos.
- **CURP:** Código único para identificar a la persona en México.
- **Genero y Tipo_Sangre:** Información médica o de identidad.
- **Fecha_Nacimiento:** Fecha formateada previamente por el usuario.
- **Estatus:** Se asigna un valor fijo de 'true' para marcar a la persona como activa.
- **Fecha_Registro y Fecha_Actualizacion:**
- Se generan dinámicamente con `DateTime.now().toISOString()` para registrar la fecha y hora actual en formato ISO 8601 (ejemplo: 2024-12-01T15:30:00Z).

Adjuntar imagen

- **if (fotografia != null):** Comprueba si el usuario seleccionó una imagen.
- **http.MultipartFile.fromPath:** Crea un archivo en el cuerpo de la solicitud usando el nombre del campo (Fotografia) y la ruta del archivo seleccionada (fotografia!.path).
- **request.files.add(...):** Adjunta la imagen al cuerpo de la solicitud para que se envíe junto con los datos.

Enviar la solicitud

- **request.send():** Envía la solicitud al servidor con los datos y archivos adjuntados.
- **await:** Espera la respuesta del servidor antes de continuar.

Manejar la respuesta

- **if (response.statusCode == 200):** Verifica si el servidor respondió con un código HTTP 200, indicando que la operación fue exitosa.
- **setState(...):** Actualiza el mensaje y el estado (success = true) para reflejar el registro exitoso.
- **En caso de error:** Lanza una excepción con un mensaje indicando que ocurrió un problema.
- **catch (error):** Actualiza el mensaje y el estado (success = false) para indicar que falló.

```

161 @override
162 Widget build(BuildContext context) {
163   return Scaffold(
164     appBar: AppBar( // AppBar ---
165
166     body: SingleChildScrollView(
167       padding: const EdgeInsets.all(16.0),
168       child: Form(
169         key: _formKey,
170         child: Column(
171           children: [
172             DropdownButtonFormField<String>( // DropdownButtonFormField ---
173               TextFormField(
174                 decoration: InputDecoration(labelText: 'Nombre', labelStyle: GoogleFonts.comicNeue(fontSize: 16) ),
175                 validator: (value) => value == null || value.isEmpty ? 'Campo obligatorio' : null,
176                 onChanged: (value) => nombre = value,
177               ), // TextFormField
178             TextFormField(
179               decoration: InputDecoration(labelText: 'Primer Apellido', labelStyle: GoogleFonts.comicNeue(fontSize: 16)),
180               validator: (value) => value == null || value.isEmpty ? 'Campo obligatorio' : null,
181               onChanged: (value) => primerApellido = value,
182             ), // TextFormField
183             TextFormField(
184               decoration: InputDecoration(labelText: 'Segundo Apellido', labelStyle: GoogleFonts.comicNeue(fontSize: 16)),
185               validator: (value) => value == null || value.isEmpty ? 'Campo obligatorio' : null,
186               onChanged: (value) => segundoApellido = value,
187             ), // TextFormField
188             TextFormField(
189               decoration: InputDecoration(labelText: 'CURP', labelStyle: GoogleFonts.comicNeue(fontSize: 16)),
190               maxLength: 18,
191               validator: (value) => value == null || value.isEmpty ? 'Campo obligatorio' : null,
192               onChanged: (value) => curp = value,
193             ), // TextFormField
194             DropdownButtonFormField<String>(
195               value: tipoSangre,
196               decoration: InputDecoration(labelText: 'Tipo Sangre', labelStyle: GoogleFonts.comicNeue(fontSize: 16)),
197               items: [---
198                 ].map((tipoSangre) {
199                   return DropdownMenuItem(value: tipoSangre, child: Text(tipoSangre, style: GoogleFonts.comicNeue(fontSize: 16)));
200                 }).toList(),
201               onChanged: (value) => setState(() => tipoSangre = value),
202             ), // DropdownButtonFormField
203             DropdownButtonFormField<String>(
204               value: genero,
205               decoration: InputDecoration(labelText: 'Genero', labelStyle: GoogleFonts.comicNeue(fontSize: 16)),
206               items: [---
207                 ].map((genero) {
208                   return DropdownMenuItem(value: genero, child: Text(genero, style: GoogleFonts.comicNeue(fontSize: 16)));
209                 }).toList(),
210               onChanged: (value) => setState(() => genero = value),
211             ), // DropdownButtonFormField
212             TextFormField(
213               decoration: InputDecoration(labelText: 'Fecha de Nacimiento', labelStyle: GoogleFonts.comicNeue(fontSize: 16)),
214               keyboardType: TextInputType.datetime,
215               controller: TextEditingController(text: fechaNacimiento),
216               validator: (value) => value == null || value.isEmpty ? 'Campo obligatorio' : null,
217               onTap: () {
218                 // Deshabilitar la edición del campo de texto y abrir el calendario
219                 FocusScope.of(context).requestFocus(FocusNode());
220                 _selectDate(context);
221               },
222             ), // TextFormField
223             const SizedBox(height: 16.0), // Espacio entre campos
224             Row( // Row ---
225
226             const SizedBox(height: 16.0),
227             ElevatedButton( // ElevatedButton ---
228               if (message.isNotEmpty)
229                 Text(
230                   message,
231                   style: TextStyle( // TextStyle ---
232
233                   ), // Text
234             ),
235           ], // Column
236         ), // Form
237       ), // SingleChildScrollView
238     ); // Scaffold
239   }
240 }

```

ILUSTRACIÓN 111. CUARTA PARTE DEL ARCHIVO REGISTER_SCREEN.DART

Scaffold y AppBar

El widget Scaffold es el contenedor principal de la interfaz de usuario. Dentro de Scaffold, se define la barra de la aplicación (AppBar) que contiene un Row con:

- Una imagen (Image.asset) que se muestra en la parte izquierda de la barra de la aplicación, probablemente el logo de la organización.
- Un título (Text), que en este caso es "Formulario de Registro", con un estilo de texto usando la fuente ComicNeue de Google Fonts.

SingleChildScrollView y Form

- SingleChildScrollView: Permite que el contenido de la página se desplace cuando el teclado aparece o cuando hay mucho contenido.
- Form: Se usa para agrupar los campos de entrada y validar los datos. Se asocia a una clave `_formKey` que se usa para validar los formularios y manejar su estado.

Campos del formulario

Se definen varios campos para la entrada de datos:

- **Campos de texto** (TextFormField) para ingresar datos como nombre, apellidos, CURP, etc.
- **Campos de selección** (DropDownButtonFormField) para elegir opciones como el título de cortesía, tipo de sangre y género.

Validación: Cada campo tiene una validación usando el método `validator`. Por ejemplo, para los campos de texto, se valida si el valor es nulo o está vacío.

Campo de fecha de nacimiento

TextFormField para la fecha de nacimiento, que usa un `TextEditingController` para manejar el valor. El campo de texto está configurado para abrir un calendario cuando el usuario hace tap en él (`onTap`), y la función `_selectDate` se encarga de abrir un selector de fechas.

Campo para mostrar y tomar fotos

En un Row, se muestra una imagen si se ha seleccionado una fotografía. Si no se ha seleccionado, muestra el texto "Sin imagen".

Botón para registrar

Un botón de registro que, al ser presionado, llama a la función registerPerson para registrar la persona. Esta función maneja el envío de datos al servidor y muestra un mensaje de éxito o error.

Mostrar mensajes

Si message no está vacío, muestra un mensaje que indica el resultado de la operación de registro (éxito o error). El color del texto cambia según el valor de success.

Estilización

- Se utiliza la fuente GoogleFonts.comicNeue para muchos de los textos.
- Se usan colores personalizados y estilos de botón como fondo, sombra y bordes redondeados para mejorar la apariencia de la interfaz.

Archivo de Arranque de la aplicación

```
main.dart x
direccion_general_flutter > lib > main.dart > HospitalApp
1 import 'package:flutter/material.dart';
2 import 'login_screean.dart';
3 import 'package:intl/date_symbol_data_local.dart'; // Importa intl para inicializar el formato de fecha
4 import 'View/home_screen.dart';
5 import 'splash_screen.dart';
6
7 void main() async {
8   WidgetsFlutterBinding.ensureInitialized(); // Asegúrate de que los widgets estén inicializados
9   await initializeDateFormatting('es_ES', null); // Inicializa el formato de fecha para español
10  runApp(const HospitalApp());
11 }
12
13 class HospitalApp extends StatelessWidget {
14   const HospitalApp({super.key});
15
16   @override
17   Widget build(BuildContext context) {
18     return MaterialApp(
19       debugShowCheckedModeBanner: false,
20       title: 'Flutter Hospital App',
21       theme: ThemeData(primarySwatch: Colors.blue),
22       initialRoute: '/', // Ruta inicial para mostrar el SplashScreen
23       routes: {
24         '/': (context) => const SplashScreen(), // SplashScreen como ruta inicial
25         '/login': (context) => const LoginScreen(), // Ruta para la pantalla de login
26         '/home': (context) => const HomeScreen(area: '', personalId: 0), // Ruta para la pantalla principal
27       },
28     ); // MaterialApp
29   }
30 }
```

ILUSTRACIÓN 112. ARCHIVO MAIN.DART

Este código es la estructura básica que da inicio a todo el proyecto en Flutter, el archivo main configura la entrada para los emuladores multiplataforma para Android, iOS, Web y otros, por tanto, siempre está presente en la construcción de cualquier proyecto de desarrollo en flutter.

Imports:

- **import 'package:flutter/material.dart';**: Importa el paquete principal de Flutter para poder utilizar sus widgets y funcionalidades.
- **import 'login_screean.dart';**: Importa el archivo login_screean.dart, que probablemente contiene la pantalla de inicio de sesión de la aplicación.
- **import 'package:intl/date_symbol_data_local.dart';**: Importa el paquete intl para inicializar los símbolos de fecha locales. En este caso, se utiliza para configurar el formato de fecha en español (España).
- **import 'View/home_screen.dart';**: Importa el archivo home_screen.dart, que probablemente contiene la pantalla principal de la aplicación.
- **import 'splash_screen.dart';**: Importa el archivo splash_screen.dart, que generalmente contiene la pantalla de bienvenida o inicio de la aplicación.

Función main:

- **WidgetsFlutterBinding.ensureInitialized();**: Este método asegura que los widgets de Flutter estén correctamente inicializados antes de que la aplicación comience a ejecutarse. Es necesario cuando se utilizan funciones async en la función main().
- **await initializeDateFormatting('es_ES', null);**: Inicializa el paquete intl con el formato de fecha para España (es_ES). Esto asegura que las fechas se muestren correctamente según las convenciones locales de ese idioma y región.
- **runApp(const HospitalApp());**: Llama a la función runApp() para ejecutar la aplicación y pasar el widget HospitalApp como raíz de la aplicación.

Clase HospitalApp:

- **class HospitalApp extends StatelessWidget**: Define la clase HospitalApp, que extiende StatelessWidget, lo que significa que la clase es un widget sin estado, es decir, no cambia después de su inicialización.
- **@override Widget build(BuildContext context)**: La función build es llamada cuando se necesita dibujar la interfaz de usuario del widget. En este caso, retorna un MaterialApp, que es el widget base para las aplicaciones Flutter con diseño Material.
- **debugShowCheckedModeBanner: false**: Desactiva la etiqueta de depuración que normalmente aparece en la esquina superior derecha de la aplicación en modo de desarrollo.

- **title: 'Flutter Hospital App':** Establece el título de la aplicación, que será utilizado por los sistemas operativos en la pantalla de inicio.
- **theme: ThemeData(primarySwatch: Colors.blue):** Define el tema de la aplicación. En este caso, establece un esquema de colores utilizando Colors.blue como el color principal.
- **initialRoute: '/':** Establece la ruta inicial de la aplicación. En este caso, la ruta inicial es /, que corresponderá al SplashScreen.
- **routes: {...}:** Define las rutas disponibles en la aplicación. Las rutas son mapeadas de la siguiente manera:
 - **/:** Muestra la pantalla de inicio o splash (SplashScreen).
 - **/login:** Muestra la pantalla de inicio de sesión (LoginScreen).
 - **/home:** Muestra la pantalla principal de la aplicación (HomeScreen), pasando dos parámetros: area y personald.

Este código configura la estructura básica de una aplicación Flutter que tiene múltiples pantallas (Splash, Login y Home) y soporta la localización para mostrar fechas en español. La navegación entre pantallas se maneja a través de rutas definidas en el MaterialApp.

Carga Previa de la Aplicación

```
splash_screen.dart X
direccion_general_flutter > lib > splash_screen.dart > _SplashScreenState > _navigateToNextScreen
1  import 'package:flutter/material.dart';
2  import 'package:shared_preferences/shared_preferences.dart';
3  import 'View/home_screen.dart';
4
5  class SplashScreen extends StatefulWidget {
6    const SplashScreen({super.key});
7
8    @override
9    _SplashScreenState createState() => _SplashScreenState(); // Cambiado de _SplashScreenS
10 }
11
12 class _SplashScreenState extends State<SplashScreen> { // Cambiado de _SplashScreenState
13   @override
14   void initState() { // Cambiado de _SplashScreenState...
15
16     Future<void> _navigateToNextScreen() async {
17       await Future.delayed(const Duration(seconds: 5)); // Tiempo de la carga
18       final sessionData = await _checkSession();
19
20       if (sessionData != null) {
21         // Navega a la pantalla principal con los datos de sesión
22         Navigator.pushReplacement(
23           context, // Don't use 'BuildContext's across async gaps. Try rewriting the code
24           MaterialPageRoute(
25             builder: (context) => HomeScreen(
26               area: sessionData['selectedArea'],
27               personaId: sessionData['personaId'],
28             ), // HomeScreen
29           ), // MaterialPageRoute
30         );
31       } else {
32         // Navega a la pantalla de inicio de sesión si no hay sesión activa
33         Navigator.pushReplacementNamed(context, '/login'); // Don't use 'BuildContext's ac
34       }
35     }
36   }
37 }
38 }
```

ILUSTRACIÓN 113. PRIMERA PARTE DEL ARCHIVO SPLASH_SCREEN.DART

Este código implementa una pantalla de inicio (SplashScreen) en una aplicación del Dirección General Hospitalaria en Flutter que verifica si hay una sesión activa antes de navegar a la siguiente pantalla.

Imports:

- **import 'package:flutter/material.dart';** Importa el paquete principal de Flutter para usar los widgets y funcionalidades de diseño Material.
- **import 'package:shared_preferences/shared_preferences.dart';** Importa el paquete shared_preferences que se utiliza para almacenar y recuperar datos persistentes de forma sencilla (como el ID de sesión y el área seleccionada).
- **import 'View/home_screen.dart';** Importa el archivo home_screen.dart, que contiene la pantalla principal de la aplicación.

Clase SplashScreen:

- **class SplashScreen extends StatefulWidget:** Define un widget de estado (StatefulWidget), lo que significa que puede cambiar de estado durante la ejecución de la aplicación.
- **@override _SplashScreenState createState() => _SplashScreenState();** Crea el estado asociado al SplashScreen mediante la clase _SplashScreenState.

Clase _SplashScreenState:

- **class _SplashScreenState extends State<SplashScreen>:** La clase _SplashScreenState define el estado de la pantalla de inicio. Esta clase maneja la lógica de la pantalla, como la navegación.
- **@override void initState():** El método initState es llamado cuando el widget es creado. Aquí se utiliza para ejecutar la función _navigateToNextScreen() al inicio, es decir, para comenzar el proceso de navegación cuando se inicia la pantalla.

Método _navigateToNextScreen:

- **Future<void> _navigateToNextScreen():** Este método se encarga de verificar la sesión después de un retraso de 5 segundos (simulando una pantalla de inicio o carga).
- **await Future.delayed(const Duration(seconds: 5));** Añade una demora de 5 segundos antes de proceder, para simular el tiempo de carga.

- **final sessionData = await _checkSession();** Llama al método `_checkSession()` para verificar si hay datos de sesión guardados.
- **if (sessionData != null):** Si se encuentran los datos de sesión, navega a la pantalla principal (HomeScreen) y pasa los datos de la sesión (como `personalId` y `selectedArea`).
- **Navigator.pushReplacement:** Este método reemplaza la pantalla actual con la nueva pantalla (en este caso, HomeScreen o la pantalla de login).
- **else:** Si no hay datos de sesión (es decir, el usuario no está autenticado), navega a la pantalla de inicio de sesión (/login).

Método `_checkSession`:

- **Future<Map<String, dynamic>?> _checkSession():** Este método verifica si hay datos de sesión almacenados en las preferencias compartidas usando el paquete `shared_preferences`.
- **SharedPreferences prefs = await SharedPreferences.getInstance();** Obtiene una instancia de `SharedPreferences`, que es una manera de almacenar pequeños datos clave-valor.
- **final int? personalId = prefs.getInt('personalId');** Recupera el valor asociado con la clave 'personalId', que se espera que sea un número entero (ID de la persona).
- **final String? selectedArea = prefs.getString('selectedArea');** Recupera el valor asociado con la clave 'selectedArea', que se espera que sea una cadena de texto.
- **if (personalId != null && selectedArea != null):** Si ambos valores son no nulos, los devuelve como un mapa con los datos de sesión.
- **return null;** Si no se encuentra el `personalId` o `selectedArea`, significa que no hay sesión activa, y el método devuelve `null`.

```

40 Future<Map<String, dynamic>?> _checkSession() async {
41   SharedPreferences prefs = await SharedPreferences.getInstance();
42   final int? personalId = prefs.getInt('personalId');
43   final String? selectedArea = prefs.getString('selectedArea');
44
45   if (personalId != null && selectedArea != null) {
46     return {'personalId': personalId, 'selectedArea': selectedArea};
47   }
48   return null;
49 }

```

ILUSTRACIÓN 114. FUTURE O PROMESA DEL INICIO DE SESIÓN PARA EL USUARIO.

Este método verifica si existen datos de sesión almacenados en las preferencias compartidas (como el ID de la persona y el área seleccionada). Si ambos valores están presentes, devuelve un mapa

con esos datos. Si no están presentes, devuelve null, lo que indica que no hay sesión activa o que el usuario no está autenticado.

```

51 @override
52 Widget build(BuildContext context) {
53   return Scaffold(
54     backgroundColor: Colors.white,
55     body: Center(
56       child: Column(
57         mainAxisAlignment: MainAxisAlignment.spaceBetween,
58         crossAxisAlignment: CrossAxisAlignment.center,
59         children: [
60           Expanded(
61             child: Column(
62               mainAxisAlignment: MainAxisAlignment.center,
63               crossAxisAlignment: CrossAxisAlignment.center,
64               children: [
65                 Image.asset('assets/HPC-DG.png', width: 350),
66                 const SizedBox(height: 90),
67                 const CircularProgressIndicator(
68                   valueColor: AlwaysStoppedAnimation<Color>(Colors.blue),
69                 ), // CircularProgressIndicator
70               ],
71             ), // Column
72           ), // Expanded
73           Padding(
74             padding: const EdgeInsets.only(bottom: 40.0),
75             child: Image.asset(
76               'assets/Jaguar Negro - Corporation.png',
77               width: 80,
78             ), // Image.asset
79           ), // Padding
80         ],
81       ), // Column
82     ), // Center
83   ); // Scaffold
84 }
85 }
86

```

ILUSTRACIÓN 115. ESTRUCTURA VISUAL DEL SPLAS_SCREEN.DART

Método build

Este método se llama cuando la vista de un widget debe construirse. En este caso, está construyendo un widget para la pantalla de inicio (Splash Screen).

Uso de Scaffold:

- **Scaffold:** Es un widget que proporciona una estructura básica de la pantalla, como una barra de navegación, un cuerpo y una fondo. En este caso, solo se usa el cuerpo (body) y el fondo.
- **backgroundColor: Colors.white:** Establece el color de fondo de la pantalla como blanco.
- **Center:** Centra el contenido dentro del cuerpo de la pantalla. Todo lo que esté dentro de este widget se colocará en el centro de la pantalla.

Uso de Column para la disposición vertical:

- **Column:** Es un widget que coloca sus hijos en una disposición vertical (uno encima del otro).

- **mainAxisAlignment: MainAxisAlignment.spaceBetween:** Alinea los hijos a lo largo del eje principal (vertical en este caso), distribuyendo el espacio de manera que haya un espacio entre ellos, pero no al principio o al final.
- **crossAxisAlignment: CrossAxisAlignment.center:** Alinea los hijos a lo largo del eje transversal (horizontal), centrando los elementos.

Primer Column dentro de Expanded

- **Expanded:** Hace que este Column ocupe el máximo espacio disponible dentro del Column principal. Asegura que el contenido dentro de este Column se expanda y se distribuya correctamente en el espacio restante.

Dentro de este Column:

- **Image.asset('assets/HPC-DG.png', width: 350):** Muestra una imagen que está ubicada en la carpeta de activos (assets/HPC-DG.png). La imagen tiene un ancho de 350 píxeles.
- **SizedBox(height: 90):** Agrega un espacio vacío de 90 píxeles de altura entre los elementos. Sirve para separar la imagen y el indicador de progreso.
- **CircularProgressIndicator:** Muestra un círculo de carga que gira (un indicador de progreso). El color del indicador se establece en azul mediante AlwaysStoppedAnimation <Color>(Colors.blue).

Imagen en la parte inferior

- **Padding:** Añade un espacio de relleno alrededor de su hijo, en este caso, solo en la parte inferior de la imagen. El valor es 40.0, lo que significa que la imagen estará separada 40 píxeles del borde inferior de la pantalla.
- **Image.asset('assets/Jaguar Negro - Corporation.png', width: 80):** Muestra otra imagen ubicada en la carpeta de activos (assets/Jaguar Negro - Corporation.png). Esta imagen tiene un ancho de 80 píxeles.

Este código construye una pantalla de inicio en la que:

En la parte superior, se muestra una imagen grande (HPC-DG.png), Justo debajo de la imagen, se muestra un círculo de carga (CircularProgressIndicator) para indicar que se está realizando alguna operación (por ejemplo, la espera mientras la aplicación carga). En la parte inferior, se muestra otra imagen pequeña (Jaguar Negro - Corporation.png), separada del borde inferior de la pantalla por un espacio de 40 píxeles.

Registro de Usuarios Primera Parte

```

user_screen.dart X
direccion_general_flutter > lib > user_screen.dart > _UserScreenState
1 import 'dart:convert';
2 import 'package:direccion_general_flutter/login_screean.dart';
3 import 'package:http/http.dart' as http;
4 import 'package:flutter/material.dart';
5 import 'package:google_fonts/google_fonts.dart';
6
7 class UserScreen extends StatefulWidget {
8   const UserScreen({super.key});
9
10  @override
11  _UserScreenState createState() => _UserScreenState(); Invalid use of a private type in a public A
12 }
13
14 class _UserScreenState extends State<UserScreen> {
15   String username = '';
16   String email = '';
17   String phoneNumber = '';
18   String password = '';
19   String repeatPassword = '';
20   String errorMessage = '';
21   String successMessage = '';
22   int? personaID; // Esto debe estar asignado a un valor válido
23
24   // Validación para los campos vacíos
25   bool validateFields() { ...
26
27   Future<void> registerUser() async {
28     // Validar que los campos no estén vacíos
29     if (!validateFields()) {
30       return; // Si alguna validación falla, no se ejecuta el registro
31     }
32
33     final user = { ...
34
35     try {
36       final response = await http.post(
37         Uri.parse('https://back-end-hospital2-0.onrender.com/tbb_usuarios/'),
38         headers: {'Content-Type': 'application/json'},
39         body: json.encode(user),
40       );
41
42       if (response.statusCode == 200) { ...
43     } else {
44       // Si la respuesta no es exitosa, intenta manejarla
45       try {
46         final errorData = json.decode(response.body);
47         setState(() { ...
48       } catch (e) {
49         setState(() { ...
50       }
51       resetForm(); // Resetear el formulario en caso de error
52     } catch (error) {
53       setState(() { ...
54       resetForm(); // Resetear el formulario en caso de error
55     }
56   }
57
58   Future<void> fetchPersonaID() async { ...
59
60   void resetForm() { ...
61
62   @override
63   void initState() { ...
64
65

```

ILUSTRACIÓN 116. ARCHIVO USER_SCREEN.DART PARA EL REGISTRO DE USUARIO.

Este código en Flutter describe una pantalla de usuario (UserScreen) que permite registrar un nuevo usuario. La lógica abarca la validación de campos, la interacción con una API para registrar el usuario y la gestión de errores.

Definición de la clase UserScreen

- **UserScreen** es un widget con estado (StatefulWidget), lo que significa que la UI cambiará dinámicamente, basándose en los cambios del estado.
- **createState()** crea y devuelve el estado de este widget, que está definido en la clase `_UserScreenState`.

Variables de estado

Estas variables almacenan la información que el usuario introduce en los campos del formulario y mensajes de error o éxito.

- **personalID** es opcional (usando int?) y almacenará el identificador de la persona. Este se obtiene al interactuar con la API para asegurar que se registre con un ID válido.

Método de validación de campos

Este método valida los campos del formulario:

- Verifica que no haya campos vacíos.
- Compara las contraseñas para asegurarse de que coinciden.
- Valida que el personalID sea válido (un entero positivo).

Si alguna validación falla, se muestra un mensaje de error.

Método para registrar al usuario

Este método maneja el registro del usuario:

- Primero valida los campos con `validateFields()`.
- Luego, construye un objeto JSON con los datos del usuario.
- Hace una solicitud HTTP POST a la API para registrar al usuario.
- Si la respuesta de la API es exitosa (código 200), muestra un mensaje de éxito, resetea el formulario y redirige a la pantalla de inicio de sesión (LoginScreen).
- Si la respuesta de la API es un error, muestra el mensaje de error correspondiente.

Método para obtener el personalID

Este método realiza una solicitud HTTP GET para obtener una lista de personas desde la API.

- Si la respuesta es exitosa, extrae el personalID del último elemento de la lista.
- Si ocurre un error, maneja el error y muestra un mensaje de error.

Método resetForm

Este método restablece todos los campos del formulario y los mensajes de error o éxito.

Método initState

- initState() es el primer método que se llama cuando el widget se inicializa. Aquí se usa para obtener el personalID de la API cuando la pantalla se carga por primera vez.

Registro de Usuarios Segunda parte

```

159 @override
160 Widget build(BuildContext context) {
161   return Scaffold(
162     appBar: AppBar(
163       title: Row(
164         children: [
165           Image.asset( // Image.asset ...
166             const SizedBox(width: 8),
167             Flexible( // Flexible ...
168               ],
169             ), // Row
170           ), // AppBar
171         body: Padding(
172           padding: const EdgeInsets.all(16.0),
173           child: Column(
174             crossAxisAlignment: CrossAxisAlignment.stretch,
175             children: [
176               TextField(
177                 onChanged: (value) => username = value,
178                 decoration: const InputDecoration(labelText: 'Usuario'),
179               ), // TextField
180               TextField(
181                 onChanged: (value) => email = value,
182                 decoration: const InputDecoration(labelText: 'Correo Electrónico'),
183                 keyboardType: TextInputType.emailAddress,
184               ), // TextField
185               TextField(
186                 onChanged: (value) => phoneNumber = value,
187                 decoration: const InputDecoration(labelText: 'Número Telefónico Móvil'),
188                 keyboardType: TextInputType.phone,
189               ), // TextField
190               TextField(
191                 onChanged: (value) => password = value,
192                 decoration: const InputDecoration(labelText: 'Contraseña'),
193                 obscureText: true,
194               ), // TextField
195               TextField(
196                 onChanged: (value) => repeatPassword = value,
197                 decoration: const InputDecoration(labelText: 'Repetir Contraseña'),
198                 obscureText: true,
199               ), // TextField
200               const SizedBox(height: 20),
201               ElevatedButton(
202                 onPressed: registerUser,
203                 child: const Text('Registrar Usuario'),
204               ), // ElevatedButton
205               const SizedBox(height: 20),
206               if (successMessage.isNotEmpty)
207                 Text(
208                   successMessage,
209                   style: const TextStyle(color: Colors.green),
210                 ), // Text
211               if (errorMessage.isNotEmpty)
212                 Text(
213                   errorMessage,
214                   style: const TextStyle(color: Colors.red),
215                 ), // Text
216             ],
217           ), // Column
218         ), // Padding
219       ); // Scaffold
220   }
221 }

```

ILUSTRACIÓN 117. CONFIGURACIÓN DEL FORMULARIO DE USER_SCREEN.DART

El código corresponde a la implementación de la interfaz gráfica para el registro de un usuario en Flutter, utilizando un Scaffold que organiza los componentes en una pantalla de registro.

Scaffold: Es el widget principal que proporciona la estructura básica de la interfaz de la aplicación de esta sección de registro de usuarios, como la barra de navegación, el cuerpo de la pantalla.

AppBar:

- El AppBar contiene el título de la pantalla, que está compuesto por un logo y el texto "Formulario de Registro de Usuario".
- El logo es una imagen cargada desde los assets del proyecto ('assets/direccion_general_logo.png').
- El texto tiene un estilo personalizado con la fuente GoogleFonts.comicNeue, y si el texto es largo, se asegura de que no se desborde utilizando overflow: TextOverflow.ellipsis y softWrap: false.

Cuerpo (Body):

- El cuerpo de la pantalla está contenido dentro de un Padding que proporciona un margen de 16 píxeles alrededor de todos los elementos dentro de la columna.
- La columna (Column) contiene una serie de widgets alineados verticalmente (crossAxisAlignment: CrossAxisAlignment.stretch):

Campos de texto:

- **Usuario:** Un campo de texto donde el usuario puede ingresar su nombre de usuario. El valor ingresado se almacena en la variable username.
- **Correo Electrónico:** Un campo de texto para ingresar el correo electrónico. El tipo de teclado está configurado para permitir solo direcciones de correo electrónico (keyboardType: TextInputType.emailAddress).
- **Número Telefónico Móvil:** Un campo de texto para ingresar el número de teléfono. El tipo de teclado está configurado para permitir solo números de teléfono (keyboardType: TextInputType.phone).
- **Contraseña:** Un campo de texto para la contraseña, con el texto oculto mediante obscureText: true.
- **Repetir Contraseña:** Otro campo para repetir la contraseña, también con el texto oculto.

Botón de Registro:

Un botón de tipo ElevatedButton que, al ser presionado, llama a la función registerUser, que es la encargada de registrar al usuario.

Mensajes de Error y Éxito:

- Si el registro es exitoso, se muestra el `successMessage` en verde.
- Si ocurre un error, se muestra el `errorMessage` en rojo. Estos mensajes de éxito o error se actualizan dinámicamente según el estado de la aplicación.

Diseño Responsivo:

- Se utiliza un `Flexible` para el título del `AppBar`, lo que permite que el texto se ajuste adecuadamente en pantallas con poco espacio horizontal.

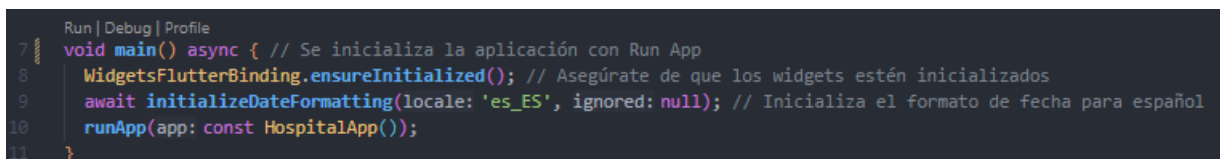
Esta sección del código proporciona una interfaz de usuario con campos de entrada para que el usuario ingrese su nombre de usuario, correo electrónico, número telefónico y contraseñas. Además, incluye un botón para registrar al usuario y mostrar mensajes de error o éxito según el resultado del registro.

Ejecución de la Aplicación

"De acuerdo con la codificación mencionada previamente, se presentarán los resultados obtenidos, mostrando las principales pantallas desarrolladas en Flutter. A continuación, se detallan las pantallas diseñadas y su funcionalidad."

Para arrancar el proyecto dentro de un dispositivo virtual o físico, es necesario, que la configuración este en modo de depuración, esto se realiza desde la configuración del dispositivo.

Para inicializar la carga del proyecto se ejecuta desde terminal el comando `flutter run` , o desde el documento `main.dart`.

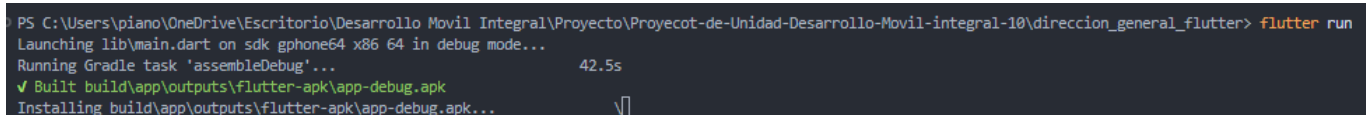


```

7  void main() async { // Se inicializa la aplicación con Run App
8    WidgetsFlutterBinding.ensureInitialized(); // Asegúrate de que los widgets estén inicializados
9    await initializeDateFormatting(locale: 'es_ES', ignored: null); // Inicializa el formato de fecha para español
10   runApp(app: const HospitalApp());
11 }

```

ILUSTRACIÓN 118. EJECUCIÓN DEL PROYECTO DESDE EL ARCHIVO MAIN.DART



```

PS C:\Users\piano\OneDrive\Escritorio\Desarrollo Movil Integral\Proyecto\Proyecot-de-Unidad-Desarrollo-Movil-integral-10\direccion_general_flutter> flutter run
Launching lib\main.dart on sdk gphone64 x86 64 in debug mode...
Running Gradle task 'assembleDebug'... 42.5s
✓ Built build\app\outputs\flutter-apk\app-debug.apk
Installing build\app\outputs\flutter-apk\app-debug.apk...

```

ILUSTRACIÓN 119. COMANDO FLUTTER RUN EN TERMINAL PARA INICIALIZAR INSTALACIÓN DE APLICACIÓN.

Pantalla de Carga antes de ejecutar la aplicación:

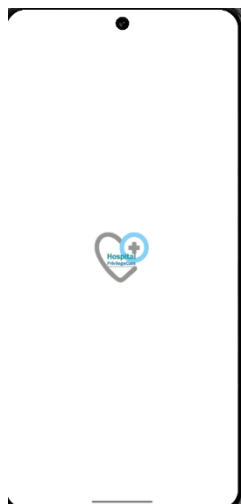


ILUSTRACIÓN 121. PANTALLA DE INICIO DE LA APLICACIÓN EN MODO CLARO.



ILUSTRACIÓN 120. PANTALLA DE INICIO DE LA APLICACIÓN EN MODO OSCURO

La ilustración 120 y 121 muestra visualmente que la aplicación se ha instalado correctamente en el dispositivo. En ella se presenta el logo de la aplicación, que permite identificarla fácilmente. Además, la temática se adapta automáticamente al entorno de configuración del teléfono, ya sea en modo claro o modo oscuro. También se ajusta al tipo y tamaño de pantalla del dispositivo móvil, ya sea físico o virtual, garantizando una experiencia óptima para el usuario.



ILUSTRACIÓN 123. LA APLICACIÓN CARGA LOS SERVICIOS QUE NECESITA PARA SU CONSUMO.



ILUSTRACIÓN 122. CARGA DE LOS SERVICIOS EN PANTALLA

La precarga de servicios es un paso fundamental para realizar las llamadas correspondientes al Back-End antes de mostrar la pantalla de inicio. Como se observa en la ilustración 122, es necesario que los registros estén habilitados previamente. Uno de los servicios iniciales es el login de la aplicación, el cual incluye un spinner donde se muestran los departamentos disponibles. Este proceso permite el acceso a los usuarios previamente registrados. Además, se incluye un botón que ofrece la opción de mantener la sesión activa incluso al cerrar la aplicación. De igual forma, se han implementado servicios para el registro mediante redes sociales, ampliando las opciones de acceso para los usuarios.

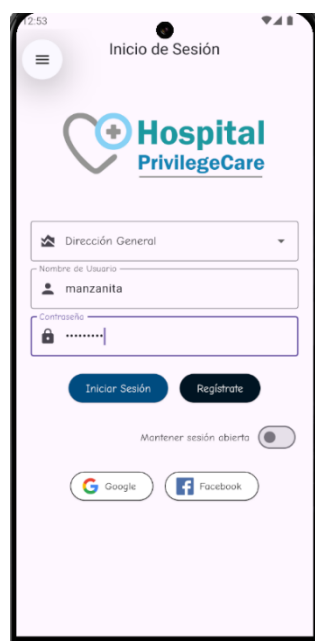


ILUSTRACIÓN 126. LOGEO DE USUARIO REGISTRADO

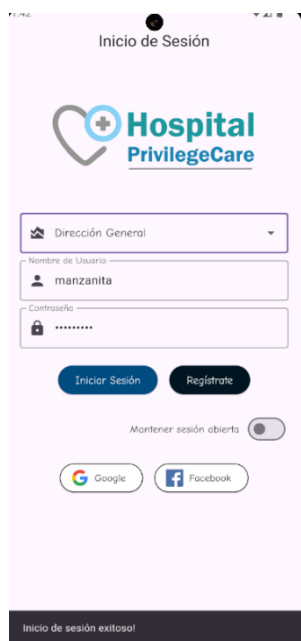


ILUSTRACIÓN 124. VALIDACIÓN DE USUARIO



ILUSTRACIÓN 125. LOGEO DE USUARIO REGISTRADO

Si un usuario se ha registrado desde el entorno web del sistema de gestión hospitalaria, puede acceder a su cuenta desde la aplicación móvil. Esto es posible porque la información del usuario ha sido validada y encontrada en la base de datos. Los datos se devuelven y se muestran en el panel de control (dashboard) del usuario. La información visible ha sido filtrada en la configuración del Front-End para garantizar que el usuario solo vea sus datos personales, como su fotografía de registro, correo electrónico y área a la que pertenece, identificándolo como parte del personal de esa área. Además, se incluye el privilegio de visualización relacionado con su área. La configuración se maneja de acuerdo con los identificadores mencionados en la sección anterior, donde se explican el Id_Usuario y el Id_Area, que son esenciales para validar y filtrar los datos que se presentan en el Front-End.

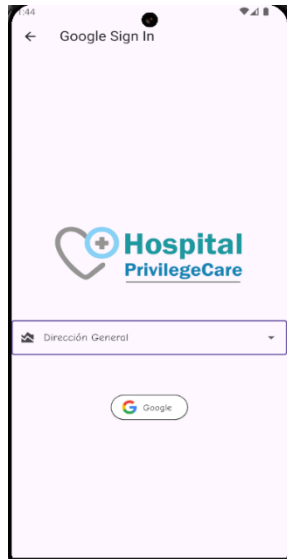


ILUSTRACIÓN 129. SELECCIÓN DE ÁREA.

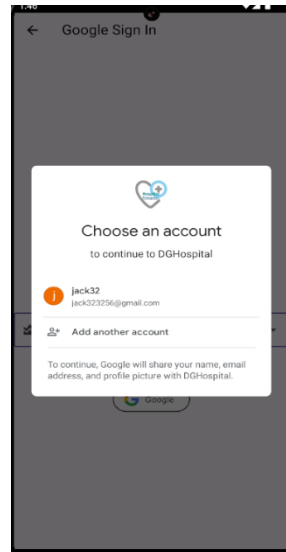


ILUSTRACIÓN 128. AUTENTIFICACIÓN DE USUARIO CON
GOOGLE.

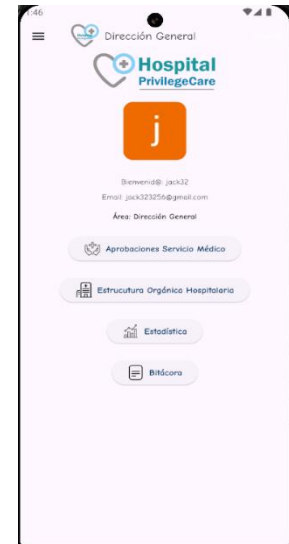


ILUSTRACIÓN 127. SE MUESTRAN LOS DATOS DEL
USUARIO DE SU CUENTA DE GOOGLE.

Si un usuario decide utilizar su cuenta personal de Google para acceder, solo necesita seleccionar el área a la que tendrá acceso. El sistema verificará la existencia de su cuenta y, una vez confirmada, le otorgará los permisos correspondientes. Este proceso también configurará su cuenta en la base de datos NoSQL de Google Firebase, donde su información será almacenada de manera segura, garantizando la integridad y privacidad de los datos del usuario en la plataforma.

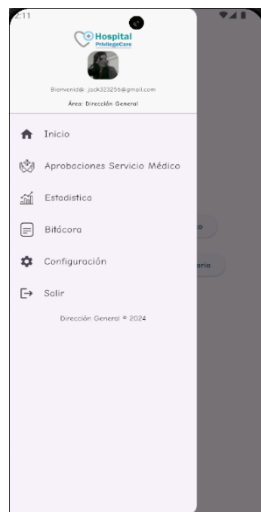


ILUSTRACIÓN 132. MENÚ LATERAL CON CREDENCIALES
DEL USUARIO



ILUSTRACIÓN 131. CONSULTA DE SOLICITUDES

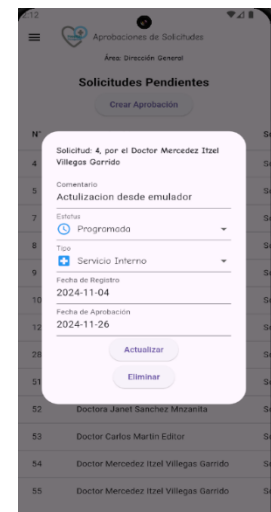


ILUSTRACIÓN 130. MANIPULACIÓN DE DATOS

El usuario registrado tiene acceso a las diferentes secciones disponibles en su panel de control (dashboard). La primera de ellas permite gestionar las solicitudes realizadas en el hospital, las cuales están identificadas por el personal que las emite. Cada solicitud se administra de manera

independiente, lo que facilita su procesamiento en tiempo real. Esto garantiza que todos los datos disponibles puedan ser consultados desde cualquier dispositivo, sin comprometer el rendimiento del sistema. La interfaz, aunque sencilla, está diseñada con elementos visuales que facilitan la identificación de tareas y la ejecución de acciones necesarias para una toma de decisiones eficiente.



ILUSTRACIÓN 135. REGISTRO DE ESTADÍSTICAS

El formulario contiene los siguientes campos: Título de Cortesía (Doctor), Nombre (Hector), Primer Apellido (Alvaro), Segundo Apellido (Moreno), CURP (EALVOSNNE00SA00NE), Tipo Sangre (O+), Género (Masculino) y Fecha de Nacimiento (2024-12-02). Hay botones para 'Tomar Foto', 'Seleccionar Imagen' y 'Registrar Persona'.

ILUSTRACIÓN 134. FORMULARIO DE REGISTRO DE DATOS

El formulario contiene los siguientes campos: Usuario (hector), Correo Electrónico (hectorin@gmail.com), Número Telefónico Móvil (44788112231), Contraseña (*****), Repetir Contraseña (*****). Hay un botón para 'Registrar Usuario'.

ILUSTRACIÓN 133. FORMULARIO DE REGISTRO DE USUARIO

La ilustración 133 ejemplifica las estadísticas generadas a partir de los datos recopilados en la gestión de peticiones. Estas estadísticas incluyen información sobre el tipo de situación registrada, clasificada según su denominación y el nivel de atención establecido. Asimismo, se detallan los estatus de las peticiones, escalados en categorías como programadas, registradas, en proceso, canceladas y aprobadas. Además, se muestra el origen de las peticiones según los departamentos involucrados, considerando el rol o nivel jerárquico de los trabajadores del hospital.

Los datos utilizados para generar estas estadísticas se recopilan a través de un formulario global de pacientes y trabajadores. Este formulario centraliza la información necesaria para asignar credenciales de acceso al sistema. Los datos recopilados se publican en una base de datos SQL en la nube, donde se procesan y almacenan de manera segura. La captura de cada dato es obligatoria, ya que son esenciales para la generación de credenciales de usuario. Tanto en la base de datos SQL como en la NoSQL, la información se almacena de forma dinámica, lo que permite al usuario operar en ambos entornos sin inconvenientes.

Limitaciones del proyecto

1. El sistema está limitado al uso exclusivo del área de Dirección General, por lo que su desarrollo se centró únicamente en esta área en particular.
2. Dado que la aplicación aún se encuentra en fase de desarrollo, el tiempo dedicado fue limitado. Por ello, se recurrió a herramientas de despliegue gratuito para el Back-End, lo que ocasiona intermitencias en la consulta de datos y ralentiza las pruebas de funcionalidad.
3. Aunque la aplicación fue desarrollada en un entorno multiplataforma, su visualización se adapta principalmente a dispositivos móviles. Si bien es posible consultar el sistema desde navegadores web, su interfaz es básica en comparación con la versión web desarrollada en Vue.js.
4. La aplicación muestra únicamente las secciones esenciales para la interacción básica, dejando fuera algunas funcionalidades disponibles en la versión web.
5. Por el momento, la instalación de la aplicación se limita a un entorno local a través del entorno de desarrollo de Flutter o mediante la generación manual del archivo .apk. Aún no está gestionada para ser publicada en la tienda de aplicaciones Google Play Store.
6. La aplicación no está disponible para dispositivos iOS debido a limitaciones en la infraestructura del proyecto, por lo que no se ha realizado ningún tipo de prueba en ese entorno.
7. El sistema solo es compatible con dispositivos que ejecuten Android 10 o versiones superiores. Las versiones inferiores no son compatibles.
8. Debido a la intermitencia en la estructura de flujo de datos del Back-End, es posible que los datos no se procesen adecuadamente, lo que podría causar que el sistema se congele hasta restablecer la conexión.
9. Aunque el sistema requiere conexión a Internet, se configuró una estructura PWA (Progressive Web App) para almacenar datos en caché y memoria virtual, lo que permite procesar datos de manera offline y sincronizarlos una vez restablecida la conexión. Sin embargo, estas funcionalidades aún están en pruebas básicas y su desempeño ha sido limitado.
10. Algunas extensiones o librerías utilizadas en Flutter presentaron problemas de incompatibilidad entre sí, lo que requirió buscar alternativas o utilizar versiones más antiguas para garantizar la funcionalidad del proyecto.

11. Compatibilidad de Plugins y Paquetes: Algunos paquetes de Flutter no son completamente compatibles con todas las plataformas (Android, iOS, Web, etc.), lo que puede requerir desarrollos adicionales o soluciones alternativas.
12. Rendimiento en Dispositivos de Gama Baja: Aunque Flutter está optimizado para el rendimiento, en dispositivos de gama baja puede haber problemas como animaciones lentas o tiempos de carga prolongados debido al alto uso de recursos gráficos.
13. Curva de Aprendizaje de Dart: Para los desarrolladores nuevos en Flutter, aprender Dart, su lenguaje de programación, puede ser un desafío inicial, especialmente si vienen de lenguajes como JavaScript o Python.
14. Falta de Soporte para Funcionalidades Avanzadas en iOS: Algunas características nativas de iOS, como ciertas animaciones o accesos profundos al sistema, no están completamente soportadas en Flutter o requieren desarrollos específicos para esa plataforma.
15. Tamaño de la Aplicación: Las aplicaciones creadas con Flutter tienden a ser más grandes en tamaño inicial en comparación con las aplicaciones nativas, lo que puede ser un problema en la integración del código con otras secciones de este.
16. Actualizaciones Frecuentes: Flutter está en constante evolución, y las actualizaciones pueden romper la compatibilidad con versiones anteriores, lo que obliga a realizar ajustes frecuentes.
17. Soporte Limitado para Widgets Personalizados Complejos: Aunque Flutter tiene una amplia variedad de widgets, crear widgets personalizados muy complejos puede ser un proceso largo y técnicamente demandante.
18. Dependencia de la Comunidad: Muchas soluciones en Flutter dependen de la comunidad de desarrolladores, y algunas librerías importantes pueden carecer de soporte activo o documentación suficiente.
19. Integraciones con Sistemas Heredados: La integración con sistemas backend más antiguos o infraestructuras no diseñadas para aplicaciones modernas puede ser complicada debido a las diferencias tecnológicas.
20. Restricciones en la Personalización Nativa: Aunque Flutter permite acceder a funcionalidades nativas mediante canales de plataforma, este proceso puede ser más complicado que el desarrollo nativo, especialmente si se requieren características muy específicas de una plataforma.

Índice de Imágenes

Ilustración 1. Comparación Entre La Problemática Y El Componente Funcional En Las Operaciones De Atención Médica.	82
Ilustración 2. servicio que incorpora las operaciones de funcionalidad hospitalaria.....	85
Ilustración 3. diagrama de caso de uso de dirección general.	92
Ilustración 4. Organigrama del Hospital General.....	99
Ilustración 5. Estructura de Dirección General.	99
Ilustración 6. servicios médicos del hospital general de culiacán.....	100
Ilustración 7. servicios de apoyo en el hospital general de culiacán.	100
Ilustración 8. servicios de relación en el hospital general de culiacán.....	100
Ilustración 9. servicios médicos del hospital general de culiacán.....	101
Ilustración 10. servicios de enfermería del hospital general de culiacán.....	101
Ilustración 11. departamento administrativos del hospital general de culiacán	102
Ilustración 12. C) vista de carga (mockup).....	104
Ilustración 13. B) vista de carga (wireframe)	104
Ilustración 14. A) vista de carga (sketch).	104
Ilustración 15. A) Vista De Inicio De Sesión (Sketch).....	105
Ilustración 16. B) Vista de inicio de sesión (wireframe).	105
Ilustración 17. C) vista de inicio de sesión (mockup).....	105
Ilustración 18. C) vista de registro de persona (mockup)	106
Ilustración 19. B) vista de registro de personas (sketch).	106
Ilustración 20. B) vista de registro de personas (wireframe).....	106
Ilustración 21. C) Lista principal conforme al rol del usuario (mockup)	107
Ilustración 22. B) vista principal conforme al rol del usuario (wireframe).....	107
Ilustración 23. A) vista principal conforme al rol del usuario (sketch).	107
Ilustración 24. C) vista de solicitudes de aprobación (mockup).	108
Ilustración 25. B) vista de solicitudes de aprobación (wireframe).	108
Ilustración 26. A) vista de solicitudes de aprobación (sketch).....	108
Ilustración 27. C) vista de solicitudes de Actividades (mockup).	109
Ilustración 28. A) vista de monitorización de actividades (sketch).	109
Ilustración 29. B) vista de monitorización de actividades (wireframe).....	109
Ilustración 30. C) vista de información estadística representada en gráficas (mockup).	110
Ilustración 31. B) vista de información estadística representada en gráficas (wireframe).	110
Ilustración 32. A) vista de información estadística representada en gráficas (sketch).	110
Ilustración 33. C) vista de registros de ejecución (mockup).	111
Ilustración 34. B) vista de registros de ejecución (wireframe).	111
Ilustración 35. A) vista de registros de ejecución (sketch).	111
Ilustración 36. C) vista de términos y condiciones (mockup).	112
Ilustración 37. A) vista de términos y condiciones (sketch).....	112
Ilustración 38. B) vista de términos y condiciones (wireframe).	112
Ilustración 39. A) vista de acerca de nosotros (sketch).....	113
Ilustración 40. B) vista de acerca de nosotros (wireframe).	113

Ilustración 41. C) vista de acerca de nosotros (Mockup)	113
Ilustración 42. Modelo ER – Aprobación – Identificación de Atributos.....	115
Ilustración 43. Clasificación de Atributos.	115
Ilustración 44. Identificación con las relaciones directas a otras tablas existentes dentro de la base de datos.	116
Ilustración 45. Estructura de la Tabla de Aprobaciones en MySQL.....	117
Ilustración 46. Invocación de los Triggers incluidos en la tabla tbb_aprobaciones.....	117
Ilustración 47. primera parte del trigger After Insert	117
Ilustración 48. Segunda parte del trigger After Insert.	118
Ilustración 49. estructura del trigger Before Update.	119
Ilustración 50. primera sección del trigger after Update.....	119
Ilustración 51. Segunda sección del trigger after Update.	120
Ilustración 52. tercera sección del trigger after Update.	121
Ilustración 53. Primera sección de trigger After Delete.	121
Ilustración 54. Segunda sección de trigger After Delete.	122
Ilustración 55. Estructuración de la tabla Bitácora.	123
Ilustración 56. Inserción de registros de prueba a la tabla tbb_aprobaciones.	123
Ilustración 57. Actualización de datos de manera manual en MySQL.....	124
Ilustración 58. Eliminación de prueba dentro de la tabla tbb_aprobaciones.....	124
Ilustración 59. Registro de movimientos realizados en la tabla Bitácora.	125
Ilustración 60. Modelo reverso de los módulos que interactuaran con la tabla tbb_aprobaciones.	126
Ilustración 61. Identificación Del Proyecto En Fastapi.	127
Ilustración 62. Configuración de los CORS del proyecto.	127
Ilustración 63. rutas establecidas que se conectan con el Routes.	128
Ilustración 64. carpeta asignada para el alojamiento de los archivos estáticos.	128
Ilustración 65. Mensaje que se mostrara en el entorno de Swagger UI.	128
Ilustración 66. Distribución de carpetas en el entorno de desarrollo del Back-End.....	129
Ilustración 67. Importación de Módulos y Clases.	131
Ilustración 68. Definición de Enumeraciones.	132
Ilustración 69. Define tipos específicos de aprobaciones, como "Servicio Interno" o "Traslados".	132
Ilustración 70. Definición de la Clase Aprobaciones.	132
Ilustración 71. Importaciones del Schemas.	133
Ilustración 72. Esquema Base (TbbAprobacionesBase).....	133
Ilustración 73. Esquemas Específicos.....	134
Ilustración 74. Importaciones que se vinculan con el crud, schemas y modelos para la tabla tbb_aprobaciones.....	135
Ilustración 75. Configuración Inicial y dependencias de Base de datos.	135
Ilustración 76. Leer Todas las Aprobaciones.	135
Ilustración 77. Leer una Aprobación Específica.	136
Ilustración 78. Crear una Nueva Aprobación.....	136
Ilustración 79. Actualizar una Aprobación.....	136
Ilustración 80. Eliminar una Aprobación.	136
Ilustración 81. configuración get_aprobaciones.	137
Ilustración 82. Configuración de creación de aprobación.....	137

Ilustración 83. Actualización de la aprobación.	138
Ilustración 84. eliminación de Aprobación.	138
Ilustración 85. Visualización de la tabla tbb_aprobaciones dentro del entorno Swagger UI.....	139
Ilustración 86. Ecpoints de tbb_personas, tbb_aprobaciones y tbd_solicitudes.	140
Ilustración 87. Configuración de permisos, claves y usuario dentro de Aiven.....	141
Ilustración 88. importación de módulos.	141
Ilustración 89. Carga de las variables de entorno.	141
Ilustración 90. Prueba de conexión.....	142
Ilustración 91. vinculación del proyecto con el repositorio almacenado en GitHub.....	143
Ilustración 92. implementación de variables de entorno para el despliegue del proyecto.	143
Ilustración 93. despliegue en línea.	143
Ilustración 94. Repliegue de Swagger UI de manera remota.	143
Ilustración 95. Estructura de Carpetas del Proyecto de la App en Flutter.....	145
Ilustración 96. Archivo Custom_Drawer.Dart.....	146
Ilustración 97. Archivo google_custom_drawer.dart.	149
Ilustración 98. Archivo google.dart.	152
Ilustración 99. Archivo Areas_screean.dart.	155
Ilustración 100. Archivo Estadisticas.dart	157
Ilustración 101. Primera parte del archivo AprobacionesSM.dart	160
Ilustración 102. Segunda Parte Del Archivo Aprobacionessm.Dart.....	164
Ilustración 103. Archivo Home_screen.dart.....	169
Ilustración 104. archivo login_screen.dart	173
Ilustración 105. Tercera parte del archivo Login_screen.dart.....	177
Ilustración 106. Cuarta parte del login_screen.dart.....	178
Ilustración 107. quinta parte del login_screen.dart.....	179
Ilustración 108. Primera parte del Archivo Register_screen.dart	180
Ilustración 109. Segunda parte del archivo Register_screen.dart	182
Ilustración 110. Tercera parte del archivo register_screen.dart.	183
Ilustración 111. Cuarta parte del archivo register_sceen.dart	185
Ilustración 112. Archivo main.dart	187
Ilustración 113. primera parte del archivo Splash_Screen.dart	189
Ilustración 114. Future o Promesa del inicio de sesión para el usuario.....	191
Ilustración 115. Estructura visual del Splas_screen.dart	192
Ilustración 116. Archivo User_screen.dart para el registro de usuario.	194
Ilustración 117. Configuración del Formulario de User_screen.dart	196
Ilustración 118. Ejecución del proyecto desde el archivo main.dart	198
Ilustración 119. Comando flutter run en terminal para inicializar instalación de aplicación.....	198
Ilustración 120. Pantalla de Inicio de la aplicación en modo oscuro	199
Ilustración 121. Pantalla de Inicio de la aplicación en modo Claro.	199
Ilustración 122. Carga de los servicios en pantalla	199
Ilustración 123. La aplicación Carga los servicios que necesita para su consumo.	199
Ilustración 124 . Validación de usuario	200
Ilustración 125. Logeo de Usuario Registrado	200

Ilustración 126. Logeo de usuario Registrado	200
Ilustración 127. Se muestran los datos del usuario de su cuenta de Google.....	201
Ilustración 128. Autenticación de usuario con Google.....	201
Ilustración 129. Selección de área.	201
Ilustración 131. Manipulación de Datos	201
Ilustración 130. Consulta de Solicitudes	201
Ilustración 132. Menú Lateral con credenciales del usuario.....	201
Ilustración 135. Formulario de Registro de Usuario	202
Ilustración 134. Formulario de registro de datos	202
Ilustración 133. Registro de estadísticas.....	202

Bibliografía

- [1] G. Soto Estrada, L. Moreno Altamirano and D. Pahua Díaz, "Panorama epidemiológico de México, principales causas de morbilidad y mortalidad," *Revista de la Facultad de Medicina (México)*, vol. 59, no. 6, 2016.
- [2] Dirección General de Información en Salud, "Guía y Formatos para el Intercambio de Información en Salud Referente al Reporte de Información al Subsistema de Prestación de Servicios," Mexico, 2014.
- [3] Secretaria de Salud, "Manual de Procedimientos Estandarizados para la Notificación Convencional de Casos Nuevos de Enfermedades," Secretaria de Salud, Ciudad de México, 2014.
- [4] Secretaria de Salud, "Sistema de Vigilancia Epidemiológica Convencional," 9-11, 2021.
- [5] Dirección General de Epidemiología, "RHOVE (Red Hospitalaria de Vigilancia Epidemiológica)," Secretaria de Salud, Mexico, 2016.
- [6] C. Ayuzo del Valle and J. C. Villafuentes González, "Uso de software de simulación para mejorar los tiempos de atención en el departamento de emergencias," *SCIELO*, p. 2, May-Jun 2018.
- [7] D. Pastor Escuredo, "Digital Epidemiology," 3, p. 19, 2020.
- [8] I. R. Rojas Crotte, "ELEMENTOS PARA EL DISEÑO DE TÉCNICAS DE INVESTIGACIÓN: UNA PROPUESTA DE DEFINICIONES Y PROCEDIMIENTOS EN LA INVESTIGACIÓN CIENTÍFICA," *Tiempo de Educar*, vol. 12, no. 24, p. 22, 2011.
- [9] X. Ferré Grau and M. I. Sánchez Segura, *Desarrollo Orientado a Objetos con UML*, Madrid: Facultad de Informática – UPM, 2012.
- [10] C. F. Jiménez Castillo, *Fundamentos de SQL*, México, D.F.: McGraw-Hill, 2010.
- [11] Microsoft, "Microsoft Soporte Técnico," [Online]. Available: <https://support.microsoft.com/es-es/topic/definir-una-relaci%C3%B3n-en-un-diagrama-de-modelo-de-base-de-datos-e81d162b-65a1-4f2e-b57c-ee8a17de877e>.
- [12] Formadores IT, "Formadores IT," 28 03 2022. [Online]. Available: <https://formadoresit.es/que-es-microsoft-visio/>.

- [13] CIE, "Centro de Estudios de Inovación Diseño y Marketing," 2014. [Online]. Available: <https://cei.es/que-es-figma/>.
- [14] Figma, "Figma Learn," 2024. [Online]. Available: <https://help.figma.com/hc/en-us>.
- [15] Microsoft, "Documentation for Visual Studio Code," 10 11 2024. [Online]. Available: <https://code.visualstudio.com/docs>.
- [16] Python Software Foundation, "Python Documentation," 10 11 2024. [Online]. Available: <https://docs.python.org/3/>.
- [17] Oracle, "MySQL Documentation," 10 11 2024. [Online]. Available: <https://dev.mysql.com/doc/>.
- [18] Unir, "Framework," 22 09 2022. [Online]. Available: <https://unirfp.unir.net/revista/ingenieria-y-tecnologia/framework/>.
- [19] FastApi, "Documentation FastApi," 2024. [Online]. Available: <https://fastapi.tiangolo.com/>.
- [20] Flutter, "Flutter documentation," - - -. [Online]. Available: <https://flutter.dev/docs>.
- [21] SQLAlchemy, "SQLAlchemy 2.0 Documentation," 15 10 2024. [Online]. Available: <https://docs.sqlalchemy.org/>.
- [22] Overview, "Overview - Database Monitoring," 10 10 2024. [Online]. Available: <https://overview.io>.
- [23] Amazon Web Services, "Amazon Amplify - Full Stack Serverless Development," 10 10 2024 . [Online]. Available: <https://aws.amazon.com/amplify>.
- [24] Render, "Render - Cloud Hosting for Developers," 2024. [Online]. Available: <https://render.com>.
- [25] Autentia, "Guía para directivos y técnicos".
- [26] Secretaría de Salud, "Administración y Organización de los Distritos de Salud (aps-i mx)," Subsecretaría de Integración y Desarrollo del Sector Salud , Ciudad de México, 2020.
- [27] J. García Allen, "Psicología y Mente," 30 05 2015. [Online]. Available: <https://psicologiymente.com/psicologia/piramide-de-maslow>.

- [28] I. Padovan, "TEORÍAS DE LA MOTIVACIÓN.," 2020. [Online]. Available: https://bdigital.uncu.edu.ar/objetos_digitales/15664/teorias-de-la-motivacin.-aplicacin-prctica.pdf.
- [29] L. Araya Castillo and M. Pedreros Gajardo, "ANÁLISIS DE LAS TEORÍAS DE MOTIVACIÓN DE CONTENIDO," 2009. [Online]. Available: <https://www.redalyc.org/pdf/153/15333870004.pdf>.
- [30] Google, "Flutter Documentation," 11 10 2024. [Online]. Available: <https://flutter.dev/docs>.
- [31] McHugh, "TechTerms," 11 03 2020. [Online]. Available: <https://www.techterms.com/definition/sql>.
- [32] Microsoft, "Microsoft," 2024. [Online]. Available: <https://www.microsoft.com/en-us/microsoft-365/word>.
- [33] Microsoft, "Ayuda y aprendizaje de Word," [Online]. Available: <https://support.microsoft.com/word>.
- [34] Overview, "Overview - Database Monitoring," 10 10 2024. [Online]. Available: <https://overview.io>.